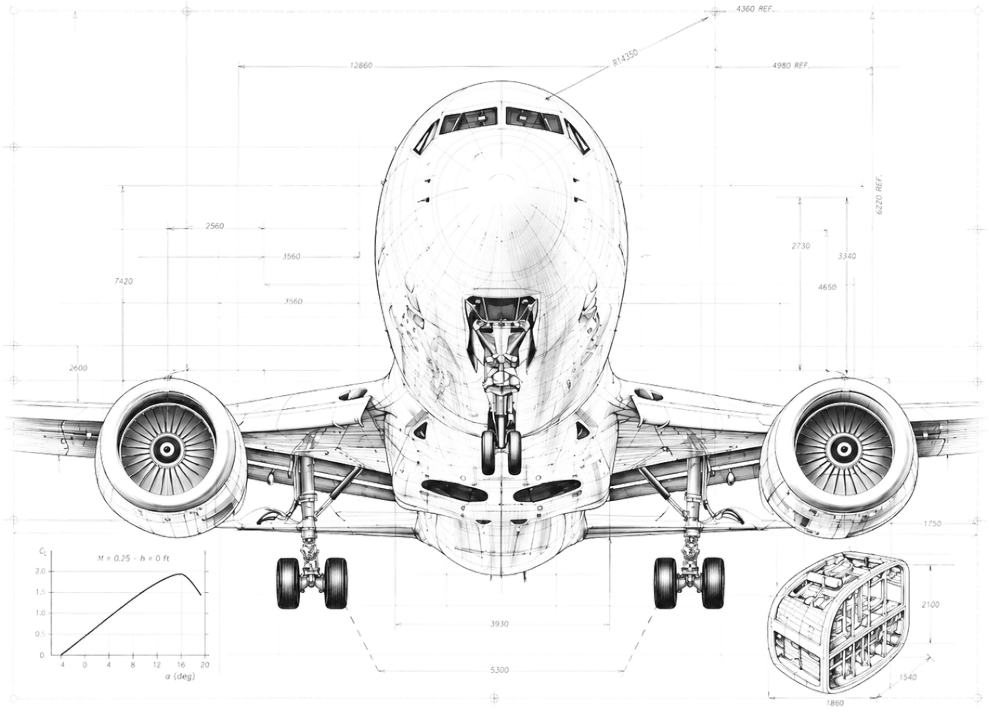
Costruzioni Aeronautiche

ITIS “E. Majorana” — Fluidodinamica numerica

Soluzione numerica delle equazioni di Navier–Stokes

Metodo di Galerkin, differenze upwind, convezione e rotazione, variabili primitive e formulazione vorticità–funzione di corrente



Monografia didattica di fluidodinamica numerica
Modulo 4: equazioni di Navier–Stokes incomprimibili

Prof. Ing. Biagio Raucci

Esempi numerici in Python (NumPy, SciPy, Matplotlib)

Indice

Presentazione **vi**

1 Il metodo di Galerkin: la sfera a numeri di Reynolds finiti **1**
1.1 Oltre il flusso di Stokes . . . 1
1.2 Le equazioni del moto assialsimmetrico . . . 1
1.3 Il metodo dei residui pesati . . . 3
1.3.1 Le condizioni al contorno . . . 4
1.3.2 Il residuo e la sua distribuzione . . . 4
1.4 Velocità, sforzi e resistenza . . . 6
1.5 La bolla di ricircolo . . . 8
1.6 Il metodo di Newton–Raphson . . . 8
1.7 Il Programma 4.1 . . . 10
1.8 Approfondimenti . . . 15
1.8.1 La derivazione simbolica delle equazioni di Galerkin . . . 15
1.8.2 La corrente radiale fra pareti inclinate (Jeffery–Hamel) . . . 17
1.8.3 Pregi e limiti del metodo . . . 21

2 Differenze upwind e viscosità artificiale **23**
2.1 Il problema del termine convettivo . . . 23
2.2 Lo schema upwind e l'analisi della perturbazione discreta . . . 23
2.3 L'analisi di Hirt e la viscosità artificiale . . . 25
2.4 Il programma: trasporto di una gaussiana . . . 26
2.5 Approfondimenti . . . 30
2.5.1 L'equazione modificata in generale . . . 30
2.5.2 Upwind = centrato + diffusione . . . 30
2.5.3 Monotonia e il teorema di Godunov . . . 31
2.5.4 Upwind in due dimensioni e falsa diffusione . . . 31

3 Convezione di Bénard e instabilità dei fluidi in rotazione **32**
3.1 Dall'analisi lineare al calcolo non lineare . . . 32
3.2 Le equazioni di Boussinesq . . . 32
3.3 Il problema del Programma 4.3 . . . 34

3.3.1 Discretizzazione . . . 34
3.3.2 Vorticità alle pareti . . . 35
3.4 Fluidi in rotazione: le equazioni assialsimmetriche . . . 40
3.4.1 La corrente di Couette circolare e la sua perturbazione . . . 43
3.4.2 Il programma: vortici di Taylor . . . 44
3.5 Approfondimenti . . . 48
3.5.1 L'avviamento in rotazione: il caffè sulla scala a chiocciola . . . 48
3.5.2 Due varianti del problema di Bénard . . . 52
3.5.3 Il tubo rotante con restringimento . . . 56

4 Schemi del secondo ordine nel tempo e sistemi tridiagonali 58
4.1 Perché il secondo ordine nel tempo . . . 58
4.2 Il metodo di Adams–Bashforth . . . 58
4.3 L'equazione di Burgers e le due condizioni di stabilità . . . 59
4.4 Il metodo di Crank–Nicolson . . . 60
4.5 Lo schema semi-implicito AB–CN . . . 60
4.6 Sistemi tridiagonali: Thomas e LU . . . 61
4.6.1 L'algoritmo di Thomas . . . 62
4.6.2 La fattorizzazione LU tridiagonale . . . 62
4.7 Il Programma 4.4: Burgers con AB–CN . . . 63
4.8 Approfondimenti . . . 67
4.8.1 La trasformazione di Cole–Hopf e la soluzione di Benton–Platzman . . . 67
4.8.2 Regioni di stabilità assoluta . . . 67
4.8.3 L'avvio di Rannacher . . . 68

5 Variabili primitive: griglia sfalsata e passi frazionari 69
5.1 Il problema della pressione . . . 69
5.2 Il metodo a passi frazionari . . . 69
5.3 Griglia collocata e griglia sfalsata . . . 71
5.4 Forma conservativa e discretizzazione MAC . . . 72
5.4.1 La versione esplicita . . . 72
5.4.2 Stabilità . . . 73
5.5 Condizioni al contorno sulla griglia sfalsata . . . 73
5.5.1 Le condizioni sulla pressione . . . 74
5.6 Il problema di Neumann per la pressione . . . 74
5.7 Il Programma 4.5: la cavità con coperchio mobile . . . 76
5.8 La cavità mossa dal vento . . . 83
5.9 Approfondimenti . . . 84
5.9.1 Lo schema semi-implicito AB–CN a passi frazionari . . . 84
5.9.2 Le proprietà di conservazione della griglia sfalsata . . . 89
5.9.3 I dati di Ghia, Ghia e Shin . . . 89

6 Formulazione vorticità–funzione di corrente: cavità e cilindro 91
6.1 L'equazione del trasporto della vorticità . . . 91
6.2 La cavità in formulazione ω–ψ . . . 92
6.2.1 La vorticità alle pareti . . . 92
6.2.2 Il sistema lineare per ψ . . . 93
6.3 Il metodo ADI di Peaceman–Rachford . . . 94
6.3.1 Il Programma 4.6 . . . 95
6.4 La corrente intorno a un cilindro circolare . . . 99
6.4.1 La trasformazione di coordinate . . . 100
6.4.2 Le condizioni al contorno . . . 100
6.4.3 La resistenza . . . 101
6.4.4 Il Programma 4.6b . . . 102
6.5 Approfondimenti . . . 108
6.5.1 Perché l'ADI converge e come scegliere ρ . . . 108
6.5.2 Formulazione ω–ψ in coordinate curvilinee ortogonali . . . 109
6.5.3 La resistenza a basso numero di Reynolds: Oseen e la scia . . . 109
6.5.4 Variabili primitive o vorticità–funzione di corrente? . . . 109

A Simboli, listati di supporto e istruzioni 110
A.1 Simboli principali . . . 110
A.2 Listati di supporto . . . 112
A.3 Come riprodurre i risultati . . . 121
A.4 Codici in rete e QR code . . . 121
A.5 Come sostituire le figure . . . 122

B Esercizi svolti 123
B.1 Esercizio 1 – Von Neumann per lo schema upwind . . . 123
B.2 Esercizio 2 – Lo schema centrato per la convezione . . . 124
B.3 Esercizio 3 – Analisi di Hirt per l'equazione della diffusione . . . 125
B.4 Esercizio 4 – Perturbazione discreta per lo schema FTCS della diffusione . . . 126
B.5 Esercizio 5 – Stabilità dello schema di Torrance . . . 127
B.6 Esercizio 6 – Fattorizzazione LU al posto dell'algoritmo di Thomas . . . 129
B.7 Esercizio 7 – Le equazioni del cilindro nelle coordinate trasformate . . . 129
B.8 Esercizio 8 – La funzione di prova di Kawaguti per piccoli numeri di Reynolds . 130
B.9 Esercizio 9 – Il problema di Neumann su quattro nodi . . . 131
B.10 Esercizio 10 (progetto) – Corrente di Jeffery–Hamel con il metodo di Galerkin . 132
B.11 Esercizio 11 (progetto) – Scatola riscaldata dall'alto con superficie libera . . . 133
B.12 Esercizio 12 (progetto) – Strato con gradiente orizzontale periodico . . . 134
B.13 Esercizio 13 (progetto) – Vortici di Taylor . . . 134
B.14 Esercizio 14 (progetto) – Tubo rotante con restringimento . . . 135
B.15 Esercizio 15 (progetto) – Il caffè sulla scala a chiocciola . . . 136

B.16 Esercizio 16 (progetto) – Cavità con lo schema AB–CN a passi frazionari 137
B.17 Esercizio 17 (progetto) – La cavità mossa dal vento 138
B.18 Esercizio 18 (progetto) – Cavità ω–ψ con il metodo ADI 138
B.19 Esercizio 19 (progetto) – Corrente intorno al cilindro 139
B.20 Esercizio 20 – Derivazione della formula della resistenza del cilindro 140

Bibliografia **142**

Elenco dei listati

1.1 Programma 4.1 – sfera con il metodo di Galerkin 10
1.2 `prog4_1b_galerkin_simbolico.py` – derivazione simbolica 16
1.3 `prog4_1c_jeffery_hamel.py` – Jeffery–Hamel con Galerkin 19
2.1 Programma 4.2 – schema upwind e viscosità artificiale 26
3.1 Programma 4.3 – convezione di Bénard non lineare 36
3.2 `prog4_3b_taylor.py` – vortici di Taylor 44
3.3 `prog4_3c_tazza.py` – avviamento in rotazione 49
3.4 `prog4_3d_benard_varianti.py` – varianti di Bénard 53
4.1 Programma 4.4 – equazione di Burgers con AB–CN 63
5.1 Programma 4.5 – cavità con coperchio mobile, griglia MAC 77
5.2 `prog4_5b_cavita_vento.py` – cavità mossa dal vento 83
5.3 `prog4_5c_cavita_abcn.py` – cavità con AB–CN a passi frazionari 85
6.1 Programma 4.6 – cavità in formulazione ω–ψ con ADI 95
6.2 Programma 4.6b – cilindro circolare in coordinate trasformate 102
A.1 `metodi_numerici.py` – modulo comune 112
A.2 `stile_grafici.py` – impostazioni di Matplotlib 115
A.3 `esercizi_svolti.py` – verifiche degli esercizi 116

Presentazione

Di che cosa parla questa monografia

Nel terzo modulo del corso le equazioni di Navier–Stokes erano state affrontate soltanto nei casi in cui la geometria o l'ordine di grandezza dei termini permettevano di eliminare la non linearità: lo strato limite, i condotti, le correnti fra pareti parallele, i flussi di Stokes. Quando il numero di Reynolds è finito e la geometria è qualsiasi, il termine convettivo $\mathbf{V} \cdot \nabla \mathbf{V}$ resta nelle equazioni e con esso restano i fenomeni che rendono interessante la meccanica dei fluidi: la *separazione* della corrente da un corpo e la scia che ne risulta, i *moti secondari* nei fluidi in rotazione, le *instabilità* che trasformano uno stato di quiete in un sistema di celle convettive o in una sequenza di vortici anulari. Questo quarto modulo è dedicato alla soluzione numerica delle equazioni complete di Navier–Stokes per il fluido incomprimibile, nelle due formulazioni che la fluidodinamica numerica ha sviluppato per il moto piano e assialsimmetrico: la formulazione in **vorticità e funzione di corrente**, in cui la pressione scompare, e quella in **variabili primitive** (velocità e pressione), in cui il vincolo di incomprimibilità richiede un trattamento speciale della pressione.

Il percorso è graduale e, come nei moduli precedenti, ogni strumento è introdotto su un problema modello prima di essere applicato alle equazioni complete:

- il **metodo di Galerkin**, che approssima la soluzione con una combinazione di funzioni note e trasforma le equazioni alle derivate parziali in un sistema algebrico non lineare risolto con Newton–Raphson; l'esempio è la corrente intorno a una sfera a numeri di Reynolds finiti, con la nascita della bolla di ricircolo e il calcolo della resistenza di attrito e di pressione;
- le **differenze upwind**, lo schema che rende stabile la discretizzazione del termine convettivo, con le tre analisi di stabilità (perturbazione discreta, von Neumann, equazione modificata di Hirt) e il concetto di **viscosità artificiale**;
- la **convezione di Bénard** non lineare e le **instabilità dei fluidi in rotazione** (vortici di Taylor, avviamento in rotazione), risolte con lo schema upwind di Torrance nella formulazione vorticità–funzione di corrente;
- gli schemi del **secondo ordine nel tempo** (Adams–Bashforth, Crank–Nicolson e il loro accoppiamento semi-implicito AB–CN), l'algoritmo di Thomas e la fattorizzazione LU per i sistemi tridiagonali, sull'equazione di Burgers;
- le **variabili primitive**: il metodo a passi frazionari (proiezione), la griglia sfalsata MAC, l'equazione di Poisson per la pressione con condizioni di Neumann e la sua singolarità; l'esempio è la cavità quadrata con coperchio mobile, confrontata con i dati di riferimento della letteratura;

- la formulazione **vorticità–funzione di corrente** per la stessa cavità, con il metodo ADI di Peaceman–Rachford e la successione di parametri di Wachspress, e per la corrente intorno a un **cilindro circolare** in coordinate trasformate: separazione, lunghezza della bolla, resistenza.

Questa monografia costituisce il quarto modulo del corso di fluidodinamica numerica. I temi seguono una progressione classica della letteratura, qui rielaborata con dimostrazioni complete, programmi in Python corredati di grafici ed esercizi interamente svolti. Ogni capitolo si chiude con una sezione di *approfondimenti* che estende il tema con risultati teorici (equazione modificata, criteri di stabilità dei fluidi in rotazione, teoria della proiezione di Helmholtz–Hodge, convergenza dell'ADI), applicazioni tecniche (strati di Ekman, resistenza di cavi e montanti, scie) e programmi aggiuntivi. Ogni risultato numerico è confrontato con una soluzione esatta o con dati di riferimento della letteratura: la tabella dei coefficienti di Galerkin è riprodotta cifra per cifra, la cavità è confrontata con Ghia, Ghia e Shin, il cilindro con Dennis e Chang e con Fornberg. La bibliografia raccoglie le fonti: i testi classici di meccanica dei fluidi computazionale [1, 2], Roache [3] per le differenze upwind e le analisi di stabilità, Chandrasekhar [15] e Drazin e Reid [16] per le instabilità, Peyret e Taylor [4] e Ferziger e Perić [5] per le variabili primitive e i metodi di proiezione, Greenspan [18] per i fluidi in rotazione, Panton [30] per la corrente intorno al cilindro.

Come è organizzata

Capitolo 1 – Il metodo di Galerkin: la sfera a numeri di Reynolds finiti.
Funzione di corrente di Stokes, equazione di Navier–Stokes in forma di vorticità, metodo dei residui pesati, polinomi di Legendre, funzione di prova di Kawaguti; sistema algebrico non lineare e metodo di Newton–Raphson; resistenza di attrito e di pressione, bolla di ricircolo. Approfondimenti: derivazione simbolica delle equazioni di Galerkin, corrente radiale di Jeffery–Hamel e sua soluzione esatta, limiti del metodo.

Capitolo 2 – Differenze upwind e viscosità artificiale.
L'equazione modello del trasporto della vorticità; lo schema upwind, l'analisi della perturbazione discreta e la proprietà trasportiva; instabilità dello schema downwind e di quello centrato; analisi di Hirt ed equazione modificata, viscosità artificiale. Approfondimenti: equazione modificata di Warming–Hyett, schema di Lax–Wendroff, numero di Péclet di cella e falsa diffusione.

Capitolo 3 – Convezione di Bénard e instabilità dei fluidi in rotazione.
Approssimazione di Boussinesq, equazioni in vorticità–funzione di corrente–temperatura, schema upwind di Torrance, vorticità di parete, criterio di stabilità; la nascita e l'evoluzione delle celle; moto assialsimmetrico con rotazione, vortici di Taylor. Approfondimenti: criterio di Rayleigh per i fluidi in rotazione, avviamento in rotazione del caffè in una tazza (strati di Ekman), varianti del problema di Bénard, tubo rotante con restringimento.

Capitolo 4 – Schemi del secondo ordine nel tempo e sistemi tridiagonali.
Adams–Bashforth e sua debole instabilità; Crank–Nicolson; lo schema semi-implicito AB–CN; algoritmo di Thomas e fattorizzazione LU; l'equazione di Burgers e la sua soluzione esatta. Approfondimenti: trasformazione di Cole–Hopf, regioni di stabilità assoluta, stabilità dell'algoritmo di Thomas.

Capitolo 5 – Variabili primitive: griglia sfalsata e passi frazionari.
Metodo a passi frazionari e proiezione di Helmholtz–Hodge; disaccoppiamento pari–dispari

e griglia MAC; forma conservativa; condizioni al contorno con i nodi fittizi; il problema di Neumann per la pressione e la sua singolarità; la cavità con coperchio mobile confrontata con i dati di Ghia et al.; la cavità mossa dal vento. Approfondimenti: schema semi-implicito AB–CN a passi frazionari, errore di separazione e forma incrementale, proprietà di conservazione della griglia sfalsata.

Capitolo 6 – Formulazione vorticità–funzione di corrente: cavità e cilindro.
Equazione del trasporto della vorticità, condizioni al contorno per la vorticità, soluzione diretta e iterativa dell'equazione di Poisson, metodo ADI di Peaceman–Rachford e parametri di Wachspress; il cilindro circolare in coordinate trasformate, separazione, bolla di ricircolo e resistenza confrontate con Dennis e Chang e con Fornberg. Approfondimenti: convergenza dell'ADI, posizione del contorno esterno, instabilità della scia e numero di Strouhal, applicazioni aeronautiche.

Appendici.
Simboli e istruzioni; i venti esercizi svolti passo per passo, fra cui dieci progetti.

Ogni capitolo segue lo stesso schema dei moduli precedenti: prima l'*intuizione fisica*, poi il *modello matematico*, quindi l'*algoritmo* e il *programma*, infine i *risultati* discussi criticamente. I riquadri colorati hanno il significato consueto:

- **teoria** (viola): definizioni, enunciati, dimostrazioni;
- **fasi** (blu): procedure e algoritmi passo per passo;
- **ipotesi** (giallo): le assunzioni semplificative del modello;
- **dati** (grigio): valori numerici usati nei calcoli;
- **risultati** (verde): esiti dei calcoli e loro verifica;
- **esempi** (arancio): applicazioni ed esempi numerici;
- **avvisi** (rosso): errori frequenti, limiti dei metodi, trappole numeriche.

I programmi

Tutti i programmi sono scritti in Python 3 e usano soltanto le librerie `numpy`, `scipy` e `matplotlib` (più `sympy` per le derivazioni simboliche). Ciascuno è autonomo, stampa i risultati principali e salva le figure nella cartella `figure/`; i moduli comuni `metodi_numerici.py` e `stile_grafici.py` sono riportati in appendice. I programmi che traducono gli algoritmi classici del capitolo sono numerati da 4.1 a 4.6; quelli di approfondimento (otto) portano lo stesso numero seguito da una lettera. I programmi sono disponibili in rete all'indirizzo `https://braucci.github.io/CFD/monografia4/codici/`: il QR code nel margine accanto a ogni listato porta direttamente al file corrispondente.

Capitolo 1

Il metodo di Galerkin: la sfera a numeri di Reynolds finiti

1.1 Oltre il flusso di Stokes

I flussi di Stokes del modulo precedente sono rari nella pratica: una gocciolina d'acqua di 0,5 mm in caduta libera ha già $Re \approx 60$, ben oltre il campo in cui l'inerzia può essere trascurata. Se la geometria del corpo non è una di quelle (pareti parallele, condotti, lamina) che eliminano la non linearità, i termini inerziali dell'equazione di Navier–Stokes restano e l'equazione resta non lineare per ogni numero di Reynolds finito. Le differenze finite, sviluppate nei capitoli successivi, permettono di affrontare il caso generale; in questo primo capitolo seguiamo invece una strada completamente diversa, in cui la soluzione non è rappresentata da valori in nodi discreti ma da una *funzione continua* con un numero finito di coefficienti incogniti. È il **metodo di Galerkin**, antenato del metodo degli elementi finiti e dei metodi spettrali: il problema scelto per illustrarlo, la corrente intorno a una sfera, è lo stesso che nel primo modulo forniva la balistica del proiettile sferico e la resistenza delle goccioline, e qui ne spiega la fisica: perché la resistenza è dovuta all'attrito a bassi Re e alla pressione ad alti Re, e come nasce la bolla di ricircolo dietro al corpo.

1.2 Le equazioni del moto assialsimmetrico

Sia (r, θ, φ) un sistema di coordinate sferiche con origine nel centro della sfera di raggio a (Fig. 1.1); lontano dalla sfera la corrente ha velocità costante U lungo l'asse x. Il moto è assialsimmetrico, cioè indipendente da φ, e si può introdurre la **funzione di corrente di Stokes** ψ, dalla quale le componenti della velocità si ricavano come

$$u_r = \frac{1}{r^2 \sin\theta}\frac{\partial \psi}{\partial \theta}, \qquad u_\theta = -\frac{1}{r \sin\theta}\frac{\partial \psi}{\partial r}. \tag{1.1}$$

Perché la funzione di corrente di Stokes soddisfa la continuità

In coordinate sferiche, per un moto indipendente da φ, la continuità del fluido incomprimibile si scrive

$$\frac{1}{r^2}\frac{\partial}{\partial r}\left(r^2 u_r\right) + \frac{1}{r \sin\theta}\frac{\partial}{\partial \theta}\left(\sin\theta\, u_\theta\right) = 0.$$

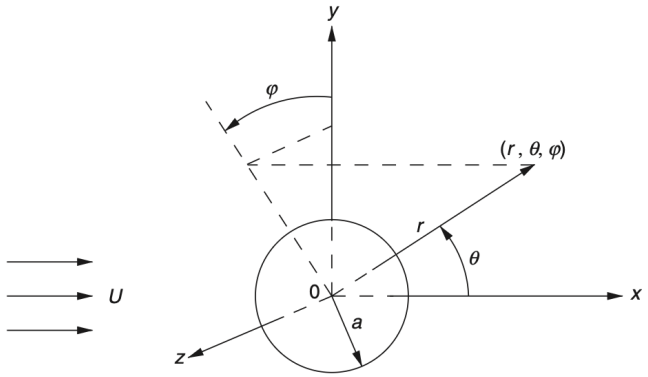


Figura 1.1: Corrente uniforme intorno a una sfera.

Sostituendo le (1.1): $r^2 u_r = \psi_\theta / \sin\theta$ e $\sin\theta\, u_\theta = -\psi_r / r$, quindi

$$\frac{1}{r^2 \sin\theta} \frac{\partial^2 \psi}{\partial r\, \partial \theta^2} - \frac{1}{r \sin\theta} \frac{1}{r} \frac{\partial^2 \psi}{\partial \theta\, \partial r^2} = 0$$

identicamente. La funzione ψ è costante lungo le linee di corrente e $2\pi\psi$ è la portata volumetrica che attraversa un cerchio di raggio $r \sin\theta$ centrato sull'asse: per questo ψ ha le dimensioni di una velocità per un'area, non di una velocità per una lunghezza come nel moto piano.

Per generalizzarla alle coordinate curvilinee, l'equazione di Navier–Stokes del fluido incomprimibile si riscrive con le identità vettoriali $\mathbf{V} \cdot \nabla \mathbf{V} = \nabla(\frac{1}{2} V^2) - \mathbf{V} \times (\nabla \times \mathbf{V})$ e $\nabla^2 \mathbf{V} = -\nabla \times \nabla \times \mathbf{V}$ (quest'ultima vale perché $\nabla \cdot \mathbf{V} = 0$):

$$\rho \left[\frac{\partial \mathbf{V}}{\partial t} - \mathbf{V} \times (\nabla \times \mathbf{V}) \right] = -\nabla \left(p + \tfrac{1}{2} \rho V^2 \right) - \mu \, \nabla \times \nabla \times \mathbf{V}. \tag{1.2}$$

Prendendo il rotore, in moto stazionario spariscono sia il gradiente sia la derivata temporale. Sostituendo le (1.1) si ottiene, dopo un calcolo laborioso ma elementare, l'equazione per la sola ψ:

$$\frac{1}{r^2 \sin\theta} \left[\frac{\partial \psi}{\partial \theta} \frac{\partial}{\partial r} - \frac{\partial \psi}{\partial r} \frac{\partial}{\partial \theta} + 2 \cot\theta \frac{\partial \psi}{\partial r} - \frac{2}{r} \frac{\partial \psi}{\partial \theta} \right] D^2 \psi = \frac{2}{Re} D^4 \psi, \tag{1.3}$$

in cui $Re = 2\rho U a / \mu$ è il numero di Reynolds basato sul *diametro*, ψ e r sono adimensionali rispetto a Ua^2 e ad a, e

$$D^2 \equiv \frac{\partial^2}{\partial r^2} + \frac{\sin\theta}{r^2} \frac{\partial}{\partial \theta} \left(\frac{1}{\sin\theta} \frac{\partial}{\partial \theta} \right) \tag{1.4}$$

è l'operatore di Stokes, l'analogo del laplaciano per la funzione di corrente assialsimmetrica: la vorticità azimutale vale $\omega_\varphi = -D^2 \psi / (r \sin\theta)$ e il flusso di Stokes del modulo precedente corrisponde a $D^4 \psi = 0$. Le condizioni al contorno sono l'aderenza sulla sfera,

$$\psi(1, \theta) = 0, \qquad \frac{\partial \psi}{\partial r}(1, \theta) = 0, \tag{1.5}$$

e la corrente uniforme all'infinito,

$$\psi(r, \theta) \to \tfrac{1}{2} r^2 \sin^2\theta \qquad \text{per } r \to \infty. \tag{1.6}$$

Conviene sostituire θ con la variabile $\zeta \equiv \cos\theta$ (da non confondere con la vorticità dei moduli precedenti: in questo capitolo ζ è una coordinata). Poiché $\partial/\partial\theta = -\sin\theta\,\partial/\partial\zeta$ e $\sin^2\theta = 1-\zeta^2$, la (1.3) diventa

$$\frac{2}{Re}D^4\psi + \frac{1}{r^2}\left[\frac{\partial\psi}{\partial\zeta}\frac{\partial}{\partial r} - \frac{\partial\psi}{\partial r}\frac{\partial}{\partial\zeta} - \frac{2\zeta}{1-\zeta^2}\frac{\partial\psi}{\partial r} - \frac{2}{r}\frac{\partial\psi}{\partial\zeta}\right]D^2\psi = 0, \tag{1.7}$$

con

$$D^2 \equiv \frac{\partial^2}{\partial r^2} + \frac{1-\zeta^2}{r^2}\frac{\partial^2}{\partial\zeta^2}, \tag{1.8}$$

e la condizione all'infinito $\psi \to \frac{1}{2}r^2(1-\zeta^2)$. Il vantaggio è che in ζ tutte le funzioni angolari diventano *polinomi*.

1.3 Il metodo dei residui pesati

Residui pesati, Galerkin e le sue varianti

Sia $\mathcal{L}[\psi] = 0$ un'equazione differenziale (anche non lineare) su un dominio D con le sue condizioni al contorno. Si cerca una soluzione approssimata

$$\tilde{\psi} = \sum_{i=1}^{n} c_i\,\Phi_i,$$

combinazione di n *funzioni di prova* Φ_i note, linearmente indipendenti e abbastanza regolari da poter essere derivate quante volte richiede l'equazione. Se le Φ_i soddisfano esattamente le condizioni al contorno, sostituendo $\tilde{\psi}$ nell'equazione si ottiene un *residuo* $\mathcal{R}(\mathbf{x}; c_1, \ldots, c_n) = \mathcal{L}[\tilde{\psi}]$ che in generale non è nullo. I coefficienti si determinano imponendo che il residuo sia ortogonale a n *funzioni peso* w_k:

$$\int_D \mathcal{R}\,w_k\,\mathrm{d}D = 0, \qquad k = 1, \ldots, n.$$

La scelta dei pesi distingue i metodi: con $w_k = \delta(\mathbf{x} - \mathbf{x}_k)$ si annulla il residuo in n punti (*collocazione*, il principio dei metodi pseudo-spettrali del modulo precedente); con $w_k = \partial\mathcal{R}/\partial c_k$ si minimizza $\int \mathcal{R}^2$ (*minimi quadrati*); con $w_k = \Phi_k$ si ha il **metodo di Galerkin** (1915): il residuo è ortogonale allo stesso spazio in cui si cerca la soluzione, cioè la soluzione approssimata è la *proiezione ortogonale* della soluzione esatta su quello spazio, nel senso che l'errore $\psi - \tilde{\psi}$, filtrato dall'operatore, non ha componenti lungo le funzioni di prova. Se le Φ_i sono i primi n elementi di un sistema completo di funzioni ortogonali, ci si aspetta che $\tilde{\psi} \to \psi$ per $n \to \infty$. Il metodo degli elementi finiti è un metodo di Galerkin con funzioni di prova a supporto compatto (polinomi a tratti); i metodi spettrali di Galerkin usano polinomi ortogonali globali.
Per un'equazione *lineare* le condizioni di ortogonalità sono un sistema lineare nei c_i; per un'equazione non lineare, come la (1.7) che è quadratica in ψ, si ottiene un sistema *algebrico non lineare*, da risolvere con un metodo iterativo. Il metodo di Galerkin trasforma dunque un problema differenziale non lineare in un problema algebrico non lineare di dimensione finita: è una strategia che ha preceduto di decenni i calcolatori.

Per il problema della sfera si scelgono funzioni di prova a variabili separate,

$$\Phi_i(r,\zeta) = f_i(r)\,[P_{i-1}(\zeta) - P_{i+1}(\zeta)], \qquad i = 1, 2, \ldots, \tag{1.9}$$

in cui $P_m(\zeta)$ sono i **polinomi di Legendre**, definiti dalla formula di Rodrigues

$$P_m(\zeta) = \frac{1}{2^m m!}\frac{\mathrm{d}^m}{\mathrm{d}\zeta^m}\left(\zeta^2-1\right)^m, \tag{1.10}$$

i cui primi quattro sono

$$P_0 = 1, \qquad P_1 = \zeta, \qquad P_2 = \tfrac{1}{2}\left(3\zeta^2-1\right), \qquad P_3 = \tfrac{1}{2}\zeta\left(5\zeta^2-3\right).$$

Le differenze $P_{i-1} - P_{i+1}$ sono scelte perché sono proporzionali a $(1-\zeta^2)P_i'(\zeta)$ e si annullano quindi in $\zeta = \pm 1$ (sull'asse di simmetria, dove ψ deve essere nulla); per $i = 1$ si ha $P_0 - P_2 = \frac{3}{2}(1-\zeta^2)$, la dipendenza angolare della corrente uniforme, e per $i = 2$ $P_1 - P_3 = \frac{5}{2}\zeta(1-\zeta^2)$, la prima correzione asimmetrica fra parte anteriore e posteriore. Grazie all'ortogonalità dei polinomi di Legendre, $\int_{-1}^{1} P_m P_n \,\mathrm{d}\zeta = 2\delta_{mn}/(2n+1)$, le Φ_i sono ortogonali in ζ:

$$\int_{-1}^{1} \Phi_i \Phi_j \,\mathrm{d}\zeta = 0 \qquad \text{per } i \neq j.$$

Seguendo Kawaguti [10] si prendono $n = 2$ termini e, per le funzioni radiali, polinomi in $1/r$:

$$\psi = \left(\tfrac{1}{2}r^2 + \frac{A_1}{r} + \frac{A_2}{r^2} + \frac{A_3}{r^3} + \frac{A_4}{r^4}\right)(1-\zeta^2) + \left(\frac{B_1}{r} + \frac{B_2}{r^2} + \frac{B_3}{r^3} + \frac{B_4}{r^4}\right)\zeta(1-\zeta^2), \tag{1.11}$$

che soddisfa automaticamente la condizione all'infinito; gli otto coefficienti A_m, B_m sono le incognite. Il primo gruppo descrive la corrente simmetrica avanti–dietro (il termine $\frac{1}{2}r^2$ è la corrente uniforme, A_1/r e A_3/r^3 sono gli stessi termini della soluzione di Stokes); il secondo gruppo, dispari in ζ, descrive l'asimmetria prodotta dall'inerzia e con essa la scia.

1.3.1 Le condizioni al contorno

L'aderenza (1.5) in $r = 1$ deve valere per ogni ζ, quindi separatamente per i due gruppi:

$$A_1 + A_2 + A_3 + A_4 = -\tfrac{1}{2}, \tag{1.12}$$
$$A_1 + 2A_2 + 3A_3 + 4A_4 = 1, \tag{1.13}$$
$$B_1 + B_2 + B_3 + B_4 = 0, \tag{1.14}$$
$$B_1 + 2B_2 + 3B_3 + 4B_4 = 0. \tag{1.15}$$

(La (1.13) si ottiene da $\partial\psi/\partial r = 0$: $r - A_1/r^2 - 2A_2/r^3 - \cdots = 0$ in $r = 1$.)

1.3.2 Il residuo e la sua distribuzione

Sostituendo la (1.11) nella (1.7) si ottiene un residuo (NS) che è un polinomio in ζ con coefficienti funzioni di r:

$$\begin{aligned}(NS) = {} & g_1(r)(1-\zeta^2) + g_2(r)\zeta(1-\zeta^2) + g_3(r)\zeta^2(1-\zeta^2) \\ & + g_4(r)(1-\zeta^2)(1-3\zeta^2) + g_5(r)\zeta(1-\zeta^2)(1-3\zeta^2) + g_6(r)\zeta^3(1-\zeta^2),\end{aligned} \tag{1.16}$$

dove, con $\alpha_m \equiv A_m/r^m$ e $\beta_m \equiv B_m/r^m$,

$$
\begin{aligned}
g_1 &= \frac{4}{r^5}\Big[\frac{2}{Re} r\,(9\alpha_2 + 35\alpha_3 + 90\alpha_4) \\
&\qquad - (2\alpha_2 + 5\alpha_3 + 9\alpha_4)(\beta_1 + \beta_2 + \beta_3 + \beta_4)\Big], \\
g_2 &= \frac{4}{r^5}\Big[- \frac{6}{Re} r\,(\beta_1 - 6\beta_3 - 21\beta_4) \\
&\qquad + \left(\tfrac{1}{2}r^2 + \alpha_1 + \alpha_2 + \alpha_3 + \alpha_4\right)(12\alpha_2 + 35\alpha_3 + 73\alpha_4) \\
&\qquad + (\beta_1 + \beta_2 + \beta_3 + \beta_4)(2\beta_1 - 3\beta_3 - 7\beta_4)\Big], \\
g_3 &= \frac{4}{r^5}\Big[\left(\tfrac{1}{2}r^2 + \alpha_1 + \alpha_2 + \alpha_3 + \alpha_4\right)(-6\beta_1 + 15\beta_3 + 42\beta_4) \\
&\qquad + (3\alpha_1 + 4\alpha_2 + 5\alpha_3 + 6\alpha_4)(-2\beta_1 + 3\beta_3 + 7\beta_4) \\
&\qquad + 3(2\alpha_2 + 5\alpha_3 + 9\alpha_4)(\beta_1 + \beta_2 + \beta_3 + \beta_4)\Big], \\
g_4 &= \frac{2}{r^5}\Big[(r^2 - \alpha_1 - 2\alpha_2 - 3\alpha_3 - 4\alpha_4)(2\beta_1 - 3\beta_3 - 7\beta_4) \\
&\qquad - (8\alpha_2 + 25\alpha_3 + 54\alpha_4)(\beta_1 + \beta_2 + \beta_3 + \beta_4)\Big], \\
g_5 &= \frac{2}{r^5}\Big[(\beta_1 + \beta_2 + \beta_3 + \beta_4)(6\beta_1 - 15\beta_3 - 42\beta_4) \\
&\qquad - (\beta_1 + 2\beta_2 + 3\beta_3 + 4\beta_4)(2\beta_1 - 3\beta_3 - 7\beta_4)\Big], \\
g_6 &= -\frac{4}{r^5}(4\beta_1 + 5\beta_2 + 6\beta_3 + 7\beta_4) \\
&\qquad \times (2\beta_1 - 3\beta_3 - 7\beta_4).
\end{aligned}
$$

Non c'è nulla di misterioso in queste espressioni, soltanto molta algebra: il Programma 4.1b le ricava e le verifica con il calcolo simbolico.

Si impone anzitutto che l'equazione di Navier–Stokes sia soddisfatta *esattamente sulla sfera*, $(NS) = 0$ in $r = 1$. Sulla superficie, per l'aderenza, tutti i termini convettivi si annullano (lo verifica il Programma 4.1b) e resta la sola parte viscosa, $2D^4\psi/Re$, che in $r = 1$ vale

$$
\frac{16}{Re}(1 - \zeta^2)\left[(9A_2 + 35A_3 + 90A_4) - 3\zeta(B_1 - 6B_3 - 21B_4)\right];
$$

annullarla per ogni ζ richiede

$$
9A_2 + 35A_3 + 90A_4 = 0, \tag{1.17}
$$

$$
B_1 - 6B_3 - 21B_4 = 0. \tag{1.18}
$$

Lontano dalla sfera il residuo non è nullo e viene distribuito nel campo con due condizioni di ortogonalità di Galerkin, usando come pesi le funzioni angolari delle due funzioni di prova e il peso radiale $1/r^k$ che rende convergenti gli integrali:

$$
\int_1^\infty \int_{-1}^1 \frac{1}{r}\,(NS)\,[P_0(\zeta) - P_2(\zeta)]\,\mathrm{d}\zeta\,\mathrm{d}r = 0, \tag{1.19}
$$

$$
\int_1^\infty \int_{-1}^1 \frac{1}{r^2}\,(NS)\,[P_1(\zeta) - P_3(\zeta)]\,\mathrm{d}\zeta\,\mathrm{d}r = 0. \tag{1.20}
$$

Svolgendo gli integrali (ancora Programma 4.1b) si ottengono due equazioni quadratiche nei coefficienti:

$$\begin{aligned}&\frac{32}{Re}\left(\tfrac{6}{5}A_2+4A_3+9A_4\right)+\tfrac{8}{35}(B_1+B_3+4B_4)-\tfrac{512}{245}A_1B_1-\tfrac{184}{35}A_2B_1-\tfrac{448}{45}A_3B_1\\&\quad-\tfrac{2848}{175}A_4B_1-\tfrac{256}{105}A_2B_2-\tfrac{32}{5}A_3B_2-\tfrac{4608}{385}A_4B_2+\tfrac{64}{21}A_1B_3+\tfrac{48}{35}A_2B_3\\&\quad-\tfrac{704}{385}A_3B_3-\tfrac{232}{35}A_4B_3+\tfrac{176}{25}A_1B_4+\tfrac{2368}{385}A_2B_4+\tfrac{392}{105}A_3B_4-\tfrac{128}{455}A_4B_4=0,\end{aligned}\tag{1.21}$$

$$\begin{aligned}&\frac{32}{21Re}(-2B_1+9B_3+28B_4)+\tfrac{16}{21}(2A_2+5A_3+9A_4)+\tfrac{128}{63}A_1A_2+\tfrac{16}{3}A_1A_3+\tfrac{768}{77}A_1A_4\\&\quad+\tfrac{64}{35}A_2^2+\tfrac{1504}{231}A_2A_3+\tfrac{32}{3}A_2A_4+\tfrac{40}{9}A_3^2+\tfrac{3424}{273}A_3A_4+\tfrac{384}{49}A_4^2-\tfrac{8}{63}B_1^2\\&\quad-\tfrac{128}{567}B_1B_2-\tfrac{16}{105}B_1B_3-\tfrac{32}{693}B_1B_4+\tfrac{64}{231}B_2B_3+\tfrac{16}{27}B_2B_4+\tfrac{8}{21}B_3^2+\tfrac{352}{273}B_3B_4+\tfrac{64}{63}B_4^2=0.\end{aligned}\tag{1.22}$$

Le otto equazioni (1.12)–(1.15), (1.17), (1.18), (1.21) e (1.22) determinano gli otto coefficienti. Le prime sei sono lineari e permettono di esprimere sei incognite in funzione delle due restanti, A_1 e B_1:

$$\begin{gathered}A_2=-\tfrac{15}{29}(8+5A_1),\qquad A_3=\tfrac{9}{29}(17+7A_1),\qquad A_4=-\tfrac{1}{58}(95+34A_1),\\B_2=-\tfrac{23}{9}B_1,\qquad B_3=\tfrac{19}{9}B_1,\qquad B_4=-\tfrac{5}{9}B_1.\end{gathered}\tag{1.23}$$

Sostituendo nelle due equazioni di Galerkin si arriva al **sistema ridotto**

$$g(A_1,B_1)\equiv\frac{9{,}931035}{Re}A_1+0{,}011713\,B_1-0{,}002546\,A_1B_1+\frac{44{,}689656}{Re}=0,\tag{1.24}$$

$$h(A_1,B_1)\equiv 0{,}050148\,A_1+\frac{2{,}201058}{Re}B_1+0{,}002421\,A_1^2-0{,}000376\,B_1^2+0{,}379018=0.\tag{1.25}$$

Due equazioni quadratiche in due incognite: tutto il problema di Navier–Stokes per la sfera si è ridotto a questo. Noti A_1 e B_1, le (1.23) danno gli altri coefficienti e la (1.11) è una funzione nota di r e θ.

Esatta sul contorno, approssimata nel campo

La funzione di prova soddisfa esattamente le condizioni al contorno ma l'equazione soltanto in media pesata, e sulla superficie del corpo. La scelta opposta (soddisfare esattamente l'equazione e distribuire l'errore sulle condizioni al contorno) è praticabile solo per equazioni lineari, di cui si conoscono famiglie di soluzioni esatte: per Navier–Stokes non lo è. La qualità del risultato dipende interamente da quanto le funzioni di prova "assomigliano" alla soluzione: con $n=2$ non ci si può aspettare di descrivere una scia lunga e sottile, e infatti vedremo che il metodo è buono per la resistenza ma mediocre per la lunghezza della bolla.

1.4 Velocità, sforzi e resistenza

Dalle (1.1) e (1.11), con $\sin\theta\,\partial/\partial\theta=-(1-\zeta^2)\partial/\partial\zeta$:

$$\frac{u_r}{U}=\left(1+\frac{2A_1}{r^3}+\frac{2A_2}{r^4}+\frac{2A_3}{r^5}+\frac{2A_4}{r^6}\right)\cos\theta+\left(\frac{B_1}{r^3}+\frac{B_2}{r^4}+\frac{B_3}{r^5}+\frac{B_4}{r^6}\right)(3\cos^2\theta-1),\tag{1.26}$$

$$\frac{u_\theta}{U}=\left(-1+\frac{A_1}{r^3}+\frac{2A_2}{r^4}+\frac{3A_3}{r^5}+\frac{4A_4}{r^6}\right)\sin\theta+\left(\frac{B_1}{r^3}+\frac{2B_2}{r^4}+\frac{3B_3}{r^5}+\frac{4B_4}{r^6}\right)\sin\theta\cos\theta.\tag{1.27}$$

La resistenza come integrale degli sforzi

Sull'elemento di superficie anulare $2\pi a \sin\theta \cdot a\,\mathrm{d}\theta$ (Fig. 1.2) agiscono lo sforzo normale τ_{rr} e lo sforzo tangenziale $\tau_{r\theta}$ (primo indice: normale alla faccia; secondo: direzione dello sforzo). La componente lungo x della forza che il fluido esercita sulla sfera è, per la normale, $-\tau_{rr}\cos\theta$ (lo sforzo normale positivo è una trazione uscente, e la resistenza è nel verso $+x$ della corrente che per θ piccolo spinge sulla faccia anteriore) e, per la tangente, $\tau_{r\theta}\sin\theta$ con il segno della convenzione sulla direzione θ crescente. Ne risultano le due parti della resistenza:

$$D_p = \int_0^\pi (\tau_{rr})_{r=a} \cos\theta \cdot 2\pi a \sin\theta \cdot a\,\mathrm{d}\theta, \tag{1.28}$$

$$D_s = -\int_0^\pi (\tau_{r\theta})_{r=a} \sin\theta \cdot 2\pi a \sin\theta \cdot a\,\mathrm{d}\theta, \tag{1.29}$$

con i segni fissati dalla convenzione della figura, che misura θ dal punto di ristagno anteriore verso valle. Le componenti del tensore degli sforzi in coordinate sferiche sono

$$\tau_{rr} = -p + 2\mu\frac{\partial u_r}{\partial r}, \qquad \tau_{r\theta} = \mu\left[r\frac{\partial}{\partial r}\left(\frac{u_\theta}{r}\right) + \frac{1}{r}\frac{\partial u_r}{\partial \theta}\right]. \tag{1.30}$$

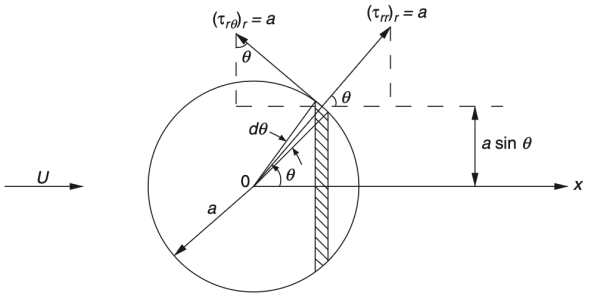


Figura 1.2: Sforzi sulla superficie della sfera.

Dalle (1.26) e (1.27) si verifica che sulla sfera $\partial u_r/\partial r = 0$ (conseguenza dell'aderenza e della continuità: se $u_r = u_\theta = 0$ su $r = 1$, la continuità impone $\partial(r^2 u_r)/\partial r = 0$), quindi lo sforzo normale è la sola pressione; e che

$$(\tau_{r\theta})_{r=a} = \frac{2\mu U}{a}\left[(2A_2 + 5A_3 + 9A_4)\sin\theta + (2B_2 + 3B_3 + 3B_4)\sin\theta\cos\theta\right]. \tag{1.31}$$

La pressione sulla superficie si ottiene integrando la componente θ dell'equazione del moto (1.2) lungo la superficie, dove i termini convettivi si annullano per l'aderenza e restano $\partial p/\partial\theta$ e il termine viscoso; il risultato è

$$(p)_{r=a} = P + \frac{\mu U}{a}\left[-2(8A_2 + 25A_3 + 54A_4)\cos\theta + 3(2B_2 + 7B_3 + 16B_4)\sin^2\theta\right], \tag{1.32}$$

con P costante di integrazione. Nell'integrale (1.28) la costante e il termine in $\sin^2\theta$ non contribuiscono (sono moltiplicati per $\cos\theta\sin\theta$, dispari rispetto a $\theta = \pi/2$); nell'integrale (1.29) non contribuisce il termine in $\sin\theta\cos\theta$. Con $\int_0^\pi \cos^2\theta\sin\theta\,\mathrm{d}\theta = 2/3$ e $\int_0^\pi \sin^3\theta\,\mathrm{d}\theta = 4/3$:

$$D_p = \tfrac{8}{3}\pi\mu a U\,(8A_2 + 25A_3 + 54A_4), \qquad D_s = \tfrac{16}{3}\pi\mu a U\,(2A_2 + 5A_3 + 9A_4). \tag{1.33}$$

Dividendo per $\frac{1}{2}\rho U^2 \cdot \pi a^2$ (pressione dinamica per area frontale) e ricordando che $\mu/(\rho U a) = 2/Re$:

$$c_{dp} = \frac{32}{3Re}(8A_2 + 25A_3 + 54A_4), \qquad c_{ds} = \frac{64}{3Re}(2A_2 + 5A_3 + 9A_4), \qquad C_D = c_{dp} + c_{ds}. \quad (1.34)$$

Verifica nel limite di Stokes

La soluzione di Stokes, $\psi = (\frac{1}{2}r^2 - \frac{3}{4}r + \frac{1}{4r})(1-\zeta^2)$, ha una forma diversa dalla (1.11) (contiene r invece di $1/r^2$ e $1/r^4$: nel limite $Re \to 0$ Kawaguti usò infatti un'altra funzione di prova, l'Esercizio 8 mostra che essa si riduce esattamente a Stokes). Si può comunque verificare la coerenza delle formule (1.34) applicandole alla soluzione di Stokes scritta come $\psi = (\frac{1}{2}r^2 + \sum a_k r^{-k})(1-\zeta^2)$ con esponenti generici: lo sforzo tangenziale di Stokes vale $\frac{3}{2}\mu U \sin\theta/a$ e la pressione $-\frac{3}{2}\mu U \cos\theta/a$, da cui $D_s = 4\pi\mu a U$, $D_p = 2\pi\mu a U$, $D = 6\pi\mu a U$: due terzi della resistenza di Stokes sono attrito e un terzo è pressione, e $C_D = 24/Re$. Vedremo che a $Re = 5$ il metodo di Galerkin dà ancora una ripartizione simile (33% di pressione), mentre a $Re = 1000$ la pressione pesa per il 64%.

1.5 La bolla di ricircolo

Al crescere del numero di Reynolds la corrente si separa dalla parte posteriore della sfera e compare un *moto secondario*: un anello vorticoso stazionario confinato dentro la linea di corrente $\psi = 0$ (Fig. 1.3). La distanza adimensionale r_s della punta della bolla, dove la velocità sull'asse si annulla, è una radice di

$$\frac{1}{2} + \frac{A_1 + B_1}{r^3} + \frac{A_2 + B_2}{r^4} + \frac{A_3 + B_3}{r^5} + \frac{A_4 + B_4}{r^6} = 0, \quad (1.35)$$

che si ottiene dalla (1.26) ponendo $u_r = 0$ in $\theta = 0$ (dietro la sfera, dove $\cos\theta = 1$ e $3\cos^2\theta - 1 = 2$). Per $r = 1$ la (1.35) è soddisfatta identicamente (aderenza); la bolla esiste se esiste una seconda radice $r_s > 1$.

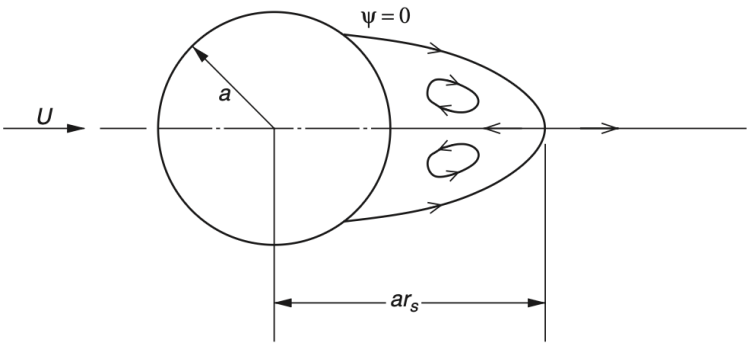


Figura 1.3: Bolla di ricircolo dietro la sfera.

1.6 Il metodo di Newton–Raphson

Sia per la (1.35) sia per il sistema (1.24)–(1.25) serve un metodo per le equazioni algebriche non lineari.

Newton–Raphson per una e per due equazioni

Sia $f(x) = 0$ con f derivabile. Da una stima x_1 si traccia la tangente in $(x_1, f(x_1))$ e se ne prende l'intersezione con l'asse x (Fig. 1.4): poiché $f'(x_1) = f_1/(x_1 - x_2)$,

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}. \tag{1.36}$$

Si arresta quando $|x_{k+1} - x_k| < \varepsilon$. *Convergenza quadratica.* Se x^* è una radice semplice ($f'(x^*) \neq 0$) e $f \in C^2$, sviluppando $0 = f(x^*) = f(x_k) + f'(x_k)(x^* - x_k) + \frac{1}{2}f''(\xi)(x^* - x_k)^2$ e dividendo per $f'(x_k)$ si ottiene

$$x^* - x_{k+1} = -\frac{f''(\xi)}{2f'(x_k)}(x^* - x_k)^2,$$

cioè l'errore al passo $k+1$ è proporzionale al *quadrato* dell'errore al passo k: il numero di cifre esatte raddoppia a ogni iterazione, purché si parta abbastanza vicino. Nel Programma 4.1 la radice r_s richiede 3–12 iterazioni.
Per un sistema $g(x, y) = 0$, $h(x, y) = 0$ il metodo di Newton completo userebbe la matrice jacobiana 2×2. La fonte adotta una variante *disaccoppiata*, più semplice da programmare:

$$x_{k+1} = x_k - \frac{g(x_k, y_k)}{(\partial g/\partial x)(x_k, y_k)}, \tag{1.37}$$

$$y_{k+1} = y_k - \frac{h(x_{k+1}, y_k)}{(\partial h/\partial y)(x_{k+1}, y_k)}, \tag{1.38}$$

in cui ogni equazione è risolta rispetto alla "propria" incognita tenendo fissa l'altra, e il valore aggiornato di x è usato subito nella seconda formula (alla maniera di Gauss–Seidel). È un'iterazione a convergenza soltanto lineare, che converge se l'accoppiamento fra le equazioni è debole, cioè se $|g_y h_x| < |g_x h_y|$ nell'intorno della soluzione; per il sistema (1.24)–(1.25) questa condizione è soddisfatta e bastano poche iterazioni. Il Programma 4.1 usa questa variante e ne confronta il risultato con il metodo di Newton completo (`scipy.optimize.fsolve`).

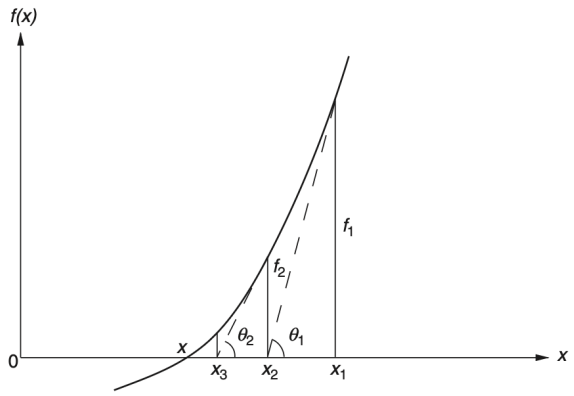


Figura 1.4: Il metodo di Newton–Raphson.

1.7 Il Programma 4.1

Procedura di calcolo

1. Per ciascun numero di Reynolds si risolve il sistema ridotto (1.24)–(1.25) con il metodo di Newton–Raphson disaccoppiato (`newtn2`), partendo dalla soluzione del Re precedente (continuazione) e, per il primo, da $A_1 = -4{,}5$, $B_1 = -0{,}5$ (stima dal limite di Stokes: $A_1 \to -3/4$ in un'altra funzione di prova; i valori qui sono diversi perché la (1.11) non contiene il termine in r).
2. Si ricavano gli altri coefficienti dalle (1.23) e i coefficienti di resistenza dalle (1.34).
3. Si cerca la radice $r_s > 1$ della (1.35) con il metodo di Newton a una variabile (`newtn1`); poiché $r = 1$ è radice per costruzione, si parte da una stima a valle e si accetta la radice solo se $r_s > 1$: altrimenti la corrente è attaccata e si pone $r_s = 1$.
4. Per $Re = 10, 100, 300$ si tracciano le linee di corrente nel semipiano meridiano, $\psi =$ cost, con le formule (1.11).

Dati del Programma 4.1

Venti numeri di Reynolds da 5 a 1000; tolleranza di Newton $\varepsilon = 10^{-6}$; confronto con la correlazione sperimentale di Schiller e Naumann, $C_D = \frac{24}{Re}(1 + 0{,}15 Re^{0{,}687})$, valida fino a $Re \approx 800$.

prog4_1_sfera.py

```
"""
prog4_1_sfera.py -- Programma 4.1: corrente attorno a una sfera a numeri di
Reynolds finiti con il metodo di Galerkin (funzione di prova di Kawaguti).

Funzione di corrente di prova (adimensionale, Re = 2 U a / nu):
    psi = (r^2/2 + A1/r + A2/r^2 + A3/r^3 + A4/r^4)(1 - z^2)
          + (B1/r + B2/r^2 + B3/r^3 + B4/r^4) z (1 - z^2),     z = cos(theta).
Le condizioni di aderenza e l'annullamento del residuo sulla sfera forniscono sei
relazioni lineari; le due condizioni di ortogonalita' di Galerkin danno il sistema
non lineare (vedi prog4_1b_galerkin_simbolico.py per la derivazione)
    g(A1, B1) = 9.931035 A1/Re + 0.011713 B1 - 0.002546 A1 B1 + 44.689656/Re = 0
    h(A1, B1) = 0.050148 A1 + 2.201058 B1/Re + 0.002421 A1^2 - 0.000376 B1^2 +
    0.379018 = 0
risolto con il metodo di Newton-Raphson (NEWTN2). Il programma calcola per
20 numeri di Reynolds i coefficienti di resistenza di pressione, di attrito e
totale, la lunghezza r_s della bolla di ricircolo (radice di u_r = 0 sull'asse,
trovata con NEWTN1) e traccia le linee di corrente per Re = 10, 100, 300.
"""
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import fsolve
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

# coefficienti del sistema ridotto (4 cifre del testo; la derivazione simbolica
# fornisce 9.931034, 0.01171243, 0.002545953, 44.68966; 0.05015221, 2.201058,
# 0.00242145, 0.0003756646, 0.3790197)
C = dict(a1=9.931035, a2=0.011713, a3=0.002546, a4=44.689656,
         b1=0.050148, b2=2.201058, b3=0.002421, b4=0.000376, b5=0.379018)


def g(x, y, Re):
    return C["a1"] * x / Re + C["a2"] * y - C["a3"] * x * y + C["a4"] / Re

```

```
33
34 def h(x, y, Re):
35     return C["b1"] * x + C["b2"] * y / Re + C["b3"] * x * x - C["b4"] * y * y +
       C["b5"]
36
37
38 def gx(x, y, Re):
39     return C["a1"] / Re - C["a3"] * y
40
41
42 def hy(x, y, Re):
43     return C["b2"] / Re - 2 * C["b4"] * y
44
45
46 def newtn1(f, fp, x, eps=1e-6, itmax=100):
47     """Newton-Raphson per una sola equazione f(x) = 0."""
48     for it in range(1, itmax + 1):
49         dx = -f(x) / fp(x)
50         x += dx
51         if abs(dx) < eps:
52             return x, it
53     raise RuntimeError("NEWTN1: convergenza non raggiunta")
54
55
56 def newtn2(x, y, Re, eps=1e-6, itmax=200):
57     """Newton-Raphson 'disaccoppiato' del testo per g(x,y) = 0, h(x,y) = 0:
58     x aggiornato con dg/dx, poi y con dh/dy usando il nuovo x."""
59     for it in range(1, itmax + 1):
60         xn = x - g(x, y, Re) / gx(x, y, Re)
61         yn = y - h(xn, y, Re) / hy(xn, y, Re)
62         dx, dy = abs(xn - x), abs(yn - y)
63         x, y = xn, yn
64         if dx < eps and dy < eps:
65             return x, y, it
66     raise RuntimeError("NEWTN2: convergenza non raggiunta")
67
68
69 def coefficienti(A1, B1):
70     A2 = -15 / 29 * (8 + 5 * A1)
71     A3 = 9 / 29 * (17 + 7 * A1)
72     A4 = -(95 + 34 * A1) / 58
73     B2 = -23 / 9 * B1
74     B3 = 19 / 9 * B1
75     B4 = -5 / 9 * B1
76     return np.array([A1, A2, A3, A4]), np.array([B1, B2, B3, B4])
77
78
79 def psi(r, z, A, B):
80     return (r**2 / 2 + A[0] / r + A[1] / r**2 + A[2] / r**3 + A[3] / r**4) * (1 -
       z**2) \
81         + (B[0] / r + B[1] / r**2 + B[2] / r**3 + B[3] / r**4) * z * (1 - z**2)
82
83
84 def ur_asse(r, A, B):
85     """u_r/U sull'asse a valle (theta = 0): (4.1.44) a meno del fattore 2."""
86     return 0.5 + (A[0] + B[0]) / r**3 + (A[1] + B[1]) / r**4 + (A[2] + B[2]) / r**5
       + (A[3] + B[3]) / r**6
87
88
89 def dur_asse(r, A, B):
```

```
    return -3 * (A[0] + B[0]) / r**4 - 4 * (A[1] + B[1]) / r**5 - 5 * (A[2] + B[2])
    / r**6 - 6 * (A[3] + B[3]) / r**7


Res = [5, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 200, 300, 400, 500, 600, 700,
    800, 900, 1000]
print("    Re        A1          B1          CDP        CDS         CD         RS
    iter")
x, y = -5.0, -0.5                       # valori di partenza (poi si usa la
    soluzione precedente)
risultati = []
for Re in Res:
    x, y, it = newtn2(x, y, Re)
    # verifica con un risolutore di libreria (Newton completo con jacobiano
    numerico)
    xs, ys = fsolve(lambda v: [g(v[0], v[1], Re), h(v[0], v[1], Re)], [x, y],
    xtol=1e-10)
    A, B = coefficienti(x, y)
    cdp = 32 / (3 * Re) * (8 * A[1] + 25 * A[2] + 54 * A[3])
    cds = 64 / (3 * Re) * (2 * A[1] + 5 * A[2] + 9 * A[3])
    # lunghezza della bolla: radice di u_r = 0 con r > 1 (se esiste)
    rs = 1.0
    rr = np.linspace(1.002, 6.0, 2000)      # si parte poco oltre la superficie
    (u_r = 0 in r = 1)
    u = ur_asse(rr, A, B)
    seg = np.nonzero((u[:-1] < 0) & (u[1:] >= 0))[0]
    if len(seg):
        rs, _ = newtn1(lambda r: ur_asse(r, A, B), lambda r: dur_asse(r, A, B),
    rr[seg[-1]])
    risultati.append((Re, x, y, cdp, cds, cdp + cds, rs, A, B))
    print(f"{Re:6.0f} {x:11.4e} {y:11.4e} {cdp:10.4e} {cds:10.4e} {cdp+cds:10.4e}
    {rs:8.4f} {it:5d}"
          f"   (fsolve: {abs(xs-x)+abs(ys-y):.1e})")

# confronto con la correlazione sperimentale di Schiller-Naumann (Re < 800)
print("\nConfronto con la correlazione sperimentale C_D = 24/Re (1 + 0,15
    Re^0,687):")
for Re, x, y, cdp, cds, cd, rs, A, B in risultati:
    if Re in (5, 10, 20, 50, 100, 200, 500, 1000):
        cde = 24 / Re * (1 + 0.15 * Re**0.687)
        print(f"   Re = {Re:5.0f}: C_D Galerkin = {cd:.3f}, sperimentale =
    {cde:.3f}, scarto {100*(cd/cde-1):+.1f} %;"
              f" quota di pressione {100*cdp/cd:.0f} %")
# separazione: sulla sfera u_r = 0 e du_r/dr = 0 (aderenza); il segno di u_r subito
# dietro la sfera e' quello di d^2 u_r/dr^2 in r = 1, che cambia segno alla
    separazione
def curvatura(Re, x0, y0):
    xx, yy, _ = newtn2(x0, y0, Re)
    A, B = coefficienti(xx, yy)
    return 12 * (A[0] + B[0]) + 20 * (A[1] + B[1]) + 30 * (A[2] + B[2]) + 42 *
    (A[3] + B[3])
for k in range(1, len(risultati)):
    if risultati[k][6] > 1.0 + 1e-9:
        Re_a, Re_b = risultati[k - 1][0], risultati[k][0]
        x0, y0 = risultati[k - 1][1], risultati[k - 1][2]
        lo, hi = Re_a, Re_b
        for _ in range(40):
            mid = 0.5 * (lo + hi)
            if curvatura(mid, x0, y0) * curvatura(lo, x0, y0) > 0:
                lo = mid
```

```
            else:
                hi = mid
        print(f"\nSeparazione (r_s > 1) fra Re = {Re_a:.0f} e {Re_b:.0f}: Re_sep =
    {0.5*(lo+hi):.2f}"
              f" (esperimenti di Taneda: circa 24)")
        break

# ------------------------------------------------------------------ figure
fig, ax = plt.subplots(1, 3, figsize=(8.6, 2.6))
xg = np.linspace(-3, 3, 301)
yg = np.linspace(0, 3, 151)
X, Y = np.meshgrid(xg, yg)
R = np.sqrt(X**2 + Y**2)
Z = np.where(R > 0, X / np.maximum(R, 1e-9), 0)
for a_, Re in zip(ax, (10, 100, 300)):
    ris = [t for t in risultati if t[0] == Re][0]
    A, B = ris[7], ris[8]
    PSI = psi(np.maximum(R, 1.0), Z, A, B)
    PSI = np.where(R < 1, np.nan, PSI)
    lv = [-0.01, -0.003, -0.0005, 0.0, 0.05, 0.15, 0.3, 0.6, 1.0, 1.5, 2.2, 3.0,
    4.0]
    a_.contour(X, Y, PSI, levels=lv, colors=[ROSSO if v < 0 else BLU for v in lv],
    linewidths=0.8)
    a_.add_patch(plt.Circle((0, 0), 1, color=GRIGIO))
    a_.set_aspect("equal")
    a_.set_xlim(-3, 3); a_.set_ylim(0, 3)
    a_.set_title(f"$Re={Re}$, $r_s={ris[6]:.2f}$".replace(".", "{,}"))
    a_.set_xlabel("$x/a$")
ax[0].set_ylabel("$y/a$")
fig.tight_layout()
salva(fig, "prog4_1_sfera_linee")

fig, ax = plt.subplots(1, 2, figsize=(7.4, 2.9))
Rv = np.array([t[0] for t in risultati])
ax[0].loglog(Rv, [t[5] for t in risultati], "o-", color=BLU, ms=3, label="Galerkin,
    $C_D$")
ax[0].loglog(Rv, [t[3] for t in risultati], "s--", color=ARANCIO, ms=3,
    label="pressione")
ax[0].loglog(Rv, [t[4] for t in risultati], "^--", color=VERDE, ms=3,
    label="attrito")
Rf = np.logspace(0.5, 3, 100)
ax[0].loglog(Rf, 24 / Rf * (1 + 0.15 * Rf**0.687), "k:", lw=1.2,
    label="Schiller--Naumann")
ax[0].loglog(Rf, 24 / Rf, color=GRIGIO, lw=0.8, label="Stokes $24/Re$")
ax[0].set_xlabel("$Re$"); ax[0].set_ylabel("$C_D$")
ax[0].legend(fontsize=6.5)
ax[1].semilogx(Rv, [t[6] for t in risultati], "o-", color=BLU, ms=3)
ax[1].set_xlabel("$Re$"); ax[1].set_ylabel("$r_s/a$")
ax[1].set_title("lunghezza della bolla di ricircolo")
fig.tight_layout()
salva(fig, "prog4_1_sfera_cd")
```

Listato 1.1: Programma 4.1 – corrente intorno a una sfera a numeri di Reynolds finiti con il metodo di Galerkin (`prog4_1_sfera.py`).

Risultati del Programma 4.1

La tabella stampata (estratto in Tab. 1.1) riproduce cifra per cifra la tabella della fonte: è la verifica che il sistema ridotto e la sua soluzione sono corretti. Il metodo di Newton disaccoppiato converge in 3–12 iterazioni; il metodo di Newton completo (`fsolve`) dà le stesse radici entro 10^{-8}.

- **Resistenza.** A $Re = 5$ la resistenza è per il 67% attrito; la quota di pressione cresce con Re e supera il 50% intorno a $Re = 90$, come anticipato nel testo; a $Re = 1000$ è il 64%. Il confronto con la correlazione di Schiller–Naumann (Fig. 1.6) è buono fra $Re = 20$ e $Re = 500$ (scarti entro il 20%), cattivo a $Re = 5$ ($+38\%$: la funzione di prova non contiene il termine in r della soluzione di Stokes) e a $Re = 1000$ (-33%: la scia reale è ormai instabile e tridimensionale).
- **Separazione.** r_s diventa maggiore di 1 fra $Re = 30$ e 40; il programma localizza il valore critico in $Re_{\text{sep}} = 39{,}1$ cercando il cambiamento di segno della curvatura di u_r sull'asse in $r = 1$ (per $r \to 1$ la (1.35) ha radice doppia alla separazione). L'esperimento di Taneda [11] dà $Re \approx 24$: la funzione di prova, con soli due termini angolari, "vede" la bolla troppo tardi e la calcola troppo corta ($r_s = 1{,}65$ a $Re = 100$ contro circa 2,2 sperimentale).
- **Linee di corrente** (Fig. 1.5). A $Re = 10$ la corrente è quasi simmetrica avanti–dietro; a $Re = 100$ e 300 la bolla è visibile e si allunga.

Tabella 1.1: Metodo di Galerkin per la sfera: coefficienti, resistenza e lunghezza della bolla (estratto; il programma stampa tutti i 20 casi).

Re	A_1	B_1	c_{dp}	c_{ds}	C_D	r_s
5	−4,4946	−0,460	3,2225	6,4095	9,6319	1,0000
10	−4,4786	−0,921	1,6449	3,2189	4,8639	1,0000
20	−4,4145	−1,849	0,8896	1,6377	2,5273	1,0000
40	−4,1621	−3,760	0,5771	0,8746	1,4517	1,0144
100	−2,6083	−10,236	0,5566	0,4870	1,0436	1,6529
300	1,6289	−26,816	0,4817	0,2870	0,7687	2,5321
1000	3,7354	−37,131	0,1887	0,1047	0,2934	2,9357

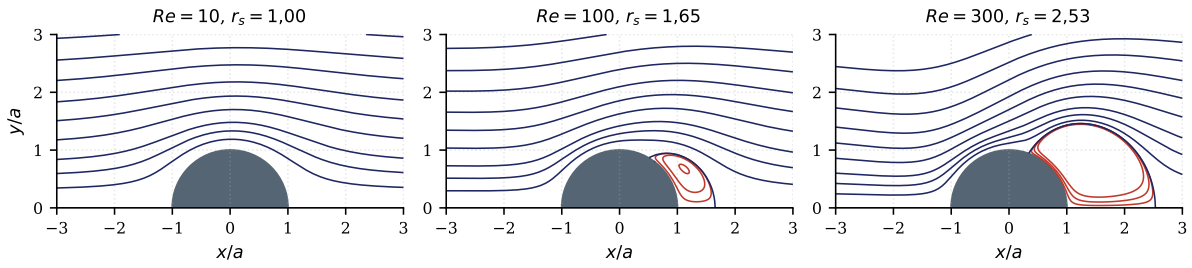


Figura 1.5: Linee di corrente nel semipiano meridiano per $Re = 10$, 100 e 300 (Programma 4.1); in rosso la linea $\psi = 0$ che delimita la bolla di ricircolo.

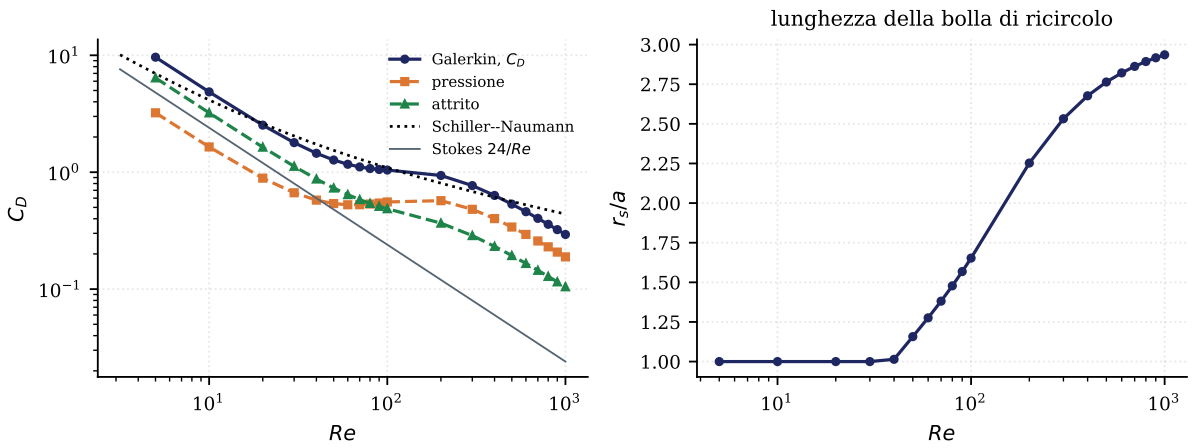


Figura 1.6: Coefficiente di resistenza della sfera: metodo di Galerkin (totale, attrito e pressione), correlazione sperimentale di Schiller–Naumann e legge di Stokes (a sinistra); lunghezza r_s della bolla di ricircolo in funzione di Re (a destra).

Riduzione della resistenza e goccioline in caduta

Il risultato che la resistenza di pressione domina per $Re \gtrsim 90$ ed è causata dalla bassa pressione nella bolla di ricircolo suggerisce la strategia classica per ridurla: *ridurre la regione separata*, affusolando il corpo (le carenature dei carrelli e dei montanti) o energizzando lo strato limite perché separi più a valle (le fossette della palla da golf, i generatori di vortici). Per la gocciolina di pioggia da 0,5 mm ($Re \approx 60$) il metodo dà $C_D \approx 1{,}17$ e una bolla appena nata ($r_s = 1{,}28$): la velocità limite che ne risulta, dall'equilibrio fra peso e resistenza $\tfrac{1}{2}\rho_a U^2 \pi a^2 C_D = \tfrac{4}{3}\pi a^3 \rho_w g$, è $U \approx 2{,}1$ m/s, coerente con le misure. Il modulo 1 usava per lo stesso problema una correlazione empirica: qui la stessa curva esce da un modello fisico.

1.8 Approfondimenti

1.8.1 La derivazione simbolica delle equazioni di Galerkin

Le espressioni dei $g_k(r)$ e le equazioni (1.21)–(1.22) sono state ricavate a mano negli anni Cinquanta; oggi un sistema di calcolo simbolico le riproduce in pochi secondi, ed è un ottimo esercizio di verifica. Il Programma 4.1b costruisce l'operatore di Navier–Stokes (1.7), lo applica alla funzione di prova, separa la parte viscosa da quella convettiva sulla sfera (mostrando che la seconda si annulla per l'aderenza e che la prima dà le (1.17)–(1.18)), calcola i due integrali di Galerkin in forma chiusa e verifica i coefficienti della fonte uno per uno; infine risolve il sistema lineare (1.23) e ottiene il sistema ridotto con nove cifre:

$$g = -0{,}002545953\, A_1 B_1 + \frac{9{,}931034}{Re} A_1 + 0{,}01171243\, B_1 + \frac{44{,}68966}{Re},$$
$$h = 0{,}00242145\, A_1^2 + 0{,}05015221\, A_1 - 0{,}0003756646\, B_1^2 + \frac{2{,}201058}{Re} B_1 + 0{,}3790197,$$

che coincide con le (1.24)–(1.25) alle sei cifre riportate. Con lo stesso programma si potrebbe aggiungere un terzo termine angolare alla funzione di prova ($n = 3$, con $P_2 - P_4$) e ricavare le nuove equazioni, cosa impraticabile a mano.


prog4_1b_galerkin_simbolico.py

```
"""
prog4_1b_galerkin_simbolico.py -- derivazione simbolica (SymPy) delle equazioni
di Galerkin per la sfera a Reynolds finito.

Partendo dalla funzione di corrente di prova di Kawaguti e dall'equazione di
Navier-Stokes in forma di funzione di corrente (coordinate r, zeta = cos theta),
    (2/Re) D^4 psi + (1/r^2) [ psi_z d/dr - psi_r d/dz - 2 z/(1-z^2) psi_r - (2/r) psi_z ] D^2 psi = 0,
    D^2 = d^2/dr^2 + (1 - z^2)/r^2 d^2/dz^2,
il programma:
 1) calcola il residuo (NS) e ne impone l'annullamento sulla sfera (r = 1),
    ottenendo le due condizioni 9 A2 + 35 A3 + 90 A4 = 0 e B1 - 6 B3 - 21 B4 = 0;
 2) impone le due condizioni di ortogonalita' di Galerkin
       int_1^oo int_-1^1 (NS) (1/r) [P0 - P2] dz dr = 0,
       int_1^oo int_-1^1 (NS) (1/r^2) [P1 - P3] dz dr = 0,
    con P_n polinomi di Legendre, e verifica i coefficienti razionali del testo;
 3) risolve le sei relazioni lineari e riduce il sistema a due equazioni in
    (A1, B1), i cui coefficienti numerici sono quelli usati in prog4_1_sfera.py.
Tempo di esecuzione: circa 15-20 s.
"""
import sympy as sp

r, z, Re = sp.symbols("r zeta Re", positive=True)
A1, A2, A3, A4, B1, B2, B3, B4 = sp.symbols("A1:5 B1:5")
psi = (r**2 / 2 + A1 / r + A2 / r**2 + A3 / r**3 + A4 / r**4) * (1 - z**2) \
    + (B1 / r + B2 / r**2 + B3 / r**3 + B4 / r**4) * z * (1 - z**2)


def D2(f):
    return sp.diff(f, r, 2) + (1 - z**2) / r**2 * sp.diff(f, z, 2)


D2psi = D2(psi)
pr, pz = sp.diff(psi, r), sp.diff(psi, z)
NS = 2 / Re * D2(D2psi) + (pz * sp.diff(D2psi, r) - pr * sp.diff(D2psi, z)
                           - 2 * z / (1 - z**2) * pr * D2psi - 2 / r * pz * D2psi) / r**2
NS = sp.expand(sp.simplify(NS))

# 1) condizioni di aderenza e residuo nullo sulla sfera
aderenza = [sp.Eq(A1 + A2 + A3 + A4, -sp.Rational(1, 2)), sp.Eq(A1 + 2 * A2 + 3 * A3 + 4 * A4, 1),
            sp.Eq(B1 + B2 + B3 + B4, 0), sp.Eq(B1 + 2 * B2 + 3 * B3 + 4 * B4, 0)]
visc = sp.factor(sp.expand(2 * D2(D2psi).subs(r, 1)))
conv = sp.expand((NS - 2 / Re * D2(D2psi)).subs(r, 1))
sol_ad = sp.solve(aderenza, [A3, A4, B3, B4])
print("Parte viscosa del residuo sulla sfera, 2 D^4 psi (r = 1):")
print("  ", visc)
print("Parte convettiva sulla sfera dopo aver imposto l'aderenza:", sp.simplify(conv.subs(sol_ad)))

# 2) ortogonalita' di Galerkin
P = lambda n: sp.legendre(n, z)
pesi = [(P(0) - P(2)) / r, (P(1) - P(3)) / r**2]
G = []
for w in pesi:
    Iz = sp.expand(sp.integrate(sp.expand(NS * w), (z, -1, 1)))
    G.append(sp.expand(sp.integrate(Iz, (r, 1, sp.oo))))
print("\nPrima equazione di Galerkin (4.1.24):")
print("  ", G[0])
```

```
print("Seconda equazione di Galerkin (4.1.25):")
print("  ", G[1])
# alcuni coefficienti da confrontare con il testo
print("\nVerifica di alcuni coefficienti: A1*B1 ->", G[0].coeff(A1 * B1), "(testo -512/245);",
      "A2/Re ->", sp.simplify(G[0].coeff(A2) * Re), "(testo 32/Re * 6/5 = 192/5);",
      "B1^2 ->", G[1].coeff(B1**2), "(testo -8/63)")

# 3) riduzione a due incognite
lineari = aderenza + [sp.Eq(9 * A2 + 35 * A3 + 90 * A4, 0), sp.Eq(B1 - 6 * B3 - 21 * B4, 0)]
sol = sp.solve(lineari, [A2, A3, A4, B2, B3, B4])
print("\nRelazioni lineari (4.1.26):")
for k, v in sol.items():
    print(f"   {k} = {v}")
E1 = sp.expand(G[0].subs(sol))
E2 = sp.expand(G[1].subs(sol))
print("\nSistema ridotto in (A1, B1):")
print("   g =", sp.N(E1, 7))
print("   h =", sp.N(E2, 7))
print("   (testo: g = 9.931035 A1/Re + 0.011713 B1 - 0.002546 A1 B1 + 44.689656/Re,")
print("           h = 0.050148 A1 + 2.201058 B1/Re + 0.002421 A1^2 - 0.000376 B1^2 + 0.379018)")

# la funzione di prova per Re piccolo (4.1.52) contiene la soluzione di Stokes
A1s, A2s, A3s, A4s = sp.symbols("a1:5")
psi_stokes = (r**2 / 2 - 3 * r / 4 + 1 / (4 * r)) * (1 - z**2)
print("\nSoluzione di Stokes: D^4 psi =", sp.simplify(D2(D2(psi_stokes))),
      "; psi(1) =", sp.simplify(psi_stokes.subs(r, 1)), "; psi_r(1) =",
    sp.simplify(sp.diff(psi_stokes, r).subs(r, 1)))
```

Listato 1.2: `prog4_1b_galerkin_simbolico.py`: residuo, equazioni di Galerkin e sistema ridotto ricavati con SymPy.

1.8.2 La corrente radiale fra pareti inclinate (Jeffery–Hamel)

Il metodo di Galerkin non è legato alla geometria sferica. Si consideri il moto piano stazionario di un fluido incomprimibile fra due pareti piane che formano un angolo 2α (Fig. 1.7), con corrente puramente radiale $u_r(r,\theta)$. Le componenti dell'equazione di Navier–Stokes in coordinate polari sono

$$\rho u_r \frac{\partial u_r}{\partial r} + \frac{\partial p}{\partial r} - \mu\left[\frac{\partial}{\partial r}\left(\frac{1}{r}\frac{\partial}{\partial r}(r u_r)\right) + \frac{1}{r^2}\frac{\partial^2 u_r}{\partial \theta^2}\right] = 0, \tag{1.39}$$

$$-\frac{1}{r}\frac{\partial p}{\partial \theta} + \frac{2\mu}{r^2}\frac{\partial u_r}{\partial \theta} = 0, \tag{1.40}$$

e la continuità $\partial(r u_r)/\partial r = 0$ dice che $r u_r$ dipende soltanto da θ. Se Q è la portata in massa per unità di larghezza,

$$2\rho \int_0^{\alpha} u_r \, r \, \mathrm{d}\theta = Q, \qquad u_r = 0 \text{ in } \theta = \pm\alpha. \tag{1.41}$$

Snyder, Spriggs e Stewart [12] scelsero le funzioni di prova

$$u_r = \frac{Q}{\rho r}\sum_{j=1}^{n} C_j(\alpha^2 - \theta^2)^j, \qquad p - p_0 = \frac{Q\mu}{\rho r^2}\left[C_0 + 2\sum_{j=1}^{n} C_j(\alpha^2 - \theta^2)^j\right], \tag{1.42}$$

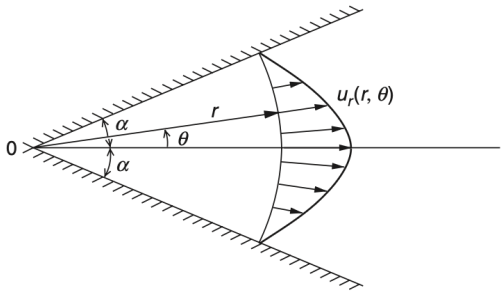


Figura 1.7: Corrente radiale fra pareti inclinate.

che soddisfano esattamente la continuità, la componente θ (1.40) e l'aderenza. Sostituite nella (1.39) lasciano il residuo

$$E(\theta) = Re \sum_{i,j} C_i C_j (\alpha^2 - \theta^2)^{i+j} + 2C_0 + 4 \sum_j C_j (\alpha^2 - \theta^2)^j + \sum_j C_j \frac{\mathrm{d}^2}{\mathrm{d}\theta^2} (\alpha^2 - \theta^2)^j, \tag{1.43}$$

con $Re = Q/\mu$, che viene distribuito con i pesi $(\alpha^2 - \theta^2)^k$:

$$\int_0^\alpha E(\theta)(\alpha^2 - \theta^2)^k \, \mathrm{d}\theta = 0, \qquad k = 1, \ldots, n; \tag{1.44}$$

con la condizione di portata, $n+1$ equazioni per le $n+1$ incognite $C_0, \ldots, C_n$.

La soluzione esatta

Ponendo $u_r = \frac{Q}{\rho r} f(\theta)$ e $p - p_0 = \frac{Q\mu}{\rho r^2}[C_0 + 2f]$, la (1.40) è soddisfatta identicamente e la (1.39) si riduce all'equazione ordinaria *non lineare*

$$f'' + 4f + Ref^2 + 2C_0 = 0, \qquad f(\pm\alpha) = 0, \qquad 2\int_0^\alpha f \, \mathrm{d}\theta = 1, \tag{1.45}$$

(il residuo (1.43) è esattamente il primo membro, valutato sulla funzione di prova). È il problema di Jeffery (1915) e Hamel (1917), una delle poche soluzioni esatte di Navier–Stokes con termini inerziali non nulli; la sua soluzione si esprime con funzioni ellittiche (Rosenhead [13], Landau e Lifshitz [14]). Moltiplicando per f' e integrando si ottiene un integrale primo, $\frac{1}{2}f'^2 + 2f^2 + \frac{1}{3}Ref^3 + 2C_0 f = \text{cost}$, da cui la quadratura ellittica. Due proprietà importanti: per pareti *convergenti* ($Re < 0$ con la convenzione di portata entrante) esiste una soluzione simmetrica per ogni $|Re|$, con profilo sempre più piatto; per pareti *divergenti* la soluzione simmetrica con sola corrente uscente esiste solo fino a un valore critico di Re, oltre il quale compare una corrente inversa alle pareti e poi la soluzione simmetrica cessa di esistere (le soluzioni diventano asimmetriche o oscillanti). Il Programma 4.1c risolve la (1.45) per tiro e la usa come riferimento per il metodo di Galerkin.

prog4_1c_jeffery_hamel.py

```
"""
prog4_1c_jeffery_hamel.py -- metodo di Galerkin per la corrente radiale fra due
pareti piane inclinate (corrente di Jeffery-Hamel), confrontato con la soluzione
esatta del problema ai limiti.

Corrente puramente radiale u_r = (Q/(rho r)) f(theta) fra le pareti theta = +-alfa,
portata Q per unita' di larghezza, Re = Q/mu. L'equazione del moto si riduce a
    f'' + 4 f + Re f^2 + 2 C0 = 0,   f(+-alfa) = 0,   2 int_0^alfa f dtheta = 1,
con C0 costante della pressione, p - p0 = (Q mu/(rho r^2)) [C0 + 2 f].
Galerkin (Snyder, Spriggs e Stewart): f = sum_{j=1}^n C_j (alfa^2 - theta^2)^j,
    residuo
E(theta) ortogonale a (alfa^2 - theta^2)^k, k = 1..n, piu' il vincolo di portata:
n + 1 equazioni (quadratiche) nelle n + 1 incognite C0, C_1..C_n, risolte con
Newton-Raphson (fsolve). Gli integrali sono calcolati in forma chiusa con SymPy.
"""
import numpy as np
import sympy as sp
from scipy.optimize import fsolve
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

RE, ALFA = 14.164, 0.36


def galerkin(n, Re=RE, alfa=ALFA):
    th = sp.symbols("theta")
    C = sp.symbols(f"C0:{n+1}")
    base = [(alfa**2 - th**2)**j for j in range(1, n + 1)]
    f = sum(C[j] * base[j - 1] for j in range(1, n + 1))
    E = sp.diff(f, th, 2) + 4 * f + Re * f**2 + 2 * C[0]
    eqs = [2 * sp.integrate(f, (th, 0, alfa)) - 1]
    eqs += [sp.integrate(sp.expand(E * base[k]), (th, 0, alfa)) for k in range(n)]
    F = sp.lambdify([C], eqs, "numpy")
    J = sp.lambdify([C], sp.Matrix(eqs).jacobian(C), "numpy")
    c0 = np.zeros(n + 1); c0[1] = 1.0 / alfa**3
    c, info, ier, msg = fsolve(lambda v: np.array(F(v), float), c0, fprime=lambda
    v: np.array(J(v), float),
                               full_output=True, xtol=1e-13)
    assert ier == 1, msg
    fnum = sp.lambdify(th, f.subs(dict(zip(C, c))), "numpy")
    return c, fnum


def esatta(Re=RE, alfa=ALFA, guida=(2.4, -11.3)):
    """Soluzione 'esatta' per tiro: incognite f(0) e C0, condizioni f(alfa) = 0 e
    2 int_0^alfa f dtheta = 1; 'guida' e' il punto di partenza (continuazione in
    Re)."""
    def integra(f0, C0):
        return solve_ivp(lambda t, y: [y[1], -4 * y[0] - Re * y[0]**2 - 2 * C0,
    y[0]], (0, alfa),
                         [f0, 0.0, 0.0], rtol=1e-11, atol=1e-13, dense_output=True)
    def residuo(v):
        s = integra(*v)
        return [s.y[0, -1], s.y[2, -1] - 0.5]
    v, info, ier, msg = fsolve(residuo, guida, xtol=1e-12, full_output=True)
    assert ier == 1, msg
    s = integra(*v)
    s.p = [v[1]]
    return s
```

```
57
58
59  sol = esatta()
60  print(f"Corrente di Jeffery-Hamel, Re = {RE}, alfa = {ALFA} rad
        ({np.degrees(ALFA):.1f} gradi):")
61  print(f"   soluzione esatta: C0 = {sol.p[0]:.6f}, f(0) = {sol.sol(0)[0]:.6f},
        f'(alfa) = {sol.sol(ALFA)[1]:.5f}")
62  th = np.linspace(0, ALFA, 181)
63  fe = sol.sol(th)[0]
64  risultati = {}
65  for n in (1, 2, 3, 4):
66      c, fnum = galerkin(n)
67      err = np.max(np.abs(fnum(th) - fe))
68      risultati[n] = (c, fnum)
69      print(f"   Galerkin n = {n}: C0 = {c[0]:.6f}, C = {np.array2string(c[1:],
        precision=4)},"
70            f" f(0) = {fnum(0.0):.6f}, errore max {err:.2e}")
71
72  # continuazione in Re: comparsa della corrente inversa e punto di svolta
73  print("\nAl crescere di Re (pareti divergenti) il profilo si gonfia al centro e si
        appiattisce vicino")
74  print("alle pareti; oltre un valore critico compare una corrente inversa (f'(alfa)
        > 0):")
75  soluzioni = {}
76  guida = (2.4, -11.3)
77  Re = RE
78  while Re < 60:
79      try:
80          s = esatta(Re, guida=guida)
81      except AssertionError:
82          print(f"   Re = {Re:7.3f}: il tiro non converge piu': la famiglia di
        soluzioni simmetriche con sola"
83                f" corrente uscente termina (punto di svolta) fra Re = {Re-0.5:.1f} e
        {Re:.1f}")
84          break
85      guida = (s.sol(0)[0], s.p[0])
86      soluzioni[round(Re, 3)] = s
87      if abs((Re - RE) % 5.0) < 1e-9:
88          print(f"   Re = {Re:7.3f}: f(0) = {s.sol(0)[0]:.4f}, C0 = {s.p[0]:8.3f},
        f'(alfa) = {s.sol(ALFA)[1]:+.4f}"
89                + ("  <- corrente inversa" if s.sol(ALFA)[1] > 0 else ""))
90      Re += 0.5
91  Re_s = [Re for Re, s in soluzioni.items() if s.sol(ALFA)[1] > 0]
92  if Re_s:
93      print(f"   prima corrente inversa alla parete fra Re = {max(Re for Re in
        soluzioni if Re not in Re_s):.1f}"
94            f" e {min(Re_s):.1f}")
95
96  fig, ax = plt.subplots(1, 2, figsize=(7.4, 2.9))
97  ax[0].plot(th, fe, "k", lw=1.8, label="esatta")
98  for n, col in zip((1, 2, 3), (GRIGIO, ARANCIO, BLU)):
99      ax[0].plot(th, risultati[n][1](th), "--", color=col, lw=1.1, label=f"Galerkin
        $n={n}$")
100 ax[0].set_xlabel("$\\theta$ (rad)")
101 ax[0].set_ylabel("$u_r\\,\\rho r/Q$")
102 ax[0].legend(fontsize=7)
103 ax[0].set_title(f"$Re={RE}$, $\\alpha={ALFA}$".replace(".", "{,}"))
104 for Re, col in zip((14.164, 19.164, 24.164, 29.164), (BLU, VERDE, ARANCIO, ROSSO)):
105     s = soluzioni[Re]
106     ax[1].plot(th, s.sol(th)[0], color=col, label=f"$Re={Re:.1f}$".replace(".",
```

```
    "{,}"))
ax[1].axhline(0, color="k", lw=0.6)
ax[1].set_xlabel("$\\theta$ (rad)")
ax[1].set_ylabel("$u_r\\,\\rho r/Q$")
ax[1].legend(fontsize=7)
ax[1].set_title("effetto del numero di Reynolds")
fig.tight_layout()
salva(fig, "prog4_1c_jeffery_hamel")
```

Listato 1.3: `prog4_1c_jeffery_hamel.py`: metodo di Galerkin con $n = 1, \dots, 4$ termini per la corrente radiale fra pareti inclinate, soluzione esatta per tiro, continuazione in Re.

Risultati del Programma 4.1c

Per $Re = 14{,}164$ e $\alpha = 0{,}36$ rad (20,6°), i valori del progetto della fonte, la soluzione esatta ha $f(0) = 2{,}4036$ e $C_0 = -11{,}319$. Il metodo di Galerkin converge rapidamente al crescere del numero di termini: l'errore massimo sul profilo è 0,32 con $n = 1$ (il profilo parabolico di Poiseuille), 0,052 con $n = 2$ (il caso del progetto, con $C_0 = -11{,}10$, $C_1 = 7{,}78$, $C_2 = 79{,}97$), 0,0060 con $n = 3$ e 0,00065 con $n = 4$: circa un fattore 10 per ogni termine aggiunto, cioè convergenza esponenziale, come ci si aspetta per funzioni di prova polinomiali e soluzione analitica (Fig. 1.8). Aumentando Re con pareti divergenti il profilo si gonfia al centro e si appiattisce alle pareti, $f'(\alpha)$ cresce da $-6{,}74$ verso zero: la corrente inversa alla parete compare fra $Re = 24{,}7$ e 25,2, e fra 29,2 e 29,7 il tiro non converge più perché la famiglia di soluzioni simmetriche termina in un punto di svolta (*fold*). È lo stesso fenomeno della separazione, visto su una soluzione esatta: il gradiente avverso di pressione del diffusore (p cresce con r) frena il fluido vicino alle pareti fino a invertirlo.

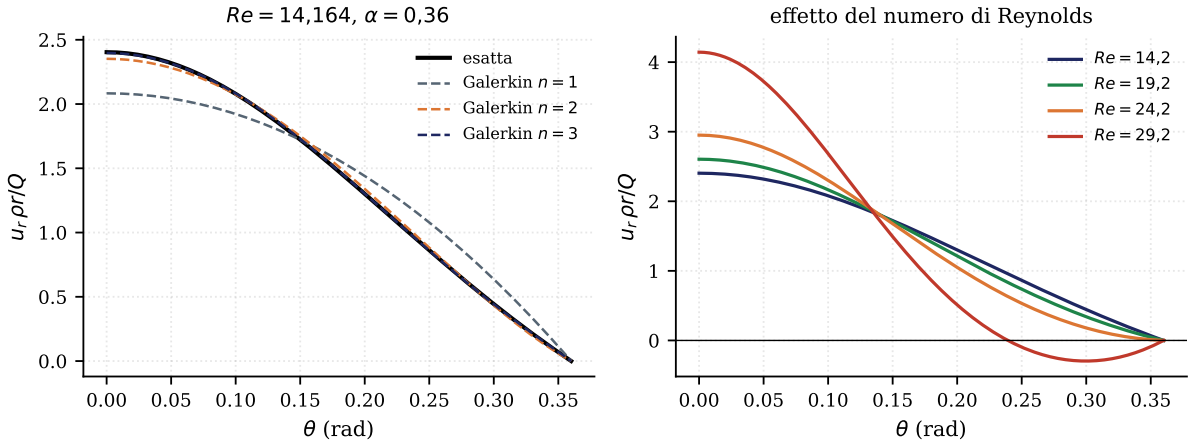


Figura 1.8: Corrente di Jeffery–Hamel: profili di velocità con $n = 1, 2, 3$ termini di Galerkin contro la soluzione esatta (a sinistra) e profili esatti al crescere di Re fino alla comparsa della corrente inversa alle pareti (a destra).

1.8.3 Pregi e limiti del metodo

Il metodo di Galerkin ha tre pregi che spiegano il suo ruolo storico e la sua discendenza (elementi finiti, metodi spettrali): la soluzione è una funzione analitica, derivabile quante volte si vuole (lo sforzo e la pressione sulla sfera sono formule, non differenze finite); la convergenza è rapida quando la soluzione è regolare e le funzioni di prova sono ben scelte; il numero di

incognite è piccolissimo (due, per la sfera). Ha però limiti altrettanto chiari: la scelta delle funzioni di prova richiede di conoscere in anticipo la forma della soluzione, e un cattivo "indovinare" dà risultati sbagliati senza alcun segnale d'allarme (la bolla della sfera); l'algebra cresce in modo esplosivo con il numero di termini; le equazioni non lineari risultanti possono avere più soluzioni o nessuna, e il metodo di Newton trova quella più vicina al punto di partenza. Per corpi di forma qualsiasi, con strati limite sottili e scie, i metodi a griglia dei capitoli seguenti sono indispensabili; il metodo di Galerkin sopravvive in essi nella forma locale degli elementi finiti, dove le funzioni di prova sono semplici polinomi su piccoli elementi e la "forma della soluzione" non va più indovinata.

Capitolo 2

Differenze upwind e viscosità artificiale

2.1 Il problema del termine convettivo

La difficoltà principale della fluidodinamica numerica è la derivata sostanziale $\mathrm{D}/\mathrm{D}t = \partial/\partial t + \mathbf{V}\cdot\nabla$, non lineare, che compare nell'equazione di Eulero, in quella di Navier–Stokes e in quella dell'energia. Nel modulo precedente le equazioni paraboliche contenevano solo la diffusione, e lo schema centrato nello spazio si comportava bene; con la convezione le cose cambiano. Per isolare il problema si considera l'equazione modello unidimensionale

$$\frac{\partial \zeta}{\partial t} + u\frac{\partial \zeta}{\partial x} = 0, \tag{2.1}$$

che descrive il trasporto della vorticità ζ da parte di una corrente di velocità u costante e positiva (si ottiene dal rotore dell'equazione di Eulero in un moto unidimensionale, come si vedrà nel Cap. 6). La soluzione esatta è $\zeta(x,t) = \zeta_0(x-ut)$: il profilo iniziale trasla rigidamente verso valle senza deformarsi. Uno schema numerico "buono" deve riprodurre questo comportamento: trasportare senza amplificare, senza propagare informazione contro corrente e, possibilmente, senza deformare.

2.2 Lo schema upwind e l'analisi della perturbazione discreta

Il piano x–t è coperto da una griglia di maglie $h \times \tau$. Approssimando la derivata temporale con la differenza in avanti e quella spaziale con la differenza *all'indietro*, cioè dal lato da cui proviene la corrente (Fig. 2.1), la (2.1) nel nodo (x_i, t_j) diventa

$$\frac{\zeta_{i,j+1} - \zeta_{i,j}}{\tau} = -u\,\frac{\zeta_{i,j} - \zeta_{i-1,j}}{h}, \tag{2.2}$$

con errore di troncamento $\mathcal{O}(\tau, h)$. È lo **schema upwind** ("a monte"). Riordinando,

$$\zeta_{i,j+1} = (1-C)\zeta_{i,j} + C\zeta_{i-1,j}, \qquad C = \frac{u\tau}{h}, \tag{2.3}$$

in cui C è il numero di Courant. Il nuovo valore è una *media pesata* dei valori nel nodo stesso e nel nodo a monte.

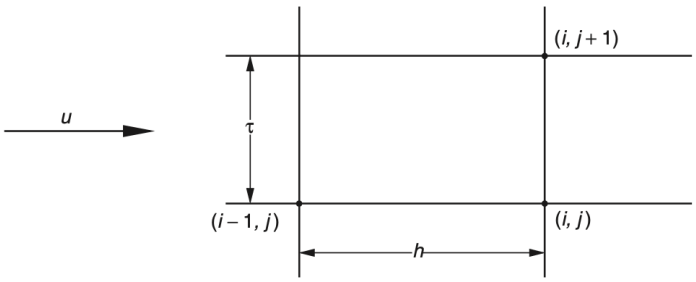


Figura 2.1: Lo schema upwind.

Analisi della perturbazione discreta

Il metodo, dovuto a Thom e Apelt [9] e sviluppato da Thoman e Szewczyk, è concettualmente diverso dall'analisi di von Neumann: invece di seguire un'onda di Fourier segue una *perturbazione puntuale*. Si assume la soluzione stazionaria $\zeta_{i,j} = 0$ per ogni i all'istante t_j e si introduce in x_m un disturbo ϵ. Dalla (2.3) al passo successivo il disturbo si trova in due nodi:

$$\zeta_{m,j+1} = (1 - C)\epsilon, \qquad \zeta_{m+1,j+1} = C\epsilon. \tag{2.4}$$

Per la stabilità si richiede che nessuno dei due superi in modulo il disturbo iniziale:

$$\left|\zeta_{m,j+1}/\epsilon\right| \leq 1, \qquad \left|\zeta_{m+1,j+1}/\epsilon\right| \leq 1. \tag{2.5}$$

La prima dà $-1 \leq 1 - C \leq 1$: la disuguaglianza di destra è automatica per $C > 0$, quella di sinistra dà $C \leq 2$. La seconda è più restrittiva, $C \leq 1$. Al secondo passo i disturbi sono

$$\zeta_{m,j+2} = (1 - C)^2\epsilon, \qquad \zeta_{m+1,j+2} = 2C(1 - C)\epsilon, \qquad \zeta_{m+2,j+2} = C^2\epsilon, \tag{2.6}$$

e in generale, dopo n passi, il disturbo nel nodo $m + k$ vale $\binom{n}{k} C^k (1 - C)^{n-k}\epsilon$: sono i termini dello sviluppo binomiale di $[C + (1 - C)]^n = 1$. Per $0 < C \leq 1$ tutti i pesi sono non negativi e la loro somma è 1, quindi nessuno di essi supera 1 e il disturbo massimo *decresce* con n (la distribuzione binomiale si allarga e si abbassa). Per $C > 1$ i pesi hanno segni alterni e la loro somma in modulo, $[C + (C - 1)]^n = (2C - 1)^n$, cresce esponenzialmente. Il criterio di stabilità è dunque

$$0 < C \leq 1. \tag{2.7}$$

È lo stesso risultato dell'analisi di von Neumann (Esercizio 1), ma ottenuto con un ragionamento diverso: qui si segue *dove va* il disturbo, non quanto si amplifica un'onda.

La proprietà trasportiva e lo schema downwind

Se nella (2.1) la derivata spaziale fosse approssimata con la differenza in avanti (*downwind*, "a valle"), si otterrebbe un'equazione della stessa forma della (2.3) ma con C *negativo*: la condizione di destra della $-1 \leq 1 - C \leq 1$ non può mai essere soddisfatta, e lo schema downwind è incondizionatamente instabile. Fisicamente: in un fluido ideale la vorticità viaggia con il fluido, quindi il valore in x_i dipende da ciò che accade a monte, non a valle; lo schema downwind usa l'informazione sbagliata.

Lo schema upwind possiede invece la **proprietà trasportiva**: una perturbazione è trasportata solo nel verso del moto. Dalla (2.4), la variazione di vorticità nel nodo perturbato e in quello a valle in un passo è

$$\zeta_{m,j+1} - \zeta_{m,j} = -\frac{u\tau}{h}\epsilon, \qquad \zeta_{m+1,j+1} - \zeta_{m+1,j} = +\frac{u\tau}{h}\epsilon, \tag{2.8}$$

mentre il nodo a monte $m-1$ non cambia: il disturbo lascia x_m e compare in x_{m+1}, come deve. Per $C = 1$ la (2.3) dà $\zeta_{i,j+1} = \zeta_{i-1,j}$: la soluzione numerica è *esatta*, perché in un passo τ il fluido percorre esattamente una maglia h e il profilo trasla di un nodo. Lo schema centrato (Esercizio 2) non ha la proprietà trasportiva: il disturbo si propaga anche a monte, e lo schema è per di più instabile per ogni $C > 0$.

2.3 L'analisi di Hirt e la viscosità artificiale

Lo schema upwind viene ora esaminato da un terzo punto di vista. Sviluppando in serie di Taylor intorno a (x_i, t_j) i due termini estremi della (2.2),

$$\zeta_{i,j+1} = \zeta_{i,j} + \tau\frac{\partial\zeta}{\partial t} + \frac{\tau^2}{2}\frac{\partial^2\zeta}{\partial t^2} + \mathcal{O}(\tau^3), \tag{2.9}$$

$$\zeta_{i-1,j} = \zeta_{i,j} - h\frac{\partial\zeta}{\partial x} + \frac{h^2}{2}\frac{\partial^2\zeta}{\partial x^2} + \mathcal{O}(h^3), \tag{2.10}$$

e sostituendo nella (2.2), si ottiene l'equazione differenziale che lo schema *effettivamente* risolve:

$$\frac{\partial\zeta}{\partial t} + \frac{\tau}{2}\frac{\partial^2\zeta}{\partial t^2} + \mathcal{O}(\tau^2) = -u\frac{\partial\zeta}{\partial x} + \frac{uh}{2}\frac{\partial^2\zeta}{\partial x^2} + \mathcal{O}(h^2). \tag{2.11}$$

La derivata seconda nel tempo si elimina con l'equazione stessa: per u costante, dalla (2.1) al primo ordine,

$$\frac{\partial^2\zeta}{\partial t^2} = \frac{\partial}{\partial t}\left(-u\frac{\partial\zeta}{\partial x}\right) = -u\frac{\partial}{\partial x}\frac{\partial\zeta}{\partial t} = u^2\frac{\partial^2\zeta}{\partial x^2}, \tag{2.12}$$

e, trattenendo i termini fino al primo ordine in τ e h,

$$\frac{\partial\zeta}{\partial t} + u\frac{\partial\zeta}{\partial x} = \mu_e\frac{\partial^2\zeta}{\partial x^2}, \qquad \mu_e = \frac{1}{2}uh\,(1-C). \tag{2.13}$$

Lo schema upwind risolve dunque, con errore del secondo ordine, non l'equazione del trasporto puro ma un'equazione di *convezione–diffusione* con un coefficiente di diffusione μ_e determinato dai passi di griglia. Poiché il termine a destra ha la stessa forma del termine viscoso dell'equazione di Navier–Stokes, μ_e si chiama **viscosità artificiale** (o numerica): il suo effetto è un'attenuazione e uno sparpagliamento artificiali dei profili. μ_e si annulla per $C = 1$ (lo schema è esatto) e quando $\tau, h \to 0$ (consistenza).

Il criterio di stabilità di Hirt

Hirt [8] osservò che un coefficiente di diffusione negativo è fisicamente impossibile: un'equazione $\zeta_t = \mu_e\zeta_{xx}$ con $\mu_e < 0$ concentra i disturbi invece di sparpagliarli, cioè li amplifica senza limite (il fattore di amplificazione dell'onda $e^{\mathrm{i}kx}$ è $e^{-\mu_e k^2 t}$, che esplode per $\mu_e < 0$ tanto più rapidamente quanto più k è grande). La richiesta $\mu_e \geq 0$ dà $C \leq 1$: lo stesso criterio (2.7). Questo procedimento, che deduce la stabilità dall'equazione

differenziale ricostruita dallo schema con gli sviluppi di Taylor, è l'**analisi di Hirt**, oggi nota anche come *metodo dell'equazione modificata* (Warming e Hyett [7], Approfondimenti). Le tre analisi (von Neumann, perturbazione discreta, Hirt) poggiano su argomenti fisici diversi e non sempre concordano: le prime due richiedono che un disturbo non cresca, ma la prima considera una distribuzione di disturbi sintetizzabile in serie di Fourier e la seconda un solo disturbo puntuale; la terza richiede che il coefficiente di diffusione sia non negativo e, tenendo solo il termine dominante dell'errore, può essere ingannata dai termini successivi. Gli Esercizi 3 e 4 mostrano un caso in cui i tre metodi danno risposte diverse per lo stesso schema. Il metodo di von Neumann resta il più usato; una valutazione comparata si trova in Roache [3].

La viscosità artificiale non è innocua

In un calcolo di Navier–Stokes con lo schema upwind la viscosità effettiva è $\nu + \mu_e$. Se la griglia è grossolana rispetto allo spessore degli strati limite, $\mu_e = uh(1-C)/2$ può superare ν: il numero di Reynolds *effettivo* della simulazione è più basso di quello nominale, gli strati limite risultano più spessi, la separazione più precoce, le scie più corte e le instabilità smorzate. Il rapporto $\mu_e/\nu = \tfrac{1}{2}(1-C)\,uh/\nu = \tfrac{1}{2}(1-C)\,Pe_h$, dove $Pe_h = uh/\nu$ è il numero di Péclet di cella: lo schema upwind è affidabile solo se Pe_h è dell'ordine dell'unità o minore. Il prezzo della stabilità è dunque una griglia fine, oppure schemi di ordine superiore (Approfondimenti).

2.4 Il programma: trasporto di una gaussiana

Il Programma 4.2 traduce in esperimenti numerici le tre analisi. Prima segue una perturbazione discreta unitaria con gli schemi upwind, centrato e downwind per vari C; poi trasporta un profilo gaussiano $\zeta_0 = \exp[-(x-x_0)^2/(2s_0^2)]$ per un tempo $t = 8$ con $u = 1$ e $h = 0{,}05$, confrontando upwind, centrato (FTCS), downwind e Lax–Wendroff (Approfondimenti); infine *misura* la viscosità artificiale: poiché una gaussiana che si diffonde con coefficiente μ_e resta gaussiana con varianza $s^2(t) = s_0^2 + 2\mu_e t$, dalla varianza del profilo trasportato si ricava μ_e e lo si confronta con la previsione (2.13).

Dati del Programma 4.2

Griglia periodica $0 \le x \le 20$ con $h = 0{,}05$ (400 nodi), $u = 1$; gaussiana centrata in $x_0 = 4$ con $s_0 = 0{,}5$; tempo finale $t = 8$ (il centro arriva in $x = 12$); numeri di Courant $C = 0{,}1$–1.

```
"""
prog4_2_upwind.py -- differenze a monte (upwind) e viscosita' artificiale per
l'equazione modello della convezione  zeta_t + u zeta_x = 0,  u > 0 costante.

1) Analisi della perturbazione discreta: un disturbo unitario in x_m a t_j si
   propaga nei passi successivi con i pesi binomiali (1-C)^k C^(n-k): si
   verifica che per 0 < C <= 1 nessun valore supera 1 e che la perturbazione
   viaggia solo verso valle (proprieta' di trasporto). Con le differenze
   centrate il disturbo si propaga anche a monte e cresce.
2) Schema upwind, downwind, centrato (FTCS) e Lax-Wendroff su un dato
   iniziale gaussiano: stabilita' e diffusione numerica.
3) Verifica dell'equazione modificata: la varianza della gaussiana cresce
   come 2 mu_e t con mu_e = u h (1 - C)/2 (viscosita' artificiale).
"""
```

prog4_2_upwind.py

```python
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

# ------------------------------------------------------------------ 1) perturbazione discreta
def propaga(schema, C, passi, N=15):
    z = np.zeros(N); z[N // 2] = 1.0            # disturbo unitario in x_m
    storia = [z.copy()]
    for _ in range(passi):
        zn = z.copy()
        if schema == "upwind":
            zn[1:] = (1 - C) * z[1:] + C * z[:-1]
        elif schema == "downwind":
            zn[:-1] = (1 + C) * z[:-1] - C * z[1:]
        elif schema == "centrato":
            zn[1:-1] = z[1:-1] - 0.5 * C * (z[2:] - z[:-2])
        z = zn
        storia.append(z.copy())
    return np.array(storia)


print("Perturbazione unitaria in x_m: valori ai passi successivi (nodi m-2 ... m+3)")
for schema, C in (("upwind", 0.5), ("upwind", 1.0), ("upwind", 1.5), ("centrato", 0.5), ("downwind", 0.5)):
    S = propaga(schema, C, 3)
    print(f"   {schema:9s} C = {C:3.1f}:")
    for k in range(4):
        print(f"      passo {k}: " + "  ".join(f"{v:+7.4f}" for v in S[k, 7 - 2:7 + 4]) + f"   max|zeta| = {np.max(np.abs(S[k])):.4f}")

# ------------------------------------------------------------------ 2) gaussiana
u, L, N = 1.0, 20.0, 400
h = L / N
x = np.arange(N) * h
x0, s0 = 4.0, 0.5
z0 = np.exp(-(x - x0)**2 / (2 * s0**2))


def integra(schema, C, t_fin):
    tau = C * h / u
    z = z0.copy()
    n = int(round(t_fin / tau))
    for _ in range(n):
        zm, zp = np.roll(z, 1), np.roll(z, -1)        # periodico (il dominio e' largo abbastanza)
        if schema == "upwind":
            z = z - C * (z - zm)
        elif schema == "centrato":
            z = z - 0.5 * C * (zp - zm)
        elif schema == "lax-wendroff":
            z = z - 0.5 * C * (zp - zm) + 0.5 * C * C * (zp - 2 * z + zm)
        elif schema == "downwind":
            z = z - C * (zp - z)
    return z, n * tau


def varianza(z):
    m = np.sum(z * x) / np.sum(z)
```

```
    return m, np.sum(z * (x - m)**2) / np.sum(z)


t_fin = 8.0
print(f"\nGaussiana trasportata per t = {t_fin} (u = 1, h = {h}):")
print("   schema          C     max      errore max    posizione    varianza   2 mu_e t + s0^2")
ris = {}
for schema, C in (("upwind", 0.25), ("upwind", 0.5), ("upwind", 0.9), ("upwind", 1.0),
                  ("lax-wendroff", 0.5), ("centrato", 0.5), ("centrato", 0.05), ("downwind", 0.5)):
    z, t = integra(schema, C, t_fin)
    ze = np.exp(-(x - x0 - u * t)**2 / (2 * s0**2))
    ris[(schema, C)] = z
    if np.all(np.isfinite(z)) and np.max(np.abs(z)) < 1e3:
        m, v = varianza(np.maximum(z, 0))
        mue = 0.5 * u * h * (1 - C)
        print(f"   {schema:13s} {C:5.2f}  {z.max():7.4f}   {np.max(np.abs(z - ze)):.3e}    {m:7.3f}     {v:7.4f}     {s0**2 + 2*mue*t:7.4f}")
    else:
        print(f"   {schema:13s} {C:5.2f}  instabile (max|zeta| = {np.nanmax(np.abs(z)):.1e})")

# ------------------------------------------------------------------ 3) verifica quantitativa della viscosita' artificiale
print("\nViscosita' artificiale misurata dalla crescita della varianza (upwind, t = 8):")
for C in (0.1, 0.25, 0.5, 0.75, 0.9, 1.0):
    z, t = integra("upwind", C, t_fin)
    m, v = varianza(z)
    mu_mis = (v - s0**2) / (2 * t)
    print(f"   C = {C:4.2f}: mu_e misurata = {mu_mis:.5f}, prevista u h (1 - C)/2 = {0.5*u*h*(1-C):.5f}")

fig, ax = plt.subplots(1, 3, figsize=(8.6, 2.8))
S = propaga("upwind", 0.5, 4, N=11)
for k, col in zip(range(5), [GRIGIO, VERDE, ARANCIO, ROSSO, BLU]):
    ax[0].plot(np.arange(11) - 5, S[k], "o-", color=col, ms=3, lw=1, label=f"passo {k}")
ax[0].set_xlabel("$i-m$"); ax[0].set_ylabel("$\\zeta/\\epsilon$"); ax[0].legend(fontsize=6.5)
ax[0].set_title("upwind, $C=0{,}5$: perturbazione discreta")
ze = np.exp(-(x - x0 - u * t_fin)**2 / (2 * s0**2))
ax[1].plot(x, ze, "k", lw=1.6, label="esatta")
for (schema, C), col in ((("upwind", 0.5), BLU), (("upwind", 0.9), VERDE), (("lax-wendroff", 0.5), ARANCIO)):
    ax[1].plot(x, ris[(schema, C)], "--", color=col, lw=1.1, label=f"{schema}, $C={C}$".replace(".", "{,}"))
ax[1].set_xlim(8, 16); ax[1].set_xlabel("$x$"); ax[1].legend(fontsize=6.5)
ax[1].set_title("gaussiana dopo $t=8$")
Cs = np.linspace(0.05, 1.0, 20)
mu_m = []
for C in Cs:
    z, t = integra("upwind", C, t_fin)
    mu_m.append((varianza(z)[1] - s0**2) / (2 * t))
ax[2].plot(Cs, mu_m, "o", color=BLU, ms=3.5, label="misurata")
ax[2].plot(Cs, 0.5 * u * h * (1 - Cs), color=ROSSO, lw=1.2, label="$uh(1-C)/2$")
ax[2].set_xlabel("$C$"); ax[2].set_ylabel("$\\mu_e$"); ax[2].legend(fontsize=7)
ax[2].set_title("viscosità artificiale")
```

```
118 fig.tight_layout()
119 salva(fig, "prog4_2_upwind")
```

Listato 2.1: Programma 4.2 – perturbazione discreta, trasporto di una gaussiana e misura della viscosità artificiale (`prog4_2_upwind.py`).

Risultati del Programma 4.2

- **Perturbazione discreta.** Con $C = 0{,}5$ il disturbo upwind si distribuisce con i pesi binomiali $(1/2)^n \binom{n}{k}$ solo verso valle e il massimo decresce (1, 0,5, 0,5, 0,375, …); con $C = 1$ trasla di un nodo per passo senza deformarsi; con $C = 1{,}5$ i pesi hanno segni alterni e il massimo cresce (1,5, 2,25, 3,375: esattamente $(2C-1)^n$). Lo schema centrato propaga il disturbo anche a monte ($-0{,}25$ in $m-1$ dopo un passo) e lo schema downwind, equivalente all'upwind con $C = -0{,}5$, cresce come $(1+2|C|)^n = 2^n$.
- **Gaussiana.** Lo schema upwind trasporta il profilo alla posizione esatta ($x = 12$) ma ne abbassa il picco: a 0,67 con $C = 0{,}25$, 0,75 con $C = 0{,}5$, 0,93 con $C = 0{,}9$, mentre con $C = 1$ l'errore è 6×10^{-15} (lo schema è esatto). Lo schema centrato con $C = 0{,}5$ ha picco 1,88 e varianza 31,8: è esploso in oscillazioni, come previsto, e perfino con $C = 0{,}05$ l'errore è del 6% dopo soli $t = 8$. Il downwind raggiunge 10^{80}. Lax–Wendroff, del secondo ordine, conserva il picco (0,996) e la varianza, con un errore massimo del 3% dovuto a piccole oscillazioni a monte (Fig. 2.2).
- **Viscosità artificiale.** La varianza misurata coincide con $s_0^2 + 2\mu_e t$ in tutte le cifre stampate, e μ_e ricavata dalla varianza coincide con $uh(1-C)/2$ per ogni C (0,0225 per $C = 0{,}1$, 0,0125 per $C = 0{,}5$, 0 per $C = 1$): l'equazione modificata (2.13) non è solo un'analisi qualitativa, è una previsione *quantitativa* del comportamento dello schema.

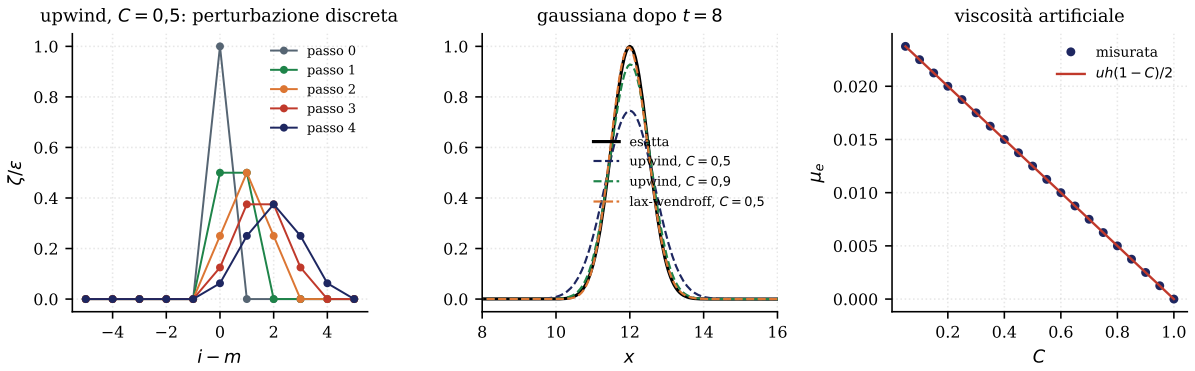


Figura 2.2: Programma 4.2: evoluzione della perturbazione discreta con lo schema upwind, $C = 0{,}5$ (a sinistra); gaussiana dopo $t = 8$ con upwind e Lax–Wendroff (al centro); viscosità artificiale misurata dalla crescita della varianza contro la previsione $uh(1-C)/2$ (a destra).

Il vortice di estremità trasportato nella scia

Un'applicazione aeronautica del trasporto della vorticità è la scia di un'ala: i vortici di estremità viaggiano con la corrente e la loro intensità si conserva (in fluido ideale) per decine di aperture alari, tanto che la separazione minima fra due aerei in avvicinamento è dettata dalla scia del precedente. Una simulazione che discretizzi il trasporto della

vorticità con lo schema upwind e $C = 0{,}5$ su una griglia con $h = 1$ m, con $U = 70$ m/s, introduce $\mu_e = \frac{1}{2} \cdot 70 \cdot 1 \cdot 0{,}5 = 17{,}5$ m^2/s: un milione di volte la viscosità dell'aria. Il vortice numerico si diffonde in pochi secondi, mentre quello reale persiste per minuti: per catturare la scia servono griglie fini e schemi di ordine superiore, o metodi lagrangiani che trasportano la vorticità con le particelle senza diffonderla.

2.5 Approfondimenti

2.5.1 L'equazione modificata in generale

Il procedimento di Hirt è stato sistematizzato da Warming e Hyett [7]. Per uno schema lineare a coefficienti costanti si sostituiscono *tutti* gli sviluppi di Taylor, si eliminano le derivate temporali di ordine superiore al primo usando ripetutamente l'equazione modificata stessa (non l'equazione originale: la differenza è del secondo ordine ma conta per i termini successivi) e si ottiene una serie

$$\zeta_t + u\zeta_x = \sum_{k\geq 2} a_k \, \partial_x^k \zeta.$$

I termini di ordine *pari* producono dissipazione (se $(-1)^{k/2+1} a_k > 0$) o antidissipazione, quelli di ordine *dispari* producono dispersione, cioè velocità di fase dipendente dalla lunghezza d'onda: ne risultano le oscillazioni a monte o a valle dei fronti ripidi che si osservano con gli schemi del secondo ordine. Per lo schema upwind $a_2 = \mu_e$ e $a_3 = -\frac{1}{6} u h^2 (1 - C)(1 - 2C)$: il termine dispersivo cambia segno a $C = 1/2$, dove le oscillazioni sono minime. Se si trattiene tutta la serie, l'equazione modificata riproduce esattamente il fattore di amplificazione di von Neumann; se si trattiene solo il primo termine, come fa Hirt, si ottiene un criterio *euristico* che può fallire: l'Esercizio 3 ne mostra un esempio istruttivo per l'equazione della diffusione, dove il primo termine non nullo è del quarto ordine.

2.5.2 Upwind = centrato + diffusione

Una identità elementare chiarisce la natura dello schema upwind. La differenza all'indietro si scrive come differenza centrata più una correzione:

$$\frac{\zeta_i - \zeta_{i-1}}{h} = \frac{\zeta_{i+1} - \zeta_{i-1}}{2h} - \frac{h}{2} \frac{\zeta_{i+1} - 2\zeta_i + \zeta_{i-1}}{h^2}.$$

Lo schema upwind è quindi *identico* allo schema centrato per l'equazione $\zeta_t + u\zeta_x = \frac{1}{2} u h \, \zeta_{xx}$: la viscosità artificiale $\frac{1}{2} uh$ è aggiunta esplicitamente, e il termine $-\frac{1}{2} u^2 \tau \zeta_{xx}$ che completa la (2.13) viene dall'avanzamento temporale di Eulero (antidiffusivo, come mostra la (2.12)). Questa è la base degli schemi "centrati con viscosità artificiale" usati nei codici per le correnti comprimibili: si parte dallo schema centrato, instabile, e si aggiunge la quantità minima di diffusione che lo stabilizza. Lo schema di **Lax–Wendroff** fa esattamente questo con la quantità $\frac{1}{2} u^2 \tau$, che cancella l'antidiffusione di Eulero:

$$\zeta_{i,j+1} = \zeta_{i,j} - \frac{C}{2}(\zeta_{i+1,j} - \zeta_{i-1,j}) + \frac{C^2}{2}(\zeta_{i+1,j} - 2\zeta_{i,j} + \zeta_{i-1,j}), \tag{2.14}$$

che è del secondo ordine in τ e h (il termine aggiunto è proprio $\frac{1}{2}\tau^2 \zeta_{tt}$ dello sviluppo di Taylor, con la (2.12)), stabile per $C \leq 1$ e privo di diffusione al primo ordine; il suo errore dominante è dispersivo, $a_3 = -\frac{1}{6} u h^2 (1 - C^2)$, da cui le oscillazioni a monte della gaussiana nel Programma 4.2.

2.5.3 Monotonia e il teorema di Godunov

Lo schema upwind con $0 < C \leq 1$ è *monotono*: i coefficienti della (2.3) sono non negativi, quindi $\min_i \zeta_{i,j} \leq \zeta_{i,j+1} \leq \max_i \zeta_{i,j}$ (il nuovo valore è una media pesata) e un profilo monotono resta monotono, senza oscillazioni né nuovi estremi. Godunov (1959) dimostrò che *nessuno schema lineare monotono può essere di ordine superiore al primo*: l'accuratezza del secondo ordine (Lax–Wendroff) e l'assenza di oscillazioni sono incompatibili per gli schemi lineari. Gli schemi moderni per le correnti con fronti ripidi (TVD, limitatori di flusso, ENO/WENO) aggirano il teorema rendendo lo schema *non lineare*: usano Lax–Wendroff dove la soluzione è regolare e passano all'upwind vicino ai fronti, con un "limitatore" che misura localmente la regolarità. Il modulo sulle correnti comprimibili li riprenderà.

2.5.4 Upwind in due dimensioni e falsa diffusione

In due dimensioni lo schema upwind, applicato componente per componente, produce una viscosità artificiale *anisotropa*. Per $C \to 0$ l'equazione modificata di $\zeta_t + u\zeta_x + v\zeta_y = 0$ è $\zeta_t + u\zeta_x + v\zeta_y = \frac{1}{2}|u|h\,\zeta_{xx} + \frac{1}{2}|v|h\,\zeta_{yy}$: un tensore di diffusione diagonale con coefficienti diversi nelle due direzioni della griglia. Per una corrente di modulo V inclinata di ϑ rispetto alla griglia ($u = V\cos\vartheta$, $v = V\sin\vartheta$) il coefficiente di diffusione nella direzione *normale* alla corrente, $\mathbf{n} = (-\sin\vartheta, \cos\vartheta)$, vale

$$\mu_\perp = \tfrac{1}{2}|u|h\sin^2\vartheta + \tfrac{1}{2}|v|h\cos^2\vartheta = \tfrac{1}{4}Vh\,\sin 2\vartheta\,(\sin\vartheta + \cos\vartheta),$$

nullo quando la corrente è allineata con la griglia e massimo ($0{,}35\,Vh$) a 45°: è la *falsa diffusione* di de Vahl Davis e Mallinson (la cui formula, $\mu_\perp = Vh\sin 2\vartheta/[4(\sin^3\vartheta + \cos^3\vartheta)]$, coincide con questa a 45°), che sparpaglia lateralmente le scie e gli strati di taglio obliqui rispetto alla griglia. Lo schema di Torrance usato nel capitolo seguente è uno schema upwind bidimensionale in forma conservativa e ne è affetto; è uno dei motivi per cui, nel problema di Bénard, le celle appaiono più "arrotondate" di quelle sperimentali e per cui la risoluzione 21×9 della fonte va considerata soltanto qualitativa.

Capitolo 3

Convezione di Bénard e instabilità dei fluidi in rotazione

3.1 Dall'analisi lineare al calcolo non lineare

Uno strato di fluido riscaldato dal basso in un campo gravitazionale è potenzialmente instabile: una particella spostata verso l'alto si trova circondata da fluido più freddo e più denso, e la spinta di Archimede la fa salire ancora; spostata verso il basso, continua a scendere. Senza una forza viscosa sufficiente a frenarla, l'instabilità si manifesta come una rete di celle convettive. Rayleigh (1916) ne fece la prima analisi; Chandrasekhar [15] ne diede la trattazione completa nella teoria *linearizzata*, e il modulo precedente ne ha risolto il problema agli autovalori: la teoria lineare prevede *quando* lo stato di quiete diventa instabile (numero di Rayleigh critico) e quale lunghezza d'onda cresce più in fretta, ma nulla dice su che cosa succede dopo, quando i disturbi non sono più piccoli e l'inerzia trascurata diventa importante. Per seguire l'evoluzione delle celle bisogna integrare le equazioni complete nel tempo: lo schema upwind del capitolo precedente, esteso a due dimensioni nella forma conservativa di Torrance, lo rende possibile. Lo stesso metodo si applica poi ai fluidi in rotazione, dove la forza centrifuga gioca il ruolo della gravità e l'instabilità di Taylor quello di Bénard.

3.2 Le equazioni di Boussinesq

Le equazioni sono la continuità, Navier–Stokes con la forza di galleggiamento, l'energia e un'equazione di stato linearizzata:

$$\rho = \rho_0 \left[1 - \alpha (T - T_0) \right], \tag{3.1}$$

con α coefficiente di dilatazione volumica (10^{-3}–10^{-4} K^{-1} per gas e liquidi ordinari) e ρ_0 la densità alla temperatura T_0.

Approssimazione di Boussinesq

Se le variazioni di temperatura non sono troppo grandi ($\alpha\,\Delta T \ll 1$), la densità può essere considerata costante *ovunque tranne nel termine di galleggiamento*, dove la piccola differenza $\rho - \rho_0$ moltiplica la grande accelerazione g e produce una forza confrontabile con le altre.

Con proprietà fisiche costanti:

$$\nabla \cdot \mathbf{V} = 0, \tag{3.2}$$

$$\rho_0 \frac{\mathrm{D}\mathbf{V}}{\mathrm{D}t} = -\mathbf{j}(\rho - \rho_0)g - \nabla p + \mu \nabla^2 \mathbf{V}, \tag{3.3}$$

$$\frac{\mathrm{D}T}{\mathrm{D}t} = \kappa \nabla^2 T, \tag{3.4}$$

dove $\mathbf{j}$ è il versore verticale (opposto a $\mathbf{g}$), p è la pressione al netto di quella idrostatica $-\rho_0 g y$ e $\kappa = k/(\rho_0 c_v)$ è la diffusività termica. Nell'equazione dell'energia è scomparsa la funzione di dissipazione: Chandrasekhar mostra che il riscaldamento viscoso è di ordine $\alpha \Delta T$ rispetto agli altri termini. L'approssimazione filtra inoltre le onde sonore (il fluido è incomprimibile) e non vale per strati di spessore confrontabile con l'altezza di scala dell'atmosfera, dove la stratificazione di base non è trascurabile.

Si considera un moto piano nel piano x–y fra due piastre $y = 0$ e $y = H$; il fluido, inizialmente a temperatura uniforme T_0, è riscaldato dal basso portando improvvisamente la piastra inferiore a T_1. La vorticità ha la sola componente z, di modulo ζ, e la continuità è soddisfatta da una funzione di corrente. Con H come lunghezza di riferimento, $\nu/H = \mu/(\rho_0 H)$ come velocità (la velocità "viscosa") e $T_1 - T_0$ come differenza di temperatura, le variabili adimensionali sono

$$X = \frac{x}{H}, \qquad Y = \frac{y}{H}, \qquad T = \frac{t}{\rho_0 H^2/\mu}, \qquad U = \frac{u}{\mu/\rho_0 H}, \qquad V = \frac{v}{\mu/\rho_0 H},$$
$$\Psi = \frac{\psi}{\mu/\rho_0}, \qquad \Omega = \frac{\zeta}{\mu/\rho_0 H^2}, \qquad \theta = \frac{T - T_0}{T_1 - T_0}. \tag{3.5}$$

D'ora in poi T indica il tempo adimensionale e θ la temperatura adimensionale.

Derivazione delle equazioni in vorticità–funzione di corrente

Si sostituisce la (3.1) nella (3.3): il termine di galleggiamento diventa $+\mathbf{j}\rho_0 \alpha g (T - T_0)$. Si prende il rotore: il gradiente di pressione scompare, $\nabla \times (\mathbf{j}\, \alpha g \rho_0 (T - T_0)) = -\mathbf{k}\, \alpha g \rho_0\, \partial T/\partial x$ con il verso dato dalla regola del prodotto vettoriale ($\nabla \times (f\mathbf{j}) = \nabla f \times \mathbf{j} = -f_x \mathbf{k}$), e per un moto piano $\nabla \times (\mathbf{V} \cdot \nabla \mathbf{V}) = \mathbf{V} \cdot \nabla \zeta$ (il termine di stiramento $\boldsymbol{\zeta} \cdot \nabla \mathbf{V}$ è nullo). Dividendo per ρ_0 e passando alle variabili (3.5), con la vorticità definita $\zeta = \partial v/\partial x - \partial u/\partial y$ cosicché $\nabla^2 \psi = -\zeta$:

$$U = \frac{\partial \Psi}{\partial Y}, \qquad V = -\frac{\partial \Psi}{\partial X}, \tag{3.6}$$

$$\Omega = -\left(\frac{\partial^2 \Psi}{\partial X^2} + \frac{\partial^2 \Psi}{\partial Y^2} \right), \tag{3.7}$$

$$\frac{\partial \Omega}{\partial T} + \frac{\partial (U\Omega)}{\partial X} + \frac{\partial (V\Omega)}{\partial Y} = Gr \frac{\partial \theta}{\partial X} + \frac{\partial^2 \Omega}{\partial X^2} + \frac{\partial^2 \Omega}{\partial Y^2}, \tag{3.8}$$

$$\frac{\partial \theta}{\partial T} + \frac{\partial (U\theta)}{\partial X} + \frac{\partial (V\theta)}{\partial Y} = \frac{1}{Pr} \left(\frac{\partial^2 \theta}{\partial X^2} + \frac{\partial^2 \theta}{\partial Y^2} \right), \tag{3.9}$$

in cui $Gr = \alpha g H^3 (T_1 - T_0)/\nu^2$ è il **numero di Grashof** (rapporto fra forze di galleggiamento e viscose) e $Pr = \nu/\kappa$ il numero di Prandtl; il loro prodotto è il numero di Rayleigh $Ra = GrPr$ usato nella teoria lineare. I termini convettivi sono scritti in *forma conservativa* $\partial(U\Omega)/\partial X + \partial(V\Omega)/\partial Y$, identica a $U\,\partial\Omega/\partial X + V\,\partial\Omega/\partial Y$ grazie alla continuità, così che le due equazioni di trasporto hanno la stessa struttura e possono essere integrate con

la stessa funzione. Il segno del termine di galleggiamento, $+Gr\,\partial\theta/\partial X$, è coerente con la definizione $\zeta = v_x - u_y$: una regione calda a destra ($\theta_X > 0$) produce vorticità positiva (antioraria), cioè fluido che sale a destra e scende a sinistra, come deve.

3.3 Il problema del Programma 4.3

Si considera acqua ($Pr = 6{,}75$) in una scatola rettangolare (Fig. 3.1). Per la simmetria rispetto all'asse Y si calcola soltanto la metà sinistra, coperta da m linee verticali e n orizzontali con maglie quadrate $h = 1/(n-1)$; nel programma $m = 21$ e $n = 9$, cioè una scatola lunga 5 volte l'altezza. All'istante $T = 0$ si ha $\theta = 0$ ovunque tranne $\theta = 1$ sulla piastra inferiore; le pareti solide restano alla loro temperatura iniziale (fondo caldo, coperchio e parete laterale freddi), sull'asse $\partial\theta/\partial X = 0$ per simmetria.

Segnaposto figura

file atteso: `immagini/fig_4_3_1.png`

Metà sinistra della scatola: griglia $i = 1, \dots, m$, $j = 1, \dots, n$ con passo h; $\theta = 1$, $\Psi = 0$ sul fondo ($j = 1$); $\theta = 0$, $\Psi = 0$ sul coperchio ($j = n$) e sulla parete laterale ($i = 1$); $\partial\theta/\partial X = 0$, $\Psi = 0$ sull'asse di simmetria ($i = m$); gravità verso il basso (Fig. 4.3.1 della fonte).

Figura 3.1: Schema del problema di Bénard non lineare.

3.3.1 Discretizzazione

Nei nodi interni le derivate spaziali, tranne i termini non lineari, sono approssimate con differenze centrate:

$$U_{i,j} = \frac{\Psi_{i,j+1} - \Psi_{i,j-1}}{2h}, \qquad V_{i,j} = -\frac{\Psi_{i+1,j} - \Psi_{i-1,j}}{2h}. \tag{3.10}$$

L'equazione di Poisson (3.7) è risolta con il sovrarilassamento (SOR) del modulo precedente, nella funzione `sorlx`, con $\Psi = 0$ sulle tre pareti solide (nessuna portata le attraversa) e sull'asse (perché il moto a destra sia l'immagine speculare di quello a sinistra); la tolleranza ERRMAX sulla somma delle variazioni è 10^{-4}. La simmetria di θ e l'antisimmetria di Ψ rispetto all'asse $i = m$ danno, per ogni j,

$$\begin{gathered} \theta_{m+1,j} = \theta_{m-1,j}, \qquad \Psi_{m,j} = 0, \qquad \Psi_{m+1,j} = -\Psi_{m-1,j}, \\ U_{m+1,j} = -U_{m-1,j}, \qquad U_{m,j} = 0, \qquad V_{m,j} = \frac{\Psi_{m-1,j}}{h}, \qquad \Omega_{m,j} = 0, \end{gathered} \tag{3.11}$$

le ultime tre dedotte dalle (3.10) e dalla (3.7).

Entrambe le equazioni di trasporto sono casi dell'**equazione generale**

$$\frac{\partial P}{\partial T} = -\frac{\partial (UP)}{\partial X} - \frac{\partial (VP)}{\partial Y} + A\frac{\partial Q}{\partial X} + B\left(\frac{\partial^2 P}{\partial X^2} + \frac{\partial^2 P}{\partial Y^2}\right), \tag{3.12}$$

con $P = \Omega$, $Q = \theta$, $A = Gr$, $B = 1$ per la vorticità e $P = \theta$, $A = 0$, $B = 1/Pr$ per la temperatura. Per i termini convettivi si adotta lo schema upwind nella forma di Torrance [17], sviluppata per la convezione naturale e per i moti rotanti. Si definiscono le velocità medie a mezzo passo in avanti e all'indietro,

$$U_f = \tfrac{1}{2}(U_{i+1,j} + U_{i,j}), \quad U_b = \tfrac{1}{2}(U_{i,j} + U_{i-1,j}), \quad V_f = \tfrac{1}{2}(V_{i,j+1} + V_{i,j}), \quad V_b = \tfrac{1}{2}(V_{i,j} + V_{i,j-1}), \tag{3.13}$$

e le combinazioni

$$P_1 = (U_f - |U_f|)P_{i+1,j} + (U_f + |U_f| - U_b + |U_b|)P_{i,j} - (U_b + |U_b|)P_{i-1,j}, \tag{3.14}$$

$$P_2 = (V_f - |V_f|)P_{i,j+1} + (V_f + |V_f| - V_b + |V_b|)P_{i,j} - (V_b + |V_b|)P_{i,j-1}, \tag{3.15}$$

$$P_3 = Q_{i+1,j} - Q_{i-1,j}, \tag{3.16}$$

$$P_4 = P_{i+1,j} + P_{i-1,j} + P_{i,j+1} + P_{i,j-1} - 4P_{i,j}. \tag{3.17}$$

Con la differenza in avanti nel tempo, $\partial P/\partial T \approx (P'_{i,j} - P_{i,j})/\Delta T$ (l'apice indica il valore a $T + \Delta T$), la (3.12) diventa

$$P'_{i,j} = P_{i,j} + \frac{\Delta T}{2h}\left(-P_1 - P_2 + A\,P_3 + \frac{2B}{h}P_4\right). \tag{3.18}$$

Perché la (3.14) è uno schema upwind conservativo

Si scriva il flusso convettivo attraverso la faccia a mezzo passo in avanti come $F_f = U_f\,P_f$, dove P_f è il valore di P "trasportato" attraverso la faccia. Lo schema upwind prende $P_f = P_{i,j}$ se $U_f > 0$ (il fluido esce dalla cella verso destra portando con sé il valore della cella) e $P_f = P_{i+1,j}$ se $U_f < 0$. Con la funzione valore assoluto le due scelte si scrivono in una formula sola:

$$F_f = \tfrac{1}{2}(U_f + |U_f|)P_{i,j} + \tfrac{1}{2}(U_f - |U_f|)P_{i+1,j},$$

e analogamente $F_b = \tfrac{1}{2}(U_b + |U_b|)P_{i-1,j} + \tfrac{1}{2}(U_b - |U_b|)P_{i,j}$ sulla faccia a monte. La divergenza discreta del flusso, $(F_f - F_b)/h$, vale esattamente $P_1/(2h)$: la (3.14) è la forma conservativa dello schema upwind (ciò che esce da una cella entra nella vicina), e il coefficiente di $P_{i,j}$, $U_f + |U_f| - U_b + |U_b|$, è la somma dei flussi uscenti. Lo schema è del primo ordine e porta con sé la viscosità artificiale del capitolo precedente, $\tfrac{1}{2}|U|h$ per direzione; l'Esercizio 5 ne ricava il criterio di stabilità

$$\Delta T \le \left[\frac{1}{2h}\left(U_f + |U_f| - U_b + |U_b| + V_f + |V_f| - V_b + |V_b|\right) + \frac{4B}{h^2}\right]^{-1}, \tag{3.19}$$

che richiede, in ogni nodo e a ogni istante, che il coefficiente del nodo centrale nella (3.18) sia non negativo (analisi quasi-lineare di Lax e Richtmyer). ΔT dipende dal campo di velocità, quindi varia durante il moto; nel Programma 4.3 si usa il valore fisso 0,0025, scelto per tentativi, e il programma verifica a posteriori che resti sotto il limite (3.19).

3.3.2 Vorticità alle pareti

Sulla parete verticale $i = 1$ si ha $U = 0$ e $\partial U/\partial Y = 0$, quindi $\Omega = \partial V/\partial X$; con una differenza in avanti a tre punti, del secondo ordine, e $V_{1,j} = 0$,

$$\Omega_{1,j} = \frac{4V_{2,j} - V_{3,j}}{2h}. \tag{3.20}$$

Sulle piastre, dove $V = 0$, $\partial V/\partial X = 0$ e $U = 0$,

$$\Omega_{i,1} = \frac{-4U_{i,2} + U_{i,3}}{2h}, \qquad \Omega_{i,n} = \frac{4U_{i,n-1} - U_{i,n-2}}{2h}. \tag{3.21}$$

Le formule a tre punti

Dallo sviluppo di Taylor $f(h) = f_0 + hf_0' + \frac{1}{2}h^2 f_0'' + \mathcal{O}(h^3)$ e $f(2h) = f_0 + 2hf_0' + 2h^2 f_0'' + \mathcal{O}(h^3)$, la combinazione $4f(h) - f(2h) = 3f_0 + 2hf_0' + \mathcal{O}(h^3)$ elimina la derivata seconda: $f_0' = [-3f_0 + 4f(h) - f(2h)]/(2h) + \mathcal{O}(h^2)$. Con $f_0 = 0$ si ottengono le (3.20)–(3.21). Rispetto alla formula di Thom del modulo precedente ($\Omega_w = -2\Psi_1/h^2$, del primo ordine) queste usano le velocità, già calcolate, e sono del secondo ordine; nel Cap. 6 si tornerà alla formula di Thom, che è più robusta.

Procedura del Programma 4.3

A ogni passo temporale:

1. note Ω e θ (dalle condizioni iniziali o dal passo precedente), si risolve la (3.7) con `sorlx` e si calcolano U, V dalle (3.10) e dalle condizioni di simmetria (3.11);
2. con la funzione `pnew` si integrano la (3.8) e la (3.9) nei nodi interni esclusi quelli dell'asse (dove $\Omega = 0$), memorizzando i nuovi valori in `omnew` e `thnew`; la temperatura sull'asse è integrata a parte con la condizione di simmetria $\theta_{m+1,j} = \theta_{m-1,j}$ (nodo specchio);
3. si aggiornano le vorticità di parete con le (3.20)–(3.21);
4. si assegnano i nuovi campi a `omega` e `theta`; agli istanti prescelti si salvano i campi e si stampa il numero di Nusselt medio sul fondo, $Nu = -\overline{\partial\theta/\partial Y}|_{Y=0}$ (rapporto fra il flusso termico effettivo e quello conduttivo puro).

Si ripete fino a NSTEP = MAXSTP.

Dati del Programma 4.3

$Pr = 6{,}75$ (acqua), $Gr = 1000$ ($Ra = 6750$, quattro volte il valore critico 1708 della teoria lineare per pareti rigide); $m = 21$, $n = 9$, $h = 1/8$; $\Delta T = 0{,}0025$; MAXSTP = 2000 (fino a $T = 5$); ERRMAX $= 10^{-4}$. Campi salvati a $T = 0{,}025$, 0,05, 0,125, 0,2, 0,25, 0,5, 0,75, 1, 2, 5.

```
"""
prog4_3_benard.py -- Programma 4.3: convezione di Benard non lineare in una
scatola rettangolare riscaldata dal basso (formulazione vorticita'-funzione di
corrente, schema upwind di Torrance, SOR per la funzione di corrente).

Variabili adimensionali (lunghezza H, velocita' nu/H, tempo H^2/nu):
    U = dPsi/dY,  V = -dPsi/dX,  Omega = -(Psi_XX + Psi_YY),
    dOmega/dT + d(U Omega)/dX + d(V Omega)/dY = Gr dtheta/dX + lap Omega,
    dtheta/dT + d(U theta)/dX + d(V theta)/dY = (1/Pr) lap theta.
Si calcola la meta' sinistra della scatola (simmetria rispetto all'asse Y):
m = 21 linee verticali (i = m sull'asse), n = 9 orizzontali, h = 1/(n-1);
theta = 1 sul fondo, 0 sul coperchio e sulla parete laterale, dtheta/dX = 0
sull'asse; Psi = 0 su tutto il contorno; vorticita' di parete dalle formule
unilatere del secondo ordine. Acqua: Pr = 6,75; Gr = 1000; dT = 0,0025.
"""
```

prog4_3_benard.py

```
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

M, N = 21, 9
H = 1.0 / (N - 1)
PR, GR = 6.75, 1000.0
DT = 0.0025
MAXSTP = 2000
ERRMAX = 1e-4
istanti = [0.025, 0.05, 0.125, 0.2, 0.25, 0.5, 0.75, 1.0, 2.0, 5.0]


def sorlx(psi, omega, h, errmax, omega_rf=None):
    """SOR per lap Psi = -Omega nei nodi interni (i = 2..M-1, j = 2..N-1), Psi = 0 sul contorno
    e sull'asse i = M. Restituisce Psi e il numero di iterazioni."""
    m, n = psi.shape
    if omega_rf is None:
        a = np.cos(np.pi / (m - 1)) + np.cos(np.pi / (n - 1))
        omega_rf = (8 - 4 * np.sqrt(4 - a * a)) / (a * a)
    for it in range(1, 10001):
        err = 0.0
        for i in range(1, m - 1):
            for j in range(1, n - 1):
                nuovo = (1 - omega_rf) * psi[i, j] + omega_rf / 4 * (
                    psi[i - 1, j] + psi[i + 1, j] + psi[i, j - 1] + psi[i, j + 1] + h * h * omega[i, j])
                err += abs(nuovo - psi[i, j])
                psi[i, j] = nuovo
        if err < errmax:
            return psi, it
    raise RuntimeError("SORLX: convergenza non raggiunta")


def pnew(P, Q, U, V, A, B, dt, h):
    """Un passo esplicito dell'equazione generale (4.3.20)
        dP/dT = -d(UP)/dX - d(VP)/dY + A dQ/dX + B lap P
    con lo schema upwind di Torrance per i termini convettivi, nei nodi interni
    (i = 1..m-2, j = 1..n-2 in indici Python). Restituisce il nuovo P nei nodi interni."""
    Uf = 0.5 * (U[2:, 1:-1] + U[1:-1, 1:-1]);  Ub = 0.5 * (U[1:-1, 1:-1] + U[:-2, 1:-1])
    Vf = 0.5 * (V[1:-1, 2:] + V[1:-1, 1:-1]);  Vb = 0.5 * (V[1:-1, 1:-1] + V[1:-1, :-2])
    Pc = P[1:-1, 1:-1]
    P1 = (Uf - abs(Uf)) * P[2:, 1:-1] + (Uf + abs(Uf) - Ub + abs(Ub)) * Pc - (Ub + abs(Ub)) * P[:-2, 1:-1]
    P2 = (Vf - abs(Vf)) * P[1:-1, 2:] + (Vf + abs(Vf) - Vb + abs(Vb)) * Pc - (Vb + abs(Vb)) * P[1:-1, :-2]
    P3 = Q[2:, 1:-1] - Q[:-2, 1:-1]
    P4 = P[2:, 1:-1] + P[:-2, 1:-1] + P[1:-1, 2:] + P[1:-1, :-2] - 4 * Pc
    return Pc + dt / (2 * h) * (-P1 - P2 + A * P3 + 2 * B * P4 / h)


def dt_massimo(U, V, B, h):
    """Limite di stabilita' (4.3.33): coefficiente del nodo centrale non negativo."""
    Uf = 0.5 * (U[2:, 1:-1] + U[1:-1, 1:-1]);  Ub = 0.5 * (U[1:-1, 1:-1] + U[:-2, 1:-1])
```

```
    Vf = 0.5 * (V[1:-1, 2:] + V[1:-1, 1:-1]);  Vb = 0.5 * (V[1:-1, 1:-1] + V[:-2, 1:-1])
    s = (Uf + abs(Uf) - Ub + abs(Ub) + Vf + abs(Vf) - Vb + abs(Vb)) / (2 * h) + 4 * B / h**2
    return 1.0 / np.max(s)


def simula(maxstp=MAXSTP, dt=DT, istanti=istanti, stampa=True):
    psi = np.zeros((M, N)); omega = np.zeros((M, N)); theta = np.zeros((M, N))
    U = np.zeros((M, N)); V = np.zeros((M, N))
    theta[:, 0] = 1.0                                   # fondo caldo
    theta[0, 0] = theta[M - 1, 0] = 1.0
    theta[0, :] = 0.0; theta[0, 0] = 1.0                  # parete laterale fredda (spigolo caldo)
    salvati, storia = {}, []
    dtmin = np.inf
    for nstep in range(1, maxstp + 1):
        # funzione di corrente e velocita'
        psi, it = sorlx(psi, omega, H, ERRMAX)
        U[:, 1:-1] = (psi[:, 2:] - psi[:, :-2]) / (2 * H)
        V[1:-1, :] = -(psi[2:, :] - psi[:-2, :]) / (2 * H)
        V[M - 1, :] = psi[M - 2, :] / H                   # asse: Psi_(m+1) = -Psi_(m-1)
        # vorticita' e temperatura nei nodi interni (esclusa la colonna dell'asse)
        omnew = omega.copy(); thnew = theta.copy()
        omnew[1:-1, 1:-1] = pnew(omega, theta, U, V, GR, 1.0, dt, H)
        thnew[1:-1, 1:-1] = pnew(theta, theta, U, V, 0.0, 1.0 / PR, dt, H)
        # temperatura sull'asse (i = M): nodo specchio i = M+1 -> M-1
        Ue = np.vstack([U[M - 2:M, :], -U[M - 2, :][None, :]])
        Ve = np.vstack([V[M - 2:M, :], V[M - 2, :][None, :]])
        The = np.vstack([theta[M - 2:M, :], theta[M - 2, :][None, :]])
        thnew[M - 1, 1:-1] = pnew(The, The, Ue, Ve, 0.0, 1.0 / PR, dt, H)[0]
        omnew[M - 1, :] = 0.0
        # vorticita' di parete
        omnew[0, 1:-1] = (4 * V[1, 1:-1] - V[2, 1:-1]) / (2 * H)
        omnew[1:-1, 0] = (-4 * U[1:-1, 1] + U[1:-1, 2]) / (2 * H)
        omnew[1:-1, N - 1] = (4 * U[1:-1, N - 2] - U[1:-1, N - 3]) / (2 * H)
        omnew[0, 0] = omnew[0, N - 1] = 0.0
        dtmin = min(dtmin, dt_massimo(U, V, 1.0 / PR, H), dt_massimo(U, V, 1.0, H))
        omega, theta = omnew, thnew
        T = nstep * dt
        storia.append((T, np.max(np.abs(psi)), it))
        for Ts in istanti:
            if abs(T - Ts) < 1e-9:
                salvati[Ts] = (psi.copy(), theta.copy(), omega.copy())
                if stampa:
                    # numero di Nusselt medio sul fondo: -dtheta/dY mediato su X (differenza in avanti del 2o ordine)
                    dth = (-3 * theta[:, 0] + 4 * theta[:, 1] - theta[:, 2]) / (2 * H)
                    Nu = -np.mean(dth[1:])
                    print(f"   T = {T:5.3f} (passo {nstep:4d}): max|Psi| = {np.max(np.abs(psi)):.4f},"
                          f" min Psi = {psi.min():+.4f}, max Psi = {psi.max():+.4f}, iter. SOR = {it:3d}, Nu = {Nu:.3f}")
    return salvati, storia, dtmin


if __name__ == "__main__":
    print(f"Benard non lineare: Pr = {PR}, Gr = {GR} (Ra = {PR*GR:.0f}), griglia
```

```
    {M}x{N}, h = {H}, dT = {DT}")
119 salvati, storia, dtmin = simula()
120 print(f"   passo temporale massimo consentito dalla (4.3.33) lungo tutta la
    simulazione: {dtmin:.4f} (usato {DT})")
121
122 # ------------------------------------------------------------ figure: storia
    temporale delle linee di corrente
123 X = np.arange(M) * H
124 Y = np.arange(N) * H
125 Xf = np.concatenate([X, 2 * X[-1] - X[-2::-1]])     # scatola intera per
    simmetria
126 fig, axs = plt.subplots(5, 2, figsize=(8.4, 7.2))
127 for a_, Ts in zip(axs.ravel(), istanti):
128     psi, theta, omega = salvati[Ts]
129     psif = np.vstack([psi, -psi[-2::-1, :]])         # Psi antisimmetrica
130     lv = np.linspace(-np.max(np.abs(psif)), np.max(np.abs(psif)), 13)
131     lv = lv[np.abs(lv) > 1e-6]
132     a_.contour(Xf, Y, psif.T, levels=lv, colors=[ROSSO if v < 0 else BLU for v
    in lv],
133                linestyles=["--" if v < 0 else "-" for v in lv], linewidths=0.8)
134     a_.set_aspect("equal")
135     a_.set_xticks([]); a_.set_yticks([])
136     a_.set_title(f"$T={Ts:g}$".replace(".", "{,}") + f",
    $|\\Psi|_{{\\max}}={np.max(np.abs(psi)):.2f}$".replace(".", "{,}"), fontsize=8)
137 fig.tight_layout()
138 salva(fig, "prog4_3_benard_storia")
139
140 # ------------------------------------------------------------ figure: isoterme e
    storia di |Psi|max
141 fig, ax = plt.subplots(1, 2, figsize=(8.0, 2.8), gridspec_kw={"width_ratios":
    [2.2, 1]})
142 psi, theta, omega = salvati[1.0]
143 thf = np.vstack([theta, theta[-2::-1, :]])
144 cs = ax[0].contour(Xf, Y, thf.T, levels=np.linspace(0.1, 0.9, 9),
    cmap="coolwarm", linewidths=0.9)
145 ax[0].clabel(cs, fmt=lambda v: f"{v:.1f}".replace(".", ","), fontsize=6)
146 ax[0].set_aspect("equal"); ax[0].set_title("isoterme a $T=1$")
147 ax[0].set_xlabel("$X$"); ax[0].set_ylabel("$Y$")
148 Ts_ = [s[0] for s in storia]
149 ax[1].plot(Ts_, [s[1] for s in storia], color=BLU)
150 ax[1].set_xlabel("$T$"); ax[1].set_ylabel("$|\\Psi|_{\\max}$")
151 ax[1].set_title("intensità del moto".replace("à", "a'"))
152 fig.tight_layout()
153 salva(fig, "prog4_3_benard_isoterme")
```

Listato 3.1: Programma 4.3 – convezione di Bénard non lineare in una scatola rettangolare con lo schema upwind di Torrance (`prog4_3_benard.py`).

Risultati del Programma 4.3

La storia delle linee di corrente (Fig. 3.2, scatola intera ricostruita per simmetria) riproduce quella descritta dalla fonte.

- A $T = 0{,}025$ un moto debole nasce nello spigolo inferiore dove la parete fredda incontra il fondo caldo ($|\Psi|_{\max} = 0{,}014$, tutta positiva, cioè antioraria nella metà sinistra: il fluido scende lungo la parete fredda e risale dopo aver lambito il fondo caldo); altrove il fluido è praticamente fermo. A $T = 0{,}05$ il moto si è rafforzato e due bolle di moto orario si formano sulle piastre a metà scatola.

- A $T = 0{,}125$ le bolle si sono unite in una cella negativa che si espande dal centro verso la parete: due celle per metà scatola. A $T = 0{,}2$ compare una terza bolla, antioraria, al centro del fondo, che a $T = 0{,}25$–$0{,}75$ si espande mentre tutte le celle si intensificano.
- A $T = 1$ (passo 400) le tre celle hanno dimensioni quasi uguali e $|\Psi|_{\max} = 1{,}14$; la cella di destra è la più intensa. In seguito la cella di sinistra si rafforza e le altre si indeboliscono: a $T = 5$ è la più intensa (1,12 contro 0,86) e il confine fra la prima e la seconda cella si è spostato a destra, più in basso che in alto, risultando curvo. Lo stato stazionario non è ancora raggiunto a $T = 5$ ma i cambiamenti sono ormai piccoli.
- Nella scatola intera il numero di celle è due all'inizio, poi quattro, infine sei; la *storia* del moto e le disuguaglianze fra le celle sono fenomeni non lineari che nessuna teoria linearizzata potrebbe prevedere.
- Il numero di Nusselt (Fig. 3.3) parte altissimo (conduzione nello strato sottilissimo iniziale: $Nu \sim 1/\sqrt{\pi T/Pr}$), scende a 2,74 a $T = 0{,}5$, risale a 3,66 quando le celle sono formate e si assesta intorno a 3,1–3,2: per $Ra = 6750$ ($4Ra_c$) le misure sullo strato indefinito danno $Nu \approx 2$ (circa 2,2 a $Ra = 10^4$); il valore più alto ottenuto qui riflette la griglia grossolana, la scatola finita con pareti fredde e il transitorio ancora in corso. Le isoterme hanno la forma "a ombrello aperto" descritta dalla fonte. Il passo massimo consentito dalla (3.19) lungo la simulazione è 0,0034: $\Delta T = 0{,}0025$ è sotto il limite con margine del 30%.

Che cosa il calcolo non può dire

La simulazione è bidimensionale, ma le celle di Bénard osservate sono esagonali o a rulli secondo le condizioni: il moto reale è tridimensionale, e il numero di celle "finale" dipende anche dalla lunghezza della scatola, dalla risoluzione e dalla viscosità artificiale dello schema upwind. Il risultato va letto come un esperimento numerico qualitativo che mostra la *dinamica* dell'instabilità, non come una previsione quantitativa. Con il programma scritto, però, si cambiano in un attimo le dimensioni della scatola, il fluido (Pr) e il riscaldamento (Gr): è il grande vantaggio del metodo numerico sulla teoria lineare, che prevede soltanto la soglia.

3.4 Fluidi in rotazione: le equazioni assialsimmetriche

Una corrente può diventare instabile anche percorrendo una traiettoria curva. Il fluido fra due cilindri coassiali rotanti a velocità diverse, per certe combinazioni, non riesce a mantenere il moto puramente circonferenziale e si organizza in una serie di vortici anulari "a ciambella": l'**instabilità di Taylor** (1923), analizzata da Taylor con una teoria linearizzata e confermata dai suoi celebri esperimenti. Le equazioni, e quindi il metodo numerico, sono molto simili a quelle del problema di Bénard.

Siano r_i, ω_i raggio e velocità angolare del cilindro interno, r_0, ω_0 quelli dell'esterno. Con $1/\omega_i$ come tempo, r_i come lunghezza e $r_i\omega_i$ come velocità di riferimento, e con Ω la componente azimutale (adimensionale) della vorticità,

$$R = \frac{r}{r_i}, \quad Z = \frac{z}{r_i}, \quad U = \frac{u_z}{r_i\omega_i}, \quad V = \frac{u_r}{r_i\omega_i}, \quad T = \frac{t}{\omega_i^{-1}}, \quad \Psi = \frac{\psi}{r_i^2(r_i\omega_i)}, \quad \Omega = \frac{-\zeta_\theta}{\omega_i}, \tag{3.22}$$

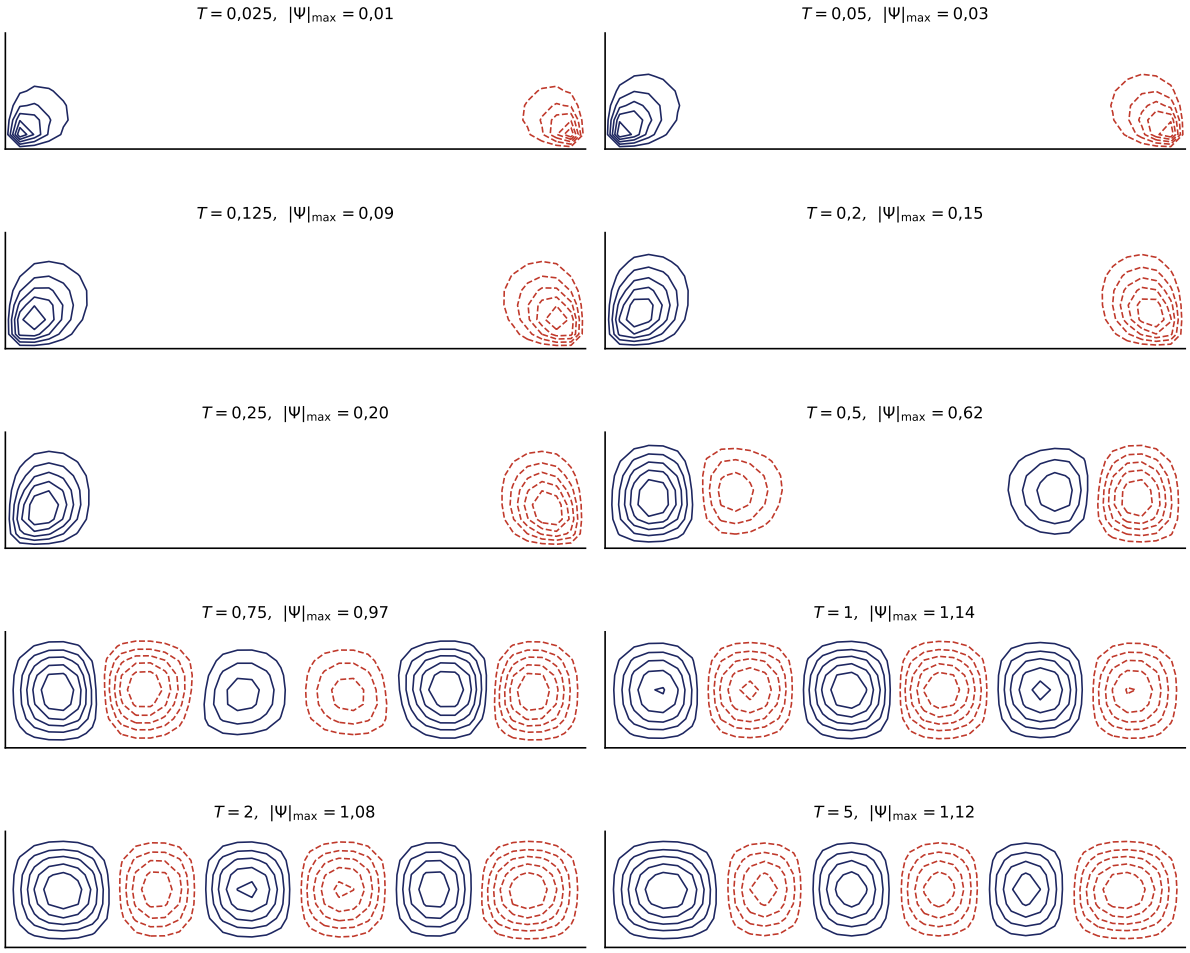


Figura 3.2: Programma 4.3: linee di corrente nella scatola intera ai dieci istanti salvati (continue: $\Psi > 0$, moto antiorario; tratteggiate: $\Psi < 0$). Il numero di celle passa da due a quattro a sei.

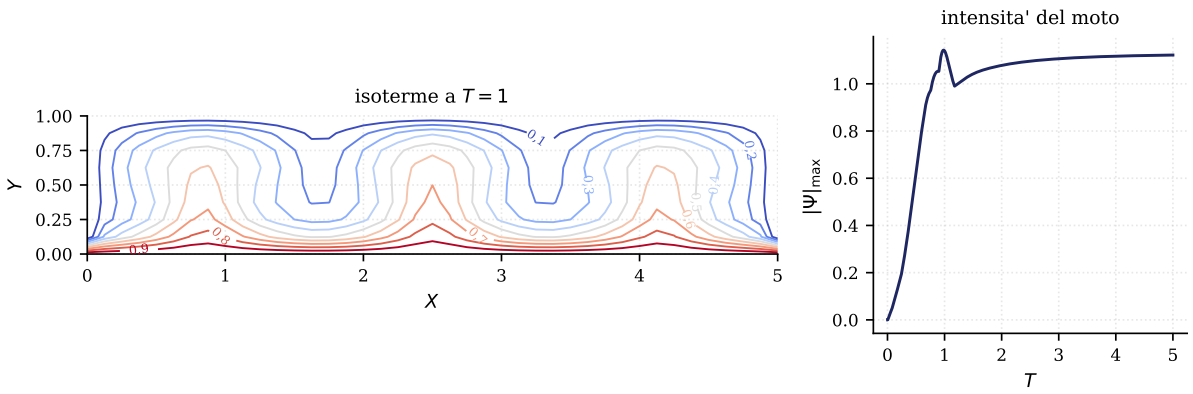


Figura 3.3: Programma 4.3: isoterme a $T = 1$ (a sinistra) e intensità del moto $|\Psi|_{\max}$ in funzione del tempo (a destra).

e, per la presenza della velocità angolare u_θ, la nuova variabile

$$W = \frac{r u_\theta}{r_i^2 \omega_i}, \tag{3.23}$$

il *momento angolare* adimensionale per unità di massa. Per un moto assialsimmetrico le equazioni sono

$$U = \frac{1}{R}\frac{\partial \Psi}{\partial R}, \qquad V = -\frac{1}{R}\frac{\partial \Psi}{\partial Z}, \tag{3.24}$$

$$\frac{\partial^2 \Psi}{\partial R^2} - \frac{1}{R}\frac{\partial \Psi}{\partial R} + \frac{\partial^2 \Psi}{\partial Z^2} = R\,\Omega, \tag{3.25}$$

$$\frac{\partial \Omega}{\partial T} = -\frac{\partial (U\Omega)}{\partial Z} - \frac{\partial (V\Omega)}{\partial R} - \frac{2W}{R^3}\frac{\partial W}{\partial Z} + \frac{1}{Re}\left(\frac{\partial^2 \Omega}{\partial R^2} + \frac{1}{R}\frac{\partial \Omega}{\partial R} - \frac{\Omega}{R^2} + \frac{\partial^2 \Omega}{\partial Z^2}\right), \tag{3.26}$$

$$\frac{\partial W}{\partial T} = -\frac{\partial (UW)}{\partial Z} - \frac{\partial (VW)}{\partial R} - \frac{VW}{R} + \frac{1}{Re}\left(\frac{\partial^2 W}{\partial R^2} - \frac{1}{R}\frac{\partial W}{\partial R} + \frac{\partial^2 W}{\partial Z^2}\right), \tag{3.27}$$

con $Re = r_i^2 \omega_i / \nu$. La (3.27) è la componente θ di Navier–Stokes; i termini non lineari delle due equazioni sono scritti di proposito nella stessa forma della (3.12), così che lo schema di Torrance si applica tale e quale (con le variabili Z, R al posto di X, Y).

Da dove vengono i termini di accoppiamento

Il termine $-(2W/R^3)\partial W/\partial Z$ nella (3.26) è il rotore (componente θ) dell'accelerazione centripeta $-u_\theta^2/r$: in variabili adimensionali $u_\theta^2/r = W^2/R^3$ e la sua derivata in Z, cambiata di segno, genera vorticità meridiana ogni volta che il momento angolare varia lungo l'asse. È l'analogo della forza di galleggiamento $Gr\,\partial\theta/\partial X$ nel problema di Bénard: la forza centrifuga fa le veci della gravità e il momento angolare quello della temperatura (a parte il segno e il fattore $2W/R^3$). Il termine $-VW/R$ nella (3.27) è il termine di Coriolis $-u_r u_\theta / r$ (conservazione del momento angolare: un anello di fluido che si sposta radialmente cambia velocità angolare); il termine $-\Omega/R^2$ nell'operatore viscoso è il termine "extra" del laplaciano vettoriale in coordinate cilindriche. La (3.25) con l'operatore $\partial_{RR} - R^{-1}\partial_R + \partial_{ZZ}$ (notare il segno meno) è la relazione fra la funzione di corrente di Stokes e la vorticità azimutale, $\zeta_\theta = -(1/r)(\psi_{rr} - \psi_r/r + \psi_{zz})$.

3.4.1 La corrente di Couette circolare e la sua perturbazione

Si consideri la corrente stazionaria fra cilindri indefiniti, con condizioni

$$W = 1 \text{ in } R = 1, \qquad W = \frac{r_0^2 \omega_0}{r_i^2 \omega_i} = A^2 B \text{ in } R = A, \qquad A = \frac{r_0}{r_i}, \quad B = \frac{\omega_0}{\omega_i}. \tag{3.28}$$

Senza moto meridiano ($U = V = 0$) la (3.27) stazionaria, $W'' - W'/R = 0$, ha la soluzione $W = aR^2 + b$:

$$W = \frac{(A^2 B - 1)R^2 + A^2(1 - B)}{A^2 - 1}, \tag{3.29}$$

cioè $u_\theta = ar + b/r$, la corrente di Couette circolare. All'istante $T = 0$ la si perturba nel piano meridiano con una debole vorticità periodica,

$$\Omega = 0{,}01 \sin \frac{2\pi Z}{\lambda}, \tag{3.30}$$

di lunghezza d'onda λ. Il moto meridiano associato, calcolato dalle (3.24)–(3.25), modifica W attraverso la (3.27), e questo a sua volta modifica Ω: si segue l'evoluzione nel tempo. Poiché la risposta a un disturbo periodico è periodica, basta calcolare un tratto di lunghezza λ con condizioni periodiche agli estremi ($S_{1,j} = S_{m,j}$, $S_{0,j} = S_{m-1,j}$ per ogni variabile S). Sulle pareti $U = V = 0$.

Il criterio di Rayleigh per i fluidi in rotazione

Per un fluido *non viscoso* in rotazione con $u_\theta(r)$, Rayleigh dimostrò che la corrente è stabile rispetto a disturbi assialsimmetrici se e solo se il quadrato del momento angolare cresce verso l'esterno:

$$\frac{\mathrm{d}}{\mathrm{d}r} (r u_\theta)^2 > 0.$$

Dimostrazione (argomento energetico). Si scambino due anelli di fluido di uguale massa, ai raggi $r_1 < r_2$, con momenti angolari per unità di massa $L_1 = r_1 u_{\theta 1}$ e $L_2 = r_2 u_{\theta 2}$. In assenza di viscosità ciascun anello conserva il proprio momento angolare: dopo lo scambio l'anello 1 si trova in r_2 con velocità L_1/r_2 e l'anello 2 in r_1 con velocità L_2/r_1. La variazione di energia cinetica (per unità di massa) è

$$\Delta E = \frac{1}{2}\left(\frac{L_1^2}{r_2^2} + \frac{L_2^2}{r_1^2}\right) - \frac{1}{2}\left(\frac{L_1^2}{r_1^2} + \frac{L_2^2}{r_2^2}\right) = \frac{1}{2}\left(L_2^2 - L_1^2\right)\left(\frac{1}{r_1^2} - \frac{1}{r_2^2}\right).$$

Se $L_2^2 > L_1^2$ lo scambio *costa* energia ($\Delta E > 0$) e non può avvenire spontaneamente: la corrente è stabile. Se $L_2^2 < L_1^2$ lo scambio libera energia, disponibile per alimentare il moto meridiano: instabilità. Per la corrente di Couette (3.29), $W = r u_\theta$ adimensionale cresce con R se $A^2 B > 1$, cioè se $\omega_0 r_0^2 > \omega_i r_i^2$. Con il cilindro esterno fermo ($B = 0$) il momento angolare *decresce* verso l'esterno e la corrente è instabile per Rayleigh per ogni velocità: è la viscosità a stabilizzarla sotto una soglia. La soglia è espressa dal **numero di Taylor**: per intercapedine stretta ($d = r_0 - r_i \ll r_i$) e cilindro esterno fermo, $Ta = 2\omega_i^2 r_i d^3/\nu^2$, con valore critico $Ta_c = 1708$, lo stesso numero di Rayleigh critico del problema di Bénard fra pareti rigide: l'analogia fra i due problemi è esatta nel limite lineare (Chandrasekhar [15], Cap. VII). Per $A = 2$, $B = 0$ e $Re = r_i^2 \omega_i/\nu = 1000$ si ha $Ta = 2Re^2 (d/r_i)^3 = 2 \times 10^6$ (l'intercapedine non è stretta, ma l'ordine di grandezza è eloquente): si è molto oltre la soglia e la teoria lineare (Taylor) prevede celle con lunghezza d'onda circa $2d = 2$, cioè sezione quasi quadrata.

3.4.2 Il programma: vortici di Taylor

Il Programma 4.3b integra le (3.24)–(3.27) con lo schema di Torrance nei nodi interni; l'equazione (3.25), con $\Psi = 0$ sulle pareti e condizioni periodiche in Z, è risolta con un metodo diretto sparso (fattorizzazione LU calcolata una volta sola: la matrice non cambia), più comodo del SOR con la periodicità. La vorticità alle pareti si ottiene dalla (3.25) con $\Psi = \partial\Psi/\partial R = 0$: $\Omega_w = \Psi_{RR}/R \approx 2\Psi_1/(Rh^2)$ (formula di Thom). Il programma stampa, agli istanti scelti, l'intensità del moto meridiano $|\Psi|_{\max}$ e il **momento torcente** sul cilindro interno, che si ricava dallo sforzo $\tau_{r\theta} = \mu r\,\mathrm{d}(u_\theta/r)/\mathrm{d}r$: in variabili adimensionali, per unità di lunghezza e in unità di $\mu r_i^2\omega_i$, $M = 2\pi\,(\partial W/\partial R - 2W/R)_{R=1}$, mediato su Z. Per la corrente di Couette (3.29) vale $M = -4\pi A^2/(A^2-1) = -16{,}76$ per $A = 2$: lo si usa come riferimento.

Dati del Programma 4.3b

$A = 2$, $B = 0$ (cilindro esterno fermo), $\lambda = 4$, $Re = 1000$; $h = 0{,}125$ (9 nodi radiali, 32 assiali), $\Delta T = 0{,}02$, fino a $T = 60$; perturbazione (3.30) con ampiezza 0,01; secondo caso con lunghezza d'onda $\lambda/2 = 2$.

prog4_3b_taylor.py

```
"""
prog4_3b_taylor.py -- instabilita' di Taylor fra cilindri coassiali rotanti
(moto assialsimmetrico, formulazione vorticita'-funzione di corrente-momento
angolare), con lo stesso schema upwind del problema di Benard.

Variabili (lunghezze riferite a r_i, velocita' a r_i omega_i, tempo a 1/omega_i):
    U = (1/R) dPsi/dR (assiale),  V = -(1/R) dPsi/dZ (radiale),  W = r
    u_theta/(r_i^2 omega_i),
    Psi_RR - Psi_R/R + Psi_ZZ = R Omega,
    dOmega/dT = -d(U Omega)/dZ - d(V Omega)/dR - (2W/R^3) dW/dZ + (1/Re)[Omega_RR +
    Omega_R/R - Omega/R^2 + Omega_ZZ],
    dW/dT     = -d(U W)/dZ - d(V W)/dR - V W/R + (1/Re)[W_RR - W_R/R + W_ZZ].
Cilindro interno R = 1 rotante (W = 1), esterno R = A = 2 fermo (W = 0); dominio
periodico in Z di lunghezza lambda = 4; Re = r_i^2 omega_i/nu = 1000; h = 0,125;
dT = 0,02. Stato iniziale: corrente di Couette circolare W = (A^2 - R^2)/(A^2 - 1)
perturbata con Omega = 0,01 sin(2 pi Z/lambda). Il programma segue la crescita
delle celle di Taylor e calcola il momento torcente sul cilindro interno.
"""
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

A, B = 2.0, 0.0
LAM, RE = 4.0, 1000.0
H, DT = 0.125, 0.02
NR = int(round((A - 1) / H)) + 1          # nodi radiali (pareti comprese)
NZ = int(round(LAM / H))                  # nodi assiali (periodici)
R = 1.0 + H * np.arange(NR)
Z = H * np.arange(NZ)
T_FIN = 60.0


def matrice_psi():
    """Operatore Psi_RR - Psi_R/R + Psi_ZZ sui nodi interni (j = 1..NR-2) x (tutti
    i k), periodico in Z."""
    n_int = NR - 2
    idx = lambda j, k: j * NZ + (k % NZ)
```

```
    Am = sp.lil_matrix((n_int * NZ, n_int * NZ))
    for j in range(n_int):
        r = R[j + 1]
        for k in range(NZ):
            p = idx(j, k)
            Am[p, p] = -4 / H**2
            Am[p, idx(j, k + 1)] += 1 / H**2
            Am[p, idx(j, k - 1)] += 1 / H**2
            if j + 1 < n_int:
                Am[p, idx(j + 1, k)] += 1 / H**2 - 1 / (2 * r * H)
            if j - 1 >= 0:
                Am[p, idx(j - 1, k)] += 1 / H**2 + 1 / (2 * r * H)
    return spla.splu(Am.tocsc())


LU = matrice_psi()


def torrance(P, U, V):
    """-d(UP)/dZ - d(VP)/dR con lo schema upwind di Torrance, nei nodi radiali interni;
    P, U, V hanno forma (NR, NZ) con Z periodico (asse 1)."""
    Pc = P[1:-1, :]
    Uf = 0.5 * (np.roll(U, -1, axis=1) + U)[1:-1, :]; Ub = 0.5 * (U + np.roll(U, 1, axis=1))[1:-1, :]
    Vf = 0.5 * (V[2:, :] + V[1:-1, :]);           Vb = 0.5 * (V[1:-1, :] + V[:-2, :])
    Pz1 = (Uf - abs(Uf)) * np.roll(P, -1, axis=1)[1:-1, :] + (Uf + abs(Uf) - Ub + abs(Ub)) * Pc \
        - (Ub + abs(Ub)) * np.roll(P, 1, axis=1)[1:-1, :]
    Pr1 = (Vf - abs(Vf)) * P[2:, :] + (Vf + abs(Vf) - Vb + abs(Vb)) * Pc - (Vb + abs(Vb)) * P[:-2, :]
    return -(Pz1 + Pr1) / (2 * H)


def simula(t_fin=T_FIN, dt=DT, ampiezza=0.01, modo=1, stampa=True):
    Rc = R[:, None]
    W = np.tile(((A * A * B - 1) * R**2 + A * A * (1 - B)) / (A * A - 1), (NZ, 1)).T
    Om = ampiezza * np.sin(2 * np.pi * modo * Z / LAM)[None, :] * np.ones((NR, 1))
    Om[0, :] = Om[-1, :] = 0.0
    Psi = np.zeros((NR, NZ)); U = np.zeros((NR, NZ)); V = np.zeros((NR, NZ))
    storia, salvati = [], {}
    istanti = (5.0, 10.0, 15.0, 20.0, 30.0, 60.0)
    n = int(round(t_fin / dt))
    for passo in range(1, n + 1):
        # funzione di corrente (Psi = 0 sulle pareti) e velocita'
        rhs = (Rc * Om)[1:-1, :].ravel()
        Psi[1:-1, :] = LU.solve(rhs).reshape(NR - 2, NZ)
        U[1:-1, :] = (Psi[2:, :] - Psi[:-2, :]) / (2 * H) / Rc[1:-1]
        V[1:-1, :] = -(np.roll(Psi, -1, axis=1) - np.roll(Psi, 1, axis=1))[1:-1, :] / (2 * H) / Rc[1:-1]
        # vorticita' alle pareti: Omega = Psi_RR/R con Psi = Psi_R = 0 -> 2 Psi_1/(R h^2)
        Om[0, :] = 2 * Psi[1, :] / (R[0] * H**2)
        Om[-1, :] = 2 * Psi[-2, :] / (R[-1] * H**2)
        # termini diffusivi e sorgenti nei nodi interni
        Rm = Rc[1:-1]
        lapOm = ((Om[2:, :] - 2 * Om[1:-1, :] + Om[:-2, :]) / H**2 + (Om[2:, :] - Om[:-2, :]) / (2 * H * Rm)
                 - Om[1:-1, :] / Rm**2 + (np.roll(Om, -1, axis=1) - 2 * Om +
```

```
    np.roll(Om, 1, axis=1))[1:-1, :] / H**2)
89      lapW = ((W[2:, :] - 2 * W[1:-1, :] + W[:-2, :]) / H**2 - (W[2:, :] - W[:-2,
    :]) / (2 * H * Rm)
90              + (np.roll(W, -1, axis=1) - 2 * W + np.roll(W, 1, axis=1))[1:-1, :]
    / H**2)
91      dWdZ = (np.roll(W, -1, axis=1) - np.roll(W, 1, axis=1))[1:-1, :] / (2 * H)
92      Om_new = Om.copy(); W_new = W.copy()
93      Om_new[1:-1, :] = Om[1:-1, :] + dt * (torrance(Om, U, V) - 2 * W[1:-1, :] /
    Rm**3 * dWdZ + lapOm / RE)
94      W_new[1:-1, :] = W[1:-1, :] + dt * (torrance(W, U, V) - V[1:-1, :] *
    W[1:-1, :] / Rm + lapW / RE)
95      Om, W = Om_new, W_new
96      T = passo * dt
97      # momento torcente sul cilindro interno (per unita' di lunghezza, in unita'
    di mu r_i^2 omega_i):
98      # tau = mu r d(u_theta/r)/dr = mu [dW/dR - 2W/R]/R  -> coppia = 2 pi R^2
    tau; media su Z
99      dWdR = (-3 * W[0, :] + 4 * W[1, :] - W[2, :]) / (2 * H)
100     coppia = 2 * np.pi * np.mean(dWdR - 2 * W[0, :] / R[0])
101     storia.append((T, np.max(np.abs(Psi)), coppia))
102     if any(abs(T - ti) < 1e-9 for ti in istanti):
103         salvati[round(T, 6)] = (Psi.copy(), W.copy(), Om.copy())
104         if stampa:
105             print(f"   T = {T:5.1f}: max|Psi| = {np.max(np.abs(Psi)):.5f},
    coppia/(mu r_i^2 omega_i) = {coppia:+.4f}")
106     if not np.isfinite(np.max(np.abs(Psi))):
107         raise RuntimeError(f"instabilita' numerica al passo {passo}")
108 return salvati, storia
109
110
111 if __name__ == "__main__":
112     coppia_couette = 2 * np.pi * (-2 * A * A / (A * A - 1))      # dW/dR - 2W/R in
    R = 1 per il moto di Couette
113     print(f"Taylor-Couette: A = {A}, B = {B}, lambda = {LAM}, Re = {RE}, griglia
    {NR}x{NZ}, dT = {DT}")
114     print(f"   coppia della corrente di Couette circolare (laminare, senza celle):
    {coppia_couette:+.4f}")
115     salvati, storia = simula()
116     Ts = np.array([s[0] for s in storia]); amp = np.array([s[1] for s in storia]);
    cop = np.array([s[2] for s in storia])
117     # tasso di crescita lineare (fra T = 2 e T = 8)
118     sel = (Ts > 2) & (Ts < 8)
119     pend = np.polyfit(Ts[sel], np.log(amp[sel]), 1)[0]
120     print(f"   tasso di crescita iniziale di |Psi|max: sigma = {pend:.4f} (unita'
    omega_i); ampiezza finale {amp[-1]:.4f}")
121     print(f"   coppia a regime {cop[-1]:+.4f}, cioe' {cop[-1]/coppia_couette:.2f}
    volte quella del moto di Couette")
122     # perturbazione di lunghezza d'onda meta' (due coppie di celle nel dominio)
123     print("\nPerturbazione iniziale con lunghezza d'onda lambda/2 = 2 (celle di
    sezione quasi quadrata):")
124     salvati2, storia2 = simula(modo=2, stampa=False)
125     Ts2 = np.array([s[0] for s in storia2]); amp2 = np.array([s[1] for s in
    storia2]); cop2 = np.array([s[2] for s in storia2])
126     sel2 = (Ts2 > 2) & (Ts2 < 8)
127     print(f"   tasso di crescita iniziale sigma = {np.polyfit(Ts2[sel2],
    np.log(amp2[sel2]), 1)[0]:.4f};"
128           f" ampiezza finale {amp2[-1]:.4f}; coppia a regime {cop2[-1]:+.4f}
    ({cop2[-1]/coppia_couette:.2f} volte Couette)")
129
130     fig, axs = plt.subplots(2, 3, figsize=(8.6, 4.6))
```

```
    for a_, Tk in zip(axs.ravel()[:5], (5.0, 15.0, 20.0, 60.0, -60.0)):
        Psi, W, Om = salvati[Tk] if Tk > 0 else salvati2[-Tk]
        Ze = np.concatenate([Z, [LAM]])
        Pe = np.concatenate([Psi, Psi[:, :1]], axis=1)
        m = np.max(np.abs(Pe))
        lv = np.linspace(-m, m, 11); lv = lv[np.abs(lv) > 1e-9]
        a_.contour(Ze, R, Pe, levels=lv, colors=[ROSSO if v < 0 else BLU for v in lv],
                   linestyles=["--" if v < 0 else "-" for v in lv], linewidths=0.8)
        a_.set_aspect("equal")
        a_.set_title((f"$T={abs(Tk):g}$, $|\\Psi|_{{\\max}}={m:.3f}$" + (" ($\\lambda/2$)" if Tk < 0 else "")).replace(".", "{,}"), fontsize=8)
        a_.set_xlabel("$Z$"); a_.set_ylabel("$R$")
    a_ = axs.ravel()[5]
    a_.semilogy(Ts, amp, color=BLU, label="$\\lambda=4$")
    a_.semilogy(Ts2, amp2, "--", color=ARANCIO, label="$\\lambda/2=2$")
    a_.set_xlabel("$T$"); a_.set_ylabel("$|\\Psi|_{\\max}$");
    a_.set_title("crescita delle celle", fontsize=8); a_.legend(fontsize=7)
    fig.tight_layout()
    salva(fig, "prog4_3b_taylor")
```

Listato 3.2: `prog4_3b_taylor.py`: instabilità di Taylor fra cilindri coassiali con lo schema di Torrance; crescita delle celle e momento torcente.

Risultati del Programma 4.3b

Il moto meridiano, seminato con ampiezza $|\Psi|_{\max} = 0{,}002$, cresce esponenzialmente con tasso $\sigma = 0{,}187\,\omega_i$ (da 0,0023 a $T = 5$ a 0,0764 a $T = 20$), poi satura a $|\Psi|_{\max} \approx 0{,}10$ e si organizza in due celle controrotanti per lunghezza d'onda, i vortici di Taylor (Fig. 3.4): fluido che sale lungo il cilindro interno in una sezione e scende in quella contigua, con il getto uscente concentrato dove le due celle convergono. Il momento torcente, inizialmente quello di Couette ($-16{,}8$), cresce fino a $-48{,}2$ a $T = 60$: le celle trasportano momento angolare attraverso l'intercapedine molto più efficacemente della diffusione viscosa, e la coppia necessaria a mantenere la rotazione aumenta di un fattore 2,9, come nei misuratori di viscosità in cui la comparsa dei vortici si rivela da un brusco aumento della coppia. Con la perturbazione di lunghezza d'onda 2 (celle di sezione quasi quadrata, quella della teoria lineare) il tasso di crescita è quasi doppio, 0,356, e la coppia finale $-51{,}7$ (3,1 volte Couette): è la lunghezza d'onda preferita, come prevede la teoria lineare, ma la saturazione non lineare rende le due ampiezze finali confrontabili.

Dove si incontra l'instabilità di Taylor

I cuscinetti a strisciamento lubrificati (albero rotante in una bronzina ferma) sono esattamente due cilindri coassiali con $B = 0$: oltre un certo numero di Taylor il film d'olio sviluppa vortici anulari che aumentano la coppia d'attrito e il calore dissipato. Lo stesso accade nelle tenute rotanti, negli scambiatori a cilindri rotanti e nei reattori chimici "Couette–Taylor", che sfruttano i vortici per mescolare. In aeronautica il problema si presenta nei cuscinetti dei motori e nelle intercapedini fra rotore e statore delle turbomacchine, dove la forza centrifuga destabilizza lo strato limite sulle pale concave (vortici di Görtler, parente stretto di quelli di Taylor).

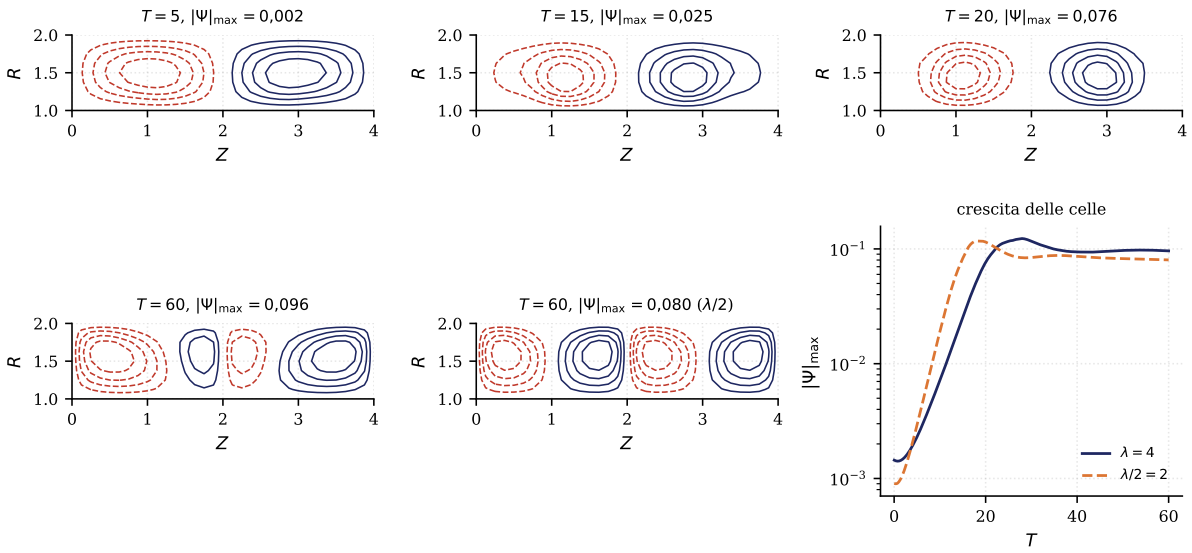


Figura 3.4: Programma 4.3b: linee di corrente meridiane dei vortici di Taylor a vari istanti ($\lambda = 4$ e, in basso al centro, $\lambda/2 = 2$) e crescita di $|\Psi|_{\max}$ nel tempo per le due lunghezze d'onda.

3.5 Approfondimenti

3.5.1 L'avviamento in rotazione: il caffè sulla scala a chiocciola

Un problema "da cartone animato" proposto da Scorer: una cameriera scende una lunga scala a chiocciola portando un vassoio con il caffè e annuncia che il caffè è "già mescolato". Ha ragione? Il moto della tazza è una traslazione più una rotazione; solo la rotazione agisce sul liquido. All'istante iniziale il caffè è fermo e la tazza riceve improvvisamente la velocità angolare ω_0, che resta costante; la superficie libera è piana e priva di sforzi. È un problema di *spin-up*, l'avviamento in rotazione di un fluido, che interessa la geofisica (oceani e atmosfera dopo un cambiamento di rotazione, nuclei planetari) e l'astrofisica, e che Greenspan [18] tratta in generale. Le equazioni sono le (3.24)–(3.27) con r_0 e $r_0\omega_0$ come riferimenti e $W = ru_\theta/(r_0^2\omega_0)$; con $r_0 = 4$ cm, $\omega_0 = 0{,}1\pi$ rad/s (tre giri al minuto) e $\nu = 10^{-6}$ m^2/s si ha $Re = r_0^2\omega_0/\nu \approx 500$. Le condizioni: sull'asse $\Psi = \Omega = W = 0$; sul fondo e sulla parete, che ruotano con la tazza, $\Psi = 0$, $W = R^2$ (rotazione rigida) e vorticità dalla formula di Thom; sulla superficie libera $\Psi = 0$, $\Omega = 0$ e $\partial W/\partial Z = 0$ (nessuno sforzo tangenziale). Il passo temporale è scelto a ogni istante dal criterio (3.19), come richiede il progetto della fonte. Il Programma 4.3c integra fino a $T = 8$, il tempo di discesa della scala (25 s).

Strati di Ekman e tempo di spin-up

In un fluido in rotazione il moto è governato dal **numero di Ekman** $E = \nu/(\omega_0 r_0^2) = 1/Re$. Subito dopo l'avviamento, sul fondo si forma uno strato viscoso di spessore $\delta_E = \sqrt{\nu/\omega_0} = r_0\sqrt{E}$ (lo *strato di Ekman*, 1,8 mm per il caffè) in cui il fluido, trascinato in rotazione, è spinto verso l'esterno dalla forza centrifuga; risale lungo la parete, rientra verso l'asse nell'interno e, rientrando, conserva il momento angolare acquisito e quindi *accelera* (come un pattinatore che chiude le braccia). È questa circolazione secondaria, il *pompaggio di Ekman*, che mette in rotazione l'interno: non la diffusione viscosa diretta, che richiederebbe un tempo $r_0^2/\nu = Re/\omega_0 = 500/\omega_0$ (27 minuti), ma un tempo molto

più breve, $\tau_{su} \sim E^{-1/2}/\omega_0 = \sqrt{Re}/\omega_0 \approx 22/\omega_0$, circa 70 s. L'avviamento da fermo è un problema non lineare, con il rientro del fluido verso l'asse limitato dalla velocità $\sim \sqrt{E}\, r_0\omega_0$ della circolazione secondaria; il tempo di discesa, $8/\omega_0$, è circa un terzo del tempo di spin-up.

prog4_3c_tazza.py

```
"""
prog4_3c_tazza.py -- avviamento in rotazione (spin-up) del caffe' in una tazza
cilindrica messa improvvisamente in rotazione con velocita' angolare omega0
(progetto "You'll find it's already stirred").

Moto assialsimmetrico in coordinate (R, Z) riferite a r0, velocita' a r0 omega0,
tempo a 1/omega0; tazza di raggio 1 e altezza 2, superficie libera piana e priva
di sforzi tangenziali. Formulazione Omega-Psi-W della corrente di Taylor:
    U = (1/R) dPsi/dR,  V = -(1/R) dPsi/dZ,  W = r u_theta/(r0^2 omega0),
    Psi_RR - Psi_R/R + Psi_ZZ = R Omega,
    dOmega/dT = -d(U Omega)/dZ - d(V Omega)/dR - (2W/R^3) dW/dZ + (1/Re) L1 Omega,
    dW/dT     = -d(U W)/dZ - d(V W)/dR - V W/R + (1/Re) L2 W,
con L1 = d_RR + d_R/R - 1/R^2 + d_ZZ e L2 = d_RR - d_R/R + d_ZZ, Re = r0^2
    omega0/nu = 500.
Condizioni: asse R = 0: Psi = Omega = W = 0; fondo e parete (rotanti con la
tazza): Psi = 0, W = R^2, Omega dalla formula di parete; superficie libera Z = 2:
Psi = 0, Omega = 0, dW/dZ = 0. Fluido inizialmente in quiete (W = 0). Schema
upwind di Torrance con passo temporale scelto a ogni passo dal criterio di
stabilita' (4.3.33), funzione di corrente con fattorizzazione LU sparsa.
"""
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

RE = 500.0
H = 0.025
ALT = 2.0
NR = int(round(1.0 / H)) + 1
NZ = int(round(ALT / H)) + 1
R = H * np.arange(NR)
Z = H * np.arange(NZ)
T_FIN = 8.0


def matrice_psi():
    """Psi_RR - Psi_R/R + Psi_ZZ nei nodi interni (j = 1..NR-2, k = 1..NZ-2), Psi =
    0 sul contorno."""
    nr, nz = NR - 2, NZ - 2
    idx = lambda j, k: j * nz + k
    A = sp.lil_matrix((nr * nz, nr * nz))
    for j in range(nr):
        r = R[j + 1]
        for k in range(nz):
            p = idx(j, k)
            A[p, p] = -4 / H**2
            if k + 1 < nz: A[p, idx(j, k + 1)] = 1 / H**2
            if k - 1 >= 0: A[p, idx(j, k - 1)] = 1 / H**2
            if j + 1 < nr: A[p, idx(j + 1, k)] = 1 / H**2 - 1 / (2 * r * H)
            if j - 1 >= 0: A[p, idx(j - 1, k)] = 1 / H**2 + 1 / (2 * r * H)
    return spla.splu(A.tocsc())


```

```
def torrance(P, U, V):
    """-d(UP)/dZ - d(VP)/dR (schema upwind di Torrance) nei nodi interni; array (NR, NZ)."""
    Pc = P[1:-1, 1:-1]
    Uf = 0.5 * (U[1:-1, 2:] + U[1:-1, 1:-1]); Ub = 0.5 * (U[1:-1, 1:-1] + U[1:-1, :-2])
    Vf = 0.5 * (V[2:, 1:-1] + V[1:-1, 1:-1]); Vb = 0.5 * (V[1:-1, 1:-1] + V[:-2, 1:-1])
    Pz = (Uf - abs(Uf)) * P[1:-1, 2:] + (Uf + abs(Uf) - Ub + abs(Ub)) * Pc - (Ub + abs(Ub)) * P[1:-1, :-2]
    Pr = (Vf - abs(Vf)) * P[2:, 1:-1] + (Vf + abs(Vf) - Vb + abs(Vb)) * Pc - (Vb + abs(Vb)) * P[:-2, 1:-1]
    return -(Pz + Pr) / (2 * H)


def dt_massimo(U, V, B):
    """Criterio (4.3.33) per il termine convettivo + diffusivo con coefficiente B."""
    Uf = 0.5 * (U[1:-1, 2:] + U[1:-1, 1:-1]); Ub = 0.5 * (U[1:-1, 1:-1] + U[1:-1, :-2])
    Vf = 0.5 * (V[2:, 1:-1] + V[1:-1, 1:-1]); Vb = 0.5 * (V[1:-1, 1:-1] + V[:-2, 1:-1])
    s = (Uf + abs(Uf) - Ub + abs(Ub) + Vf + abs(Vf) - Vb + abs(Vb)) / (2 * H) + 4 * B / H**2
    return 1.0 / np.max(s)


def simula(t_fin=T_FIN, istanti=(0.5, 1.0, 2.0, 4.0, 8.0), stampa=True, sicurezza=0.8):
    LU = matrice_psi()
    Rc = R[:, None]
    Psi = np.zeros((NR, NZ)); Om = np.zeros((NR, NZ)); W = np.zeros((NR, NZ))
    U = np.zeros((NR, NZ)); V = np.zeros((NR, NZ))
    W[-1, :] = 1.0; W[:, 0] = R**2                          # parete e fondo rotanti
    T, salvati, storia = 0.0, {}, []
    prossimo = 0
    while T < t_fin - 1e-12:
        # funzione di corrente e velocita'
        Psi[1:-1, 1:-1] = LU.solve((Rc * Om)[1:-1, 1:-1].ravel()).reshape(NR - 2, NZ - 2)
        U[1:-1, 1:-1] = (Psi[2:, 1:-1] - Psi[:-2, 1:-1]) / (2 * H) / Rc[1:-1]
        V[1:-1, 1:-1] = -(Psi[1:-1, 2:] - Psi[1:-1, :-2]) / (2 * H) / Rc[1:-1]
        # vorticita' sulle pareti solide (Psi = Psi_n = 0 -> Omega = 2 Psi_1/(R h^2)); asse e superficie libera: 0
        Om[-1, 1:-1] = 2 * Psi[-2, 1:-1] / (R[-1] * H**2)
        Om[1:-1, 0] = 2 * Psi[1:-1, 1] / (R[1:-1] * H**2)
        Om[0, :] = 0.0; Om[:, -1] = 0.0
        # superficie libera: dW/dZ = 0
        W[:, -1] = W[:, -2]
        # passo temporale dal criterio di stabilita' (il piu' restrittivo fra le due equazioni)
        dt = sicurezza * min(dt_massimo(U, V, 1.0 / RE), dt_massimo(U, V, 1.0 / RE))
        if prossimo < len(istanti):
            dt = min(dt, istanti[prossimo] - T)
        Rm = Rc[1:-1]
        lapOm = ((Om[2:, 1:-1] - 2 * Om[1:-1, 1:-1] + Om[:-2, 1:-1]) / H**2 + (Om[2:, 1:-1] - Om[:-2, 1:-1]) / (2 * H * Rm)
                 - Om[1:-1, 1:-1] / Rm**2 + (Om[1:-1, 2:] - 2 * Om[1:-1, 1:-1] + Om[1:-1, :-2]) / H**2)
        lapW = ((W[2:, 1:-1] - 2 * W[1:-1, 1:-1] + W[:-2, 1:-1]) / H**2 - (W[2:,
```

```
    1:-1] - W[:-2, 1:-1]) / (2 * H * Rm)
98                  + (W[1:-1, 2:] - 2 * W[1:-1, 1:-1] + W[1:-1, :-2]) / H**2)
99          dWdZ = (W[1:-1, 2:] - W[1:-1, :-2]) / (2 * H)
100         Om_n = Om.copy(); W_n = W.copy()
101         Om_n[1:-1, 1:-1] = Om[1:-1, 1:-1] + dt * (torrance(Om, U, V) - 2 * W[1:-1,
    1:-1] / Rm**3 * dWdZ + lapOm / RE)
102         W_n[1:-1, 1:-1] = W[1:-1, 1:-1] + dt * (torrance(W, U, V) - V[1:-1, 1:-1] *
    W[1:-1, 1:-1] / Rm + lapW / RE)
103         Om, W = Om_n, W_n
104         T += dt
105         # momento angolare totale del fluido rispetto a quello della rotazione
    rigida (W = R^2)
106         L = np.sum((W * Rc)[1:-1, 1:-1]) / np.sum((Rc**3)[1:-1] * np.ones((1, NZ -
    2)))
107         storia.append((T, dt, np.max(np.abs(Psi)), L))
108         if prossimo < len(istanti) and abs(T - istanti[prossimo]) < 1e-10:
109             salvati[istanti[prossimo]] = (Psi.copy(), W.copy(), Om.copy())
110             if stampa:
111                 print(f"   T = {T:4.1f}: dT = {dt:.5f}, max|Psi| =
    {np.max(np.abs(Psi)):.5f},"
112                       f" momento angolare / rotazione rigida = {L:.3f}, u_theta
    sull'asse di meta' altezza (R = 0,5): "
113                       f"{W[NR//2, NZ//2]/R[NR//2]:.3f} (rigido: {R[NR//2]:.3f})")
114             prossimo += 1
115     return salvati, storia
116
117
118 if __name__ == "__main__":
119     print(f"Spin-up nella tazza: Re = {RE:.0f}, griglia {NR}x{NZ} (h = {H}),
    altezza {ALT}")
120     print(f"   tempo di spin-up di Ekman stimato: sqrt(Re) = {np.sqrt(RE):.1f}
    unita' di 1/omega0;"
121           f" tempo di diffusione viscosa Re = {RE:.0f}")
122     print(f"   discesa della scala: T = 8 corrisponde a {8/(0.1*np.pi):.1f} s con
    omega0 = 0,1 pi rad/s")
123     salvati, storia = simula()
124     Ts = np.array([s[0] for s in storia]); Ls = np.array([s[3] for s in storia])
125     print(f"   frazione di momento angolare acquisita a T = 8: {Ls[-1]:.3f}")
126
127     fig, axs = plt.subplots(1, 5, figsize=(8.6, 3.4), gridspec_kw={"width_ratios":
    [1, 1, 1, 1, 1.6]})
128     for a_, Tk in zip(axs[:4], (1.0, 2.0, 4.0, 8.0)):
129         Psi, W, Om = salvati[Tk]
130         m = np.max(np.abs(Psi))
131         lv = np.linspace(-m, m, 11); lv = lv[np.abs(lv) > 1e-12]
132         a_.contour(R, Z, Psi.T, levels=lv, colors=[ROSSO if v < 0 else BLU for v in
    lv],
133                    linestyles=["--" if v < 0 else "-" for v in lv], linewidths=0.7)
134         cs = a_.contour(R, Z, (W / np.maximum(R, 1e-9)[:, None]).T, levels=[0.1,
    0.3, 0.5, 0.7, 0.9], colors=GRIGIO, linewidths=0.6)
135         a_.clabel(cs, fontsize=5, fmt=lambda v: f"{v:.1f}".replace(".", ","))
136         a_.set_aspect("equal"); a_.set_xlabel("$R$")
137         a_.set_title(f"$T={Tk:g}$, $|\\Psi|_{{\\max}}={m:.4f}$".replace(".",
    "{,}"), fontsize=7)
138         a_.set_xticks([0, 0.5, 1]); a_.set_yticks([0, 1, 2])
139     axs[0].set_ylabel("$Z$")
140     axs[4].plot(Ts, Ls, color=BLU)
141     axs[4].set_xlabel("$T$"); axs[4].set_ylabel("momento angolare / rotazione
    rigida")
142     axs[4].set_title("avviamento", fontsize=7)
```

```
    fig.tight_layout()
    salva(fig, "prog4_3c_tazza")
```

Listato 3.3: `prog4_3c_tazza.py`: spin-up del caffè in una tazza messa in rotazione, con passo temporale adattato al criterio di stabilità.

Risultati del Programma 4.3c

Con $h = 0{,}025$ (41×81 nodi, due celle nello strato di Ekman) il passo scelto dal criterio di stabilità oscilla fra 0,012 e 0,034. La circolazione secondaria nasce nello spigolo fra fondo e parete e si allarga progressivamente (Fig. 3.5): a $T = 8$ occupa il terzo esterno della tazza, mentre l'interno è ancora fermo (a metà raggio e metà altezza u_θ è l'1,5% del valore di rotazione rigida). Il momento angolare totale del liquido, in frazione di quello della rotazione rigida, cresce come $\sqrt{T}$ nelle prime fasi (diffusione dalle pareti) e raggiunge 0,51 a $T = 8$. Con la griglia $h = 0{,}05$ il risultato è 0,50: la frazione è insensibile alla risoluzione dello strato di Ekman, mentre lo sono i dettagli della circolazione. La risposta alla domanda è dunque *no*: dopo la discesa la metà esterna del caffè ruota con la tazza ma il centro è ancora fermo, e non c'è mescolamento nel senso della cameriera (che d'altra parte, se il caffè fosse in rotazione rigida, non sarebbe affatto mescolato: nella rotazione rigida il fluido non si deforma). Il mescolamento vero lo fa la circolazione secondaria, che porta il fluido dal fondo alla superficie lungo la parete: è lo stesso meccanismo per cui le foglie di tè si raccolgono al centro del fondo quando si smette di mescolare (problema di Einstein, 1926), con la circolazione di segno opposto nel rallentamento.

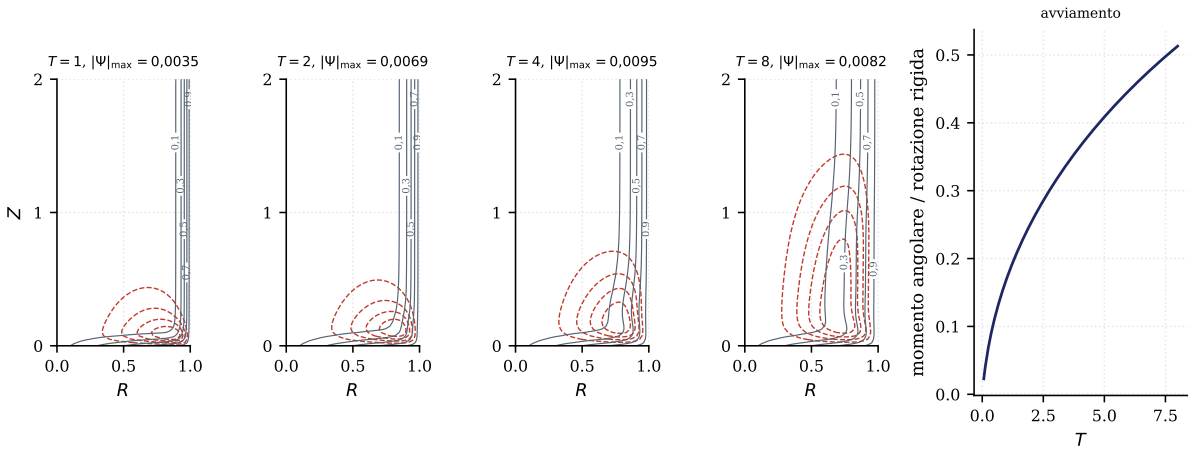


Figura 3.5: Programma 4.3c: linee di corrente della circolazione secondaria (blu/rosso) e linee di uguale velocità angolare u_θ / R (grigio, da 0,1 a 0,9) nel semipiano meridiano della tazza a $T = 1, 2, 4, 8$; a destra la frazione di momento angolare acquisita.

3.5.2 Due varianti del problema di Bénard

Il Programma 4.3 si adatta con poche righe a situazioni diverse. Il Programma 4.3d ne considera due, proposte dalla fonte come progetti.

1. **Scatola riscaldata dall'alto con superficie libera.** All'inizio $\theta = 0$ ovunque tranne $\theta = 1$ sulla superficie superiore, che è una superficie libera rigida (piana) e priva di sforzi,

quindi con $\Omega = 0$; le pareti restano alle temperature iniziali. È un modello di una regione d'oceano fra due iceberg riscaldata dal sole. La stratificazione media è *stabile* (caldo sopra): non c'è instabilità di Bénard, ma il gradiente orizzontale di temperatura negli spigoli superiori, dove la superficie calda incontra la parete fredda, genera vorticità direttamente dal termine $Gr\,\partial\theta/\partial X$. Il moto nasce negli spigoli e si estende verso il centro: a $T = 5$ due celle (una per metà scatola) occupano tutta la lunghezza, con $|\Psi|_{\max} = 0{,}46$, metà di quelle di Bénard; il fluido scende lungo le pareti fredde e risale al centro sotto la superficie (Fig. 3.6, colonna a).

2. **Strato con gradiente orizzontale periodico.** Due piastre orizzontali indefinite; sulla superiore si impone la temperatura periodica $\theta = 1 + 0{,}5\sin(2\pi X/\lambda)$, sull'inferiore $\theta = 0$; condizioni periodiche in X su una lunghezza $\lambda = 2{,}5$. Modella la circolazione prodotta da un riscaldamento differenziale orizzontale (fra equatore e poli, con stratificazione verticale stabile). Il moto è formato da due celle controrotanti per lunghezza d'onda, con fluido che sale sotto la parte calda della piastra e scende sotto quella fredda; l'intensità $|\Psi|_{\max} \approx 0{,}21$ si stabilizza già a $T = 1$ (Fig. 3.6, colonna b). Il programma usa un risolutore diretto sparso per l'equazione di Poisson con la periodicità e la funzione `pnew` del Programma 4.3 su array estesi con una colonna fantasma per lato.

prog4_3d_benard_varianti.py

```
"""
prog4_3d_benard_varianti.py -- due varianti del problema di Benard risolte con lo
stesso schema del Programma 4.3 (upwind di Torrance, formulazione Omega-Psi):

(a) scatola riscaldata dall'alto: theta = 1 sulla superficie superiore, che e'
    una superficie libera piana e priva di sforzi (Omega = 0); theta = 0 sul fondo
    e sulla parete laterale; fluido inizialmente in quiete. La stratificazione
    media e' stabile; il moto nasce dal gradiente orizzontale di temperatura
    negli spigoli superiori (modello di una regione di oceano fra due iceberg
    riscaldata dal sole). Meta' sinistra della scatola, asse di simmetria i = m.
(b) strato fra due piastre orizzontali infinite con temperatura periodica sulla
    piastra superiore, theta = 1 + 0,5 sin(2 pi X/lambda), lambda = 2,5, theta = 0
    sulla piastra inferiore; condizioni periodiche in X (modello della circolazione
    generata da gradienti termici orizzontali). Pr = 6,75, Gr = 1000.
"""
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt
from prog4_3_benard import pnew, sorlx, dt_massimo, M, N, H, PR, GR
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO


# ---------------------------------------------------------------- (a) riscaldamento dall'alto
def scatola_dall_alto(maxstp=2000, dt=0.0025, istanti=(0.05, 0.25, 1.0, 5.0), errmax=1e-4):
    psi = np.zeros((M, N)); omega = np.zeros((M, N)); theta = np.zeros((M, N))
    U = np.zeros((M, N)); V = np.zeros((M, N))
    theta[:, N - 1] = 1.0                                  # superficie superiore calda
    theta[0, :] = 0.0                                      # parete laterale fredda (spigolo superiore freddo)
    salvati, storia = {}, []
    for nstep in range(1, maxstp + 1):
        psi, it = sorlx(psi, omega, H, errmax)
        U[:, 1:-1] = (psi[:, 2:] - psi[:, :-2]) / (2 * H)
        V[1:-1, :] = -(psi[2:, :] - psi[:-2, :]) / (2 * H)
```

```
        V[M - 1, :] = psi[M - 2, :] / H
        omnew = omega.copy(); thnew = theta.copy()
        omnew[1:-1, 1:-1] = pnew(omega, theta, U, V, GR, 1.0, dt, H)
        thnew[1:-1, 1:-1] = pnew(theta, theta, U, V, 0.0, 1.0 / PR, dt, H)
        Ue = np.vstack([U[M - 2:M, :], -U[M - 2, :][None, :]])
        Ve = np.vstack([V[M - 2:M, :], V[M - 2, :][None, :]])
        The = np.vstack([theta[M - 2:M, :], theta[M - 2, :][None, :]])
        thnew[M - 1, 1:-1] = pnew(The, The, Ue, Ve, 0.0, 1.0 / PR, dt, H)[0]
        omnew[M - 1, :] = 0.0
        omnew[0, 1:-1] = (4 * V[1, 1:-1] - V[2, 1:-1]) / (2 * H)          # parete
    laterale
        omnew[1:-1, 0] = (-4 * U[1:-1, 1] + U[1:-1, 2]) / (2 * H)          # fondo
        omnew[:, N - 1] = 0.0                                              #
    superficie libera senza sforzi
        omega, theta = omnew, thnew
        T = nstep * dt
        storia.append((T, np.max(np.abs(psi))))
        for Ts in istanti:
            if abs(T - Ts) < 1e-9:
                salvati[Ts] = (psi.copy(), theta.copy())
                print(f"   (a) T = {T:5.2f}: max|Psi| = {np.max(np.abs(psi)):.4f},
    min Psi = {psi.min():+.4f},"
                      f" max Psi = {psi.max():+.4f}, iter. SOR = {it}")
    return salvati, storia


# ------------------------------------------------------------------ (b) gradiente
    orizzontale periodico
LAM = 2.5
NX = int(round(LAM / H))                  # nodi periodici in X (X_i = i h, i =
    0..NX-1)


def matrice_periodica():
    """lap Psi nei nodi j = 1..N-2 (tutti gli i, periodici), Psi = 0 su j = 0 e j =
    N-1."""
    nj = N - 2
    idx = lambda i, j: (i % NX) * nj + (j - 1)
    A = sp.lil_matrix((NX * nj, NX * nj))
    for i in range(NX):
        for j in range(1, N - 1):
            p = idx(i, j)
            A[p, p] = -4 / H**2
            A[p, idx(i + 1, j)] += 1 / H**2
            A[p, idx(i - 1, j)] += 1 / H**2
            if j + 1 <= N - 2: A[p, idx(i, j + 1)] += 1 / H**2
            if j - 1 >= 1: A[p, idx(i, j - 1)] += 1 / H**2
    return spla.splu(A.tocsc())


def estesa(S):
    """Aggiunge una colonna fantasma per lato (periodicita' in X)."""
    return np.vstack([S[-1:, :], S, S[:1, :]])


def strato_periodico(maxstp=2000, dt=0.0025, istanti=(0.05, 0.25, 1.0, 5.0)):
    X = H * np.arange(NX)
    LU = matrice_periodica()
    psi = np.zeros((NX, N)); omega = np.zeros((NX, N)); theta = np.zeros((NX, N))
    U = np.zeros((NX, N)); V = np.zeros((NX, N))
```

```
    theta[:, N - 1] = 1.0 + 0.5 * np.sin(2 * np.pi * X / LAM)
    salvati, storia = {}, []
    for nstep in range(1, maxstp + 1):
        psi[:, 1:-1] = LU.solve((-omega[:, 1:-1]).ravel()).reshape(NX, N - 2)
        U[:, 1:-1] = (psi[:, 2:] - psi[:, :-2]) / (2 * H)
        V = -(np.roll(psi, -1, axis=0) - np.roll(psi, 1, axis=0)) / (2 * H)
        omnew = omega.copy(); thnew = theta.copy()
        omnew[:, 1:-1] = pnew(estesa(omega), estesa(theta), estesa(U), estesa(V),
    GR, 1.0, dt, H)
        thnew[:, 1:-1] = pnew(estesa(theta), estesa(theta), estesa(U), estesa(V),
    0.0, 1.0 / PR, dt, H)
        omnew[:, 0] = (-4 * U[:, 1] + U[:, 2]) / (2 * H)                  # piastra
    inferiore
        omnew[:, N - 1] = (4 * U[:, N - 2] - U[:, N - 3]) / (2 * H)       # piastra
    superiore
        omega, theta = omnew, thnew
        T = nstep * dt
        storia.append((T, np.max(np.abs(psi))))
        for Ts in istanti:
            if abs(T - Ts) < 1e-9:
                salvati[Ts] = (psi.copy(), theta.copy())
                print(f"   (b) T = {T:5.2f}: max|Psi| = {np.max(np.abs(psi)):.4f},
    min Psi = {psi.min():+.4f},"
                      f" max Psi = {psi.max():+.4f}")
    return salvati, storia, X


if __name__ == "__main__":
    print(f"Varianti del problema di Benard: Pr = {PR}, Gr = {GR}, h = {H}, dT =
    0.0025")
    print("(a) scatola riscaldata dall'alto con superficie libera (meta' sinistra,
    21x9):")
    sa, storia_a = scatola_dall_alto()
    print(f"(b) strato periodico, lambda = {LAM} ({NX} nodi in X, periodici):")
    sb, storia_b, X = strato_periodico()

    Xa = np.arange(M) * H; Y = np.arange(N) * H
    Xf = np.concatenate([Xa, 2 * Xa[-1] - Xa[-2::-1]])
    fig, axs = plt.subplots(4, 2, figsize=(8.4, 6.0))
    for riga, Ts in enumerate((0.05, 0.25, 1.0, 5.0)):
        psi, theta = sa[Ts]
        psif = np.vstack([psi, -psi[-2::-1, :]]); thf = np.vstack([theta,
    theta[-2::-1, :]])
        m = np.max(np.abs(psif)); lv = np.linspace(-m, m, 13); lv = lv[np.abs(lv) >
    1e-7]
        a_ = axs[riga, 0]
        a_.contour(Xf, Y, psif.T, levels=lv, colors=[ROSSO if v < 0 else BLU for v
    in lv],
                   linestyles=["--" if v < 0 else "-" for v in lv], linewidths=0.7)
        a_.contour(Xf, Y, thf.T, levels=np.linspace(0.1, 0.9, 5), colors=GRIGIO,
    linewidths=0.5)
        a_.set_aspect("equal"); a_.set_xticks([]); a_.set_yticks([])
        a_.set_title(f"(a) $T={Ts:g}$, $|\\Psi|_{{\\max}}={m:.3f}$".replace(".",
    "{,}"), fontsize=8)
        psi, theta = sb[Ts]
        Xe = np.concatenate([X, [LAM]]); pe = np.vstack([psi, psi[:1]]); te =
    np.vstack([theta, theta[:1]])
        m = np.max(np.abs(pe)); lv = np.linspace(-m, m, 13); lv = lv[np.abs(lv) >
    1e-7]
        a_ = axs[riga, 1]
```

```
135         a_.contour(Xe, Y, pe.T, levels=lv, colors=[ROSSO if v < 0 else BLU for v in
    lv],
136                    linestyles=["--" if v < 0 else "-" for v in lv], linewidths=0.7)
137         a_.contour(Xe, Y, te.T, levels=np.linspace(0.2, 1.4, 7), colors=GRIGIO,
    linewidths=0.5)
138         a_.set_aspect("equal"); a_.set_xticks([]); a_.set_yticks([])
139         a_.set_title(f"(b) $T={Ts:g}$, $|\\Psi|_{{\\max}}={m:.3f}$".replace(".",
    "{,}"), fontsize=8)
140     fig.tight_layout()
141     salva(fig, "prog4_3d_benard_varianti")
```

Listato 3.4: `prog4_3d_benard_varianti.py`: scatola riscaldata dall'alto con superficie libera e strato con gradiente orizzontale periodico.

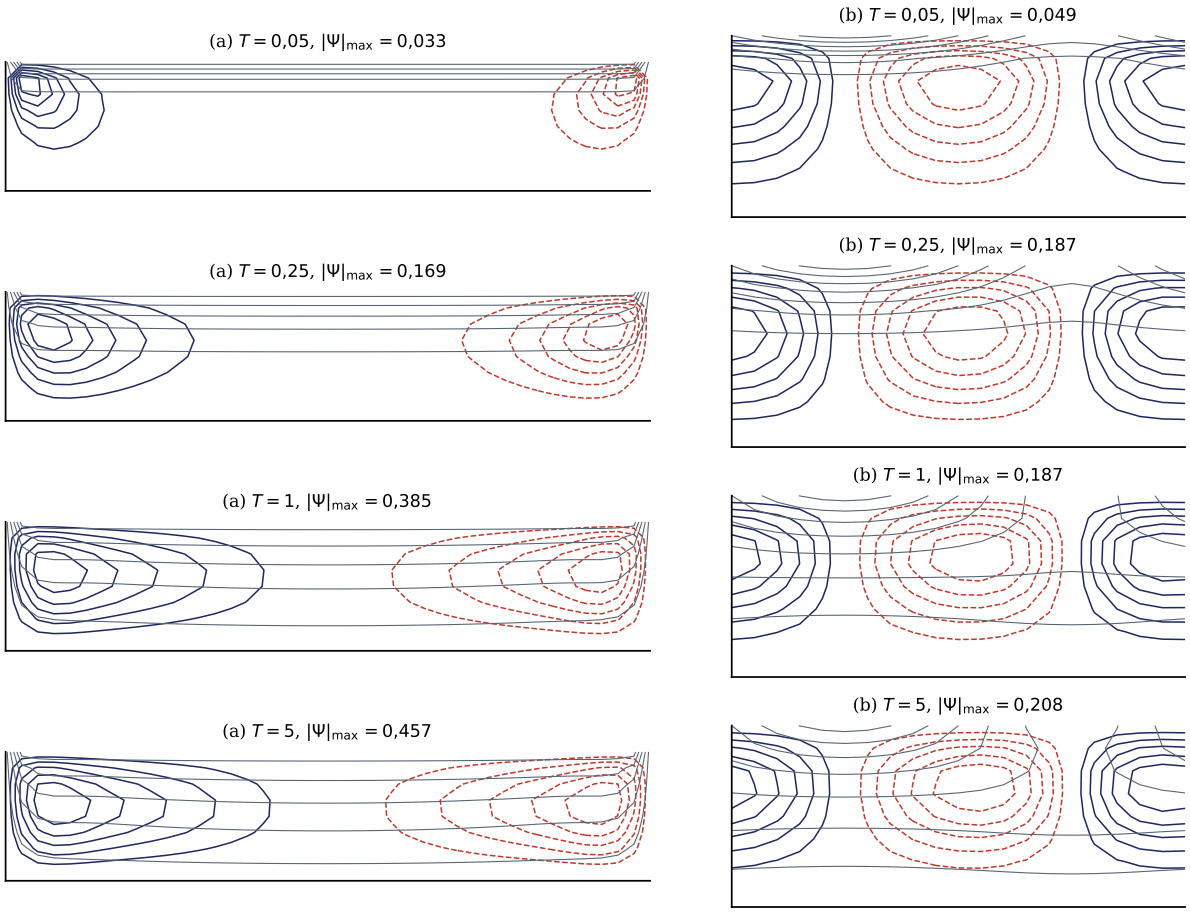


Figura 3.6: Programma 4.3d: linee di corrente (blu/rosso) e isoterme (grigio) a $T = 0{,}05$, 0,25, 1, 5 per la scatola riscaldata dall'alto (colonna a) e per lo strato periodico (colonna b).

3.5.3 Il tubo rotante con restringimento

Un ultimo progetto della fonte, qui soltanto formulato (Esercizio 14), riguarda la corrente stazionaria in un tubo rotante con un brusco restringimento (Fig. 3.7): all'ingresso il fluido ruota rigidamente con il tubo, ω_0, e avanza con velocità uniforme u_0. Con r_0 e u_0 come riferimenti e $W = ru_\theta/(r_0^2\omega_0)$, le equazioni sono le (3.24)–(3.27) con la (3.26) sostituita da

$$\frac{\partial \Omega}{\partial T} = -\frac{\partial (U\Omega)}{\partial Z} - \frac{\partial (V\Omega)}{\partial R} - \frac{1}{2Ro^2}\frac{W}{R^3}\frac{\partial W}{\partial Z} + \frac{1}{Re}\left(\frac{\partial^2 \Omega}{\partial R^2} + \frac{1}{R}\frac{\partial \Omega}{\partial R} - \frac{\Omega}{R^2} + \frac{\partial^2 \Omega}{\partial Z^2}\right), \tag{3.31}$$

dove $Re = u_0 r_0/\nu$ e $Ro = u_0/(2r_0\omega_0)$ è il **numero di Rossby**, rapporto fra moto assiale e rotazione: il fattore $1/(2Ro)^2$ misura quanto la rotazione pesa sul moto meridiano. Le condizioni: $\Psi = 0$ sull'asse, $\Psi = \frac{1}{2}$ sulla parete (portata adimensionale), $\Psi = \frac{1}{2}R^2$ all'ingresso, $\partial\Psi/\partial Z = 0$ all'uscita, aderenza sulla parete rotante, e sull'asse $U_{i,1} = 2\Psi_{i,2}/h^2$ (sviluppo di $\Psi \approx \frac{1}{2}UR^2$ vicino all'asse). Il risultato atteso, per $Re = 50$ e Ro decrescente da 2 a 0,2, è che a piccoli numeri di Rossby la corrente si stacca dall'asse formando una bolla di ricircolo a monte del restringimento: è il fenomeno della *rottura del vortice* (vortex breakdown), che si osserva nei vortici d'estremità delle ali a delta e nei tubi rotanti, previsto già da Yih per un fluido ideale che entra in un pozzo.

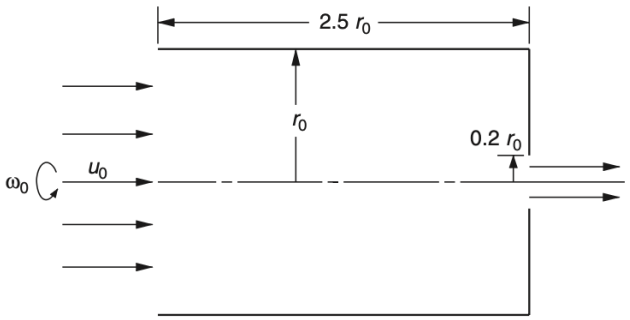


Figura 3.7: Tubo rotante con restringimento.

Capitolo 4

Schemi del secondo ordine nel tempo e sistemi tridiagonali

4.1 Perché il secondo ordine nel tempo

Gli schemi dei capitoli precedenti sono del secondo ordine nello spazio ma del *primo* nel tempo (Eulero esplicito). Per molti problemi basta, soprattutto quando interessa lo stato stazionario e il transitorio è solo un mezzo per raggiungerlo. Ma quando la dipendenza dal tempo è essa stessa il risultato (correnti turbolente e di transizione, scie oscillanti, transitori rapidi) l'accuratezza temporale conta quanto quella spaziale, e un errore $\mathcal{O}(\Delta t)$ richiederebbe passi proibitivamente piccoli. In questo capitolo si introducono due metodi del secondo ordine in entrambe le variabili, largamente usati per le correnti incomprimibili a bassa velocità: il metodo **di Adams–Bashforth** (esplicito) e quello **di Crank–Nicolson** (implicito), e il loro accoppiamento *semi-implicito* AB–CN, che sarà il motore dei risolutori di Navier–Stokes dei capitoli seguenti. Poiché la parte implicita produce sistemi tridiagonali, si presentano anche i due algoritmi che li risolvono in un numero di operazioni proporzionale alle incognite: l'algoritmo di Thomas e la fattorizzazione LU.

4.2 Il metodo di Adams–Bashforth

Si consideri l'equazione della convezione non lineare

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = 0. \tag{4.1}$$

Sviluppando in serie di Taylor intorno al nodo (i, n) (apice: livello temporale; pedice: nodo spaziale),

$$u_i^{n+1} = u_i^n + (u_t)_i^n \Delta t + (u_{tt})_i^n \frac{\Delta t^2}{2} + \mathcal{O}\left(\Delta t^3\right), \tag{4.2}$$

e approssimando la derivata seconda con una differenza all'indietro,

$$(u_{tt})_i^n = \frac{(u_t)_i^n - (u_t)_i^{n-1}}{\Delta t} + \mathcal{O}(\Delta t), \tag{4.3}$$

si ottiene, dopo aver diviso per Δt,

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{3}{2}\left(\frac{\partial u}{\partial t}\right)_i^n - \frac{1}{2}\left(\frac{\partial u}{\partial t}\right)_i^{n-1} + \mathcal{O}\left(\Delta t^2\right). \tag{4.4}$$

Definendo $H \equiv \partial u/\partial t = -u\,\partial u/\partial x$,

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{3}{2}H_i^n - \frac{1}{2}H_i^{n-1} + \mathcal{O}(\Delta t^2) : \tag{4.5}$$

è il **metodo di Adams–Bashforth** (AB) del secondo ordine. È esplicito (il nuovo valore si calcola direttamente) e con differenze centrate nello spazio è $\mathcal{O}(\Delta t^2, \Delta x^2)$; richiede però *due* livelli di dati, n e $n-1$: al primo passo si usa un metodo a un livello, Eulero esplicito $(u_i^{n+1} - u_i^n)/\Delta t = H_i^n$, che è del primo ordine ma solo per quel passo.

La debole instabilità di Adams–Bashforth

Applicato all'equazione lineare $u_t = \lambda u$ (con λ l'autovalore dell'operatore spaziale discretizzato: per la convezione con differenze centrate $\lambda = -\mathrm{i}(u/\Delta x)\sin\beta$ è *immaginario puro*), il metodo AB dà la ricorrenza $u^{n+1} = u^n + z(\frac{3}{2}u^n - \frac{1}{2}u^{n-1})$ con $z = \lambda\Delta t$. Cercando $u^n = \varrho^n$ si ottiene il polinomio caratteristico

$$\varrho^2 - \left(1 + \tfrac{3}{2}z\right)\varrho + \tfrac{1}{2}z = 0,$$

con due radici: la *principale*, $\varrho_1 = 1 + z + \frac{1}{2}z^2 + \mathcal{O}(z^3)$, che approssima e^z al secondo ordine, e la *spuria*, $\varrho_2 = \frac{1}{2}z + \mathcal{O}(z^2)$, piccola, introdotta dal secondo livello. Per $z = \mathrm{i}\theta$ immaginario puro, sviluppando il prodotto $\varrho_1\bar{\varrho}_1$ fino al quarto ordine (i termini del secondo ordine si cancellano esattamente):

$$|\varrho_1|^2 = 1 + \tfrac{1}{2}\theta^4 + \mathcal{O}(\theta^6) > 1.$$

Il modulo della radice principale supera l'unità, ma solo di una quantità $\mathcal{O}(\Delta t^4)$ per passo (la fonte scrive $|\varrho| = 1 + \mathcal{O}(\Delta t^2)$ intendendo l'eccesso per unità di tempo $\theta^4/\Delta t \sim \Delta t^3$: l'ordine esatto è comunque piccolo): il metodo AB per la convezione pura è *debolmente, ma incondizionatamente, instabile*. In un calcolo lungo l'amplificazione resta limitata se il numero di Courant $C = u_{\max}\Delta t/\Delta x \le 1$, e l'aggiunta di un termine diffusivo (λ con parte reale negativa) la elimina del tutto. In pratica, con la diffusione fisica o con la viscosità artificiale, AB è usato con successo; per la convezione pura si preferisce Adams–Bashforth del terzo ordine o Runge–Kutta, le cui regioni di stabilità contengono un tratto dell'asse immaginario. L'Esercizio 6 e il Programma 4.4 mostrano il comportamento di AB sull'equazione di Burgers, dove la diffusione lo stabilizza.

4.3 L'equazione di Burgers e le due condizioni di stabilità

Un modello più realistico di Navier–Stokes è l'equazione di Burgers, convezione non lineare più diffusione:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}. \tag{4.6}$$

Con $H \equiv \nu u_{xx} - u u_x$ e il metodo AB con differenze centrate si ha ancora la (4.5), ora $\mathcal{O}(\Delta t^2, \Delta x^2)$. Devono essere soddisfatte *due* condizioni di stabilità, convettiva e diffusiva, ricavate dall'analisi di von Neumann:

$$C = \frac{u_{\max}\Delta t}{\Delta x} \le 1, \qquad d \equiv \frac{\nu\Delta t}{\Delta x^2} \le \frac{1}{2}, \tag{4.7}$$

e si sceglie il Δt più piccolo. La differenza è essenziale: raffinando la griglia, il Δt convettivo decresce *linearmente* con Δx, quello diffusivo *quadraticamente*. Dove servono griglie fini (strati

limite, strati di taglio) la condizione diffusiva diventa di gran lunga la più severa: con Δx dimezzato il passo si divide per quattro. Per rilassarla si tratta il termine diffusivo in modo *implicito*: essendo lineare a viscosità costante, produce un sistema a banda risolubile con metodi diretti efficienti.

4.4 Il metodo di Crank–Nicolson

Per l'equazione della diffusione $u_t = \nu u_{xx}$ si usa la differenza "trapezoidale". Dagli sviluppi in avanti e all'indietro

$$u_i^{n+1} = u_i^n + (u_t)_i^n \Delta t + (u_{tt})_i^n \frac{\Delta t^2}{2} + (u_{ttt})_i^n \frac{\Delta t^3}{6} + \dots \tag{4.8}$$

$$u_i^n = u_i^{n+1} - (u_t)_i^{n+1} \Delta t + (u_{tt})_i^{n+1} \frac{\Delta t^2}{2} - (u_{ttt})_i^{n+1} \frac{\Delta t^3}{6} + \dots \tag{4.9}$$

sottraendo la seconda dalla prima e usando $(u_{tt})^{n+1} = (u_{tt})^n + \Delta t (u_{ttt})^n + \mathcal{O}(\Delta t^2)$, i termini in u_{tt} si cancellano e resta

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{1}{2}(u_t)_i^{n+1} + \frac{1}{2}(u_t)_i^n + \mathcal{O}(\Delta t^2) : \tag{4.10}$$

la media delle derivate ai due livelli (regola del trapezio) è del secondo ordine. Sostituendo $u_t = \nu u_{xx}$ e discretizzando con differenze centrate,

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{\nu}{2\Delta x^2} \left(u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1} + u_{i+1}^n - 2u_i^n + u_{i-1}^n \right) + \mathcal{O}(\Delta t^2, \Delta x^2) , \tag{4.11}$$

cioè, con $d = \nu \Delta t / \Delta x^2$,

$$-\frac{d}{2} u_{i+1}^{n+1} + (1+d) u_i^{n+1} - \frac{d}{2} u_{i-1}^{n+1} = \frac{d}{2} u_{i+1}^n - (d-1) u_i^n + \frac{d}{2} u_{i-1}^n : \tag{4.12}$$

un sistema tridiagonale a ogni passo. Il metodo di **Crank–Nicolson** (CN), già incontrato nel modulo precedente, è incondizionatamente stabile nel senso di von Neumann, per la diffusione come per la convezione.

Applicato all'equazione di Burgers completa, CN richiederebbe il termine convettivo al livello $n+1$,

$$\frac{1}{2} \left(u \frac{\partial u}{\partial x} \right)^{n+1} = \frac{1}{4\Delta x} u_i^{n+1} \left(u_{i+1}^{n+1} - u_{i-1}^{n+1} \right) , \tag{4.13}$$

che è *non lineare* nelle incognite: servirebbe una linearizzazione locale (si "ritarda" il coefficiente u_i^{n+1} al livello n in un passo predittore e lo si aggiorna in un correttore; una iterazione basta di solito) a costo di un intero ciclo di calcolo in più, pesante nei problemi multidimensionali.

4.5 Lo schema semi-implicito AB–CN

La via più pratica e diffusa per Navier–Stokes è trattare i termini *convettivi* con Adams–Bashforth, esplicito, e quelli *diffusivi* con Crank–Nicolson, implicito: lo schema **AB–CN**. Il sistema da risolvere a ogni passo resta *lineare* (nessuna iterazione né linearizzazione), e la restrizione diffusiva sul passo scompare: resta solo quella convettiva, $C \leq 1$, che nei problemi

a parete, dove la griglia è fine vicino alle pareti ma le velocità vi sono piccole, è molto meno limitante. Con il termine convettivo in forma conservativa,

$$H = -u\frac{\partial u}{\partial x} = -\frac{1}{2}\frac{\partial u^2}{\partial x}, \qquad \frac{\partial u}{\partial t} = H + \nu\frac{\partial^2 u}{\partial x^2}, \tag{4.14}$$

si scrive

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{3}{2}H_i^n - \frac{1}{2}H_i^{n-1} + \frac{1}{2}\nu\left(\frac{\partial^2 u}{\partial x^2}\right)^{n+1} + \frac{1}{2}\nu\left(\frac{\partial^2 u}{\partial x^2}\right)^{n} + \mathcal{O}\left(\Delta t^2, \Delta x^2\right), \tag{4.15}$$

e, portando a sinistra l'incognita,

$$u_i^{n+1} - \frac{1}{2}\nu\Delta t\left(\frac{\partial^2 u}{\partial x^2}\right)^{n+1} = u_i^n + \frac{3}{2}\Delta t H_i^n - \frac{1}{2}\Delta t H_i^{n-1} + \frac{1}{2}\nu\Delta t\left(\frac{\partial^2 u}{\partial x^2}\right)^{n} \equiv \mathrm{RHS}_i^n. \tag{4.16}$$

Con le differenze centrate:

$$-\frac{d}{2}u_{i-1}^{n+1} + (1+d)u_i^{n+1} - \frac{d}{2}u_{i+1}^{n+1} = \mathrm{RHS}_i^n, \tag{4.17}$$

$$\mathrm{RHS}_i^n = \frac{d}{2}\left(u_{i-1}^n + u_{i+1}^n\right) + (1-d)u_i^n + \frac{1}{2}\Delta t\left(3H_i^n - H_i^{n-1}\right), \qquad H_i^n = -\frac{(u^2)_{i+1}^n - (u^2)_{i-1}^n}{4\Delta x}. \tag{4.18}$$

Procedura AB–CN

1. Si genera il secondo livello di dati iniziali con Eulero esplicito sul termine convettivo (CN sulla diffusione).
2. Si valutano H_i^n e H_i^{n-1} con le differenze centrate.
3. Si calcola RHS_i^n dalla (4.18).
4. Si formano i coefficienti del primo membro della (4.17) (costanti, se ν e la griglia lo sono).
5. Si impongono le condizioni al contorno (i valori noti agli estremi passano a destra).
6. Si risolve il sistema tridiagonale.
7. Si torna al punto 2 e si avanza nel tempo.
8. Il passo è limitato dalla sola condizione convettiva $C \leq 1$, da rispettare rigorosamente.

4.6 Sistemi tridiagonali: Thomas e LU

La discretizzazione centrata della diffusione produce sistemi $\mathbf{A}\mathbf{x} = \mathbf{f}$ con matrice tridiagonale,

$$\mathbf{A} = \begin{bmatrix} a_1 & c_1 & & & \\ b_2 & a_2 & c_2 & & \\ & \ddots & \ddots & \ddots & \\ & & b_{n-1} & a_{n-1} & c_{n-1} \\ & & & b_n & a_n \end{bmatrix}. \tag{4.19}$$

4.6.1 L'algoritmo di Thomas

È l'eliminazione di Gauss che sfrutta la struttura: *eliminazione in avanti*,

$$a_j \leftarrow a_j - \frac{b_j}{a_{j-1}} c_{j-1}, \qquad f_j \leftarrow f_j - \frac{b_j}{a_{j-1}} f_{j-1}, \qquad j = 2, \dots, n, \tag{4.20}$$

e *sostituzione all'indietro*,

$$x_n = \frac{f_n}{a_n}, \qquad x_{n-j} = \frac{f_{n-j} - c_{n-j} x_{n-j+1}}{a_{n-j}}, \qquad j = 1, \dots, n-1. \tag{4.21}$$

Il numero di operazioni è proporzionale a n (circa $8n$), contro n^3 dell'eliminazione su matrice piena, e la memoria è $3n$. Attenzione: l'eliminazione sovrascrive la diagonale; se la stessa matrice serve al passo successivo, i coefficienti vanno ricalcolati o salvati.

4.6.2 La fattorizzazione LU tridiagonale

Si cerca $\mathbf{A} = \mathbf{L}\mathbf{U}$ con $\mathbf{L}$ bidiagonale inferiore (diagonale l_j, sottodiagonale b_j, la stessa di $\mathbf{A}$) e $\mathbf{U}$ bidiagonale superiore *unitaria* (diagonale 1, sopradiagonale u_j). Per una matrice 4×4,

$$\mathbf{L}\mathbf{U} = \begin{bmatrix} l_1 & l_1 u_1 & 0 & 0 \\ b_2 & b_2 u_1 + l_2 & l_2 u_2 & 0 \\ 0 & b_3 & b_3 u_2 + l_3 & l_3 u_3 \\ 0 & 0 & b_4 & b_4 u_3 + l_4 \end{bmatrix},$$

e il confronto elemento per elemento con $\mathbf{A}$ dà l'algoritmo

$$l_1 = a_1, \qquad u_{j-1} = \frac{c_{j-1}}{l_{j-1}}, \qquad l_j = a_j - b_j u_{j-1}, \qquad j = 2, \dots, n. \tag{4.22}$$

La soluzione di $\mathbf{L}\mathbf{U}\mathbf{x} = \mathbf{f}$ procede in due sostituzioni: $\mathbf{L}\mathbf{z} = \mathbf{f}$ (in avanti) e $\mathbf{U}\mathbf{x} = \mathbf{z}$ (all'indietro),

$$z_1 = \frac{f_1}{l_1}, \qquad z_j = \frac{f_j - b_j z_{j-1}}{l_j}, \qquad j = 2, \dots, n, \tag{4.23}$$

$$x_n = z_n, \qquad x_j = z_j - u_j x_{j+1}, \qquad j = n-1, \dots, 1. \tag{4.24}$$

Il vantaggio è evidente quando la matrice è costante nel tempo: la fattorizzazione (4.22) ($\sim 3n$ operazioni) si fa una volta sola, e a ogni passo restano solo le sostituzioni (4.23)–(4.24) ($\sim 5n$). È il caso dello schema AB–CN a viscosità costante, e, nei capitoli seguenti, dei sistemi (non più tridiagonali, ma sparsi) dell'equazione di Poisson: la stessa idea, fattorizzare una volta e sostituire molte, è alla base dei risolutori diretti sparsi usati nei Programmi 4.5–4.6b.

Quando Thomas e LU sono stabili

L'eliminazione senza scambi di righe è sicura se la matrice è *diagonalmente dominante*, $|a_j| \geq |b_j| + |c_j|$ con disuguaglianza stretta almeno in una riga. Per induzione: se $|l_{j-1}| \geq |c_{j-1}|$ allora $|u_{j-1}| \leq 1$ e $|l_j| \geq |a_j| - |b_j||u_{j-1}| \geq |a_j| - |b_j| \geq |c_j| > 0$: nessun pivot si annulla e nessun moltiplicatore supera 1, quindi gli errori di arrotondamento non si amplificano. La matrice di Crank–Nicolson (4.17), con diagonale $1 + d$ e fuori diagonale $-d/2$, è dominante per ogni $d > 0$ ($1 + d > d$): Thomas e LU sono stabili per qualunque passo temporale, coerentemente con la stabilità incondizionata dello schema. Lo stesso vale per lo schema implicito di Eulero ($1 + 2d$ contro $2d$). Per le matrici del

laplaciano discreto con condizioni di Dirichlet la dominanza è debole (uguaglianza nelle righe interne) ma stretta nelle righe adiacenti al contorno, e questo basta.

4.7 Il Programma 4.4: Burgers con AB–CN

Il Programma 4.4 risolve l'equazione di Burgers (4.6) con $\nu = 1$ su $-6 \le x \le 6$, partendo da $t_0 = 0{,}01$ con la soluzione esatta di Benton e Platzman (Approfondimenti),

$$u(x,t) = -\frac{2\sinh x}{\cosh x - e^{-t}}, \tag{4.25}$$

che per $t \to 0$ tende a una singolarità in $x = 0$ (il denominatore si annulla) e per $t \to \infty$ a $-2\tanh x$; le condizioni di Dirichlet agli estremi sono date dalla stessa soluzione. Il programma confronta Eulero esplicito, AB esplicito e AB–CN al variare di Δt, misura l'ordine di convergenza nel tempo dello schema AB–CN e confronta i tempi di calcolo di Thomas e LU (Esercizio 6).

Dati del Programma 4.4

$\nu = 1$, 101 nodi ($\Delta x = 0{,}12$), da $t = 0{,}01$ a $t = 1$; Δt da 0,002 a 0,05 (numero di Courant da 0,03 a 0,83 con $u_{\max} = 2$; d da 0,14 a 3,5). Ordine di convergenza: 401 nodi, Δt da 0,02 a 0,00125, confronto con la soluzione a $\Delta t = 10^{-4}$ sulla stessa griglia (che isola l'errore temporale). Thomas contro LU: 2001 nodi, 1000 passi.

prog4_4_burgers_abcn.py

```
"""
prog4_4_burgers_abcn.py -- Programma 4.4: equazione di Burgers viscosa
    u_t + u u_x = nu u_xx,   -6 <= x <= 6,
con lo schema semi-implicito AB-CN: Adams-Bashforth del secondo ordine sul termine
convettivo (forma conservativa H = -(u^2/2)_x, differenze centrate) e
Crank-Nicolson sul termine diffusivo; a ogni passo un sistema tridiagonale
    -d/2 u_(i-1) + (1 + d) u_i - d/2 u_(i+1) = RHS_i,    d = nu dt/dx^2,
risolto con l'algoritmo di Thomas oppure, poiche' la matrice e' costante,
con la fattorizzazione LU calcolata una volta sola (Esercizio 6).
Soluzione esatta di Benton-Platzman (nu = 1): u = -2 sinh x/(cosh x - e^{-t}),
condizioni di Dirichlet agli estremi date dalla soluzione esatta.
Il programma confronta i tre schemi (Eulero esplicito, AB esplicito, AB-CN),
misura l'ordine di convergenza nel tempo e il costo dei risolutori.
"""
import time
import numpy as np
import matplotlib.pyplot as plt
from metodi_numerici import thomas
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

XL, XR = -6.0, 6.0
NU = 1.0


def esatta(x, t):
    return -2.0 * np.sinh(x) / (np.cosh(x) - np.exp(-t))


def lu_tridiag(a, b, c):
    """Fattorizzazione LU di una matrice tridiagonale (a sotto, b diag, c sopra):
    L bidiagonale inferiore (diag l, sotto a), U bidiagonale superiore unitaria
    (sopra u)."""
```

```
    n = len(b)
    l = np.empty(n); u = np.empty(n - 1)
    l[0] = b[0]
    for j in range(1, n):
        u[j - 1] = c[j - 1] / l[j - 1]
        l[j] = b[j] - a[j] * u[j - 1]
    return l, u


def lu_risolvi(a, l, u, f):
    n = len(l)
    z = np.empty(n); x = np.empty(n)
    z[0] = f[0] / l[0]
    for j in range(1, n):
        z[j] = (f[j] - a[j] * z[j - 1]) / l[j]
    x[-1] = z[-1]
    for j in range(n - 2, -1, -1):
        x[j] = z[j] - u[j] * x[j + 1]
    return x


def H(u, dx):
    """H = -(u^2/2)_x con differenze centrate (nodi interni)."""
    return -(u[2:]**2 - u[:-2]**2) / (4 * dx)


def integra(schema, nx, dt, t_fin, t0=0.01, risolutore="thomas"):
    """Integra da t0 (dato iniziale esatto, regolare) a t_fin. schema: 'eulero',
    'ab', 'abcn'."""
    x = np.linspace(XL, XR, nx); dx = x[1] - x[0]
    u = esatta(x, t0)
    d = NU * dt / dx**2
    n = int(round((t_fin - t0) / dt))
    t = t0
    Hprec = None
    n_int = nx - 2
    if schema == "abcn":
        a = np.full(n_int, -d / 2); b = np.full(n_int, 1 + d); c = np.full(n_int,
    -d / 2)
        if risolutore == "lu":
            l, uu = lu_tridiag(a, b, c)
    for _ in range(n):
        Hn = H(u, dx)
        lap = (u[2:] - 2 * u[1:-1] + u[:-2]) / dx**2
        if Hprec is None or schema == "eulero":
            conv = Hn                                          # Eulero esplicito
    (primo passo)
        else:
            conv = 1.5 * Hn - 0.5 * Hprec                      # Adams-Bashforth
        t_new = t + dt
        if schema in ("eulero", "ab"):
            un = u.copy()
            un[1:-1] = u[1:-1] + dt * (conv + NU * lap)
        else:
            rhs = u[1:-1] + dt * conv + 0.5 * NU * dt * lap
            rhs[0] += 0.5 * d * esatta(XL, t_new)              # condizioni di
    Dirichlet al nuovo livello
            rhs[-1] += 0.5 * d * esatta(XR, t_new)
            un = np.empty_like(u)
            un[1:-1] = thomas(a, b, c, rhs) if risolutore == "thomas" else
```

```
    lu_risolvi(a, l, uu, rhs)
        un[0] = esatta(XL, t_new); un[-1] = esatta(XR, t_new)
        Hprec = Hn
        u = un
        t = t_new
        if not np.all(np.isfinite(u)) or np.max(np.abs(u)) > 1e3:
            return x, None, t
    return x, u, t


if __name__ == "__main__":
    T_FIN = 1.0
    nx = 101
    x = np.linspace(XL, XR, nx); dx = x[1] - x[0]
    print(f"Burgers viscosa, nu = {NU}, {nx} nodi (dx = {dx:.3f}), da t = 0,01 a t = {T_FIN}")
    print("   schema    dt       C=u dt/dx   d=nu dt/dx^2   errore max")
    for schema in ("eulero", "ab", "abcn"):
        for dt in (0.002, 0.005, 0.01, 0.02, 0.05):
            x, u, t = integra(schema, nx, dt, T_FIN)
            C = 2 * dt / dx; d = NU * dt / dx**2
            err = "instabile" if u is None else f"{np.max(np.abs(u - esatta(x, t))):.3e}"
            print(f"   {schema:7s} {dt:6.3f}    {C:6.3f}       {d:6.3f}    {err}")

    print("\nOrdine di convergenza nel tempo (AB-CN, 401 nodi; errore rispetto alla soluzione con dt = 1e-4")
    print("sulla stessa griglia, che isola l'errore temporale da quello spaziale, pari a 1,4e-4):")
    x_rif, u_rif, _ = integra("abcn", 401, 1e-4, T_FIN)
    err_prec = None
    errs = []
    for dt in (0.02, 0.01, 0.005, 0.0025, 0.00125):
        x, u, t = integra("abcn", 401, dt, T_FIN)
        err = np.max(np.abs(u - u_rif))
        errs.append(err)
        print(f"   dt = {dt:8.5f}: errore {err:.3e}" + (f", ordine {np.log2(err_prec/err):.2f}" if err_prec else ""))
        err_prec = err
    print(f"   errore spaziale residuo (dt = 1e-4 contro soluzione esatta): {np.max(np.abs(u_rif - esatta(x_rif, T_FIN))):.2e}")

    print("\nThomas contro LU (fattorizzazione una volta sola), AB-CN con 2001 nodi, 1000 passi:")
    for ris in ("thomas", "lu"):
        t0 = time.perf_counter()
        x, u, t = integra("abcn", 2001, 0.001, T_FIN, risolutore=ris)
        print(f"   {ris:7s}: {time.perf_counter()-t0:6.2f} s, errore {np.max(np.abs(u - esatta(x, t))):.3e}")

    fig, ax = plt.subplots(1, 2, figsize=(7.4, 2.9))
    xe = np.linspace(XL, XR, 400)
    for t, col in zip((0.01, 0.25, 1.0), (GRIGIO, ARANCIO, BLU)):
        ax[0].plot(xe, esatta(xe, t), color=col, label=f"$t={t:g}$".replace(".", "{,}"))
    x, u, t = integra("abcn", 51, 0.02, T_FIN)
    ax[0].plot(x, u, "o", color=BLU, ms=2.5, mfc="none", label="AB--CN, 51 nodi")
    ax[0].set_xlabel("$x$"); ax[0].set_ylabel("$u$"); ax[0].legend(fontsize=7)
    dts = np.array([0.02, 0.01, 0.005, 0.0025, 0.00125])
```

```
137     ax[1].loglog(dts, errs, "o-", color=BLU, label="AB--CN")
138     ax[1].loglog(dts, errs[0] * (dts / dts[0])**2, ":", color=GRIGIO,
        label="$\\Delta t^2$")
139     ax[1].set_xlabel("$\\Delta t$"); ax[1].set_ylabel("errore massimo a $t=1$");
        ax[1].legend(fontsize=7)
140     fig.tight_layout()
141     salva(fig, "prog4_4_burgers_abcn")
```

Listato 4.1: Programma 4.4 – equazione di Burgers viscosa con lo schema semi-implicito AB–CN, algoritmo di Thomas e fattorizzazione LU (`prog4_4_burgers_abcn.py`).

Risultati del Programma 4.4

- **Stabilità.** Eulero esplicito e AB esplicito sono stabili per $\Delta t = 0{,}002$ e $0{,}005$ ($d \le 0{,}35$) e instabili da $\Delta t = 0{,}01$ ($d = 0{,}69 > 1/2$): la condizione diffusiva, non quella convettiva ($C = 0{,}17$), li ferma. AB–CN è stabile per tutti i Δt provati, fino a $d = 3{,}5$ e $C = 0{,}83$, con errore massimo fra $1{,}6 \times 10^{-3}$ e $2{,}4 \times 10^{-3}$: l'errore è dominato dalla discretizzazione spaziale, e *diminuire* Δt non lo riduce.
- **Accuratezza.** A parità di $\Delta t = 0{,}005$ l'errore di AB ($3{,}1 \times 10^{-3}$) è inferiore a quello di Eulero ($6{,}9 \times 10^{-3}$), e AB–CN ($2{,}3 \times 10^{-3}$) è il migliore.
- **Ordine di convergenza.** Sulla griglia fine, dimezzando Δt da 0,01 a 0,00125, l'errore temporale si riduce di fattori 3,8, 4,0, 4,0: ordine 1,94, 1,99, 2,01. Il secondo ordine è confermato; il primo dimezzamento (da 0,02) dà un fattore maggiore perché a $\Delta t = 0{,}02$ l'avvio con Eulero pesa ancora (Fig. 4.1).
- **Thomas contro LU.** Con 2001 nodi e 1000 passi la fattorizzazione una tantum fa risparmiare circa un quarto del tempo (3,2 s contro 2,5 s in Python puro), e le due soluzioni coincidono alla precisione di macchina: Thomas ed LU sono *lo stesso* procedimento, con la differenza che LU conserva i moltiplicatori.

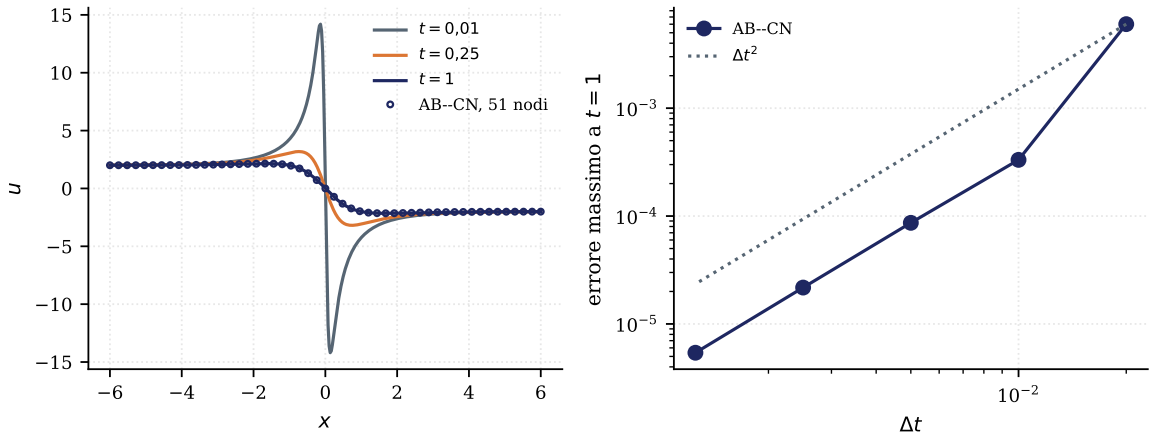


Figura 4.1: Programma 4.4: soluzione esatta di Benton–Platzman a tre istanti e soluzione AB–CN con 51 nodi (a sinistra); errore temporale in funzione di Δt con la retta di pendenza 2 (a destra).

Stabile non vuol dire accurato

Lo schema AB–CN resta stabile con $d = 3{,}5$, cioè con un passo sette volte superiore al limite esplicito, e l'errore non cresce: una tentazione a usare passi enormi. Ma la stabilità

incondizionata vale per la parte *diffusiva*; quella convettiva resta esplicita e il numero di Courant deve restare sotto 1, e per di più Crank–Nicolson con $d \gg 1$ non smorza i modi a corta lunghezza d'onda (il fattore di amplificazione tende a -1) e li trasporta inalterati, oscillanti nel tempo, come si vedrà nel Cap. 5 con la cavità a $Re = 1$. Il passo va scelto per l'accuratezza che si vuole, non per il massimo che lo schema tollera.

4.8 Approfondimenti

4.8.1 La trasformazione di Cole–Hopf e la soluzione di Benton–Platzman

L'equazione di Burgers è l'unica equazione non lineare della fluidodinamica che si risolve esattamente in generale. Cole (1951) e Hopf (1950) osservarono che la sostituzione

$$u = -2\nu \frac{\partial}{\partial x} \ln \phi = -2\nu \frac{\phi_x}{\phi} \tag{4.26}$$

trasforma la (4.6) nell'equazione *lineare* del calore $\phi_t = \nu \phi_{xx}$. *Verifica*: da $u = -2\nu\phi_x/\phi$ si calcolano $u_t = -2\nu(\phi_{xt}\phi - \phi_x\phi_t)/\phi^2$, $u_x = -2\nu(\phi_{xx}\phi - \phi_x^2)/\phi^2$ e u_{xx}; sostituendo nella (4.6) e raccogliendo si trova che essa equivale a $\partial_x[(\phi_t - \nu\phi_{xx})/\phi] = 0$, soddisfatta se ϕ risolve l'equazione del calore. Ogni soluzione dell'equazione del calore genera una soluzione di Burgers. Per $\nu = 1$ si prenda $\phi = e^t \cosh x - 1$: è soluzione, perché $\phi_t = e^t \cosh x = \phi_{xx}$, e la (4.26) dà

$$u = -2\frac{\phi_x}{\phi} = -\frac{2e^t \sinh x}{e^t \cosh x - 1} = -\frac{2 \sinh x}{\cosh x - e^{-t}},$$

cioè la (4.25). È una delle soluzioni catalogate da Benton e Platzman [31]: a $t = 0$ il denominatore si annulla in $x = 0$ e la soluzione è singolare (un "dipolo" di velocità), per $t > 0$ è regolare e tende a $-2\tanh x$, un profilo che la convezione tende a ripidire e la diffusione a spianare, in equilibrio. Per $\nu \to 0$ la stessa trasformazione fornisce la soluzione con onda d'urto, e la soluzione di Burgers converge alla soluzione debole corretta dell'equazione non viscosa.

4.8.2 Regioni di stabilità assoluta

Per un metodo multipasso applicato a $u_t = \lambda u$ la *regione di stabilità assoluta* è l'insieme dei $z = \lambda\Delta t$ nel piano complesso per cui tutte le radici del polinomio caratteristico hanno modulo ≤ 1. Per Eulero esplicito è il disco $|1 + z| \leq 1$; per AB2 è una regione ovale che tocca l'origine e si estende fino a $z = -1$ sull'asse reale negativo (diffusione: $d \leq 1/4$ per il modo più corto, $\lambda\Delta t = -4d$) ma *non* contiene alcun tratto dell'asse immaginario, da cui l'instabilità debole per la convezione; per Crank–Nicolson è l'intero semipiano $\Re z \leq 0$ (A-stabilità), con $|\varrho| \to 1$ per $|z| \to \infty$: incondizionatamente stabile ma senza smorzamento dei modi rigidi, a differenza di Eulero implicito ($\varrho = 1/(1 - z) \to 0$: L-stabilità). Lo schema AB–CN eredita la A-stabilità per la parte diffusiva e l'instabilità debole per quella convettiva; l'analisi combinata per l'equazione di convezione–diffusione (si veda Peyret e Taylor [4]) mostra che un piccolo termine diffusivo esplicito o implicito basta a riportare il modulo sotto 1 purché C non sia troppo grande, e in pratica $C \leq 0{,}5$–$0{,}8$ è sicuro.

4.8.3 L'avvio di Rannacher

Il difetto di Crank–Nicolson con dati iniziali irregolari (la singolarità di Benton–Platzman a $t \to 0$, o l'avviamento impulsivo di una parete) è l'oscillazione non smorzata dei modi corti. Un rimedio semplice e del secondo ordine è l'*avvio di Rannacher*: i primi due o quattro passi si fanno con Eulero implicito (a passo dimezzato per conservare l'ordine), che smorza fortemente i modi rigidi, e poi si passa a Crank–Nicolson. Il modulo precedente lo ha applicato all'avviamento della corrente in un canale; nel Cap. 5 se ne vedrà la necessità con la cavità a $Re = 1$, dove AB–CN con $d = 33$ non raggiunge un residuo piccolo proprio per queste oscillazioni.

Capitolo 5

Variabili primitive: griglia sfalsata e passi frazionari

5.1 Il problema della pressione

Nelle formulazioni dei capitoli precedenti la pressione era scomparsa prendendo il rotore. In **variabili primitive** (velocità e pressione) essa resta, ma con un ruolo singolare: non ha un'equazione di evoluzione propria. Le equazioni del fluido incomprimibile, adimensionalizzate con la lunghezza L e la velocità U_0,

$$\nabla \cdot \mathbf{V} = 0, \tag{5.1}$$

$$\frac{\partial \mathbf{V}}{\partial t} + \mathbf{V} \cdot \nabla \mathbf{V} = -\nabla P + \frac{1}{Re} \nabla^2 \mathbf{V}, \tag{5.2}$$

con $P = p/\rho$ e $Re = U_0 L/\nu$, sono tre equazioni per tre incognite (u, v, P), ma la continuità è un *vincolo* istantaneo sulla velocità, non un'equazione per P: la pressione è il moltiplicatore di Lagrange che costringe il campo di velocità a restare solenoidale. Un buon risolutore deve soddisfare la (5.1) *alla precisione di macchina* in ogni punto e a ogni istante: la conservazione della massa non ammette approssimazioni. Il modo più efficiente per imporlo è il **metodo a passi frazionari** (o di proiezione), di cui esistono molte varianti (Chorin [19], Kim e Moin [20], Orszag e Kells); qui si segue quella di Huser e Biringen [21]. Un'alternativa diffusa nei codici commerciali, non trattata, è l'algoritmo SIMPLE di Patankar e Spalding [23], predittore–correttore basato sui volumi finiti su griglia sfalsata. Si definisce il termine convettivo in forma conservativa, $\mathbf{H} = \mathbf{V} \cdot \nabla \mathbf{V} = \nabla \cdot (\mathbf{V}\mathbf{V})$ (identità che vale per la (5.1)).

5.2 Il metodo a passi frazionari

Nel *passo predittore*, da t^n a un livello intermedio $\hat{t}$, si trascura il gradiente di pressione:

$$\frac{\hat{\mathbf{V}} - \mathbf{V}^n}{\Delta t} = -\frac{3}{2}\mathbf{H}^n + \frac{1}{2}\mathbf{H}^{n-1} + \frac{1}{2Re}\nabla^2 \left(\hat{\mathbf{V}} + \mathbf{V}^n\right), \tag{5.3}$$

con Adams–Bashforth sulla convezione e Crank–Nicolson sulla diffusione. L'equazione completa da t^n a t^{n+1} sarebbe

$$\frac{\mathbf{V}^{n+1} - \mathbf{V}^n}{\Delta t} = -\frac{3}{2}\mathbf{H}^n + \frac{1}{2}\mathbf{H}^{n-1} + \frac{1}{2Re}\nabla^2 \left(\mathbf{V}^{n+1} + \mathbf{V}^n\right) - \nabla P^{n+1}. \tag{5.4}$$

Sottraendo si ottiene il *passo correttore*:

$$\frac{\mathbf{V}^{n+1}-\hat{\mathbf{V}}}{\Delta t}=\frac{1}{2Re}\nabla^2\left(\mathbf{V}^{n+1}-\hat{\mathbf{V}}\right)-\nabla P^{n+1}\equiv-\nabla\phi^{n+1}. \tag{5.5}$$

Lo scalare ϕ^{n+1} contiene la pressione e il residuo viscoso; è semplicemente l'operatore che rende solenoidale il campo al livello $n+1$. Prendendo la divergenza della (5.5) e imponendo $\nabla\cdot\mathbf{V}^{n+1}=0$:

$$\nabla^2\phi^{n+1}=\frac{\nabla\cdot\hat{\mathbf{V}}}{\Delta t}. \tag{5.6}$$

Perché funziona: la decomposizione di Helmholtz–Hodge

Ogni campo vettoriale $\hat{\mathbf{V}}$ abbastanza regolare su un dominio limitato D si decompone in modo unico nella somma di un campo solenoidale tangente al contorno e del gradiente di uno scalare:

$$\hat{\mathbf{V}}=\mathbf{V}^{n+1}+\Delta t\,\nabla\phi,\qquad \nabla\cdot\mathbf{V}^{n+1}=0,\qquad \mathbf{V}^{n+1}\cdot\mathbf{n}=0\ \text{su}\ \partial D.$$

Esistenza: si prende ϕ soluzione del problema di Neumann $\Delta t\nabla^2\phi=\nabla\cdot\hat{\mathbf{V}}$ con $\Delta t\,\partial\phi/\partial n=\hat{\mathbf{V}}\cdot\mathbf{n}$ sul contorno (risolubile perché $\int_D\nabla\cdot\hat{\mathbf{V}}\,\mathrm{d}D=\oint\hat{\mathbf{V}}\cdot\mathbf{n}\,\mathrm{d}l$, teorema della divergenza); allora $\mathbf{V}^{n+1}=\hat{\mathbf{V}}-\Delta t\nabla\phi$ ha divergenza nulla e componente normale nulla. *Unicità e ortogonalità*: le due componenti sono ortogonali nel prodotto scalare $\int_D\mathbf{a}\cdot\mathbf{b}\,\mathrm{d}D$, perché $\int_D\mathbf{V}^{n+1}\cdot\nabla\phi\,\mathrm{d}D=\oint\phi\,\mathbf{V}^{n+1}\cdot\mathbf{n}\,\mathrm{d}l-\int_D\phi\nabla\cdot\mathbf{V}^{n+1}\mathrm{d}D=0$; una decomposizione in componenti ortogonali è unica. Il passo correttore (5.5) è dunque la *proiezione ortogonale* di $\hat{\mathbf{V}}$ sul sottospazio dei campi solenoidali: fra tutti i campi a divergenza nulla, $\mathbf{V}^{n+1}$ è il più vicino a $\hat{\mathbf{V}}$ in norma L^2. Da qui il nome di *metodo di proiezione* (Chorin, 1968). Il prezzo è un errore di separazione (splitting) fra pressione e viscosità, $\mathcal{O}(\Delta t)$ in uno strato sottile vicino alle pareti (Approfondimenti).

Le tre fasi del metodo a passi frazionari

1. **Predittore.** Si risolve la (5.3) per $\hat{\mathbf{V}}$:

$$\frac{\hat{\mathbf{V}}}{\Delta t}-\frac{1}{2Re}\nabla^2\hat{\mathbf{V}}=-\frac{3}{2}\mathbf{H}^n+\frac{1}{2}\mathbf{H}^{n-1}+\frac{1}{2Re}\nabla^2\mathbf{V}^n+\frac{\mathbf{V}^n}{\Delta t}. \tag{5.7}$$

   Per la componente x, con differenze centrate e $\Delta x=\Delta y=h$,

$$-\hat{u}_{i+1,j}-\hat{u}_{i-1,j}-\hat{u}_{i,j+1}-\hat{u}_{i,j-1}+(4+\beta_1)\hat{u}_{i,j}=(\mathrm{RHS}_u^n)_{i,j},\qquad \beta_1^{-1}\equiv\frac{\Delta t}{2Reh^2}, \tag{5.8}$$

   con $\mathrm{RHS}_u^n=-\frac{3}{2}\Delta t\beta_1H_u^n+\frac{1}{2}\Delta t\beta_1H_u^{n-1}+(u_{i+1,j}^n-2u_{i,j}^n+u_{i-1,j}^n+u_{i,j+1}^n-2u_{i,j}^n+u_{i,j-1}^n)+\beta_1u_{i,j}^n$: un'equazione ellittica di Helmholtz, risolubile con i metodi iterativi del secondo modulo o, per griglie fino a circa 100×100, con un metodo diretto (LU). Analogamente per $\hat{v}$.

2. **Pressione.** Si risolve la (5.6) con differenze centrate,

$$\phi_{i+1,j}+\phi_{i-1,j}+\phi_{i,j+1}+\phi_{i,j-1}-4\phi_{i,j}=\frac{h^2}{\Delta t}\left(\frac{\partial\hat{u}}{\partial x}+\frac{\partial\hat{v}}{\partial y}\right)_{i,j}, \tag{5.9}$$

   con condizioni di Neumann su tutto il contorno: il sistema è singolare (Sez. 5.6).

3. **Correzione.** Si aggiorna la velocità con la (5.5):

$$u^{n+1}_{i+1/2,j} = \hat{u}_{i+1/2,j} - \Delta t \left(\frac{\partial \phi}{\partial x}\right)^{n+1}_{i,j}, \qquad v^{n+1}_{i,j+1/2} = \hat{v}_{i,j+1/2} - \Delta t \left(\frac{\partial \phi}{\partial y}\right)^{n+1}_{i,j}. \quad (5.10)$$

Gli indici semi-interi si riferiscono alla griglia sfalsata introdotta nel seguito.

5.3 Griglia collocata e griglia sfalsata

La discretizzazione segue la griglia sfalsata (*staggered*) del metodo MAC (*marker-and-cell*) di Harlow e Welch [22]. Il motivo si capisce su un problema unidimensionale ridotto ai soli termini temporale e di pressione, $u_t = -P_x$, con continuità $u_x = 0$. Su una griglia *collocata*, in cui u e P stanno negli stessi nodi (Fig. 5.1), con integrazione implicita e differenze centrate,

$$u^{n+1}_i = u^n_i - \frac{\Delta t}{2\Delta x}\left(P^{n+1}_{i+1} - P^{n+1}_{i-1}\right); \quad (5.11)$$

derivando in x e imponendo $(u_x)^{n+1}_i = 0$ si ottiene $\Delta t\, P_{xx} = u_x$, cioè, con differenze centrate,

$$P^{n+1}_{i+1} - 2P^{n+1}_i + P^{n+1}_{i-1} = \frac{\Delta x}{2\Delta t}\left(u^n_{i+1} - u^n_{i-1}\right). \quad (5.12)$$

Nella (5.12) P_i non dipende da u_i e nella (5.11) u_i non dipende da P_i: i nodi pari e dispari sono *disaccoppiati*. Un campo di pressione a scacchiera, $P_i = (-1)^i$, ha gradiente centrato nullo ed è invisibile alle equazioni: può crescere indisturbato e produce le oscillazioni di alta frequenza che richiedono viscosità artificiale per essere soppresse.

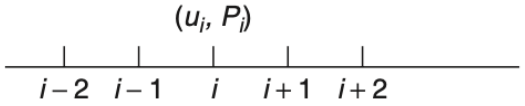


Figura 5.1: Griglia collocata.

Sulla griglia *sfalsata* (Fig. 5.2) le velocità stanno a metà fra i nodi di pressione. L'equazione del moto scritta in $i + \frac{1}{2}$,

$$u^{n+1}_{i+1/2} = u^n_{i+1/2} - \frac{\Delta t}{\Delta x}\left(P^{n+1}_{i+1} - P^{n+1}_i\right), \quad (5.13)$$

derivata e con $(u_x)^{n+1}_{i+1/2} = 0$ dà

$$P^{n+1}_{i+1} - 2P^{n+1}_i + P^{n+1}_{i-1} = \frac{\Delta x}{\Delta t}\left(u^n_{i+1/2} - u^n_{i-1/2}\right): \quad (5.14)$$

i nodi P in i e i nodi u in $i \pm \frac{1}{2}$ sono sempre accoppiati, e il modo a scacchiera ha gradiente non nullo. A numeri di Reynolds alti con differenze centrate possono restare oscillazioni, ma di ampiezza minore e a Re più alti che su griglia collocata.

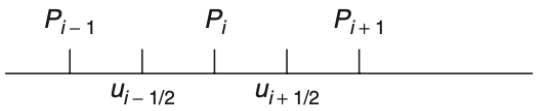


Figura 5.2: Griglia sfalsata.

5.4 Forma conservativa e discretizzazione MAC

I termini convettivi si scrivono in forma conservativa, che conserva massa e quantità di moto anche nella forma discreta (Roache [3]): dall'identità $\mathbf{V}\cdot\nabla\mathbf{V} = \nabla\cdot(\mathbf{V}\mathbf{V}) - \mathbf{V}\nabla\cdot\mathbf{V}$ e dalla continuità,

$$\frac{\partial u}{\partial t}+\frac{\partial u^2}{\partial x}+\frac{\partial (uv)}{\partial y}=\frac{1}{Re}\left(\frac{\partial^2 u}{\partial x^2}+\frac{\partial^2 u}{\partial y^2}\right)-\frac{\partial P}{\partial x},\qquad \frac{\partial v}{\partial t}+\frac{\partial (uv)}{\partial x}+\frac{\partial v^2}{\partial y}=\frac{1}{Re}\left(\frac{\partial^2 v}{\partial x^2}+\frac{\partial^2 v}{\partial y^2}\right)-\frac{\partial P}{\partial y}. \tag{5.15}$$

Nella configurazione MAC (Fig. 5.3) la pressione sta al centro della cella (i,j), u sulle facce verticali $(i\pm\frac{1}{2},j)$ e v su quelle orizzontali $(i,j\pm\frac{1}{2})$. L'equazione di u si scrive nei nodi u, quella di v nei nodi v:

$$\left(\frac{\partial u}{\partial t}\right)_{i+1/2,j}=\frac{u^{n+1}_{i+1/2,j}-u^{n}_{i+1/2,j}}{\Delta t},\qquad \left(\frac{\partial P}{\partial x}\right)_{i+1/2,j}=\frac{P_{i+1,j}-P_{i,j}}{\Delta x}, \tag{5.16}$$

$$\left(\frac{\partial^2 u}{\partial x^2}\right)_{i+1/2,j}=\frac{u_{i-1/2,j}-2u_{i+1/2,j}+u_{i+3/2,j}}{\Delta x^2}, \tag{5.17}$$

$$\left(\frac{\partial u^2}{\partial x}\right)_{i+1/2,j}=\frac{u^2_{i+1,j}-u^2_{i,j}}{\Delta x},\qquad u_{i+1,j}=\tfrac{1}{2}\left(u_{i+1/2,j}+u_{i+3/2,j}\right), \tag{5.18}$$

$$\left(\frac{\partial (uv)}{\partial y}\right)_{i+1/2,j}=\frac{(uv)_{i+1/2,j+1/2}-(uv)_{i+1/2,j-1/2}}{\Delta y}, \tag{5.19}$$

$$u_{i+1/2,j+1/2}=\tfrac{1}{2}\left(u_{i+1/2,j}+u_{i+1/2,j+1}\right),\qquad v_{i+1/2,j+1/2}=\tfrac{1}{2}\left(v_{i,j+1/2}+v_{i+1,j+1/2}\right). \tag{5.20}$$

Le quantità che non sono definite nel nodo in cui servono si ottengono per *media*, e i prodotti sono *prodotti di medie*, non medie di prodotti: (uv) nel vertice della cella è il prodotto dei valori interpolati nel vertice. Il gradiente di pressione risulta naturalmente centrato sul nodo u con passo Δx, senza il disaccoppiamento della griglia collocata; e la divergenza nella cella (i,j), $(u_{i+1/2,j}-u_{i-1/2,j})/\Delta x+(v_{i,j+1/2}-v_{i,j-1/2})/\Delta y$, è un bilancio di flussi attraverso le quattro facce: la conservazione della massa vale *per cella*, non per nodo.

5.4.1 La versione esplicita

Per semplicità il Programma 4.5 usa Eulero esplicito sia per i termini convettivi sia per quelli viscosi, $\mathcal{O}(\Delta t, h^2)$. Il predittore per u diventa

$$\begin{aligned}\frac{\hat{u}_{i+1/2,j}-u^n_{i+1/2,j}}{\Delta t}=&\frac{(u^n_{i,j})^2-(u^n_{i+1,j})^2}{\Delta x}+\frac{(uv)^n_{i+1/2,j-1/2}-(uv)^n_{i+1/2,j+1/2}}{\Delta y}\\&+\frac{u^n_{i+3/2,j}-2u^n_{i+1/2,j}+u^n_{i-1/2,j}}{Re\Delta x^2}+\frac{u^n_{i+1/2,j+1}-2u^n_{i+1/2,j}+u^n_{i+1/2,j-1}}{Re\Delta y^2},\end{aligned} \tag{5.21}$$

e analogamente per $\hat{v}$. In questo caso avanzando da t^n a $\hat{t}$ senza pressione e da t^n a t^{n+1} con la pressione, i termini convettivi e viscosi, valutati entrambi al livello n, si cancellano *esattamente*

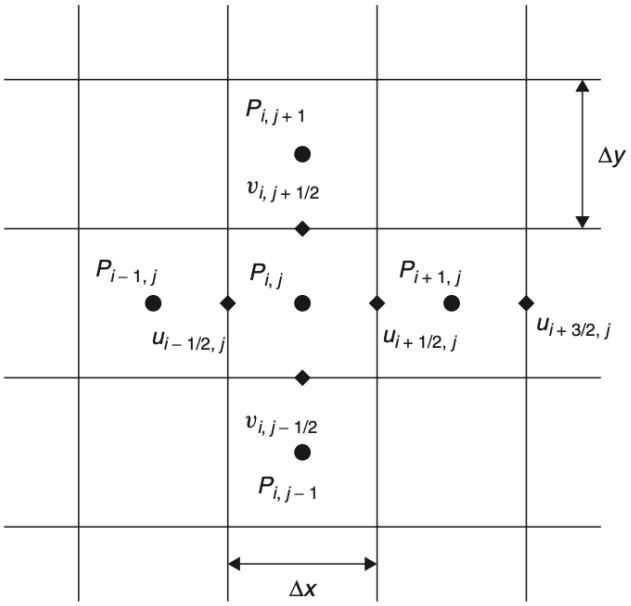


Figura 5.3: La griglia sfalsata MAC.

nella differenza:

$$\frac{\mathbf{V}^{n+1} - \hat{\mathbf{V}}}{\Delta t} = -\nabla P^{n+1}, \qquad \nabla^2 P^{n+1} = \frac{\nabla \cdot \hat{\mathbf{V}}}{\Delta t} : \tag{5.22}$$

lo scalare che proietta è la *pressione termodinamica* stessa (a meno della densità), e non serve ricavarla da ϕ come nello schema semi-implicito.

5.4.2 Stabilità

Per il risolutore esplicito valgono le condizioni pratiche di Peyret e Taylor [4],

$$\tfrac{1}{4}(|u| + |v|)^2 \Delta t Re \le 1, \qquad \frac{\Delta t}{Re\,\Delta x^2} \le \tfrac{1}{4} \quad (\Delta x = \Delta y), \tag{5.23}$$

oppure si controllano le due quantità

$$c_1 = \max_{i,j} \left(\frac{|u_{i,j}|}{\Delta x_i} + \frac{|v_{i,j}|}{\Delta y_j} \right) \Delta t \lesssim 0{,}5, \qquad c_2 = \max_{i,j} \frac{\Delta t}{Re} \left(\frac{1}{\Delta x_i^2} + \frac{1}{\Delta y_j^2} \right) \lesssim 0{,}1. \tag{5.24}$$

La seconda delle (5.23) è la condizione diffusiva bidimensionale ($d \le 1/4$: con due direzioni il limite si dimezza rispetto a $1/2$); la prima combina convezione e diffusione: per la convezione–diffusione lineare con FTCS la condizione è $u^2 \Delta t/(2\nu) \le 1$ nella direzione della velocità, cioè $C^2 \le 2d$, ed è questa che limita il passo a Re alti. Il Programma 4.5 usa $\Delta t = 0{,}4 \min\{Re h^2/4,\ h/(2u_{\max})\}$.

5.5 Condizioni al contorno sulla griglia sfalsata

Con il metodo a passi frazionari non servono condizioni al contorno esplicite sulla pressione, ma quelle sulle velocità sono più delicate che su griglia collocata, perché le componenti *parallele*

a una parete non hanno nodi sulla parete. Vicino a una parete orizzontale (Fig. 5.4) la velocità u di parete, u_Γ, si impone con la media fra il primo nodo interno e un *nodo fittizio* esterno:

$$u_\Gamma = \tfrac{1}{2}\left(u_{i+1/2,1} + u_{i+1/2,-1}\right) \quad \Rightarrow \quad u_{i+1/2,-1} = 2u_\Gamma - u_{i+1/2,1}, \tag{5.25}$$

equivalente a un'estrapolazione lineare ($\mathcal{O}(h^2)$); sono possibili estrapolazioni di ordine superiore, come $u_{i+1/2,-1} = \frac{1}{3}(u_{i+1/2,2} - 6u_{i+1/2,1} + 8u_\Gamma)$. Sulla parete orizzontale esistono invece i nodi v, dove $v = 0$ si impone direttamente; su una parete verticale i ruoli si scambiano. Il nodo fittizio entra nel termine viscoso $\partial^2 u/\partial y^2$ della prima fila di nodi u e, attraverso le medie, nei termini convettivi: è così che il moto del coperchio ($u_\Gamma = 1$) si trasmette al fluido.

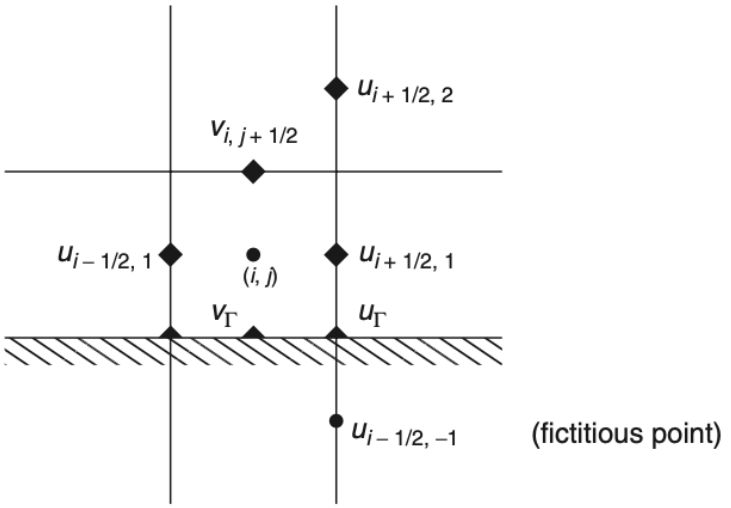


Figura 5.4: Condizioni al contorno per u su una parete orizzontale.

5.5.1 Le condizioni sulla pressione

Dalla (5.22), sulle pareti dove $\mathbf{V}^{n+1} = \mathbf{V}^n = \hat{\mathbf{V}}$ (la velocità normale è assegnata e non cambia), segue $\partial P^{n+1}/\partial n = 0$: condizioni di **Neumann omogenee** su tutto il contorno, la cui accuratezza è discussa da Gresho e da Kim e Moin [20]. Nella cella adiacente a una parete orizzontale (Fig. 5.5) l'equazione di Poisson si scrive con il nodo fittizio $P_{i,-1} = P_{i,1}$ e con $\hat{v}_\Gamma = v_\Gamma^{n+1} = 0$:

$$\frac{P_{i,2} - 2P_{i,1} + P_{i,-1}}{\Delta y^2} + \frac{P_{i+1,1} - 2P_{i,1} + P_{i-1,1}}{\Delta x^2} = \frac{1}{\Delta t}\left(\frac{\hat{u}_{i+1/2,1} - \hat{u}_{i-1/2,1}}{\Delta x} + \frac{\hat{v}_{i,3/2} - \hat{v}_\Gamma}{\Delta y}\right), \tag{5.26}$$

e analogamente per una parete verticale.

5.6 Il problema di Neumann per la pressione

Un'equazione ellittica con condizioni di Neumann su tutto il contorno è *indeterminata*: la matrice del sistema ha un autovalore nullo, le equazioni sono linearmente dipendenti e il sistema non ha soluzione unica (Mitchell [6]). La soluzione è definita a meno di una costante additiva, che non ha effetto sul gradiente di pressione, l'unica cosa che compare nell'equazione del moto. Il rimedio è fissare la pressione in un nodo di riferimento ($P = 0$): il sistema diventa non singolare e la soluzione è unica a meno di quella costante.

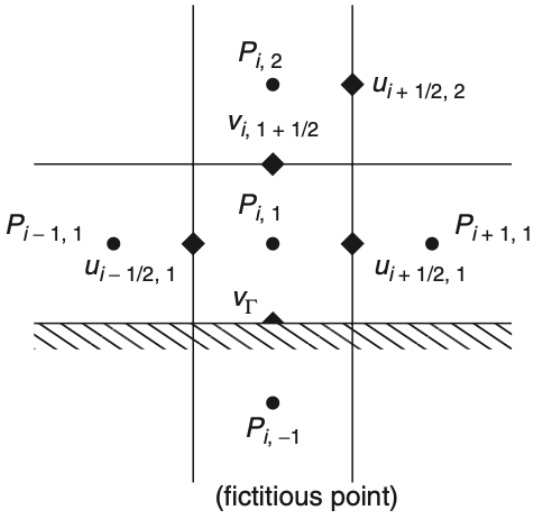


Figura 5.5: Condizioni al contorno per la pressione.

La condizione di compatibilità

Per $\nabla^2\phi = f$ in S con $\partial\phi/\partial n = g$ sul contorno L, il teorema della divergenza impone

$$\int_S f \,\mathrm{d}S = \int_S \nabla^2\phi \,\mathrm{d}S = \oint_L \frac{\partial\phi}{\partial n}\,\mathrm{d}l = \oint_L g \,\mathrm{d}l : \tag{5.27}$$

se termine noto e dati al contorno non soddisfano questa *condizione di compatibilità*, il problema non ha soluzione. Nella forma discreta: sommando tutte le equazioni (5.9) con le condizioni di Neumann, i termini ϕ si cancellano a coppie (ogni $\phi_{i,j}$ compare con $+1$ nelle equazioni dei quattro vicini e con -4 nella propria; con i nodi fittizi uguali agli interni il conto torna anche al bordo) e resta $\sum_{i,j} h^2 f_{i,j} = 0$: il termine noto deve avere somma nulla, cioè deve essere ortogonale al vettore nullo $(1, 1, \ldots, 1)$ della matrice. Sulla griglia MAC la condizione è soddisfatta *automaticamente*: il termine noto è la divergenza discreta di $\hat{\mathbf{V}}$, somma di flussi attraverso le facce; sommando su tutte le celle i flussi interni si cancellano a coppie e restano i flussi attraverso il contorno, dove la velocità normale è quella di parete ($\hat{v}_\Gamma = 0$): $\sum_{i,j} \nabla \cdot_h \hat{\mathbf{V}} = 0$ identicamente. Su griglia collocata, invece, il termine noto va corretto a mano.

Un sistema 4×4

Per il quadrato unitario con $N = 2$ (quattro nodi interni, Fig. 5.6) le equazioni $\phi_{i+1,j} + \phi_{i-1,j} + \phi_{i,j+1} + \phi_{i,j-1} - 4\phi_{i,j} = h^2 f_{i,j}$, con i nodi di contorno eliminati tramite le differenze in avanti $\phi_{1,0} = \phi_{1,1} - h g_{1,0}$ ecc., diventano

$$\begin{bmatrix} -2 & 1 & 1 & 0 \\ 1 & -2 & 0 & 1 \\ 1 & 0 & -2 & 1 \\ 0 & 1 & 1 & -2 \end{bmatrix} \begin{bmatrix} \phi_{11} \\ \phi_{12} \\ \phi_{21} \\ \phi_{22} \end{bmatrix} = \begin{bmatrix} r_{11} \\ r_{12} \\ r_{21} \\ r_{22} \end{bmatrix}, \qquad r_{11} = h^2 f_{11} + h g_{01} + h g_{10}, \ \ldots$$

Gli autovalori della matrice sono -4, -2, -2 e 0 (l'Esercizio 9 li calcola; la fonte riporta -4 per i primi tre, un refuso): uno nullo, matrice singolare di rango 3, con vettore nullo $(1, 1, 1, 1)$. Fissando $\phi_{11} = 0$ (prima riga sostituita da $\phi_{11} = 0$) gli autovalori

diventano $-0{,}5858$, -2, $-3{,}4142$ (e 1 per la riga fittizia): tutti non nulli, soluzione unica, ma il numero di condizionamento resta alto (rcond $= 0{,}0625$) e cresce con la griglia molto più che per il problema di Dirichlet: per $N = 32$ il rapporto fra gli autovalori estremi è $1{,}7 \times 10^4$ contro 440. I metodi iterativi convergono lentamente e, per griglie moderate, conviene un metodo diretto sparso con fattorizzazione una tantum: è la scelta dei Programmi 4.5–4.6b.

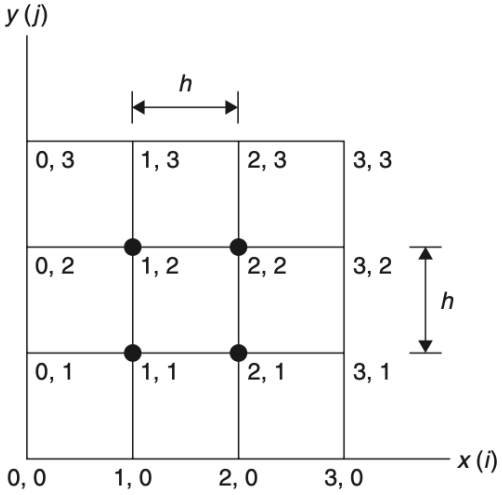


Figura 5.6: Schema per il problema di Neumann su quattro nodi.

5.7 Il Programma 4.5: la cavità con coperchio mobile

La cavità quadrata il cui coperchio scorre con velocità U (Fig. 5.7) è il problema di riferimento della fluidodinamica numerica incomprimibile: geometria banale, condizioni al contorno semplici, ma una soluzione ricca (vortice principale, vortici secondari negli spigoli, singolarità negli spigoli superiori dove u vale 0 e 1 a distanza nulla) e dati di confronto accuratissimi, quelli di Ghia, Ghia e Shin [24] su griglia 129×129. Nel modulo precedente la si era risolta nel limite di Stokes; qui $Re = UL/\nu$ è finito.

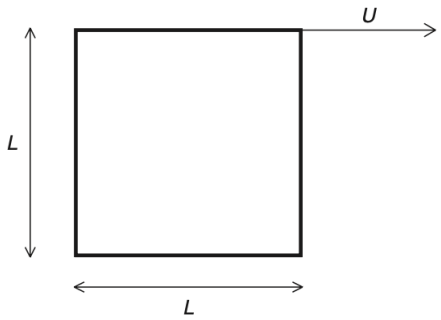


Figura 5.7: La cavità con coperchio mobile.

Struttura del Programma 4.5

Griglia MAC con N celle per lato ($h = 1/N$); array u di forma $(N+1)\times(N+2)$ (nodi u sulle facce verticali, con le due file fittizie $j = 0$ e $j = N+1$), v di forma $(N+2)\times(N+1)$, P di forma $N \times N$. A ogni passo:

1. nodi fittizi: $u_{\cdot,0} = -u_{\cdot,1}$ (fondo fermo), $u_{\cdot,N+1} = 2U - u_{\cdot,N}$ (coperchio), $v_{0,\cdot} = -v_{1,\cdot}$, $v_{N+1,\cdot} = -v_{N,\cdot}$ (pareti laterali);
2. predittore esplicito (5.21) in forma vettoriale (NumPy), con le medie (5.18)–(5.19);
3. divergenza di $\hat{\mathbf{V}}$ per cella, soluzione dell'equazione di Poisson con il risolutore diretto sparso (`scipy.sparse.linalg.splu`, fattorizzazione calcolata una sola volta) con $P = 0$ nella prima cella;
4. correzione (5.10); residuo $\max|\mathbf{V}^{n+1} - \mathbf{V}^n|/\Delta t$; si arresta quando il residuo scende sotto 10^{-6} (stato stazionario) o al tempo massimo.

Alla fine si calcolano la funzione di corrente nei vertici delle celle integrando u dal fondo, $\psi_{i,j} = \psi_{i,j-1} + h\,u_{i,j-1/2}$ (con $\psi = 0$ sulle pareti: il valore ottenuto sulla parete superiore, 10^{-16}, è una verifica della conservazione della massa), la vorticità $\omega = \partial v/\partial x - \partial u/\partial y$ nei vertici, e si confrontano ψ e ω nel centro del vortice principale e il profilo $u(0{,}5, y)$ con i dati di Ghia et al.

Dati del Programma 4.5

$Re = 1$, 100, 400 su griglia 65×65 ($N = 64$) e $Re = 1000$ su 81×81; $\Delta t = 0{,}4\min\{Re h^2/4,\ h/2\}$; tolleranza 10^{-6} sul residuo; tempi massimi 5, 30, 60, 120.

prog4_5_cavita_mac.py

```
"""
prog4_5_cavita_mac.py -- Programma 4.5: cavita' quadrata con coperchio mobile,
equazioni di Navier-Stokes in variabili primitive, griglia sfalsata MAC e metodo
a passi frazionari con avanzamento esplicito di Eulero.

Passo 1 (predizione): u^ = u^n + dt [ -( (u^2)_x + (uv)_y ) + lap u / Re ],  v^
    analoga.
Passo 2 (pressione):  lap p = div u^ / dt,  dp/dn = 0 sul contorno (equazione di
    Poisson di
                      Neumann: matrice singolare, si fissa p = 0 in una cella; la
    griglia MAC
                      soddisfa automaticamente la condizione di compatibilita').
Passo 3 (correzione): u^(n+1) = u^ - dt grad p,  con div u^(n+1) = 0 a meno
    dell'arrotondamento.
Le quantita' non definite nel nodo in cui servono sono ottenute per media; le
    condizioni di
aderenza sulle pareti parallele alla velocita' si impongono con i nodi fittizi
(u_(-1) = 2 u_parete - u_1). L'equazione di Poisson e' risolta con un metodo
    diretto sparso
(fattorizzazione LU calcolata una sola volta). Si calcolano funzione di corrente e
    vorticita'
nei vertici delle celle e si confrontano i valori al centro del vortice principale
    con
i dati di Ghia, Ghia e Shin (1982).
"""
import time
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
```

```
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

GHIA = {   # psi_min, omega al centro, x_c, y_c (Ghia, Ghia e Shin 1982, griglia 129x129)
    100: (-0.103423, -3.16646, 0.6172, 0.7344),
    400: (-0.113909, -2.29469, 0.5547, 0.6055),
    1000: (-0.117929, -2.04968, 0.5313, 0.5625),
}
# profilo u(0,5, y) di Ghia et al. per Re = 100, 400, 1000 (alcuni punti)
GHIA_U = {
    "y": [0.0, 0.0547, 0.1016, 0.1719, 0.2813, 0.4531, 0.5, 0.6172, 0.7344, 0.8516, 0.9531, 0.9766, 1.0],
    100: [0.0, -0.03717, -0.06434, -0.10150, -0.15662, -0.21090, -0.20581, -0.13641, 0.00332, 0.23151, 0.68717, 0.84123, 1.0],
    400: [0.0, -0.08186, -0.14612, -0.24299, -0.32726, -0.17119, -0.11477, 0.02135, 0.16256, 0.29093, 0.55892, 0.75837, 1.0],
    1000: [0.0, -0.18109, -0.29730, -0.38289, -0.27805, -0.10648, -0.06080, 0.05702, 0.18719, 0.33304, 0.46604, 0.65928, 1.0],
}


def matrice_pressione(N, h):
    """Laplaciano a cinque punti sulle celle di pressione con dp/dn = 0 (nodi fittizi
    uguali a quelli interni); la prima cella e' fissata a p = 0."""
    n = N * N
    idx = lambda i, j: i * N + j
    A = sp.lil_matrix((n, n))
    for i in range(N):
        for j in range(N):
            p = idx(i, j)
            diag = 0.0
            for (a, b) in ((i + 1, j), (i - 1, j), (i, j + 1), (i, j - 1)):
                if 0 <= a < N and 0 <= b < N:
                    A[p, idx(a, b)] = 1.0 / h**2
                    diag -= 1.0 / h**2
            A[p, p] = diag
    A[0, :] = 0.0; A[0, 0] = 1.0                    # livello di riferimento della pressione
    return spla.splu(A.tocsc())


def cavita(Re, N, t_max, tol=1e-6, u_cop=1.0, cfl=0.4, stampa=True, sforzo=None, u_max=1.0):
    """Griglia MAC con N celle per lato. u: (N+1) x (N+2) (colonne = nodi u in x, righe
    fittizie j = 0 e j = N+1); v: (N+2) x (N+1); p: N x N.
    Se 'sforzo' non e' None, sulla superficie superiore si impone du/dy = sforzo (superficie
    libera mossa dal vento) invece della velocita' u_cop; u_max serve per il passo temporale."""
    h = 1.0 / N
    u = np.zeros((N + 1, N + 2)); v = np.zeros((N + 2, N + 1)); p = np.zeros((N, N))
    LU = matrice_pressione(N, h)
    dt = min(cfl * 0.25 * Re * h * h, cfl * 0.5 * h / max(u_max, 1e-9))
    t, passo, res = 0.0, 0, np.inf
    storia = []
    t0 = time.perf_counter()
    while t < t_max and res > tol:
```

```
        # nodi fittizi: aderenza sulle pareti orizzontali (per u) e verticali (per v)
        u[:, 0] = -u[:, 1]                                  # fondo fermo
        u[:, N + 1] = (2 * u_cop - u[:, N]) if sforzo is None else (u[:, N] + h * sforzo)   # coperchio mobile / sforzo imposto
        v[0, :] = -v[1, :]; v[N + 1, :] = -v[N, :]    # pareti laterali ferme
        # --- predizione (Eulero esplicito, forma conservativa)
        # u-momento nei nodi u interni i = 1..N-1 (x), j = 1..N (y)
        ue = 0.5 * (u[1:, 1:-1] + u[:-1, 1:-1])           # u sui centri cella (N, N)
        duudx = (ue[1:, :]**2 - ue[:-1, :]**2) / h        # (N-1, N)
        uc = 0.5 * (u[1:-1, 1:] + u[1:-1, :-1])           # u sui vertici (N-1, N+1)
        vc = 0.5 * (v[1:-1, :][1:, :] + v[1:-1, :][:-1, :])    # v sui vertici (N-1, N+1)
        duvdy = (uc[:, 1:] * vc[:, 1:] - uc[:, :-1] * vc[:, :-1]) / h
        lapu = (u[2:, 1:-1] - 2 * u[1:-1, 1:-1] + u[:-2, 1:-1] + u[1:-1, 2:] - 2 * u[1:-1, 1:-1] + u[1:-1, :-2]) / h**2
        us = u.copy()
        us[1:-1, 1:-1] = u[1:-1, 1:-1] + dt * (-duudx - duvdy + lapu / Re)
        # v-momento nei nodi v interni i = 1..N (x), j = 1..N-1 (y)
        ve = 0.5 * (v[1:-1, 1:] + v[1:-1, :-1])           # v sui centri cella (N, N)
        dvvdy = (ve[:, 1:]**2 - ve[:, :-1]**2) / h        # (N, N-1)
        uc2 = 0.5 * (u[:, 1:-1][:, 1:] + u[:, 1:-1][:, :-1])  # u sui vertici (N+1, N-1)
        vc2 = 0.5 * (v[1:, 1:-1] + v[:-1, 1:-1])          # v sui vertici (N+1, N-1)
        duvdx = (uc2[1:, :] * vc2[1:, :] - uc2[:-1, :] * vc2[:-1, :]) / h
        lapv = (v[2:, 1:-1] - 2 * v[1:-1, 1:-1] + v[:-2, 1:-1] + v[1:-1, 2:] - 2 * v[1:-1, 1:-1] + v[1:-1, :-2]) / h**2
        vs = v.copy()
        vs[1:-1, 1:-1] = v[1:-1, 1:-1] + dt * (-duvdx - dvvdy + lapv / Re)
        # --- pressione
        div = (us[1:, 1:-1] - us[:-1, 1:-1]) / h + (vs[1:-1, 1:] - vs[1:-1, :-1]) / h
        rhs = (div / dt).ravel(); rhs[0] = 0.0
        p = LU.solve(rhs).reshape(N, N)
        # --- correzione
        un = us.copy(); vn = vs.copy()
        un[1:-1, 1:-1] -= dt * (p[1:, :] - p[:-1, :]) / h
        vn[1:-1, 1:-1] -= dt * (p[:, 1:] - p[:, :-1]) / h
        res = max(np.max(np.abs(un - u)), np.max(np.abs(vn - v))) / dt
        u, v = un, vn
        t += dt; passo += 1
        if passo % 200 == 0:
            storia.append((t, res))
    divf = np.max(np.abs((u[1:, 1:-1] - u[:-1, 1:-1]) / h + (v[1:-1, 1:] - v[1:-1, :-1]) / h))
    durata = time.perf_counter() - t0
    # funzione di corrente e vorticita' nei vertici (N+1) x (N+1): psi_(j+1) = psi_j + h u_(i+1/2, j+1/2)
    psi = np.zeros((N + 1, N + 1))
    for j in range(1, N + 1):
        psi[:, j] = psi[:, j - 1] + h * u[:, j]
    om = np.zeros((N + 1, N + 1))
    om[1:-1, 1:-1] = (v[2:-1, 1:-1] - v[1:-2, 1:-1]) / h - (u[1:-1, 2:-1] - u[1:-1, 1:-2]) / h
    if stampa:
        k = np.unravel_index(np.argmin(psi), psi.shape)
        print(f"   Re = {Re:5.0f}, griglia {N+1}x{N+1}: {passo} passi (dt = {dt:.2e}), t = {t:.1f}, residuo {res:.1e},"
              f" max|div u| = {divf:.1e}, {durata:.1f} s")
        print(f"      psi_min = {psi.min():.5f} in ({k[0]*h:.4f}, {k[1]*h:.4f}),
```

```
    omega_c = {om[k]:.4f}, "
120             f"psi sulla parete superiore: max|psi| = {np.max(np.abs(psi[:,
    N])):.1e}")
121         if Re in GHIA:
122             g = GHIA[Re]
123             print(f"      Ghia et al. (129x129): psi_min = {g[0]:.5f} in ({g[2]},
    {g[3]}), omega_c = {g[1]:.4f}")
124     return dict(Re=Re, N=N, h=h, u=u, v=v, p=p, psi=psi, om=om, t=t, passi=passo,
    storia=storia)
125
126
127 if __name__ == "__main__":
128     print("Cavita' quadrata con coperchio mobile, griglia MAC, passi frazionari
    espliciti")
129     casi = [(1, 64, 5.0), (100, 64, 30.0), (400, 64, 60.0), (1000, 80, 120.0)]
130     ris = {}
131     for Re, N, tmax in casi:
132         ris[Re] = cavita(Re, N, tmax)
133     # profili di velocita' sulla mezzeria verticale
134     print("\nVelocita' u sulla mezzeria x = 0,5 (griglia MAC: colonna i = N/2):")
135     print("      y     " + "".join(f"  Re={Re:4d} (calc./Ghia)" for Re in (100,
    400, 1000)))
136     for k, y in enumerate(GHIA_U["y"]):
137         riga = f"   {y:6.4f} "
138         for Re in (100, 400, 1000):
139             r = ris[Re]; N = r["N"]; h = r["h"]
140             yy = (np.arange(N + 2) - 0.5) * h          # quote dei nodi u (con
    fittizi)
141             uu = np.interp(y, yy, r["u"][N // 2, :])
142             riga += f"   {uu:+.4f}/{GHIA_U[Re][k]:+.4f}   "
143         print(riga)
144
145     fig, axs = plt.subplots(2, 4, figsize=(8.8, 4.4))
146     for col, Re in enumerate((1, 100, 400, 1000)):
147         r = ris[Re]; N = r["N"]
148         xg = np.linspace(0, 1, N + 1)
149         lv = [-0.1175, -0.115, -0.11, -0.1, -0.09, -0.07, -0.05, -0.03, -0.01,
    -1e-4, 1e-5, 1e-4, 5e-4, 1e-3, 1.5e-3, 3e-3]
150         axs[0, col].contour(xg, xg, r["psi"].T, levels=lv, colors=[ROSSO if v > 0
    else BLU for v in lv], linewidths=0.7)
151         axs[0, col].set_aspect("equal"); axs[0, col].set_title(f"$\\psi$,
    $Re={Re}$", fontsize=8)
152         lo = [-5, -4, -3, -2, -1, -0.5, 0, 0.5, 1, 2, 3, 4, 5]
153         axs[1, col].contour(xg, xg, r["om"].T, levels=lo, colors=[ROSSO if v > 0
    else BLU for v in lo], linewidths=0.7)
154         axs[1, col].set_aspect("equal"); axs[1, col].set_title(f"$\\omega$,
    $Re={Re}$", fontsize=8)
155         for a_ in axs[:, col]:
156             a_.set_xticks([]); a_.set_yticks([])
157     fig.tight_layout()
158     salva(fig, "prog4_5_cavita_mac")
159
160     fig, ax = plt.subplots(1, 2, figsize=(7.4, 2.9))
161     for Re, col in zip((100, 400, 1000), (BLU, VERDE, ROSSO)):
162         r = ris[Re]; N = r["N"]; h = r["h"]
163         yy = (np.arange(N + 2) - 0.5) * h
164         ax[0].plot(r["u"][N // 2, 1:-1], yy[1:-1], color=col, label=f"$Re={Re}$")
165         ax[0].plot(GHIA_U[Re], GHIA_U["y"], "o", color=col, ms=3, mfc="none")
166     ax[0].set_xlabel("$u(0{,}5,y)$"); ax[0].set_ylabel("$y$");
    ax[0].legend(fontsize=7)
```

```
    ax[0].set_title("profili sulla mezzeria (cerchi: Ghia et al.)", fontsize=8)
    for Re, col in zip((100, 400, 1000), (BLU, VERDE, ROSSO)):
        st = np.array(ris[Re]["storia"])
        ax[1].semilogy(st[:, 0], st[:, 1], color=col, label=f"$Re={Re}$")
    ax[1].set_xlabel("$t$"); ax[1].set_ylabel("$\\max|u^{n+1}-u^n|/\\Delta t$");
    ax[1].legend(fontsize=7)
    ax[1].set_title("avvicinamento allo stato stazionario", fontsize=8)
    fig.tight_layout()
    salva(fig, "prog4_5_cavita_profili")
```

Listato 5.1: Programma 4.5 – cavità quadrata con coperchio mobile in variabili primitive: griglia MAC, passi frazionari espliciti, risolutore diretto per la pressione (`prog4_5_cavita_mac.py`).

Risultati del Programma 4.5

- **Conservazione della massa.** La divergenza massima del campo finale è 10^{-14}–10^{-13}: zero di macchina, come promesso dal metodo a passi frazionari su griglia sfalsata.
- **Confronto con Ghia et al.** (Tab. 5.1). A $Re = 100$ il minimo di ψ è $-0{,}1032$ contro $-0{,}1034$ (0,2%), nella stessa posizione $(0{,}61, 0{,}73)$; la vorticità al centro $-3{,}10$ contro $-3{,}17$ (2%: la vorticità è una derivata, più sensibile alla griglia). A $Re = 400$ e 1000 gli scarti su $\psi_{\min}$ sono 1,7% e 2,1%, con griglie da 2 a 4 volte più grossolane di quella di riferimento. I profili $u(0{,}5, y)$ (Fig. 5.9) coincidono con i cerchi di Ghia entro 0,01 a $Re = 100$ e 400 e entro 0,01–0,02 a $Re = 1000$, dove il picco negativo vicino al fondo ($-0{,}37$ contro $-0{,}38$) richiederebbe più risoluzione.
- **Struttura della corrente** (Fig. 5.8). A $Re = 1$ la soluzione è simmetrica rispetto alla mezzeria verticale (come nel flusso di Stokes) e la vorticità si concentra negli spigoli superiori, dove il problema è singolare per il doppio valore di u; a $Re = 100$ la simmetria si rompe, il centro del vortice si sposta verso l'angolo superiore destro e nasce il vortice secondario in basso a destra; a $Re = 400$ e 1000 il centro scende verso il centro geometrico, i vortici secondari crescono in entrambi gli spigoli inferiori e una "lingua" di vorticità si stacca dalla parete destra e penetra nella cavità.
- **Avviamento.** Il residuo decresce esponenzialmente (Fig. 5.9) con un tempo caratteristico che cresce con Re: $t \approx 22$ per $Re = 100$, 50 per 400, 112 per 1000; il numero di passi (da 9000 a 45 000) e il tempo di calcolo (da 8 a 60 s) sono il prezzo dello schema esplicito, che l'Approfondimento sullo schema AB–CN riduce.

Tabella 5.1: Funzione di corrente e vorticità nel centro del vortice principale: Programma 4.5, tabella della fonte (stesse griglie) e Ghia, Ghia e Shin (griglia 129×129).

Re	griglia	ψ_c (4.4)	ψ_c (fonte)	ψ_c (Ghia)	ω_c (4.4)	ω_c (fonte)	ω_c (Ghia)
1	65×65	$-0{,}1000$	$-0{,}099$	–	$-3{,}217$	$-3{,}207$	–
100	65×65	$-0{,}1032$	$-0{,}101$	$-0{,}1034$	$-3{,}099$	$-3{,}157$	$-3{,}166$
400	65×65	$-0{,}1120$	$-0{,}107$	$-0{,}1139$	$-2{,}257$	$-2{,}609$	$-2{,}295$
1000	81×81	$-0{,}1154$	$-0{,}113$	$-0{,}1179$	$-2{,}013$	$-2{,}191$	$-2{,}050$

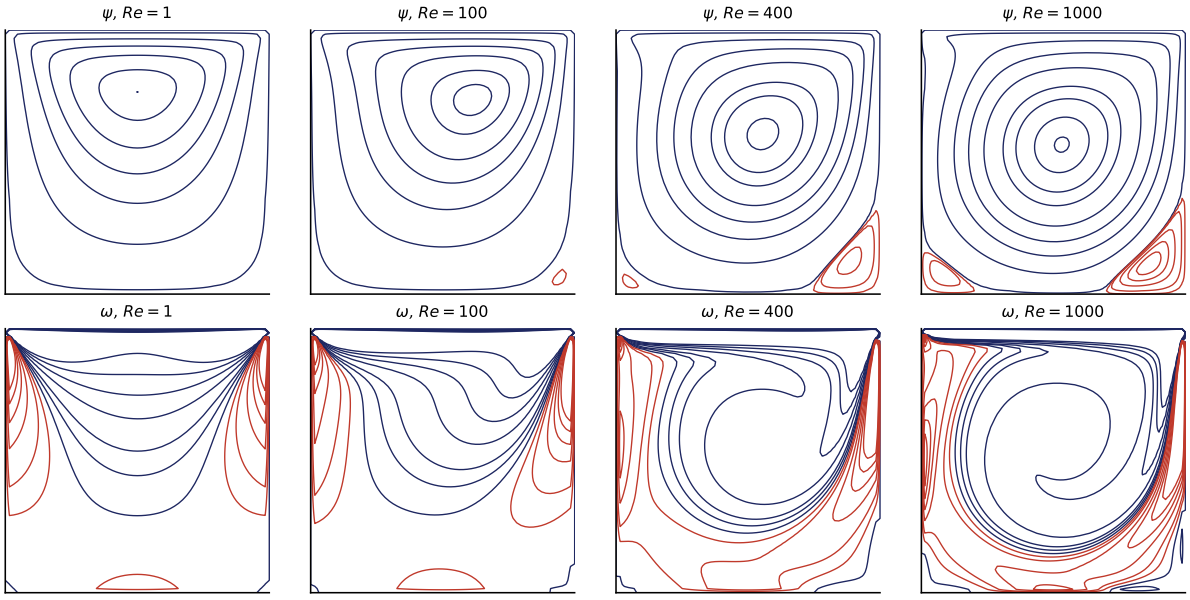


Figura 5.8: Programma 4.5: linee di corrente (sopra) e linee di uguale vorticità (sotto) per $Re = 1, 100, 400, 1000$.

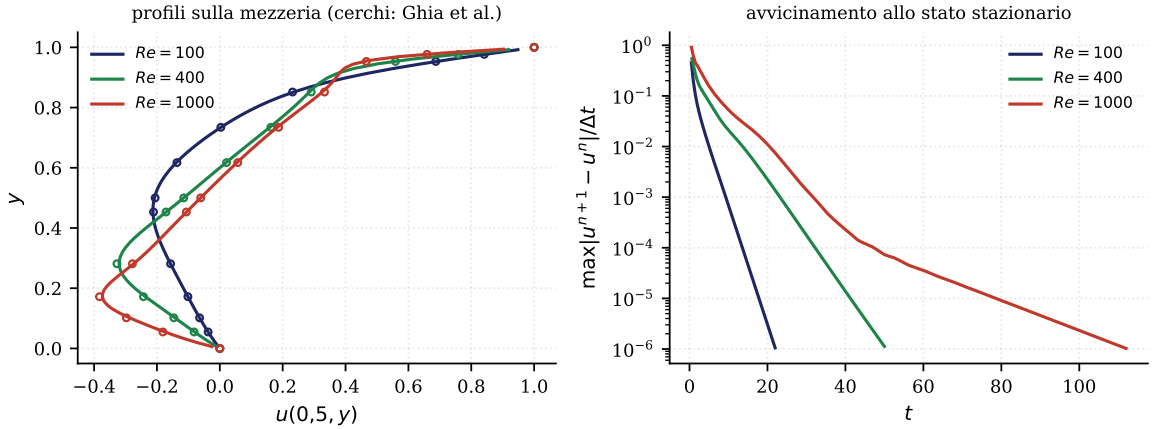


Figura 5.9: Programma 4.5: profili di u sulla mezzeria verticale confrontati con i dati di Ghia et al. (cerchi) e decadimento del residuo durante l'avviamento.

La singolarità degli spigoli

Negli spigoli superiori la velocità tangenziale passa da 1 (coperchio) a 0 (parete) in un punto: la vorticità e lo sforzo vi sono infiniti ($\sim 1/r$) e la pressione singolare. Nessuna griglia può "convergere" lì, e i valori di ω nelle prime celle dipendono dalla risoluzione. Per fortuna la singolarità è integrabile e non influenza la soluzione lontano dagli spigoli, come dimostra l'accordo con Ghia et al.; ma confrontare le vorticità *di parete* vicino agli spigoli fra griglie diverse non ha senso, e la coppia richiesta per muovere il coperchio è formalmente infinita. Nelle simulazioni accurate si regolarizza il profilo di velocità del coperchio, per esempio con $u_\Gamma = 16x^2(1-x)^2$.

5.8 La cavità mossa dal vento

Il secondo progetto della fonte sostituisce al coperchio rigido una *superficie libera* su cui il vento esercita uno sforzo tangenziale costante $\tau = \rho_a u_*^2$, dove u_* è la velocità d'attrito dell'aria: è un modello del trasferimento di quantità di moto dall'aria all'acqua (laghi, bacini, vasche) quando si trascurano le onde. Adimensionalizzando con u_* e con la profondità L si ha $Re = u_* L/\nu$ e, sulla superficie, la condizione sulla *derivata* invece che sulla velocità:

$$\mu \frac{\partial u}{\partial y} = \tau \quad \Rightarrow \quad \frac{\partial u}{\partial y} = \frac{\rho_a}{\rho} Re \qquad \text{(in unità di } u_*,\ L), \tag{5.28}$$

con $\rho_a/\rho \approx 1/1000$ per aria su acqua; $v = 0$ (superficie piana). Sulla griglia MAC la condizione si impone con il nodo fittizio $u_{\cdot,N+1} = u_{\cdot,N} + h\,\partial u/\partial y$ invece della (5.25): una riga di codice (parametro `sforzo` della funzione `cavita`). Il Programma 4.5b la applica, con il vento diretto verso sinistra, per $Re = 100$ e 2000 (valori rappresentativi di una vasca di laboratorio e di uno stagno).


prog4_5b_cavita_vento.py

```
"""
prog4_5b_cavita_vento.py -- cavita' mossa dal vento (shear-driven cavity): sulla
superficie superiore, invece della velocita', e' imposto lo sforzo tangenziale
costante tau = rho_a u*^2 esercitato dall'aria. Con la velocita' d'attrito u* e
l'altezza L come scale, Re = u* L/nu e la condizione diventa
    du/dy = -(rho_a/rho) Re,   v = 0   (vento da destra verso sinistra),
con rho_a/rho = 1/1000 (aria su acqua). Si usa lo stesso risolutore MAC del
Programma 4.5 (prog4_5_cavita_mac.py) su una griglia 41 x 41, per Re = 100 e 2000.
"""
import numpy as np
import matplotlib.pyplot as plt
from prog4_5_cavita_mac import cavita
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

RAPPORTO = 1.0 / 1000.0          # rho_aria / rho_acqua
ris = {}
print("Cavita' mossa dal vento (sforzo imposto in superficie), griglia 41x41:")
for Re, tmax in ((100, 60.0), (2000, 400.0)):
    g = -RAPPORTO * Re
    r = cavita(Re, 40, tmax, tol=1e-6, sforzo=g, u_max=max(0.3, abs(g) * 0.5),
    stampa=False)
    psi = r["psi"]; N = r["N"]; h = r["h"]
    u_sup = 0.5 * (r["u"][:, N] + r["u"][:, N + 1])
    k_min = np.unravel_index(np.argmin(psi), psi.shape); k_max =
    np.unravel_index(np.argmax(psi), psi.shape)
    print(f"   Re = {Re:5d}: du/dy = {g:+.2f}; {r['passi']} passi, t =
    {r['t']:.1f};"
          f" psi_min = {psi.min():+.5f} in ({k_min[0]*h:.3f}, {k_min[1]*h:.3f}),
    psi_max = {psi.max():+.5f},"
          f" Delta psi = {psi.max()-psi.min():.5f}; velocita' in superficie: max
    |u| = {np.max(np.abs(u_sup)):.4f}")
    ris[Re] = r

fig, axs = plt.subplots(1, 3, figsize=(8.6, 2.8), gridspec_kw={"width_ratios": [1,
    1, 1.1]})
for a_, Re in zip(axs[:2], (100, 2000)):
    r = ris[Re]; N = r["N"]; xg = np.linspace(0, 1, N + 1)
    m = np.max(np.abs(r["psi"]))
    lv = np.linspace(-m, m, 15); lv = lv[np.abs(lv) > 1e-9]
    a_.contour(xg, xg, r["psi"].T, levels=lv, colors=[ROSSO if v < 0 else BLU for v
    in lv],
               linestyles=["--" if v < 0 else "-" for v in lv], linewidths=0.8)
```

```
    a_.set_aspect("equal"); a_.set_title(f"$\\psi$, $Re={Re}$", fontsize=8)
    a_.set_xticks([]); a_.set_yticks([])
for Re, col in zip((100, 2000), (BLU, ROSSO)):
    r = ris[Re]; N = r["N"]; h = r["h"]
    yy = (np.arange(N + 2) - 0.5) * h
    axs[2].plot(r["u"][N // 2, 1:-1], yy[1:-1], color=col, label=f"$Re={Re}$")
axs[2].axvline(0, color="k", lw=0.6)
axs[2].set_xlabel("$u(0{,}5,y)/u_*$"); axs[2].set_ylabel("$y$");
    axs[2].legend(fontsize=7)
fig.tight_layout()
salva(fig, "prog4_5b_cavita_vento")
```

Listato 5.2: `prog4_5b_cavita_vento.py`: cavità con sforzo tangenziale imposto sulla superficie libera.

Risultati del Programma 4.5b

Con griglia 41×41: a $Re = 100$ la superficie si muove con velocità massima $0{,}014 u_*$ e l'intensità della circolazione è $\Delta\psi = 1{,}3 \times 10^{-3}$; a $Re = 2000$ la velocità superficiale è $0{,}15 u_*$ e $\Delta\psi = 1{,}4 \times 10^{-2}$ (Fig. 5.10). La corrente ha la stessa struttura della cavità con coperchio (un vortice principale che riempie la cavità, con il centro spostato nel verso del vento), ma le velocità sono molto minori: lo sforzo $\rho_a u_*^2$ è piccolo, e l'acqua risponde con una velocità superficiale u_s tale che lo sforzo viscoso interno $\mu u_s/\delta$ lo equilibri. Il profilo $u(0{,}5, y)$ mostra la corrente di ritorno sotto la superficie, come nei laghi sotto vento: una corrente superficiale nel verso del vento e un flusso di ritorno in profondità, che riporta l'acqua accumulata sottovento. A $Re = 2000$ l'avviamento richiede $t = 254$ (50 000 passi): il tempo di diffusione $L^2/\nu = Re$ in unità di L/u_*.

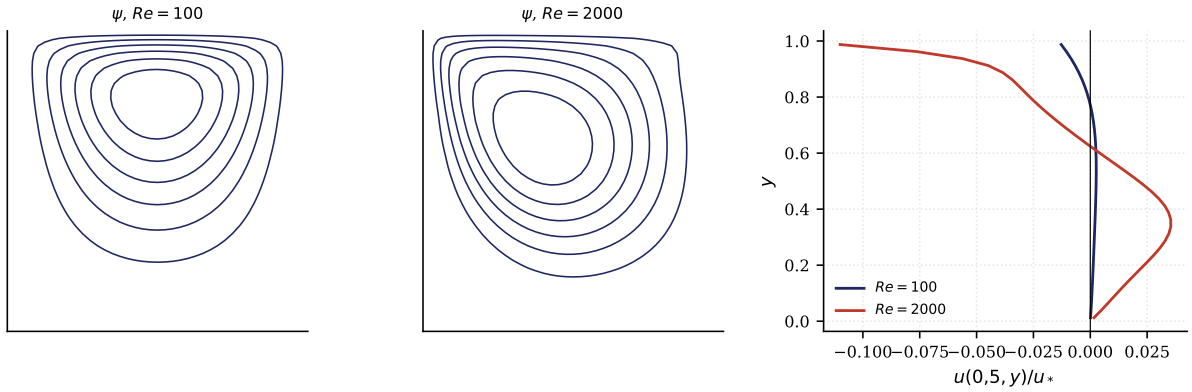


Figura 5.10: Programma 4.5b: linee di corrente della cavità mossa dal vento per $Re = 100$ e 2000 e profili di u/u_* sulla mezzeria.

5.9 Approfondimenti

5.9.1 Lo schema semi-implicito AB–CN a passi frazionari

Il primo progetto della fonte chiede di risolvere la cavità con lo schema semi-implicito del Cap. 4: Adams–Bashforth per la convezione, Crank–Nicolson per la diffusione, passi frazionari su griglia MAC 21×21. Il Programma 4.5c lo realizza: a ogni passo risolve due equazioni di Helmholtz (5.8) (una per componente) e una di Poisson; tutte e tre hanno matrice costante e

vengono fattorizzate una sola volta. Le condizioni di aderenza con i nodi fittizi entrano nella matrice di Helmholtz (il nodo fittizio $-u_1$ modifica la diagonale della prima fila) e nel termine noto (il contributo $2U/h^2$ del coperchio). Il passo temporale è limitato dalla sola condizione convettiva, $\Delta t = 0{,}5h$.


prog4_5c_cavita_abcn.py

```
"""
prog4_5c_cavita_abcn.py -- cavita' con coperchio mobile su griglia MAC con lo schema
semi-implicito AB-CN (Adams-Bashforth del secondo ordine per i termini convettivi,
Crank-Nicolson per quelli viscosi) e metodo a passi frazionari (progetto 1a):

  (I - dt/(2Re) L) u^ = u^n + dt (3/2 H^n - 1/2 H^(n-1)) + dt/(2Re) L u^n,
  lap p = div u^ / dt,      u^(n+1) = u^ - dt grad p,

con L il laplaciano discreto comprensivo delle condizioni di aderenza imposte con i
nodi fittizi (u_0 = -u_1 sul fondo, u_(N+1) = 2 U - u_N sul coperchio). I tre sistemi
lineari (u^, v^, p) hanno matrici costanti: fattorizzazione LU sparsa una volta sola.
Il passo temporale e' limitato soltanto dalla condizione convettiva (CFL), non da
quella diffusiva dt <= Re h^2/4: il vantaggio e' grande a basso Re e su griglie fini.
Confronto con il Programma 4.5 (esplicito) e con Ghia et al. (1982).
"""
import time
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt
from prog4_5_cavita_mac import matrice_pressione, GHIA, cavita
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

TABELLA = {1: (-0.099, -3.207), 100: (-0.101, -3.157), 400: (-0.107, -2.609), 1000: (-0.113, -2.191)}  # 65x65 (81x81)


def matrice_viscosa(N, h, coef, comp):
    """I - coef * L sui nodi interni della componente comp ('u': (N-1) x N nodi, 'v': N x (N-1)).
    Le pareti ortogonali alla componente sono nodi di Dirichlet (contributo nullo); su quelle
    parallele il nodo fittizio vale -(nodo interno) [+ termine noto], cioe' -1/h^2 in diagonale."""
    nx, ny = (N - 1, N) if comp == "u" else (N, N - 1)
    idx = lambda i, j: i * ny + j
    A = sp.lil_matrix((nx * ny, nx * ny))
    for i in range(nx):
        for j in range(ny):
            p = idx(i, j)
            diag = -4.0 / h**2
            for (a, b) in ((i + 1, j), (i - 1, j), (i, j + 1), (i, j - 1)):
                if 0 <= a < nx and 0 <= b < ny:
                    A[p, idx(a, b)] = coef * (-1.0 / h**2)
                else:
                    tangente = (comp == "u" and (b < 0 or b >= ny)) or (comp == "v" and (a < 0 or a >= nx))
                    if tangente:
                        diag -= 1.0 / h**2          # nodo fittizio = -nodo interno (+ termine noto)
            A[p, p] = 1.0 - coef * diag
    return spla.splu(A.tocsc())
```

```
47
48
49  def convettivi(u, v, N, h):
50      """Termini convettivi conservativi H_u = -((u^2)_x + (uv)_y), H_v = -((uv)_x +
        (v^2)_y) nei nodi interni."""
51      ue = 0.5 * (u[1:, 1:-1] + u[:-1, 1:-1]); duudx = (ue[1:, :]**2 - ue[:-1, :]**2)
        / h
52      uc = 0.5 * (u[1:-1, 1:] + u[1:-1, :-1]); vc = 0.5 * (v[1:-1, :][1:, :] +
        v[1:-1, :][:-1, :])
53      duvdy = (uc[:, 1:] * vc[:, 1:] - uc[:, :-1] * vc[:, :-1]) / h
54      ve = 0.5 * (v[1:-1, 1:] + v[1:-1, :-1]); dvvdy = (ve[:, 1:]**2 - ve[:, :-1]**2)
        / h
55      uc2 = 0.5 * (u[:, 1:-1][:, 1:] + u[:, 1:-1][:, :-1]); vc2 = 0.5 * (v[1:, 1:-1]
        + v[:-1, 1:-1])
56      duvdx = (uc2[1:, :] * vc2[1:, :] - uc2[:-1, :] * vc2[:-1, :]) / h
57      return -duudx - duvdy, -duvdx - dvvdy
58
59
60  def lap(w, h):
61      return (w[2:, 1:-1] + w[:-2, 1:-1] + w[1:-1, 2:] + w[1:-1, :-2] - 4 * w[1:-1,
        1:-1]) / h**2
62
63
64  def cavita_abcn(Re, N, t_max, dt=None, tol=1e-6, cfl=0.5, stampa=True,
        incrementale=False):
65      """incrementale = True: forma a correzione di pressione incrementale (Kim-Moin,
        Van Kan): nel
66      predittore compare -dt grad p^n e si risolve per l'incremento phi, p^(n+1) =
        p^n + phi; a regime
67      phi -> 0 e l'errore di separazione (splitting) O(dt) dello schema non
        incrementale scompare."""
68      h = 1.0 / N
69      u = np.zeros((N + 1, N + 2)); v = np.zeros((N + 2, N + 1)); p = np.zeros((N, N))
70      if dt is None:
71          dt = cfl * h                                          # solo il limite
        convettivo (u_max = 1)
72      coef = dt / (2 * Re)
73      LUp = matrice_pressione(N, h)
74      LUu = matrice_viscosa(N, h, coef, "u"); LUv = matrice_viscosa(N, h, coef, "v")
75      Hu_prec = Hv_prec = None
76      t, passo, res = 0.0, 0, np.inf
77      t0 = time.perf_counter()
78      while t < t_max and res > tol:
79          u[:, 0] = -u[:, 1]; u[:, N + 1] = 2.0 - u[:, N]
80          v[0, :] = -v[1, :]; v[N + 1, :] = -v[N, :]
81          Hu, Hv = convettivi(u, v, N, h)
82          if Hu_prec is None:
83              cu, cv = Hu, Hv                                   # primo passo: Eulero
        esplicito
84          else:
85              cu, cv = 1.5 * Hu - 0.5 * Hu_prec, 1.5 * Hv - 0.5 * Hv_prec
86          # termini noti: parte esplicita di Crank-Nicolson (con i nodi fittizi
        correnti) e, nella parte
87          # implicita, il contributo noto del coperchio 2U/h^2 sui nodi adiacenti (j
        = N)
88          rhs_u = u[1:-1, 1:-1] + dt * cu + coef * lap(u, h)
89          rhs_u[:, -1] += coef * 2.0 / h**2
90          rhs_v = v[1:-1, 1:-1] + dt * cv + coef * lap(v, h)
91          if incrementale:
92              rhs_u -= dt * (p[1:, :] - p[:-1, :]) / h
```

```
            rhs_v -= dt * (p[:, 1:] - p[:, :-1]) / h
        us = u.copy(); vs = v.copy()
        us[1:-1, 1:-1] = LUu.solve(rhs_u.ravel()).reshape(N - 1, N)
        vs[1:-1, 1:-1] = LUv.solve(rhs_v.ravel()).reshape(N, N - 1)
        div = (us[1:, 1:-1] - us[:-1, 1:-1]) / h + (vs[1:-1, 1:] - vs[1:-1, :-1]) / h
        rhs = (div / dt).ravel(); rhs[0] = 0.0
        phi = LUp.solve(rhs).reshape(N, N)
        un = us.copy(); vn = vs.copy()
        un[1:-1, 1:-1] -= dt * (phi[1:, :] - phi[:-1, :]) / h
        vn[1:-1, 1:-1] -= dt * (phi[:, 1:] - phi[:, :-1]) / h
        p = p + phi if incrementale else phi
        res = max(np.max(np.abs(un - u)), np.max(np.abs(vn - v))) / dt
        Hu_prec, Hv_prec = Hu, Hv
        u, v = un, vn
        t += dt; passo += 1
        if not np.isfinite(res):
            break
    durata = time.perf_counter() - t0
    psi = np.zeros((N + 1, N + 1))
    for j in range(1, N + 1):
        psi[:, j] = psi[:, j - 1] + h * u[:, j]
    om = np.zeros((N + 1, N + 1))
    om[1:-1, 1:-1] = (v[2:-1, 1:-1] - v[1:-2, 1:-1]) / h - (u[1:-1, 2:-1] - u[1:-1, 1:-2]) / h
    k = np.unravel_index(np.argmin(psi), psi.shape)
    if stampa:
        print(f"   Re = {Re:5.0f}, griglia {N+1}x{N+1}, dt = {dt:.4f} (d = dt/(Re h^2) = {dt/(Re*h*h):.2f},"
              f" limite esplicito 0,25): {passo} passi, t = {t:.1f}, residuo {res:.1e}, {durata:.1f} s")
        print(f"      psi_min = {psi.min():.5f} in ({k[0]*h:.3f}, {k[1]*h:.3f}), omega_c = {om[k]:.4f}"
              + (f";  Tabella (65x65): {TABELLA[Re][0]:.3f}, {TABELLA[Re][1]:.3f}" if Re in TABELLA else "")
              + (f";  Ghia: {GHIA[Re][0]:.4f}, {GHIA[Re][1]:.4f}" if Re in GHIA else ""))
    return dict(Re=Re, N=N, h=h, u=u, v=v, p=p, psi=psi, om=om, t=t, passi=passo, dt=dt, durata=durata, res=res)


if __name__ == "__main__":
    print("Cavita' con coperchio mobile, griglia MAC, schema semi-implicito AB-CN a passi frazionari")
    print("Griglia 21x21 (progetto):")
    ris = {}
    for Re in (100, 400):
        ris[(Re, 20)] = cavita_abcn(Re, 20, 60.0)
    print("\nGriglia 65x65, confronto con lo schema esplicito (Programma 4.5) a parita' di griglia:")
    for Re, tmax, dt in ((1, 3.0, 0.002), (100, 30.0, None), (400, 60.0, None)):
        r = cavita_abcn(Re, 64, tmax, dt=dt)
        ris[(Re, 64)] = r
        t0 = time.perf_counter()
        e = cavita(Re, 64, tmax, stampa=False)
        print(f"      esplicito: {e['passi']} passi, {time.perf_counter()-t0:.1f} s, psi_min = {e['psi'].min():.5f}"
              f"  (rapporto fra i passi: {e['passi']/r['passi']:.0f})")
    print("\nErrore di separazione (splitting) a regime, Re = 1, 65x65 (riferimento esplicito psi_min = -0,10002):")
```

```
    print("   lo schema non incrementale produce uno strato limite numerico di spessore ~ sqrt(dt/Re)")
    for dt in (0.0005, 0.001, 0.002, 0.004, 0.008):
        with np.errstate(all="ignore"):
            r1 = cavita_abcn(1, 64, 3.0, dt=dt, stampa=False)
            r2 = cavita_abcn(1, 64, 3.0, dt=dt, stampa=False, incrementale=True)
        print(f"   dt = {dt:.4f} (d = {dt*4096:5.1f}, sqrt(dt/Re)/h = {np.sqrt(dt)*64:4.1f} celle): non incrementale psi_min = {r1['psi'].min():.5f}"
              f" (residuo {r1['res']:.1e}); incrementale psi_min = {r2['psi'].min():.5f} (residuo {r2['res']:.1e})")
    print("\nStabilita' al crescere di dt (Re = 100, 65x65): d = dt/(Re h^2), C = dt/h")
    for dt in (0.005, 0.01, 0.02, 0.03, 0.04):
        with np.errstate(all="ignore"):
            r = cavita_abcn(100, 64, 30.0, dt=dt, stampa=False)
        print(f"   dt = {dt:.3f}: d = {dt/(100/64**2):5.2f}, C = {dt*64:.2f} -> "
              + ("instabile" if not np.isfinite(r['res']) else f"psi_min = {r['psi'].min():.5f} ({r['passi']} passi, {r['durata']:.1f} s)"))

    fig, axs = plt.subplots(1, 3, figsize=(8.4, 2.9))
    for a_, key in zip(axs[:2], ((100, 20), (400, 20))):
        r = ris[key]; x = np.linspace(0, 1, r["N"] + 1)
        lv = [-0.11, -0.1, -0.09, -0.07, -0.05, -0.03, -0.01, -1e-4, 1e-5, 1e-4, 5e-4, 1e-3]
        a_.contour(x, x, r["psi"].T, levels=lv, colors=[ROSSO if v > 0 else BLU for v in lv], linewidths=0.7)
        a_.set_aspect("equal"); a_.set_title(f"$\\psi$, $Re={key[0]}$, $21\\times21$", fontsize=8)
        a_.set_xticks([0, 0.5, 1]); a_.set_yticks([0, 0.5, 1])
    from prog4_5_cavita_mac import GHIA_U
    for Re, col in ((100, BLU), (400, ARANCIO)):
        for N, ls in ((20, "--"), (64, "-")):
            r = ris[(Re, N)]
            u = r["u"]; h = r["h"]
            uc = 0.5 * (u[N // 2, :] + u[N // 2, :]) if N % 2 == 0 else None
            yv = np.linspace(0, 1, N + 1)
            u_mid = 0.5 * (u[N // 2, 1:] + u[N // 2, :-1])       # u(0,5, y) nei vertici
            axs[2].plot(u_mid, yv, ls, color=col, lw=0.9, label=f"$Re={Re}$, ${N+1}\\times{N+1}$")
        axs[2].plot(GHIA_U[Re], GHIA_U["y"], "o", color=col, ms=2.5, mfc="none")
    axs[2].set_xlabel("$u(0{,}5,y)$"); axs[2].set_ylabel("$y$"); axs[2].legend(fontsize=6)
    axs[2].set_title("profili sulla mezzeria (cerchi: Ghia et al.)", fontsize=8)
    fig.tight_layout()
    salva(fig, "prog4_5c_cavita_abcn")
```

Listato 5.3: `prog4_5c_cavita_abcn.py`: cavità con lo schema semi-implicito AB–CN a passi frazionari, forma incrementale e non incrementale, errore di separazione.

Risultati del Programma 4.5c

Sulla griglia 21×21 del progetto, $Re = 100$ dà $\psi_c = -0{,}1000$, $\omega_c = -2{,}98$ (Ghia: $-0{,}1034$, $-3{,}17$) e $Re = 400$ dà $-0{,}0989$, $-2{,}10$ ($-0{,}1139$, $-2{,}29$): con 20 celle per lato l'errore è del 3% e del 13%, e il tempo di calcolo è inferiore al secondo. Sulla griglia 65×65 i risultati coincidono con quelli dello schema esplicito ($\psi_c = -0{,}10319$ contro $-0{,}10318$ a $Re = 100$) con un numero di passi 3 volte minore a $Re = 100$, 2 volte a $Re = 400$ e *56 volte* a $Re = 1$, dove la condizione diffusiva è la più severa: è esattamente il vantaggio

annunciato nel Cap. 4. A $Re = 100$ lo schema resta stabile fino a $\Delta t = 0{,}03$ ($C = 1{,}9$ nominale, ma le velocità reali sono inferiori a 1 quasi ovunque) e diventa instabile a $\Delta t = 0{,}04$: la condizione convettiva è l'unica che resta.

L'errore di separazione e la forma incrementale

Il metodo a passi frazionari non è esatto a regime: nella forma *non incrementale* (il predittore ignora del tutto la pressione) lo stato stazionario del sistema discreto non coincide con quello delle equazioni di Navier–Stokes discretizzate direttamente, ma ne differisce di un termine $\mathcal{O}(\Delta t)$ concentrato in uno *strato limite numerico* alle pareti, di spessore $\sqrt{\nu \Delta t} = \sqrt{\Delta t / Re}$. L'origine è il passo correttore: la componente tangenziale di $\Delta t \nabla \phi$ non si annulla sulla parete (si impone solo $\partial \phi / \partial n = 0$), quindi $\mathbf{V}^{n+1}$ ha uno scorrimento tangenziale spurio di ordine $\Delta t \, \partial \phi / \partial \tau$ che la viscosità smorza in uno strato di spessore $\sqrt{\nu \Delta t}$. Il Programma 4.5c lo misura a $Re = 1$: con $\Delta t = 0{,}008$ ($\sqrt{\Delta t / Re} = 5{,}7$ celle) $\psi_c = -0{,}0972$ invece di $-0{,}1000$, un errore del 3% che decresce linearmente con Δt ($-0{,}0996$ a $\Delta t = 0{,}002$). Il rimedio di Kim e Moin [20] e di Van Kan è la forma *incrementale*: il predittore include il gradiente della pressione *vecchia*, $-\Delta t \nabla P^n$, e la Poisson si risolve per l'incremento ϕ, con $P^{n+1} = P^n + \phi$. A regime $\phi \to 0$ e l'errore di separazione scompare: il programma ottiene $\psi_c = -0{,}10002$ per ogni Δt, identico allo schema esplicito. Con entrambe le forme, a $\Delta t = 0{,}008$ ($d = 33$) il residuo si ferma a 4×10^{-3}: sono le oscillazioni non smorzate di Crank–Nicolson innescate dall'avviamento impulsivo del coperchio (Cap. 4), curabili con l'avvio di Rannacher.

5.9.2 Le proprietà di conservazione della griglia sfalsata

La griglia MAC ha una struttura algebrica notevole: il gradiente discreto $\mathbf{G}$ (dalle celle di pressione alle facce) e la divergenza discreta $\mathbf{D}$ (dalle facce alle celle) sono l'uno il *trasposto negativo* dell'altro, $\mathbf{D} = -\mathbf{G}^{\mathsf{T}}$, esattamente come nel continuo $\int_S \phi \nabla \cdot \mathbf{V} \, \mathrm{d}S = -\int_S \mathbf{V} \cdot \nabla \phi \, \mathrm{d}S$ per campi tangenti al contorno. Ne seguono tre conseguenze. (i) La matrice di Poisson della pressione, $\mathbf{D}\mathbf{G} = -\mathbf{G}^{\mathsf{T}}\mathbf{G}$, è simmetrica e semidefinita negativa, con il solo vettore nullo costante: la singolarità è "innocua" e la compatibilità è automatica. (ii) Il gradiente di pressione non fa lavoro su un campo solenoidale, $\sum \mathbf{V} \cdot \mathbf{G}P = -\sum P \, \mathbf{D}\mathbf{V} = 0$: la pressione non altera l'energia cinetica discreta, come nel continuo. (iii) Con la forma conservativa dei termini convettivi (prodotti di medie) anche la convezione conserva l'energia cinetica discreta nel limite non viscoso, se la discretizzazione è *skew-symmetric* (Morinishi et al.): è il motivo per cui le simulazioni dirette della turbolenza usano ancora oggi griglie sfalsate e differenze centrate, senza alcuna viscosità artificiale. Su griglia collocata, per ottenere le stesse proprietà, Rhie e Chow introdussero un'interpolazione speciale delle velocità sulle facce, che è di fatto una griglia sfalsata "nascosta".

5.9.3 I dati di Ghia, Ghia e Shin

Il lavoro di Ghia, Ghia e Shin (1982) risolse la cavità in formulazione vorticità–funzione di corrente con il metodo multigriglia su griglie fino a 257×257 per Re da 100 a 10 000, tabulando i profili di velocità sulle mezzerie e le caratteristiche dei vortici. È diventato il banco di prova universale dei risolutori incomprimibili: ogni nuovo codice viene confrontato con queste tabelle. Vale la pena ricordare che oltre $Re \approx 8000$ la corrente stazionaria nella cavità è instabile (biforcazione di Hopf verso una soluzione periodica), e i valori tabulati a $Re = 10\,000$ sono

soluzioni stazionarie del sistema discreto ottenute con un metodo che non ne verifica la stabilità: un promemoria che convergenza del metodo e realizzabilità fisica sono cose diverse.

Capitolo 6

Formulazione vorticità–funzione di corrente: cavità e cilindro

6.1 L'equazione del trasporto della vorticità

L'alternativa alle variabili primitive, già usata nel Cap. 3, elimina la pressione prendendo il rotore dell'equazione del moto. Si parte dalla forma vettoriale

$$\frac{\partial \mathbf{V}}{\partial t} + (\mathbf{V}\cdot\nabla)\mathbf{V} = -\nabla\frac{p}{\rho} + \nu\nabla^2\mathbf{V}, \tag{6.1}$$

e si usa l'identità $(\mathbf{V}\cdot\nabla)\mathbf{V} = \nabla(\frac{1}{2}\mathbf{V}\cdot\mathbf{V}) - \mathbf{V}\times(\nabla\times\mathbf{V})$:

$$\frac{\partial \mathbf{V}}{\partial t} + \nabla\left(\tfrac{1}{2}V^2\right) - \mathbf{V}\times(\nabla\times\mathbf{V}) = -\nabla\frac{p}{\rho} + \nu\nabla^2\mathbf{V}. \tag{6.2}$$

Con la vorticità $\boldsymbol{\omega} = \nabla\times\mathbf{V}$ e poiché il rotore di un gradiente è nullo, il rotore della (6.2) dà

$$\frac{\partial \boldsymbol{\omega}}{\partial t} - \nabla\times(\mathbf{V}\times\boldsymbol{\omega}) = \nu\nabla^2\boldsymbol{\omega}, \tag{6.3}$$

e con l'identità $\nabla\times(\mathbf{V}\times\boldsymbol{\omega}) = \mathbf{V}(\nabla\cdot\boldsymbol{\omega}) - \boldsymbol{\omega}(\nabla\cdot\mathbf{V}) - (\mathbf{V}\cdot\nabla)\boldsymbol{\omega} + (\boldsymbol{\omega}\cdot\nabla)\mathbf{V}$, in cui $\nabla\cdot\boldsymbol{\omega} = 0$ (divergenza di un rotore), $\nabla\cdot\mathbf{V} = 0$ (continuità) e $(\boldsymbol{\omega}\cdot\nabla)\mathbf{V} = 0$ per un moto piano (la vorticità è normale al piano del moto, lungo cui nulla varia), si ottiene l'**equazione del trasporto della vorticità**

$$\frac{\partial \boldsymbol{\omega}}{\partial t} + (\mathbf{V}\cdot\nabla)\boldsymbol{\omega} = \nu\nabla^2\boldsymbol{\omega}. \tag{6.4}$$

In due dimensioni l'unica componente non nulla è $\omega = \partial v/\partial x - \partial u/\partial y$:

$$\frac{\partial \omega}{\partial t} + u\frac{\partial \omega}{\partial x} + v\frac{\partial \omega}{\partial y} = \nu\nabla^2\omega, \tag{6.5}$$

e con la funzione di corrente $u = \partial\psi/\partial y$, $v = -\partial\psi/\partial x$,

$$\nabla^2\psi = -\omega. \tag{6.6}$$

Le due incognite sono ω e ψ; la pressione, se serve, si ricava a posteriori. Il termine $(\boldsymbol{\omega}\cdot\nabla)\mathbf{V}$ scartato è lo *stiramento dei vortici*, il meccanismo con cui in tre dimensioni la vorticità si intensifica allungando i tubi vorticosi: la sua assenza rende la turbolenza bidimensionale qualitativamente diversa da quella tridimensionale, e la formulazione ω–ψ è, per questo, essenzialmente bidimensionale (o assialsimmetrica).

6.2 La cavità in formulazione ω–ψ

Si riprende la cavità con coperchio mobile del capitolo precedente. Senza pressione non serve la griglia sfalsata: si usa una griglia *collocata*, con ω e ψ negli stessi nodi. Con differenze centrate ed Eulero esplicito, la (6.5) adimensionale ($Re = UL/\nu$, $h = \Delta x = \Delta y$) diventa

$$\begin{aligned}\frac{\omega_{i,j}^{n+1} - \omega_{i,j}^{n}}{\Delta t} + u_{i,j}^{n}\frac{\omega_{i+1,j}^{n} - \omega_{i-1,j}^{n}}{2h} + v_{i,j}^{n}\frac{\omega_{i,j+1}^{n} - \omega_{i,j-1}^{n}}{2h} \\ = \frac{1}{h^2 Re}\left(\omega_{i+1,j}^{n} + \omega_{i-1,j}^{n} + \omega_{i,j+1}^{n} + \omega_{i,j-1}^{n} - 4\omega_{i,j}^{n}\right),\end{aligned} \tag{6.7}$$

la (6.6) al nuovo livello

$$\frac{\psi_{i+1,j}^{n+1} - 2\psi_{i,j}^{n+1} + \psi_{i-1,j}^{n+1}}{h^2} + \frac{\psi_{i,j+1}^{n+1} - 2\psi_{i,j}^{n+1} + \psi_{i,j-1}^{n+1}}{h^2} = -\omega_{i,j}^{n+1}, \tag{6.8}$$

e le velocità

$$u_{i,j}^{n+1} = \frac{\psi_{i,j+1}^{n+1} - \psi_{i,j-1}^{n+1}}{2h}, \qquad v_{i,j}^{n+1} = -\frac{\psi_{i+1,j}^{n+1} - \psi_{i-1,j}^{n+1}}{2h}. \tag{6.9}$$

Sequenza di integrazione

1. Condizioni iniziali ($\omega = 0$ nei nodi interni, coperchio avviato impulsivamente).
2. Calcolo di ω^{n+1} nei nodi interni dalla (6.7).
3. Soluzione della (6.8) per ψ^{n+1}, con un metodo diretto o iterativo.
4. Aggiornamento di u, v dalle (6.9).
5. Aggiornamento dei valori di ω sul contorno, che dipendono dalla soluzione, con ψ^{n+1}.
6. Arresto al tempo voluto o al criterio di convergenza; altrimenti si torna al punto 2.
7. Controllo del numero di Courant: se cresce rapidamente, si ricomincia con passo dimezzato.

6.2.1 La vorticità alle pareti

Nel modulo precedente le condizioni per la vorticità erano state ricavate con formule di ordine superiore; qui si usa la forma più semplice, che impiega solo ψ nella direzione normale alla parete. Vicino alla parete inferiore ferma (Fig. 6.1), con $\psi_{i,1}$ sulla parete e $\psi_{i,2}$ nel primo nodo interno,

$$\psi_{i,2}^{n+1} = \psi_{i,1}^{n+1} + \left(\frac{\partial\psi}{\partial y}\right)_{i,1}^{n+1}\Delta y + \left(\frac{\partial^2\psi}{\partial y^2}\right)_{i,1}^{n+1}\frac{\Delta y^2}{2} + \dots \tag{6.10}$$

Sulla parete $\psi_{i,1} = 0$ (nessuna portata la attraversa) e $(\partial\psi/\partial y)_{i,1} = u_{i,1} = 0$ (aderenza). Dalla definizione $\omega = \partial v/\partial x - \partial u/\partial y$, poiché $v = 0$ lungo tutta la parete anche $\partial v/\partial x = 0$, e quindi $\omega_{i,1} = -(\partial u/\partial y)_{i,1} = -(\partial^2\psi/\partial y^2)_{i,1}$. La (6.10) si riduce a $\psi_{i,2} = -\omega_{i,1}\Delta y^2/2$:

$$\omega_{i,1}^{n+1} = -\frac{\psi_{i,2}^{n+1}}{\Delta y^2/2}. \tag{6.11}$$

Sulla parete superiore mobile ($j = J$, $u_W = 1$), sviluppando verso il basso, $\psi_{i,J-1} = \psi_{i,J} - u_W\Delta y - \omega_{i,J}\Delta y^2/2$, da cui

$$\omega_{i,J}^{n+1} = \frac{-\psi_{i,J-1}^{n+1} - u_W\Delta y}{\Delta y^2/2}. \tag{6.12}$$

È la formula di Thom (1933), del primo ordine; Roache [3] osserva che in pratica dà risultati accurati quanto le forme di ordine superiore, ed è più robusta (le formule a più punti possono rendere instabile l'iterazione).

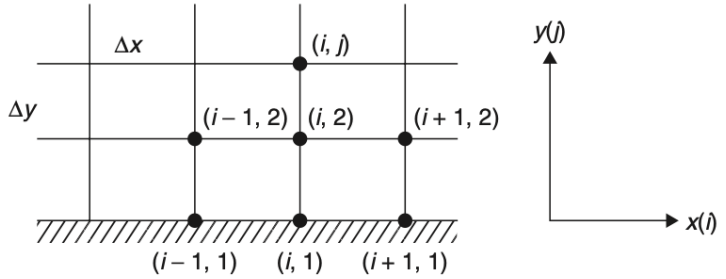


Figura 6.1: Griglia vicino alla parete inferiore.

6.2.2 Il sistema lineare per ψ

Per una griglia con $N \times N$ nodi interni la (6.8) è un sistema di N^2 equazioni. Numerando i nodi per righe, per $N = 3$ (Fig. 6.2) si ha $\mathbf{A}\boldsymbol{\psi} = \mathbf{f}$ con

$$\mathbf{A} = \begin{bmatrix} \mathbf{B} & -\mathbf{I} & \mathbf{0} \\ -\mathbf{I} & \mathbf{B} & -\mathbf{I} \\ \mathbf{0} & -\mathbf{I} & \mathbf{B} \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 4 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 4 \end{bmatrix}, \tag{6.13}$$

e termine noto $h^2\omega_{i,j}$ più i valori di contorno di ψ adiacenti a ciascun nodo. La matrice è a banda con $2N + 1$ bande non nulle (semilarghezza di banda N), simmetrica e definita positiva. La fattorizzazione LU costa $\sim n^3/2$ operazioni per una matrice piena di ordine $n = N^2$, ma per una matrice a banda $\sim nN^2 = N^4$ e le sostituzioni $\sim nN = N^3$: fattorizzando una volta sola, come nel Cap. 4, ogni passo costa solo le sostituzioni. In alternativa i metodi iterativi: il SOR del secondo modulo o il metodo ADI che segue. A differenza del problema di Neumann per la pressione, la matrice di ψ non è singolare (condizioni di Dirichlet) e nessun metodo richiede precauzioni.

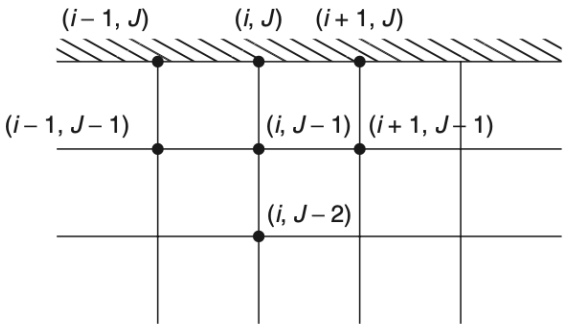


Figura 6.2: Griglia per l'equazione della funzione di corrente.

6.3 Il metodo ADI di Peaceman–Rachford

Il metodo *implicito a direzioni alternate* (ADI) risolve un'equazione ellittica introducendo uno pseudo-tempo: per l'equazione di Laplace $T_{xx} + T_{yy} = 0$ si integra

$$\frac{\partial T}{\partial t} = \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \tag{6.14}$$

fino allo stato stazionario, che è la soluzione cercata. L'accuratezza del transitorio non interessa: conta solo arrivare in fretta a $t \to \infty$. Lo schema di Peaceman e Rachford [25] avanza in due mezzi passi, ciascuno implicito in una sola direzione:

$$\frac{T_{i,j}^{n+1/2} - T_{i,j}^{n}}{\Delta t_1} = \frac{T_{i+1,j}^{n+1/2} - 2T_{i,j}^{n+1/2} + T_{i-1,j}^{n+1/2}}{\Delta x^2} + \frac{T_{i,j+1}^{n} - 2T_{i,j}^{n} + T_{i,j-1}^{n}}{\Delta y^2}, \tag{6.15}$$

$$\frac{T_{i,j}^{n+1} - T_{i,j}^{n+1/2}}{\Delta t_1} = \frac{T_{i+1,j}^{n+1/2} - 2T_{i,j}^{n+1/2} + T_{i-1,j}^{n+1/2}}{\Delta x^2} + \frac{T_{i,j+1}^{n+1} - 2T_{i,j}^{n+1} + T_{i,j-1}^{n+1}}{\Delta y^2}. \tag{6.16}$$

Con $\beta_r = \Delta x/\Delta y$ e $\rho = \Delta x^2/\Delta t_1$ (il parametro di iterazione):

$$T_{i+1,j}^{n+1/2} - (2+\rho)T_{i,j}^{n+1/2} + T_{i-1,j}^{n+1/2} = \beta_r^2 \left[-T_{i,j+1}^{n} + \left(2 - \frac{\rho}{\beta_r^2}\right) T_{i,j}^{n} - T_{i,j-1}^{n} \right], \tag{6.17}$$

$$T_{i,j+1}^{n+1} - \left(2 + \frac{\rho}{\beta_r^2}\right) T_{i,j}^{n+1} + T_{i,j-1}^{n+1} = \frac{1}{\beta_r^2} \left[-T_{i+1,j}^{n+1/2} + (2-\rho)T_{i,j}^{n+1/2} - T_{i-1,j}^{n+1/2} \right]. \tag{6.18}$$

Per l'equazione di Poisson $\nabla^2 T = \xi$ (in pseudo-tempo $T_t = \nabla^2 T - \xi$) e $\Delta x = \Delta y$ si aggiunge $-h^2 \xi_{i,j}$ a ciascun membro destro:

$$T_{i+1,j}^{n+1/2} - (2+\rho)T_{i,j}^{n+1/2} + T_{i-1,j}^{n+1/2} = -T_{i,j+1}^{n} + (2-\rho)T_{i,j}^{n} - T_{i,j-1}^{n} + h^2 \xi_{i,j}, \tag{6.19}$$

$$T_{i,j+1}^{n+1} - (2+\rho)T_{i,j}^{n+1} + T_{i,j-1}^{n+1} = -T_{i+1,j}^{n+1/2} + (2-\rho)T_{i,j}^{n+1/2} - T_{i-1,j}^{n+1/2} + h^2 \xi_{i,j} \tag{6.20}$$

(il segno del termine di sorgente dipende dalla convenzione: con $T_t = \nabla^2 T - \xi$ e portando il termine noto a destra si ottiene $+h^2\xi$; per $\nabla^2 \psi = -\omega$ si pone $\xi = -\omega$). I due sistemi si risolvono prima per righe e poi per colonne: ciascuno è *tridiagonale* e si risolve con Thomas o LU (la matrice è la stessa per tutte le righe, se ρ è fisso). La convergenza dipende fortemente da ρ: per l'equazione di Laplace nel quadrato il metodo converge per ogni $\rho > 0$ fisso purché lo stesso valore sia usato nei due mezzi passi; per scelte pratiche basta $\rho \approx 8/N$ (Ferziger), e la velocità massima si ottiene con una *successione* di parametri.

La successione di parametri di Wachspress

Per il quadrato con M intervalli per lato ($M-1$ nodi interni per direzione) Wachspress [26] propose la successione ciclica

$$r_{k+1} = \frac{1}{\rho_{k+1}} = \frac{1}{2\cos^2(\pi/2M)} \left[\cot^2 \frac{\pi}{2M}\right]^{k/(k_0-1)}, \qquad k = 0, 1, \ldots, k_0 - 1, \tag{6.21}$$

in cui $k_0 \geq 2$ è il più piccolo intero tale che

$$(\sqrt{2}-1)^{k_0-1} \leq \tan \frac{\pi}{2M}. \tag{6.22}$$

Per $M = 11$ si ha $k_0 = 4$ e $r = 0{,}51033610,\ 1{,}85949096,\ 6{,}77535187,\ 24{,}68707504$; per $M = 32$ (la griglia del progetto, 31×31 nodi interni) $k_0 = 5$ e $r = 0{,}501,\ 2{,}26,\ 10{,}2,\ 46{,}0,\ 208$.

L'idea: gli autovalori dell'operatore discreto per direzione vanno da $\lambda_{\min} = 4\sin^2(\pi/2M)$ a $\lambda_{\max} = 4\cos^2(\pi/2M)$, e un singolo passo ADI con parametro ρ smorza al meglio i modi con $\lambda \approx \rho$ (fattore di riduzione $|(\rho-\lambda)/(\rho+\lambda)|^2$ per il modo con autovalori λ in entrambe le direzioni); una successione geometrica di ρ_k che copre l'intervallo $[\lambda_{\min}, \lambda_{\max}]$ smorza *tutti* i modi in un ciclo, e il numero di parametri necessario cresce solo come $\log M$. Il ciclo si ripete finché il criterio di convergenza (per esempio tre ordini di grandezza di riduzione dell'errore) non è soddisfatto. L'Approfondimento sulla convergenza dell'ADI ne dà la dimostrazione.

6.3.1 Il Programma 4.6

Il Programma 4.6 realizza il progetto della fonte: cavità in formulazione ω–ψ con 31×31 nodi interni, Eulero esplicito per la vorticità e, per la funzione di corrente, il metodo ADI di Peaceman–Rachford con $\rho = 8/M$ fisso oppure con la successione di Wachspress, confrontato con il SOR a parametro ottimo. L'equazione (6.8) va risolta a convergenza *a ogni passo temporale*: il costo del risolutore ellittico domina il calcolo, e la scelta del parametro è decisiva.

Dati del Programma 4.6

$M = 32$ intervalli ($h = 1/32$), $Re = 1$, 100, 400; $\Delta t = 0{,}4\min\{Reh^2/4, h\}$; tolleranza 10^{-6} sulla variazione massima di ψ fra due iterazioni; criterio di stato stazionario $\max|\Delta\omega|/\Delta t < 10^{-5}$.

prog4_6_cavita_omega_psi.py

```
"""
prog4_6_cavita_omega_psi.py -- Programma 4.6: cavita' con coperchio mobile nella
formulazione vorticita'-funzione di corrente, griglia collocata, Eulero esplicito
per l'equazione del trasporto della vorticita',
    omega_t + u omega_x + v omega_y = lap omega / Re,     lap psi = -omega,
vorticita' di parete al primo ordine (omega_w = -2 psi_1/h^2 - 2 u_w/h sul
    coperchio).
L'equazione di Poisson per psi e' risolta a ogni passo con il SOR oppure con il
    metodo
ADI di Peaceman-Rachford usato come iterazione (pseudo-tempo), con parametro fisso
rho ~ 8/N oppure con la successione di Wachspress. Griglia con M = 32 intervalli
(31 x 31 nodi interni), Re = 1, 100, 400; confronto con Ghia et al. e con il
Programma 4.5 (variabili primitive).
"""
import time
import numpy as np
from scipy.linalg import solve_banded
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

GHIA = {100: (-0.103423, -3.16646), 400: (-0.113909, -2.29469), 1000: (-0.117929,
    -2.04968)}


def wachspress(M):
    """Successione di parametri r_k = 1/rho_k di Wachspress per il quadrato con M
    intervalli."""
    k0 = 2
    while (np.sqrt(2) - 1)**(k0 - 1) > np.tan(np.pi / (2 * M)):
        k0 += 1
```

```
    return [1 / (2 * np.cos(np.pi / (2 * M))**2) * (1 / np.tan(np.pi / (2 * M)))**(2 * k / (k0 - 1)) for k in range(k0)]


def tridiag_cost(a, b, c, D):
    """Risolve, per ogni colonna di D, il sistema tridiagonale a coefficienti costanti (a sotto, b diag, c sopra)."""
    n = D.shape[0]
    ab = np.zeros((3, n))
    ab[0, 1:] = c; ab[1, :] = b; ab[2, :-1] = a
    return solve_banded((1, 1), ab, D)


def poisson_adi(psi, f, h, rho, tol, it_max=500):
    """Iterazione ADI (Peaceman-Rachford) per lap psi = f (f = -omega), psi assegnata sul contorno.
    rho = h^2/dt_pseudo, scalare oppure successione ciclica. Restituisce psi e il numero di doppi passi."""
    n = psi.shape[0] - 2
    seq = np.atleast_1d(rho)
    for it in range(1, it_max + 1):
        r = seq[(it - 1) % len(seq)]
        vecchio = psi.copy()
        # mezzo passo implicito in x (sistemi lungo le righe j fissato): psi_(i+1) - (2+r) psi_i + psi_(i-1) = rhs
        rhs = -(psi[1:-1, 2:] + psi[1:-1, :-2]) + (2 - r) * psi[1:-1, 1:-1] + h * h * f[1:-1, 1:-1]
        rhs[0, :] -= psi[0, 1:-1]; rhs[-1, :] -= psi[-1, 1:-1]
        psi_s = psi.copy()
        psi_s[1:-1, 1:-1] = tridiag_cost(1.0, -(2 + r), 1.0, rhs)
        # mezzo passo implicito in y (sistemi lungo le colonne i fissato)
        rhs = -(psi_s[2:, 1:-1] + psi_s[:-2, 1:-1]) + (2 - r) * psi_s[1:-1, 1:-1] + h * h * f[1:-1, 1:-1]
        rhs[:, 0] -= psi[1:-1, 0]; rhs[:, -1] -= psi[1:-1, -1]
        psi[1:-1, 1:-1] = tridiag_cost(1.0, -(2 + r), 1.0, rhs.T).T
        if np.max(np.abs(psi - vecchio)) < tol:
            return psi, it
    return psi, it_max


def poisson_sor(psi, f, h, omega_rf, tol, it_max=5000):
    n = psi.shape[0]
    for it in range(1, it_max + 1):
        err = 0.0
        for i in range(1, n - 1):
            for j in range(1, n - 1):
                nuovo = (1 - omega_rf) * psi[i, j] + omega_rf / 4 * (
                    psi[i - 1, j] + psi[i + 1, j] + psi[i, j - 1] + psi[i, j + 1] - h * h * f[i, j])
                err = max(err, abs(nuovo - psi[i, j]))
                psi[i, j] = nuovo
        if err < tol:
            return psi, it
    return psi, it_max


def cavita_omega_psi(Re, M, t_max, risolutore="adi", rho=None, tol_psi=1e-6, tol_stat=1e-5, cfl=0.4):
    h = 1.0 / M
    n = M + 1
```

```
    x = np.linspace(0, 1, n)
    psi = np.zeros((n, n)); om = np.zeros((n, n)); u = np.zeros((n, n)); v = np.zeros((n, n))
    u[:, -1] = 1.0
    dt = min(cfl * 0.25 * Re * h * h, cfl * h)
    if rho is None:
        rho = 8.0 / M
    a = 2 * np.cos(np.pi / M)
    om_sor = (8 - 4 * np.sqrt(4 - a * a)) / (a * a)
    t, passo, res, it_tot = 0.0, 0, np.inf, 0
    t0 = time.perf_counter()
    while t < t_max and res > tol_stat:
        # vorticita' di parete (primo ordine)
        om[1:-1, 0] = -2 * psi[1:-1, 1] / h**2
        om[1:-1, -1] = -2 * psi[1:-1, -2] / h**2 - 2 * 1.0 / h
        om[0, 1:-1] = -2 * psi[1, 1:-1] / h**2
        om[-1, 1:-1] = -2 * psi[-2, 1:-1] / h**2
        # trasporto della vorticita' (Eulero esplicito, differenze centrate)
        omn = om.copy()
        omn[1:-1, 1:-1] = om[1:-1, 1:-1] + dt * (
            -u[1:-1, 1:-1] * (om[2:, 1:-1] - om[:-2, 1:-1]) / (2 * h)
            - v[1:-1, 1:-1] * (om[1:-1, 2:] - om[1:-1, :-2]) / (2 * h)
            + (om[2:, 1:-1] + om[:-2, 1:-1] + om[1:-1, 2:] + om[1:-1, :-2] - 4 * om[1:-1, 1:-1]) / (h * h * Re))
        res = np.max(np.abs(omn[1:-1, 1:-1] - om[1:-1, 1:-1])) / dt
        om = omn
        # funzione di corrente
        if risolutore == "adi":
            psi, it = poisson_adi(psi, -om, h, rho, tol_psi)
        else:
            psi, it = poisson_sor(psi, -om, h, om_sor, tol_psi)
        it_tot += it
        u[1:-1, 1:-1] = (psi[1:-1, 2:] - psi[1:-1, :-2]) / (2 * h)
        v[1:-1, 1:-1] = -(psi[2:, 1:-1] - psi[:-2, 1:-1]) / (2 * h)
        t += dt; passo += 1
    durata = time.perf_counter() - t0
    return dict(Re=Re, M=M, x=x, psi=psi, om=om, u=u, v=v, t=t, passi=passo, it_medie=it_tot / max(passo, 1),
                durata=durata, res=res, dt=dt)


if __name__ == "__main__":
    M = 32
    print(f"Cavita' in formulazione omega-psi, M = {M} intervalli ({M-1}x{M-1} nodi interni)")
    print(f"   parametri di Wachspress (M = {M}): r_k = " + ", ".join(f"{r:.4f}" for r in wachspress(M)))
    print(f"   esempio del testo, M = 11: r_k = " + ", ".join(f"{r:.8f}" for r in wachspress(11)))
    ris = {}
    for Re, tmax in ((1, 2.0), (100, 30.0), (400, 60.0)):
        r = cavita_omega_psi(Re, M, tmax, "adi")
        ris[Re] = r
        k = np.unravel_index(np.argmin(r["psi"]), r["psi"].shape)
        print(f"   Re = {Re:4d}: {r['passi']} passi (dt = {r['dt']:.2e}), t = {r['t']:.1f}, residuo {r['res']:.1e},"
              f" {r['it_medie']:.1f} doppi passi ADI per passo, {r['durata']:.1f} s")
        print(f"      psi_min = {r['psi'].min():.5f} in ({k[0]/M:.4f}, {k[1]/M:.4f}), omega_c = {r['om'][k]:.4f}"
```

```
            + (f";  Ghia: {GHIA[Re][0]:.5f}, {GHIA[Re][1]:.4f}" if Re in GHIA else ""))
    # confronto fra i risolutori per l'equazione di Poisson (un singolo problema: omega finale di Re = 100)
    print("\nRisolutori per lap psi = -omega (omega finale di Re = 100, tolleranza 1e-6 sulla variazione massima):")
    om = ris[100]["om"]; h = 1.0 / M
    for nome, fun in (("SOR, omega ottimo", lambda p: poisson_sor(p, -om, h, (8 - 4 * np.sqrt(4 - (2 * np.cos(np.pi / M))**2)) / (2 * np.cos(np.pi / M))**2, 1e-6)),
                      ("ADI, rho = 8/M fisso", lambda p: poisson_adi(p, -om, h, 8.0 / M, 1e-6)),
                      ("ADI, rho = 1 fisso", lambda p: poisson_adi(p, -om, h, 1.0, 1e-6)),
                      ("ADI, successione di Wachspress", lambda p: poisson_adi(p, -om, h, 1 / np.array(wachspress(M)), 1e-6))):
        t0 = time.perf_counter()
        p, it = fun(np.zeros((M + 1, M + 1)))
        print(f"   {nome:32s}: {it:4d} iterazioni, {time.perf_counter()-t0:.3f} s, psi_min = {p.min():.6f}")
    # la stessa cavita' con il SOR nel ciclo temporale (Re = 100) per confronto dei tempi
    r = cavita_omega_psi(100, M, 30.0, "sor")
    print(f"\n   Re = 100 con SOR nel ciclo temporale: {r['it_medie']:.1f} iterazioni per passo, {r['durata']:.1f} s"
          f" (ADI: {ris[100]['it_medie']:.1f}, {ris[100]['durata']:.1f} s); psi_min = {r['psi'].min():.5f}")

    fig, axs = plt.subplots(2, 3, figsize=(8.0, 5.0))
    for col, Re in enumerate((1, 100, 400)):
        r = ris[Re]; x = r["x"]
        lv = [-0.115, -0.11, -0.1, -0.09, -0.07, -0.05, -0.03, -0.01, -1e-4, 1e-5, 1e-4, 5e-4, 1e-3, 1.5e-3]
        axs[0, col].contour(x, x, r["psi"].T, levels=lv, colors=[ROSSO if v > 0 else BLU for v in lv], linewidths=0.7)
        lo = [-5, -4, -3, -2, -1, -0.5, 0, 0.5, 1, 2, 3, 4, 5]
        axs[1, col].contour(x, x, r["om"].T, levels=lo, colors=[ROSSO if v > 0 else BLU for v in lo], linewidths=0.7)
        axs[0, col].set_title(f"$\\psi$, $Re={Re}$", fontsize=8); axs[1, col].set_title(f"$\\omega$, $Re={Re}$", fontsize=8)
        for a_ in axs[:, col]:
            a_.set_aspect("equal"); a_.set_xticks([]); a_.set_yticks([])
    fig.tight_layout()
    salva(fig, "prog4_6_cavita_omega_psi")
```

Listato 6.1: Programma 4.6 – cavità con coperchio mobile in formulazione vorticità–funzione di corrente: Eulero esplicito, SOR e ADI di Peaceman–Rachford con parametri di Wachspress (`prog4_6_cavita_omega_psi.py`).

Risultati del Programma 4.6

- **Soluzioni.** A $Re = 100$, $\psi_c = -0{,}1004$ e $\omega_c = -3{,}21$ (Ghia: $-0{,}1034$, $-3{,}17$), a $Re = 400$ $\psi_c = -0{,}0941$ e $\omega_c = -2{,}11$ ($-0{,}1139$, $-2{,}29$): con 31×31 nodi interni e la formula di Thom del primo ordine l'errore è del 3% a $Re = 100$ e del 17% a $Re = 400$, dove gli strati limite sono sottili. Le linee di corrente e di vorticità (Fig. 6.3) hanno la stessa struttura della Fig. 5.8.
- **Confronto fra i risolutori** (stesso problema di Poisson, ω finale di $Re = 100$, partenza da zero, tolleranza 10^{-6}): SOR con ω ottimo 72 iterazioni; ADI con

$\rho = 8/M$ fisso 61 doppi passi; ADI con $\rho = 1$ fisso 205; ADI con la successione di Wachspress **11** doppi passi (poco più di due cicli di 5 parametri). In tempo di calcolo l'ADI vettorializzato è 10–50 volte più rapido del SOR scritto con cicli espliciti.

- **Nel ciclo temporale** lo stato precedente è un'ottima stima iniziale e bastano 4–6 doppi passi ADI per passo temporale contro 9 iterazioni SOR; il tempo totale a $Re = 100$ passa da 36 s (SOR) a 2 s (ADI).

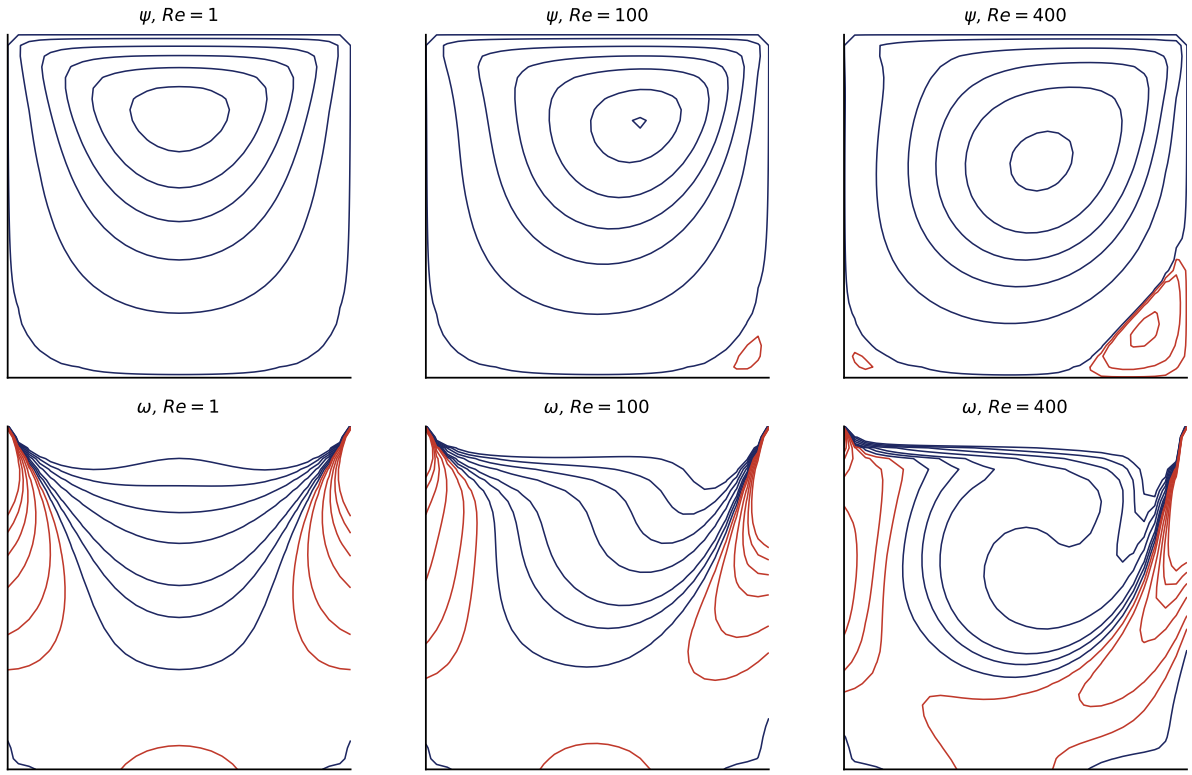


Figura 6.3: Programma 4.6: linee di corrente e di vorticità per $Re = 1, 100, 400$ con 31×31 nodi interni.

6.4 La corrente intorno a un cilindro circolare

L'ultimo esempio è una corrente *esterna*: un cilindro circolare di raggio a investito da una corrente uniforme U_∞ (Fig. 6.4). È il problema classico della scia a basso numero di Reynolds, con la formulazione di Peyret e Taylor [4]. In coordinate polari, con $Re = 2U_\infty a/\nu$ basato sul diametro e grandezze adimensionali rispetto ad a e U_∞,

$$\frac{\partial^2\psi}{\partial r^2} + \frac{1}{r}\frac{\partial\psi}{\partial r} + \frac{1}{r^2}\frac{\partial^2\psi}{\partial\theta^2} = -\omega, \tag{6.23}$$

$$\frac{\partial\omega}{\partial t} + v_r\frac{\partial\omega}{\partial r} + \frac{v_\theta}{r}\frac{\partial\omega}{\partial\theta} = \frac{2}{Re}\left(\frac{\partial^2\omega}{\partial r^2} + \frac{1}{r}\frac{\partial\omega}{\partial r} + \frac{1}{r^2}\frac{\partial^2\omega}{\partial\theta^2}\right), \tag{6.24}$$

con $v_r = (1/r)\partial\psi/\partial\theta$ e $v_\theta = -\partial\psi/\partial r$. Il cilindro è in quiete per $t < 0$; a $t = 0$ la corrente uniforme viene applicata impulsivamente. Si risolvono le equazioni non stazionarie con aderenza sulla superficie e corrente uniforme all'infinito. Per $Re \lesssim 40$ la corrente resta

simmetrica rispetto all'asse e stazionaria; oltre, la soluzione biforca rapidamente verso il regime oscillante della scia di von Kármán. Si studiano i casi simmetrici, sfruttando la simmetria, e l'effetto di *Re* sulla lunghezza L/d della bolla di ricircolo e sulla resistenza.

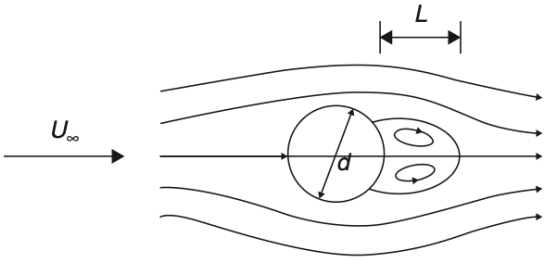


Figura 6.4: Geometria della corrente intorno al cilindro.

6.4.1 La trasformazione di coordinate

La difficoltà è il dominio *illimitato*. La si risolve con la trasformazione algebrica di Ta Phuoc Loc [27],

$$r = \frac{\bar{r}}{a} = e^{\pi\xi}, \qquad \theta = \pi\eta, \tag{6.25}$$

che manda la corona $1 \le r \le e^{\pi\xi_\infty}$ nel rettangolo $0 \le \xi \le \xi_\infty$, $0 \le \eta \le 1$ (semipiano superiore; Fig. 6.5). Basta $\xi_\infty = 1$, cioè $r_\infty = e^\pi = 23{,}14$, abbastanza lontano dal cilindro. La trasformazione è *conforme* (è la mappa $\ln z$ a meno di fattori): poiché $\partial\eta/\partial\theta = 1/\pi$ e $\partial\xi/\partial r = 1/(\pi r) = e^{-\pi\xi}/\pi$,

$$v_r = \frac{1}{r}\frac{\partial\psi}{\partial\theta} = \frac{1}{\pi e^{\pi\xi}}\frac{\partial\psi}{\partial\eta}, \qquad v_\theta = -\frac{\partial\psi}{\partial r} = -\frac{1}{\pi e^{\pi\xi}}\frac{\partial\psi}{\partial\xi}, \tag{6.26}$$

e il laplaciano si trasforma senza termini del primo ordine (l'Esercizio 7 lo verifica con SymPy):

$$\frac{\partial\omega}{\partial t} + \frac{1}{g(\xi)}\left(\frac{\partial\psi}{\partial\eta}\frac{\partial\omega}{\partial\xi} - \frac{\partial\psi}{\partial\xi}\frac{\partial\omega}{\partial\eta}\right) = \frac{2}{Re}\frac{1}{g(\xi)}\left(\frac{\partial^2\omega}{\partial\xi^2} + \frac{\partial^2\omega}{\partial\eta^2}\right), \tag{6.27}$$

$$\frac{\partial^2\psi}{\partial\xi^2} + \frac{\partial^2\psi}{\partial\eta^2} = -g(\xi)\,\omega, \tag{6.28}$$

$$g(\xi) = \pi^2 e^{2\pi\xi}. \tag{6.29}$$

La (6.27) è parabolica nel tempo ed ellittica nello spazio; la (6.28) è un'equazione di Poisson con coefficiente g che cresce esponenzialmente con ξ: le celle della griglia uniforme nel piano (ξ,η) corrispondono a celle fisiche sempre più grandi allontanandosi dal cilindro, e la spaziatura radiale alla parete vale πh in unità di a.

6.4.2 Le condizioni al contorno

a. **Superficie solida** A–B ($\xi = 0$, $j = 1$): $\psi = 0$ e aderenza, $v_r = v_\theta = 0$, cioè $\partial\psi/\partial\eta = \partial^2\psi/\partial\eta^2 = 0$ e $\partial\psi/\partial\xi = 0$; con $g(0) = \pi^2$ la (6.28) si riduce a $\omega = -\pi^{-2}\partial^2\psi/\partial\xi^2$, e lo sviluppo di Taylor $\psi_{i,2} = \frac{1}{2}(\partial^2\psi/\partial\xi^2)_{i,1}\Delta\xi^2$ dà la formula di Thom

$$\omega_{i,1}^{n+1} = -\frac{2\psi_{i,2}^{n+1}}{\pi^2\Delta\xi^2}, \qquad \psi_{i,1} = 0. \tag{6.30}$$

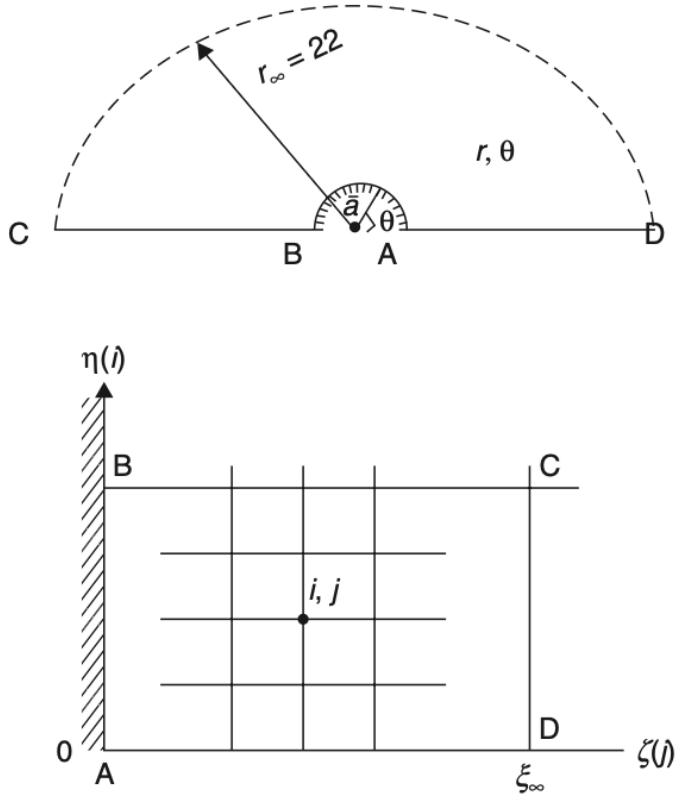


Figura 6.5: Il cilindro nelle coordinate trasformate.

b. **Assi di simmetria** C–B **e** A–D ($\eta = 1$ e $\eta = 0$): $\psi = 0$, $\omega = 0$.

c. **Contorno esterno** C–D ($\xi = \xi_\infty$, $j = J$): corrente irrotazionale, $\omega_{i,J} = 0$, e funzione di corrente della corrente potenziale intorno al cilindro, $\psi = r\sin\theta(1 - 1/r^2)$, che nelle nuove variabili vale

$$\psi_{i,J} = 2\sin(\pi\eta)\sinh(\pi\xi_\infty). \tag{6.31}$$

6.4.3 La resistenza

Il coefficiente di resistenza, riferito alla pressione dinamica e al diametro, è

$$C_D = \frac{D}{\frac{1}{2}\rho U_\infty^2 d} = \frac{D_s + D_p}{\rho U_\infty^2 a}, \tag{6.32}$$

somma dell'attrito D_s e della resistenza di pressione D_p:

$$\frac{D_s}{\rho U_\infty^2 a} + \frac{D_p}{\rho U_\infty^2 a} = -\frac{4\pi}{Re}\int_0^1 \omega\sin(\pi\eta)\,\mathrm{d}\eta - 2\pi\int_0^1 P\cos(\pi\eta)\,\mathrm{d}\eta, \tag{6.33}$$

con $P = p/(\rho U_\infty^2)$ la pressione adimensionale sulla superficie e ω la vorticità di parete (l'Esercizio 20 ricava la formula dalle componenti dell'equazione del moto sulla superficie). La pressione si ottiene dalla componente azimutale dell'equazione del moto valutata sulla superficie, dove per l'aderenza restano solo il gradiente di pressione e il termine viscoso:

$$\left(\frac{\partial P}{\partial \eta}\right)_{i,1} = \frac{2}{Re}\left(\frac{\partial \omega}{\partial \xi}\right)_{i,1}, \tag{6.34}$$

integrata con la regola dei trapezi,

$$P_{i,1} = P_{i-1,1} + \frac{\Delta\eta}{Re}\left[\left(\frac{\partial\omega}{\partial\xi}\right)_{i,1} + \left(\frac{\partial\omega}{\partial\xi}\right)_{i-1,1}\right], \tag{6.35}$$

a partire dal nodo $(1,1)$. La fonte suggerisce di assumervi la pressione di ristagno ($P_{1,1} = \frac{1}{2}$ nelle unità scelte); in realtà la costante di integrazione è irrilevante per la resistenza, perché $\int_0^1 \cos(\pi\eta)\,\mathrm{d}\eta = 0$: il programma pone $P = 0$ nel punto posteriore e riferisce il coefficiente di pressione al punto anteriore solo per la rappresentazione grafica.

6.4.4 Il Programma 4.6b

Il Programma 4.6b integra le (6.27)–(6.28) con Eulero esplicito e differenze centrate per ω, e con il risolutore diretto sparso (fattorizzazione una tantum) per ψ. Il passo temporale è il minimo fra il limite diffusivo alla parete, $Re\pi^2h^2/8$ (dove g è minimo e la diffusione adimensionale massima), e il limite convettivo valutato sul campo corrente, con fattore di sicurezza 0,5. Partendo dalla corrente potenziale ($\omega = 0$) si integra fino allo stato stazionario (residuo max $|\partial\omega/\partial t| < 10^{-5}$) e si calcolano: l'angolo di separazione θ_s (dove la vorticità di parete cambia segno, misurato dal punto posteriore), la lunghezza della bolla L (dove la velocità sull'asse posteriore si annulla, cioè dove $\psi_{2,j}$ cambia segno), la resistenza dalla (6.33). I risultati sono confrontati con i calcoli di riferimento di Dennis e Chang [28] e di Fornberg [29] e con il valore sperimentale del numero di Reynolds di separazione (5–7).

Dati del Programma 4.6b

$h = \Delta\xi = \Delta\eta = 1/60$ (61×61 nodi, $\Delta r/a = 0{,}052$ alla parete), $\xi_\infty = 1$; $Re = 2$, 10, 20, 36 per le figure; $Re = 4$–8 per la separazione; sensibilità alla griglia ($N = 40$, 60, 80) e alla posizione del contorno esterno ($\xi_\infty = 0{,}75$–$1{,}5$, cioè $r_\infty/a = 10{,}6$–111) a $Re = 20$.

prog4_6b_cilindro.py

```
"""
prog4_6b_cilindro.py -- corrente uniforme intorno a un cilindro circolare a basso
numero di Reynolds (2 <= Re <= 40, Re = U d/nu), formulazione vorticita'-funzione
di corrente nelle coordinate trasformate di Ta Phuoc Loc
    r/a = exp(pi xi),   theta = pi eta,          g(xi) = pi^2 exp(2 pi xi),
    omega_t + (psi_eta omega_xi - psi_xi omega_eta)/g = (2/Re) (omega_xixi +
    omega_etaeta)/g,
    psi_xixi + psi_etaeta = -g omega.
Meta' superiore del dominio (0 <= eta <= 1), 0 <= xi <= xi_inf = 1 (r_inf = e^pi =
    23,1 a).
Condizioni: psi = omega = 0 sull'asse di simmetria; sulla superficie psi = 0 e
omega = -2 psi_(i,2)/(pi^2 h^2); sul contorno esterno omega = 0 e psi = corrente
potenziale 2 sin(pi eta) sinh(pi xi_inf). Avviamento impulsivo (psi potenziale,
omega = 0). Eulero esplicito con differenze centrate per omega; l'equazione di
Poisson, a coefficienti costanti, e' risolta a ogni passo con la fattorizzazione
LU sparsa calcolata una volta sola.
Risultati: Re di separazione, lunghezza della bolla L/d, angolo di separazione,
coefficiente di resistenza (attrito + pressione) dalla (4.6.60), confronto con
Dennis e Chang (1970) e Fornberg (1980).
"""
import time
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO
```

```
25
26 XI_INF = 1.0
27 # Dennis e Chang (1970): Re, C_D, lunghezza della scia L/a dal punto posteriore,
       angolo di separazione (gradi)
28 DENNIS = {5: (4.116, None, None), 7: (3.421, None, None), 10: (2.846, 0.53, 29.6),
29           20: (2.045, 1.88, 43.7), 40: (1.522, 4.69, 53.8)}
30 FORNBERG = {20: (2.0001, 1.82), 40: (1.4980, 4.48)}   # Re: (C_D, L/a)
31
32
33 def matrice_poisson(N, Nx, h):
34     """psi_xixi + psi_etaeta sui nodi interni (i = 1..N-1 in eta, j = 1..Nx-1 in
       xi), Dirichlet ovunque."""
35     n, nx = N - 1, Nx - 1
36     idx = lambda i, j: (i - 1) * nx + (j - 1)
37     A = sp.lil_matrix((n * nx, n * nx))
38     for i in range(1, N):
39         for j in range(1, Nx):
40             p = idx(i, j)
41             A[p, p] = -4.0 / h**2
42             for ii, jj in ((i + 1, j), (i - 1, j), (i, j + 1), (i, j - 1)):
43                 if 1 <= ii <= N - 1 and 1 <= jj <= Nx - 1:
44                     A[p, idx(ii, jj)] = 1.0 / h**2
45     return spla.splu(A.tocsc())
46
47
48 def cilindro(Re, N=60, t_max=200.0, tol=1e-5, cfl=0.5, stampa=True, xi_inf=XI_INF):
49     """N intervalli in eta (h = 1/N); in xi lo stesso passo h fino a xi_inf (Nx =
       xi_inf N intervalli)."""
50     h = 1.0 / N
51     Nx = int(round(xi_inf * N))
52     eta = np.linspace(0, 1, N + 1); xi = h * np.arange(Nx + 1)
53     g = np.pi**2 * np.exp(2 * np.pi * xi)[None, :]           # g(xi), forma (1,
       Nx+1)
54     psi_pot = 2 * np.sin(np.pi * eta)[:, None] * np.sinh(np.pi * xi)[None, :]
55     psi = psi_pot.copy()
56     om = np.zeros((N + 1, Nx + 1))
57     LU = matrice_poisson(N, Nx, h)
58     # passo temporale: limite diffusivo alla parete (g = pi^2) e limite convettivo
59     dt_diff = Re * np.pi**2 * h**2 / 8
60     t, passo, res = 0.0, 0, np.inf
61     t0 = time.perf_counter()
62     storia = []
63     while t < t_max and res > tol:
64         # vorticita' sulla superficie (primo ordine) e sul contorno esterno / asse
65         om[:, 0] = -2 * psi[:, 1] / (np.pi**2 * h**2)
66         om[0, :] = om[-1, :] = 0.0; om[:, -1] = 0.0
67         psi_xi = (psi[1:-1, 2:] - psi[1:-1, :-2]) / (2 * h)
68         psi_eta = (psi[2:, 1:-1] - psi[:-2, 1:-1]) / (2 * h)
69         gi = g[:, 1:-1]
70         vel = max(np.max(np.abs(psi_xi / gi)), np.max(np.abs(psi_eta / gi)))
71         dt = cfl * min(dt_diff, h / max(vel, 1e-12))
72         om_xi = (om[1:-1, 2:] - om[1:-1, :-2]) / (2 * h)
73         om_eta = (om[2:, 1:-1] - om[:-2, 1:-1]) / (2 * h)
74         lap = (om[1:-1, 2:] + om[1:-1, :-2] + om[2:, 1:-1] + om[:-2, 1:-1] - 4 *
       om[1:-1, 1:-1]) / h**2
75         incr = (-(psi_eta * om_xi - psi_xi * om_eta) + 2.0 / Re * lap) / gi
76         om[1:-1, 1:-1] += dt * incr
77         res = np.max(np.abs(incr))
78         # funzione di corrente: psi_xixi + psi_etaeta = -g omega, contorno esterno
       potenziale
```

```
        rhs = -(g * om)[1:-1, 1:-1].copy()
        rhs[:, -1] -= psi_pot[1:-1, -1] / h**2
        psi[1:-1, 1:-1] = LU.solve(rhs.ravel()).reshape(N - 1, Nx - 1)
        t += dt; passo += 1
        if passo % 200 == 0:
            storia.append((t, coeff_resistenza(om, Re, h)[0], bolla(psi, xi)))
    r = analisi(psi, om, Re, h, eta, xi)
    r.update(t=t, passi=passo, res=res, durata=time.perf_counter() - t0,
    storia=storia, psi=psi, om=om,
             eta=eta, xi=xi, N=N, dt=dt, xi_inf=xi_inf)
    if stampa:
        print(f"   Re = {Re:5.1f}: {passo:6d} passi, t = {t:6.1f}, residuo
    {res:.1e} ({r['durata']:.0f} s);"
              f" C_D = {r['cd']:.3f} (attrito {r['cds']:.3f}, pressione
    {r['cdp']:.3f});"
              f" theta_s = {r['theta_s']:5.1f} gradi, L/a = {r['L']:.3f} (L/d =
    {r['L']/2:.3f})")
    return r


def coeff_resistenza(om, Re, h):
    """(4.6.60): C_D = -(4 pi/Re) int omega sin(pi eta) deta - 2 pi int P cos(pi
    eta) deta,
    con dP/deta = (2/Re) domega/dxi sulla superficie (4.6.61)-(4.6.62), P(eta = 0)
    = 0 (la costante
    additiva non contribuisce alla resistenza). Integrazione con la regola dei
    trapezi."""
    N = om.shape[0] - 1
    eta = np.linspace(0, 1, N + 1)
    om_w = om[:, 0]
    dom_dxi = (-3 * om[:, 0] + 4 * om[:, 1] - om[:, 2]) / (2 * h)
    P = np.concatenate([[0.0], np.cumsum(h / Re * (dom_dxi[1:] + dom_dxi[:-1]))])
    cds = -4 * np.pi / Re * np.trapezoid(om_w * np.sin(np.pi * eta), eta)
    cdp = -2 * np.pi * np.trapezoid(P * np.cos(np.pi * eta), eta)
    return cds + cdp, cds, cdp, P


def bolla(psi, xi):
    """Lunghezza della bolla di ricircolo L/a lungo l'asse posteriore (eta = 0): la
    velocita' sull'asse
    e' u = psi_eta/(pi r) ~ psi_(1,j)/(h pi r); la bolla termina dove psi_(1,j)
    cambia segno."""
    s = psi[1, :]
    j = np.where((s[:-1] < 0) & (s[1:] >= 0))[0]
    if len(j) == 0:
        return 0.0
    j = j[0]
    xs = xi[j] + (xi[j + 1] - xi[j]) * (-s[j]) / (s[j + 1] - s[j])
    return np.exp(np.pi * xs) - 1.0


def analisi(psi, om, Re, h, eta, xi):
    cd, cds, cdp, P = coeff_resistenza(om, Re, h)
    om_w = om[:, 0]
    # angolo di separazione dal punto posteriore: omega_w cambia segno (omega_w > 0
    nella bolla, eta piccolo)
    k = np.where((om_w[1:-1] > 0) & (om_w[2:] <= 0))[0]
    if len(k):
        k = k[0] + 1
        es = eta[k] + h * om_w[k] / (om_w[k] - om_w[k + 1])
```

```
        theta_s = 180.0 * es
    else:
        theta_s = 0.0
    return dict(Re=Re, cd=cd, cds=cds, cdp=cdp, P=P, om_w=om_w, theta_s=theta_s,
    L=bolla(psi, xi))


def disegna_linee(ax, r, titolo):
    eta, xi, psi = r["eta"], r["xi"], r["psi"]
    R = np.exp(np.pi * xi)[None, :]; TH = np.pi * eta[:, None]
    X = R * np.cos(TH); Y = R * np.sin(TH)
    lv_pos = [0.05, 0.2, 0.5, 1.0, 1.5, 2.0, 3.0]
    lv_neg = [-0.08, -0.04, -0.02, -0.01, -0.004, -0.001]
    ax.contour(X, Y, psi, levels=lv_pos, colors=BLU, linewidths=0.7)
    ax.contour(X, Y, psi, levels=lv_neg, colors=ROSSO, linewidths=0.7,
    linestyles="--")
    ax.contour(X, -Y, psi, levels=lv_pos, colors=BLU, linewidths=0.7)
    ax.contour(X, -Y, psi, levels=lv_neg, colors=ROSSO, linewidths=0.7,
    linestyles="--")
    ax.add_patch(plt.Circle((0, 0), 1.0, color=GRIGIO))
    ax.set_xlim(-3, 10); ax.set_ylim(-3, 3); ax.set_aspect("equal")
    ax.set_title(titolo, fontsize=8); ax.set_xlabel("$x/a$"); ax.set_ylabel("$y/a$")


if __name__ == "__main__":
    N = 60
    print(f"Cilindro circolare: griglia {N+1}x{N+1} (h = 1/{N}), xi_inf = {XI_INF}
    (r_inf/a = {np.exp(np.pi*XI_INF):.2f})")
    print("   spaziatura radiale alla parete: Delta r/a = pi h =", f"{np.pi/N:.4f}")
    ris = {}
    for Re, tmax in ((2, 40.0), (10, 80.0), (20, 150.0), (36, 250.0)):
        ris[Re] = cilindro(Re, N, tmax)
        if Re in DENNIS:
            d = DENNIS[Re]
            print(f"             Dennis-Chang: C_D = {d[0]:.3f}" + (f", theta_s =
    {d[2]} gradi, L/a = {d[1]}" if d[1] else ""))
    print("\nRicerca del numero di Reynolds di separazione (prima comparsa di
    omega_w > 0 sul punto posteriore):")
    sep = {}
    for Re in (4.0, 5.0, 6.0, 7.0, 8.0):
        r = cilindro(Re, N, 60.0, stampa=False)
        sep[Re] = (r["om_w"][1], r["L"], r["cd"])
        print(f"   Re = {Re:3.0f}: omega_w vicino al punto posteriore =
    {r['om_w'][1]:+.4f}, L/a = {r['L']:.3f}, C_D = {r['cd']:.3f}"
              + (f" (Dennis-Chang {DENNIS[int(Re)][0]:.3f})" if int(Re) in DENNIS
    else ""))
    Res = sorted(sep); vals = [sep[R][0] for R in Res]
    k = np.where((np.array(vals[:-1]) < 0) & (np.array(vals[1:]) >= 0))[0]
    if len(k):
        k = k[0]; Re_s = Res[k] + (Res[k + 1] - Res[k]) * (-vals[k]) / (vals[k + 1]
    - vals[k])
        print(f"   -> Re di separazione (interpolazione lineare): {Re_s:.1f}
    (esperimenti: 5-7; Dennis-Chang: 6,2)")
    print("\nPosizione del contorno esterno (Re = 20, h = 1/60):")
    for xinf in (0.75, 1.0, 1.25, 1.5):
        r = cilindro(20, N, 150.0, stampa=False, xi_inf=xinf) if xinf != 1.0 else
    ris[20]
        print(f"   xi_inf = {xinf:.2f} (r_inf/a = {np.exp(np.pi*xinf):6.1f}): C_D =
    {r['cd']:.3f}, theta_s = {r['theta_s']:.1f}, L/a = {r['L']:.3f}")
    print("\nSensibilita' alla griglia (Re = 20):")
```

```
    r40 = cilindro(20, 40, 150.0, stampa=False); r80 = cilindro(20, 80, 150.0, stampa=False)
    for r in (r40, ris[20], r80):
        print(f"   N = {r['N']:3d}: C_D = {r['cd']:.3f}, theta_s = {r['theta_s']:.1f}, L/a = {r['L']:.3f}, {r['durata']:.0f} s")

    # ------------------------------------------------------------------ figure
    fig, axs = plt.subplots(2, 2, figsize=(8.0, 4.6))
    for a_, Re in zip(axs.ravel(), (2, 10, 20, 36)):
        r = ris[Re]
        disegna_linee(a_, r, f"$Re={Re}$: $C_D={r['cd']:.2f}$, $L/d={r['L']/2:.2f}$".replace(".", "{,}"))
    fig.tight_layout()
    salva(fig, "prog4_6b_cilindro_linee")

    fig, ax = plt.subplots(1, 3, figsize=(8.4, 2.7))
    for Re, col in zip((2, 10, 20, 36), (GRIGIO, VERDE, ARANCIO, BLU)):
        r = ris[Re]
        th = 180 * r["eta"]
        ax[0].plot(th, -r["om_w"] * 2 / Re, color=col, label=f"$Re={Re}$")
        ax[1].plot(th, 2 * (r["P"] - r["P"][-1]) + 1.0 + 0 * th, color=col)   # C_p riferito al punto anteriore (ipotesi C_p = 1 ... vedi testo)
    ax[0].axhline(0, color="k", lw=0.5); ax[0].set_xlabel("$\\theta$ dal punto posteriore [gradi]")
    ax[0].set_ylabel("$c_f = -2\\omega_w/Re$"); ax[0].legend(fontsize=7); ax[0].set_title("attrito superficiale", fontsize=8)
    ax[1].set_xlabel("$\\theta$ [gradi]"); ax[1].set_ylabel("$C_p$")
    ax[1].set_title("pressione superficiale", fontsize=8)
    Re_d = sorted(DENNIS); ax[2].plot(Re_d, [DENNIS[R][0] for R in Re_d], "s", color=GRIGIO, mfc="none", label="Dennis--Chang (1970)")
    ax[2].plot(sorted(FORNBERG), [FORNBERG[R][0] for R in sorted(FORNBERG)], "^", color=GRIGIO, mfc="none", label="Fornberg (1980)")
    Re_all = sorted(list(ris) + list(sep))
    cd_all = [ris[R]["cd"] if R in ris else sep[R][2] for R in Re_all]
    ax[2].plot(Re_all, cd_all, "o-", color=BLU, ms=3, label="presente calcolo")
    ax[2].set_xscale("log"); ax[2].set_xlabel("$Re$"); ax[2].set_ylabel("$C_D$"); ax[2].legend(fontsize=7)
    ax[2].set_title("resistenza", fontsize=8)
    fig.tight_layout()
    salva(fig, "prog4_6b_cilindro_cd")
```

Listato 6.2: Programma 4.6b – corrente intorno a un cilindro circolare in formulazione ω–ψ e coordinate trasformate: separazione, bolla di ricircolo, resistenza (`prog4_6b_cilindro.py`).

Risultati del Programma 4.6b

- **Separazione.** La vorticità di parete nel primo nodo dal punto posteriore passa da negativa (corrente attaccata) a positiva fra $Re = 6$ e 7; l'interpolazione dà $Re_{\text{sep}} = 6{,}3$, in accordo con gli esperimenti (5–7) e con il valore 6,2 di Dennis e Chang. Al crescere di Re il punto di separazione avanza: $\theta_s = 29{,}3°$ a $Re = 10$, $43{,}5°$ a 20, $52{,}1°$ a 36 (Dennis–Chang: $29{,}6°$, $43{,}7°$, $53{,}8°$ a $Re = 40$).
- **Bolla di ricircolo.** $L/d = 0{,}24$, 0,88, 1,86 a $Re = 10$, 20, 36: la lunghezza cresce quasi linearmente con Re, come negli esperimenti di Taneda e nei dati raccolti da Panton [30] ($L/d \approx 0{,}05(Re - 6)$). A $Re = 20$, $L/a = 1{,}77$ contro 1,88 (Dennis–Chang) e 1,82 (Fornberg).
- **Resistenza.** $C_D = 7{,}87$, 3,03, 2,16, 1,67 a $Re = 2$, 10, 20, 36, con la quota di pressione

che cresce dal 51% al 65%. Rispetto a Dennis e Chang (2,846 a $Re = 10$, 2,045 a 20) i valori sono del 6% più alti.

- **Effetto del contorno esterno.** La differenza non è di griglia: raffinando da $N = 40$ a 80 il C_D a $Re = 20$ cambia di 0,2%. È invece la posizione del contorno esterno: allontanandolo da $r_\infty = 10{,}6a$ a $23a$, $51a$, $111a$ il C_D scende da 2,45 a 2,16, 2,05, 2,02 e L/a sale a 1,83, convergendo ai valori di Fornberg (2,000, 1,82), che usò $r_\infty \approx 600a$. La corrente potenziale imposta sul contorno esterno sovrastima la velocità vicino al corpo (la scia reale a basso Re è larga e lenta, e la perturbazione decade solo come $1/\sqrt{r}$): confinare il dominio "accelera" la corrente e aumenta la resistenza. Un risultato di validazione importante: l'errore sistematico del 6% non è dello schema, ma del modello di contorno.

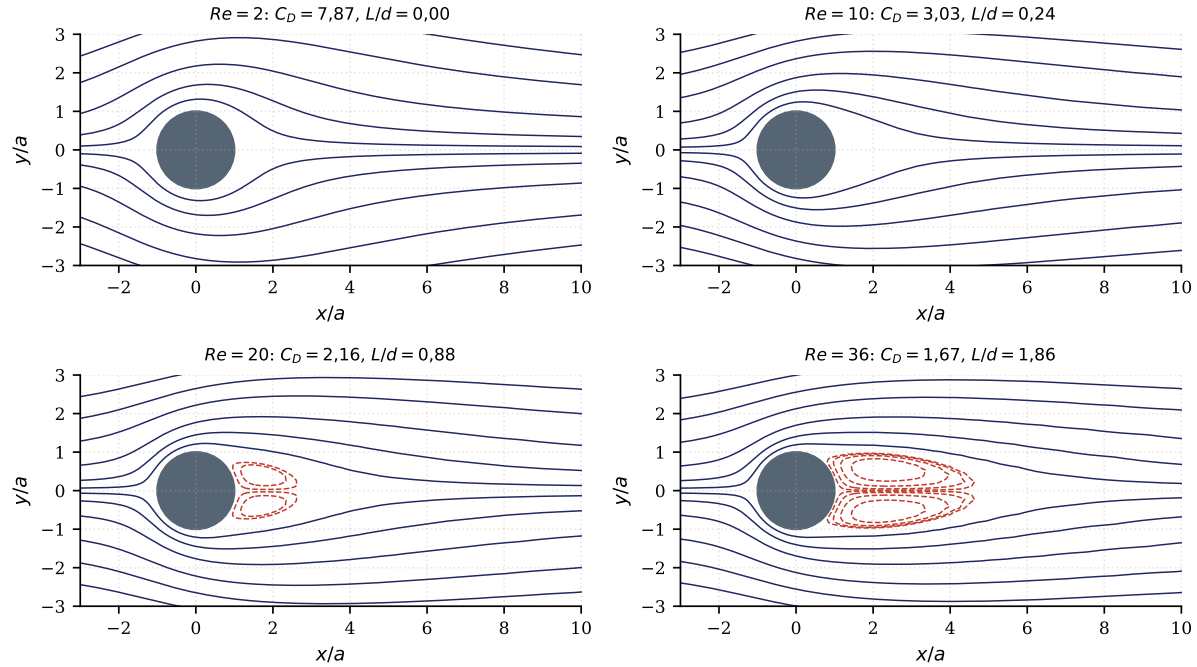


Figura 6.6: Programma 4.6b: linee di corrente intorno al cilindro per $Re = 2$, 10, 20, 36 (ricostruite per simmetria); in rosso tratteggiato la bolla di ricircolo ($\psi < 0$).

Cavi, antenne e montanti

I cilindri in corrente trasversale sono ovunque in aeronautica: cavi dei controlli e delle antenne, tubi di Pitot, montanti dei carrelli, tiranti dei biplani e degli ultraleggeri. A Re di qualche decina (un filo da 0,1 mm a 5 m/s) la resistenza per unità di lunghezza è $\frac{1}{2}\rho U^2 d\, C_D$ con $C_D \approx 2$: dieci volte quella di un profilo alare di pari spessore. Per questo i tiranti dei biplani erano profilati ("streamline wires") e i cavi esposti sono oggi carenati. A Re più alti (10^2–10^5) il C_D scende a circa 1,2 e la scia diventa oscillante: il distacco alternato dei vortici produce una forza trasversale periodica di frequenza $f = St\, U/d$ con numero di Strouhal $St \approx 0{,}2$, responsabile del "canto" dei cavi nel vento e delle vibrazioni per risonanza (il fenomeno che fece oscillare le prime linee elettriche e il ponte di Tacoma). Il programma non può coglierla perché impone la simmetria: oltre $Re \approx 47$ la soluzione simmetrica esiste ancora ma è instabile.

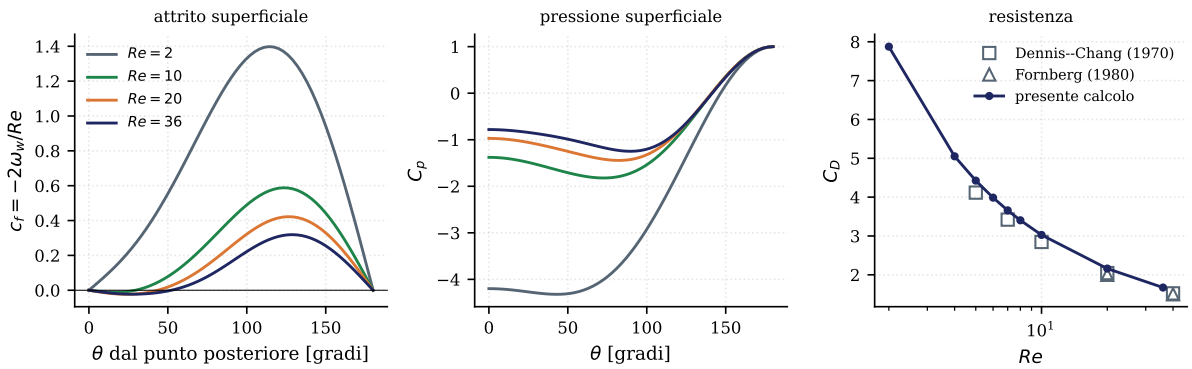


Figura 6.7: Programma 4.6b: coefficiente d'attrito superficiale $c_f = -2\omega_w/Re$ (a sinistra) e coefficiente di pressione (al centro, con $C_p = 1$ nel punto anteriore) in funzione dell'angolo dal punto posteriore; coefficiente di resistenza in funzione di Re confrontato con Dennis–Chang e Fornberg (a destra).

6.5 Approfondimenti

6.5.1 Perché l'ADI converge e come scegliere ρ

Per il problema di Dirichlet nel quadrato gli operatori discreti $\mathbf{A}_x$ (differenze seconde in x) e $\mathbf{A}_y$ (in y) *commutano* e hanno gli stessi autovettori, i modi $\sin(p\pi x_i)\sin(q\pi y_j)$, con autovalori $\lambda_p = 4\sin^2(p\pi/2M)$ e λ_q, $p, q = 1, \dots, M-1$. Scrivendo i due mezzi passi (6.17)–(6.18) in forma matriciale, $(\rho\mathbf{I} + \mathbf{A}_x)\mathbf{T}^{n+1/2} = (\rho\mathbf{I} - \mathbf{A}_y)\mathbf{T}^{n} + \dots$ e $(\rho\mathbf{I} + \mathbf{A}_y)\mathbf{T}^{n+1} = (\rho\mathbf{I} - \mathbf{A}_x)\mathbf{T}^{n+1/2} + \dots$, l'errore $\mathbf{e}^n$ (differenza dalla soluzione esatta del sistema discreto) si propaga con la matrice

$$\mathbf{M}(\rho) = (\rho\mathbf{I} + \mathbf{A}_y)^{-1}(\rho\mathbf{I} - \mathbf{A}_x)(\rho\mathbf{I} + \mathbf{A}_x)^{-1}(\rho\mathbf{I} - \mathbf{A}_y),$$

i cui autovalori, grazie alla commutatività, sono

$$\mu_{pq}(\rho) = \frac{\rho - \lambda_p}{\rho + \lambda_p}\cdot\frac{\rho - \lambda_q}{\rho + \lambda_q}.$$

Poiché $\lambda > 0$, ogni fattore ha modulo < 1 per ogni $\rho > 0$: il metodo converge per qualunque parametro fisso, come affermato. Il fattore $|(\rho - \lambda)/(\rho + \lambda)|$ è nullo per $\lambda = \rho$ e tende a 1 per $\lambda \ll \rho$ o $\lambda \gg \rho$: un solo ρ smorza bene solo i modi vicini a ρ. Con un ciclo di parametri $\rho_1, \dots, \rho_{k_0}$ il fattore totale è $\prod_k \mu_{pq}(\rho_k)$ e il problema diventa: scegliere i ρ_k in modo da rendere minimo $\max_{\lambda \in [\lambda_{\min}, \lambda_{\max}]} \prod_k |(\rho_k - \lambda)/(\rho_k + \lambda)|$. È un problema di approssimazione di Chebyshev, la cui soluzione esatta (Wachspress, 1957; Jordan) si esprime con funzioni ellittiche; la successione geometrica (6.21), con rapporto costante fra parametri consecutivi, ne è un'ottima approssimazione e garantisce un fattore di riduzione per ciclo inferiore a $(\sqrt{2}-1)^2 \approx 0{,}17$ per ogni modo, da cui il criterio (6.22). Il numero di iterazioni per raggiungere una tolleranza fissata cresce come $\log M$ (contro M per il SOR ottimo e M^2 per Jacobi): per griglie grandi il vantaggio è enorme, e l'ADI con parametri ottimi è stato per decenni il risolutore ellittico più efficiente, prima dei metodi multigriglia. Per domini non rettangolari o coefficienti variabili gli operatori non commutano più e l'analisi non vale; il metodo di solito converge ancora, ma senza garanzie né parametri ottimi.

6.5.2 Formulazione ω–ψ in coordinate curvilinee ortogonali

La trasformazione (6.25) è un caso particolare: per qualunque mappa conforme $z = x + \mathrm{i}y = F(\zeta)$, $\zeta = \xi + \mathrm{i}\eta$, con fattore di scala $J = |F'(\zeta)|^2$, le equazioni ω–ψ diventano

$$\frac{\partial \omega}{\partial t} + \frac{1}{J}\left(\frac{\partial \psi}{\partial \eta}\frac{\partial \omega}{\partial \xi} - \frac{\partial \psi}{\partial \xi}\frac{\partial \omega}{\partial \eta}\right) = \frac{\nu}{J}\left(\frac{\partial^2 \omega}{\partial \xi^2} + \frac{\partial^2 \omega}{\partial \eta^2}\right), \qquad \frac{\partial^2 \psi}{\partial \xi^2} + \frac{\partial^2 \psi}{\partial \eta^2} = -J\omega,$$

perché il laplaciano e lo jacobiano del termine convettivo si trasformano entrambi con il solo fattore $1/J$ (il laplaciano è invariante conforme a meno di J). Per il cilindro $F = e^{\pi\zeta}$ e $J = \pi^2 e^{2\pi\xi} = g$. Con la trasformazione di Joukowski si ottengono nello stesso modo le equazioni per un profilo alare, e questo è stato per anni il modo standard di calcolare le correnti viscose intorno ai profili a basso Re (Mehta, Ta Phuoc Loc); le griglie strutturate curvilinee dei codici moderni ne sono la generalizzazione non conforme, con termini metrici in più.

6.5.3 La resistenza a basso numero di Reynolds: Oseen e la scia

Per $Re \to 0$ il cilindro non ammette una soluzione di Stokes (paradosso di Stokes, modulo 3): l'inerzia, per quanto piccola, conta lontano dal corpo. La correzione di Oseen, che linearizza la convezione con la velocità asintotica, dà

$$C_D = \frac{8\pi}{Re\,[2{,}002 - \ln Re]} \qquad (Re = U_\infty d/\nu \lesssim 1),$$

cioè $C_D = 9{,}6$ a $Re = 2$, mentre il calcolo dà 7,9 e Tritton misurò circa 6,5: a $Re = 2$ la formula di Oseen è già fuori dal suo campo e sovrastima, e il calcolo risente ancora del contorno esterno, che a basso Re dovrebbe stare a centinaia di diametri perché la scia di Oseen, larga $\sim \sqrt{\nu x/U}$, si allarga senza limite. Per $Re \gtrsim 10$ i calcoli di Dennis e Chang e di Fornberg, e le misure di Tritton, concordano entro pochi percento e il Programma 4.6b, con il contorno a $111a$, li raggiunge. La bolla cresce linearmente con Re fino a $Re \approx 47$, dove una biforcazione di Hopf la rende instabile: nasce la scia di von Kármán, e la resistenza media smette di decrescere ($C_D \approx 1{,}5$ a $Re = 50$, $\approx 1{,}2$ fino a $Re \approx 2 \times 10^5$, poi la crisi della resistenza quando lo strato limite diventa turbolento).

6.5.4 Variabili primitive o vorticità–funzione di corrente?

I due capitoli hanno risolto lo stesso problema con le due formulazioni, e il confronto è istruttivo. La formulazione ω–ψ ha due incognite invece di tre, nessun vincolo da imporre (la continuità è automatica), nessun problema di Neumann singolare e condizioni di Dirichlet per ψ; ma richiede le condizioni al contorno sulla vorticità, che non sono date e vanno ricavate dalla soluzione (formula di Thom), e non si estende alle tre dimensioni se non con il potenziale vettore, a costo di sei incognite. Le variabili primitive hanno condizioni al contorno naturali (velocità assegnate), si estendono senza modifiche a tre dimensioni, danno la pressione direttamente e, su griglia sfalsata, conservano massa ed energia; ma richiedono il risolutore di Poisson per la pressione, con il suo problema di Neumann, e il metodo a passi frazionari con il suo errore di separazione. Nella pratica moderna le variabili primitive dominano (sono l'unica scelta in tre dimensioni e nei codici generali), mentre la formulazione ω–ψ resta preziosa per i problemi piani e assialsimmetrici, per la stabilità e per i metodi a vortici, dove la vorticità è la variabile naturale.

Appendice A

Simboli, listati di supporto e istruzioni

A.1 Simboli principali

Tabella A.1: Principali simboli della monografia e nomi corrispondenti nei programmi.

Simbolo	Programma	Significato
Metodo di Galerkin (Cap. 1)		
ψ, $\zeta = \cos\theta$	`psi`, `z`	funzione di corrente di Stokes e variabile angolare
D^2, D^4	`D2`	operatore di Stokes e suo quadrato
A_m, B_m	`A`, `B`	coefficienti della funzione di prova di Kawaguti
$P_m(\zeta)$	–	polinomi di Legendre
$g(x,y)$, $h(x,y)$	`g`, `h`	sistema ridotto in A_1, B_1
c_{dp}, c_{ds}, C_D	`cdp`, `cds`, `cd`	coefficienti di resistenza di pressione, di attrito e totale
r_s	`rs`	lunghezza della bolla di ricircolo (dal centro, in raggi)
$C_0, \ldots, C_n$	`C`	coefficienti di Galerkin per Jeffery–Hamel
Differenze upwind (Cap. 2)		
$C = u\tau/h$	`C`	numero di Courant
G, $\beta = kh$	–	fattore di amplificazione e fase per nodo
μ_e	`mu_e`	viscosità artificiale $uh(1-C)/2$
$R = \nu\tau/h^2$	`R`	parametro di stabilità diffusivo
Bénard e fluidi in rotazione (Cap. 3)		
X, Y, T, U, V, Ψ, Ω, θ	`X, Y, T, U, V, psi, omega, theta`	variabili adimensionali (3.5)
Gr, Pr, Ra	`GR`, `PR`	numeri di Grashof, Prandtl, Rayleigh
U_f, U_b, V_f, V_b	`Uf`, `Ub`, `Vf`, `Vb`	velocità medie a mezzo passo (schema di Torrance)
$P_1, \ldots, P_4$	`P1`–`P4`	termini dello schema (3.18)
Nu	`Nu`	numero di Nusselt
R, Z, W	`R`, `Z`, `W`	coordinate meridiane e momento angolare adimensionale
$A = r_0/r_i$, $B = \omega_0/\omega_i$, λ	`A`, `B`, `LAM`	parametri di Taylor–Couette
Ta, E, Ro	–	numeri di Taylor, Ekman, Rossby

Tabella A.2: Simboli della monografia (continuazione).

Simbolo	Programma	Significato
Schemi nel tempo (Cap. 4)		
H	`H`	termine convettivo $-\frac{1}{2}\partial(u^2)/\partial x$
$d = \nu\Delta t/\Delta x^2$	`d`	numero di diffusione
a_j, b_j, c_j	`a, b, c`	diagonale, sottodiagonale, sopradiagonale
l_j, u_j, z_j	`l, u, z`	fattori LU e vettore ausiliario
Variabili primitive (Cap. 5)		
$u_{i+1/2,j}$, $v_{i,j+1/2}$, $P_{i,j}$	`u[i,j], v[i,j], p[i,j]`	velocità sulle facce e pressione al centro della cella MAC
$\hat{\mathbf{V}}$, ϕ	`us, vs, phi`	velocità intermedia e scalare di proiezione
β_1	–	$2Reh^2/\Delta t$ nell'equazione di Helmholtz
u_Γ, u_{-1}	`u_cop, u[:,0]`	velocità di parete e nodo fittizio
ψ_c, ω_c	`psi.min(), om[k]`	valori nel centro del vortice principale
u_*, τ	`sforzo`	velocità d'attrito e sforzo del vento
Vorticità–funzione di corrente (Cap. 6)		
ω, ψ	`om, psi`	vorticità e funzione di corrente, $\nabla^2\psi = -\omega$
ρ, $r_k = 1/\rho_k$, k_0	`rho, wachspress(M)`	parametro ADI e successione di Wachspress
ξ, η, $g(\xi)$	`xi, eta, g`	coordinate trasformate del cilindro e fattore di scala
θ_s, L	`theta_s, L`	angolo di separazione e lunghezza della bolla
P, c_f	`P, –`	pressione adimensionale $p/(\rho U_\infty^2)$ e coefficiente d'attrito

A.2 Listati di supporto


metodi_numerici.py

```
"""
metodi_numerici.py -- strumenti comuni della monografia 3 (flussi viscosi).

Contiene:
  * trid(C)              eliminazione di Gauss per sistemi tridiagonali (forma a 4 colonne)
  * thomas(a, b, c, d)   algoritmo di Thomas in memoria compatta
  * rk4_passo(f, x, y, h) un passo di Runge-Kutta del quarto ordine per y' = f(x, y)
  * rk4_integra(...)     integrazione a passo costante con memorizzazione della soluzione
  * liebmann(F, Q, h, errmax)  Gauss-Seidel per lap(F) = Q su griglia quadrata
  * sor_poisson(...)     sovrarilassamento successivo per lap(F) = Q
  * omega_ottimo(m, n)   fattore di sovrarilassamento ottimo per un rettangolo
  * cheb(N)              matrice di differenziazione di Chebyshev e nodi di Gauss-Lobatto
  * numeri_onda(N, L)    numeri d'onda della trasformata discreta di Fourier
"""
import numpy as np


# ---------------------------------------------------------------------------
# 1. Sistemi tridiagonali
# ---------------------------------------------------------------------------
def trid(C):
    """Risolve il sistema tridiagonale scritto riga per riga come
         C[i,0]*f[i-1] + C[i,1]*f[i] + C[i,2]*f[i+1] = C[i,3],  i = 0..n-1
    (C[0,0] e C[n-1,2] devono essere gia' stati azzerati dalle condizioni al
    contorno). Eliminazione in avanti per combinazione di righe, poi
    sostituzione all'indietro. Restituisce il vettore f (C non viene alterata).
    """
    C = np.array(C, dtype=float)          # copia di lavoro
    n = C.shape[0]
    for i in range(1, n - 1):             # eliminazione di C[i,0], i = 1..n-2
        c2 = C[i, 1] * C[i - 1, 1] - C[i, 0] * C[i - 1, 2]
        c3 = C[i, 2] * C[i - 1, 1]
        c4 = C[i, 3] * C[i - 1, 1] - C[i, 0] * C[i - 1, 3]
        C[i, 1], C[i, 2], C[i, 3] = c2, c3, c4
    f = np.zeros(n)
    # ultime due equazioni risolte insieme
    f[n - 1] = ((C[n - 1, 3] * C[n - 2, 1] - C[n - 1, 0] * C[n - 2, 3]) /
                (C[n - 1, 1] * C[n - 2, 1] - C[n - 1, 0] * C[n - 2, 2]))
    for j in range(n - 2, -1, -1):        # sostituzione all'indietro
        f[j] = (C[j, 3] - C[j, 2] * f[j + 1]) / C[j, 1]
    return f


def thomas(a, b, c, d):
    """Algoritmo di Thomas per  a[i] x[i-1] + b[i] x[i] + c[i] x[i+1] = d[i].
    a[0] e c[-1] non vengono usati. Funziona anche con coefficienti complessi."""
    n = len(d)
    tipo = np.result_type(a, b, c, d, float)
    cp = np.zeros(n, dtype=tipo)
    dp = np.zeros(n, dtype=tipo)
    cp[0] = c[0] / b[0]
    dp[0] = d[0] / b[0]
    for i in range(1, n):                 # eliminazione in avanti
        den = b[i] - a[i] * cp[i - 1]
        cp[i] = c[i] / den if i < n - 1 else 0.0
```

```
        dp[i] = (d[i] - a[i] * dp[i - 1]) / den
    x = np.zeros(n, dtype=tipo)
    x[-1] = dp[-1]
    for i in range(n - 2, -1, -1):        # sostituzione all'indietro
        x[i] = dp[i] - cp[i] * x[i + 1]
    return x


# ---------------------------------------------------------------------------
# 2. Runge-Kutta del quarto ordine per sistemi  y' = f(x, y)
# ---------------------------------------------------------------------------
def rk4_passo(f, x, y, h):
    """Un passo RK4: restituisce y(x + h)."""
    k1 = h * f(x, y)
    k2 = h * f(x + h / 2, y + k1 / 2)
    k3 = h * f(x + h / 2, y + k2 / 2)
    k4 = h * f(x + h, y + k3)
    return y + (k1 + 2 * k2 + 2 * k3 + k4) / 6.0


def rk4_integra(f, y0, x_fin, h, x0=0.0):
    """Integra y' = f(x, y) da x0 a x_fin con passo h costante.
    Restituisce i vettori x (n+1) e la matrice Y (n+1, dim)."""
    n = int(round((x_fin - x0) / h))
    x = x0 + h * np.arange(n + 1)
    Y = np.zeros((n + 1, len(y0)))
    Y[0] = y0
    for i in range(n):
        Y[i + 1] = rk4_passo(f, x[i], Y[i], h)
    return x, Y


# ---------------------------------------------------------------------------
# 3. Equazione di Poisson  F_xx + F_yy = Q  su griglia di passo h.
#    F[i, j] = valore nel nodo (x_i, y_j); i valori sul contorno restano fissi.
# ---------------------------------------------------------------------------
def liebmann(F, Q, h, errmax, azzera=True):
    """Metodo di Liebmann (Gauss-Seidel) sui nodi interni di un rettangolo;
    si ferma quando la somma dei |variazioni| di un'iterazione e' <= errmax.
    Se azzera=True i nodi interni partono da zero (come la subroutine LIEBMN).
    Restituisce (F, numero di iterazioni)."""
    M, N = F.shape
    if azzera:
        F[1:-1, 1:-1] = 0.0
    it = 0
    while True:
        it += 1
        err = 0.0
        for i in range(1, M - 1):
            for j in range(1, N - 1):
                nuovo = 0.25 * (F[i - 1, j] + F[i + 1, j] + F[i, j - 1] +
                                F[i, j + 1] - h * h * Q[i, j])
                err += abs(nuovo - F[i, j])
                F[i, j] = nuovo
        if err <= errmax:
            return F, it


def sor_poisson(F, Q, h, omega, errmax, nodi, prima_del_nodo=None, it_max=100000):
    """Sovrarilassamento successivo sui nodi 'nodi' (lista di coppie (i, j),
```

```
    visitati nell'ordine dato). prima_del_nodo(F, i, j), se fornita, viene
    chiamata prima di aggiornare ogni nodo (serve ad aggiornare i nodi fittizi
    delle condizioni di derivata). Restituisce (F, iterazioni)."""
    for it in range(1, it_max + 1):
        err = 0.0
        for i, j in nodi:
            if prima_del_nodo is not None:
                prima_del_nodo(F, i, j)
            gs = 0.25 * (F[i - 1, j] + F[i + 1, j] + F[i, j - 1] + F[i, j + 1] - h
    * h * Q[i, j])
            nuovo = (1.0 - omega) * F[i, j] + omega * gs
            err += abs(nuovo - F[i, j])
            F[i, j] = nuovo
        if err <= errmax:
            return F, it
    raise RuntimeError("SOR: convergenza non raggiunta")


def omega_ottimo(m, n):
    """Fattore di sovrarilassamento ottimo per un rettangolo con m x n linee di
    griglia (come nei programmi originali):
    omega = (8 - 4 sqrt(4 - a^2)) / a^2,  a = cos(pi/m) + cos(pi/n)."""
    a = np.cos(np.pi / m) + np.cos(np.pi / n)
    return (8.0 - 4.0 * np.sqrt(4.0 - a * a)) / (a * a)


# ---------------------------------------------------------------------------
# 4. Metodi spettrali
# ---------------------------------------------------------------------------
def cheb(N):
    """Nodi di Chebyshev-Gauss-Lobatto y_j = cos(j pi / N), j = 0..N (da +1 a -1)
    e matrice di differenziazione D (N+1 x N+1): (D u)_j approssima u'(y_j).
    Elementi fuori diagonale: D_jk = c_j (-1)^(j+k) / (c_k (y_j - y_k)), c_0 = c_N
    = 2;
    la diagonale si ottiene imponendo che la derivata di una costante sia nulla
    (somma per righe nulla), che e' anche la scelta numericamente piu' stabile."""
    if N == 0:
        return np.zeros((1, 1)), np.array([1.0])
    j = np.arange(N + 1)
    y = np.cos(np.pi * j / N)
    c = np.ones(N + 1)
    c[0] = c[-1] = 2.0
    c = c * (-1.0) ** j
    Y = np.tile(y, (N + 1, 1)).T
    dY = Y - Y.T
    D = np.outer(c, 1.0 / c) / (dY + np.eye(N + 1))
    D = D - np.diag(D.sum(axis=1))
    return D, y


def numeri_onda(N, L):
    """Numeri d'onda k_n = 2 pi n / L nell'ordine usato da numpy.fft
    (n = 0, 1, ..., N/2-1, -N/2, ..., -1)."""
    return 2.0 * np.pi * np.fft.fftfreq(N, d=L / N)
```

Listato A.1: `metodi_numerici.py`: sistemi tridiagonali, Runge–Kutta, metodi iterativi per l'equazione di Poisson e altre funzioni comuni ai moduli del corso.

stile_grafici.py

```python
"""
stile_grafici.py -- impostazioni grafiche comuni (palette "della casa").
Importato da tutti gli script della monografia.
"""
import os
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# Palette coerente con raucci-aero.sty
BLU      = "#1E2761"
ARANCIO  = "#DC7633"
ROSSO    = "#C0392B"
VERDE    = "#1E8449"
GIALLO   = "#B7950B"
VIOLA    = "#6C3483"
GRIGIO   = "#566573"
PALETTE  = [BLU, ARANCIO, VERDE, VIOLA, ROSSO, GIALLO, GRIGIO]

plt.rcParams.update({
    "font.family": "serif",
    "font.size": 9,
    "axes.labelsize": 9,
    "axes.titlesize": 9,
    "legend.fontsize": 8,
    "xtick.labelsize": 8,
    "ytick.labelsize": 8,
    "axes.prop_cycle": matplotlib.cycler(color=PALETTE),
    "axes.grid": True,
    "grid.alpha": 0.3,
    "grid.linestyle": ":",
    "lines.linewidth": 1.4,
    "figure.dpi": 120,
    "savefig.bbox": "tight",
    "savefig.pad_inches": 0.02,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "legend.frameon": False,
})

# Le figure vengono salvate nella cartella "figure" accanto a quella degli script
# (creata se manca): cosi' ogni programma funziona anche se scaricato da solo.
FIG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "figure")


def salva(fig, nome):
    """Salva la figura in PDF nella cartella delle figure del progetto LaTeX."""
    os.makedirs(FIG_DIR, exist_ok=True)
    from matplotlib.ticker import NullFormatter
    for ax in fig.get_axes():              # niente etichette secondarie sugli assi
    logaritmici
        if ax.get_xscale() == "log":
            ax.xaxis.set_minor_formatter(NullFormatter())
        if ax.get_yscale() == "log":
            ax.yaxis.set_minor_formatter(NullFormatter())
    fig.savefig(os.path.join(FIG_DIR, f"{nome}.pdf"))
    plt.close(fig)
    print(f"  -> figura salvata: {nome}.pdf")
```

Listato A.2: `stile_grafici.py`: impostazioni comuni di Matplotlib.

esercizi_svolti.py

```
"""
esercizi_svolti.py -- verifiche numeriche e simboliche degli esercizi del Modulo 4.

Es. 1  Stabilita' di von Neumann dello schema upwind (fattore di amplificazione).
Es. 2  Schema FTCS per la convezione: instabile per ogni C > 0 e privo della
       proprieta' trasportiva (analisi della perturbazione discreta).
Es. 3  Analisi di Hirt (equazione modificata) per gli schemi FTCS e BTCS
       dell'equazione della diffusione: derivazione simbolica dei coefficienti e
       verifica sperimentale dei limiti di stabilita'.
Es. 4  Analisi della perturbazione discreta dello schema FTCS per la diffusione.
Es. 5  Coefficienti dello schema upwind di Torrance e criterio di stabilita' (4.3.33).
Es. 6  Fattorizzazione LU contro algoritmo di Thomas (equazione di Burgers AB-CN).
Es. 7  Coordinate trasformate del cilindro: verifica simbolica del laplaciano e
       del termine convettivo (4.6.44)-(4.6.46).
Es. 8  Funzione di prova di Kawaguti per piccoli Re: riduzione alla soluzione di Stokes.
Es. 9  Problema di Neumann per l'equazione di Poisson: matrice singolare, fissaggio
       di un nodo, autovalori e condizionamento.
"""
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO


def titolo(s):
    print("\n" + "=" * 78 + "\n" + s + "\n" + "=" * 78)


# ============================================================== Es. 1: von Neumann per lo schema upwind
titolo("Es. 1 -- schema upwind: fattore di amplificazione G = 1 - C (1 - e^{-i beta})")
C, beta = sp.symbols("C beta", real=True)
G = 1 - C * (1 - sp.exp(-sp.I * beta))
G2 = sp.simplify(sp.expand(G * sp.conjugate(G), complex=True))
G2 = sp.simplify(G2.rewrite(sp.cos))
print("   |G|^2 =", G2, "  =  1 - 2 C (1 - C)(1 - cos beta)")
print("   verifica:", sp.simplify(G2 - (1 - 2 * C * (1 - C) * (1 - sp.cos(beta)))) == 0)
Cs = np.array([0.25, 0.5, 0.75, 1.0, 1.1, 1.5])
bs = np.linspace(0, np.pi, 721)
print("   massimo di |G| sul cerchio unitario: " + ", ".join(
    f"C={c:g}: {np.max(np.abs(1 - c*(1 - np.exp(-1j*bs)))):.3f}" for c in Cs))

# ============================================================== Es. 2: FTCS per la convezione
titolo("Es. 2 -- schema FTCS per zeta_t + u zeta_x = 0: G = 1 - i C sin beta")
print("   |G|^2 = 1 + C^2 sin^2 beta >= 1 per ogni C != 0: stabile solo per C -> 0")
print("   massimo di |G|: " + ", ".join(f"C={c:g}: {np.sqrt(1 + c*c):.3f}" for c in (0.1, 0.5, 1.0)))
print("   perturbazione discreta: zeta = 0 ovunque, zeta_m = eps all'istante j (C = 0,5):")
z = np.zeros(11); z[5] = 1.0
for passo in range(3):
    zn = z.copy(); zn[1:-1] = z[1:-1] - 0.25 * (z[2:] - z[:-2])
    z = zn
    print(f"      j+{passo+1}: " + " ".join(f"{v:+6.3f}" for v in z[2:9]) + "  (nodi m-3 .. m+3)")
```

```
print("   la perturbazione raggiunge anche i nodi a monte (m-1, m-2, ...): nessuna proprieta' trasportiva.")
print("   con lo schema upwind (C = 0,5):")
z = np.zeros(11); z[5] = 1.0
for passo in range(3):
    zn = z.copy(); zn[1:] = z[1:] - 0.5 * (z[1:] - z[:-1])
    z = zn
    print(f"      j+{passo+1}: " + " ".join(f"{v:+6.3f}" for v in z[2:9]) + "  (nodi m-3 .. m+3)")

# ============================================================ Es. 3: analisi di Hirt per la diffusione
titolo("Es. 3 -- analisi di Hirt: equazione modificata degli schemi FTCS e BTCS per u_t = nu u_yy")
y, t, nu, tau, h = sp.symbols("y t nu tau h", positive=True)
u = sp.Function("u")(y, t)


def serie(f, dy, dtt, ordine=5):
    """Sviluppo di Taylor di u(y + dy, t + dtt) fino all'ordine indicato (in dy e dtt)."""
    s = 0
    for n in range(ordine + 1):
        for k in range(n + 1):
            s += sp.diff(u, y, k, t, n - k) * dy**k * dtt**(n - k) / (sp.factorial(k) * sp.factorial(n - k))
    return s


def equazione_modificata(implicito):
    liv = tau if implicito else 0
    lhs = (serie(u, 0, tau) - u) / tau
    rhs = nu * (serie(u, h, liv) - 2 * serie(u, 0, liv) + serie(u, -h, liv)) / h**2
    E = sp.expand(lhs - rhs)                                  # = 0 e' l'equazione modificata
    # eliminazione delle derivate temporali di ordine > 1 con l'equazione stessa (iterativamente)
    utt = sp.diff(u, t, 2); uyyt = sp.diff(u, y, 2, t)
    # al primo ordine in tau: u_t = nu u_yy + O(tau, h^2) => u_tt = nu u_yyt = nu^2 u_yyyy, u_yyt = nu u_yyyy
    E = E.subs({utt: nu**2 * sp.diff(u, y, 4), uyyt: nu * sp.diff(u, y, 4)})
    E = E.subs(sp.diff(u, t, 3), 0).subs(sp.diff(u, y, 2, t, 2), 0).subs(sp.diff(u, y, 4, t), 0)
    E = sp.expand(E)
    # si trattengono i termini fino a O(tau, h^2)
    coeff4 = -E.coeff(sp.diff(u, y, 4))
    return sp.simplify(coeff4)


R = sp.symbols("R", positive=True)
for nome, impl in (("FTCS (esplicito)", False), ("BTCS (implicito)", True)):
    c4 = equazione_modificata(impl)
    c4R = sp.simplify(c4.subs(tau, R * h**2 / nu))
    print(f"   {nome}: u_t = nu u_yy + [{sp.factor(c4R)}] u_yyyy + ...   "
          f"(= nu h^2/2 ({'1/6 - R' if not impl else '1/6 + R'}))")
print("   Per la (4.2.23) il coefficiente del termine del quarto ordine e' nu h^2 (1/6 - R)/2.")
print("   Verifica sperimentale (FTCS, onda sin(pi y) su [0,1], 41 nodi, t = 0,1):")
Ny = 41; yy = np.linspace(0, 1, Ny); hh = yy[1] - yy[0]
for Rr in (0.1, 1 / 6, 0.3, 0.45, 0.5, 0.55):
```

```
    dt = Rr * hh**2
    uu = np.sin(np.pi * yy)
    n = int(round(0.1 / dt))
    for _ in range(n):
        uu[1:-1] = uu[1:-1] + Rr * (uu[2:] - 2 * uu[1:-1] + uu[:-2])
        if np.max(np.abs(uu)) > 1e3:
            break
    err = np.max(np.abs(uu - np.sin(np.pi * yy) * np.exp(-np.pi**2 * n * dt)))
    print(f"      R = {Rr:.3f}: " + ("instabile" if not np.isfinite(err) or err > 1
    else f"errore massimo {err:.2e}"))
print("   -> lo schema e' stabile fino a R = 1/2 (von Neumann), non fino a R =
    1/6.")

# ============================================================ Es. 4: perturbazione
    discreta FTCS diffusione
titolo("Es. 4 -- perturbazione discreta per u_(i,j+1) = u_(i,j) + R (u_(i+1,j) - 2
    u_(i,j) + u_(i-1,j))")
print("   una perturbazione eps in i = m da' u_m = (1 - 2R) eps, u_(m+-1) = R eps
    al passo successivo;")
print("   la risposta nel nodo perturbato non cambia segno se 1 - 2R >= 0, cioe' R
    <= 1/2;")
print("   |1 - 2R| <= 1 (sola ampiezza) darebbe R <= 1, condizione insufficiente:")
for Rr in (0.4, 0.5, 0.6, 0.8):
    z = np.zeros(41); z[20] = 1.0
    for _ in range(200):
        z[1:-1] = z[1:-1] + Rr * (z[2:] - 2 * z[1:-1] + z[:-2])
    print(f"      R = {Rr}: max|u| dopo 200 passi = {np.max(np.abs(z)):.3e}")

# ============================================================ Es. 5: coefficienti
    dello schema di Torrance
titolo("Es. 5 -- schema upwind di Torrance: coefficienti a1..a5 e criterio
    (4.3.33)")
Uf, Ub, Vf, Vb, dT, hh_, Pr = sp.symbols("U_f U_b V_f V_b Delta_T h Pr", real=True)
B = 1 / Pr
a1 = -dT / (2 * hh_) * (Uf - sp.Abs(Uf)) + dT * B / hh_**2
a2 = dT / (2 * hh_) * (Ub + sp.Abs(Ub)) + dT * B / hh_**2
a4 = -dT / (2 * hh_) * (Vf - sp.Abs(Vf)) + dT * B / hh_**2
a5 = dT / (2 * hh_) * (Vb + sp.Abs(Vb)) + dT * B / hh_**2
a3 = 1 - dT / (2 * hh_) * (Uf + sp.Abs(Uf) - Ub + sp.Abs(Ub) + Vf + sp.Abs(Vf) - Vb
    + sp.Abs(Vb)) - 4 * dT * B / hh_**2
print("   a1 =", a1, "\n   a2 =", a2, "\n   a4 =", a4, "\n   a5 =", a5, "\n   a3
    =", a3)
print("   a1, a2, a4, a5 >= 0 perche' |U| - U >= 0 e |U| + U >= 0;")
print("   a1+...+a5 = 1 - (Delta T/h)(Uf - Ub + Vf - Vb):", sp.simplify(a1 + a2 +
    a3 + a4 + a5 - (1 - dT / hh_ * (Uf - Ub + Vf - Vb))) == 0,
      "-> = 1 (consistenza) se il campo discreto e' solenoidale")
print("   a3 >= 0  <=>  Delta T <= [ (Uf+|Uf| - Ub+|Ub| + Vf+|Vf| - Vb+|Vb|)/(2h) +
    4/(h^2 Pr) ]^(-1)   (4.3.33)")
from prog4_3_benard import simula as benard, DT as DT_B
print("   Verifica sul problema di Benard (Programma 4.3), 400 passi:")
with np.errstate(all="ignore"):
    sal, sto, dtmin = benard(maxstp=400, stampa=False)
    print(f"      passo massimo consentito dalla (4.3.33) nei primi 400 passi:
    {dtmin:.5f} (usato {DT_B})")
    for fatt in (0.9, 1.1, 1.5):
        try:
            s2, st2, _ = benard(maxstp=int(400 * DT_B / (fatt * dtmin)), dt=fatt *
    dtmin, stampa=False)
            m = max(v[1] for v in st2)
            print(f"      dT = {fatt:.1f} x limite = {fatt*dtmin:.5f}: " +
```

```
    ("instabile (overflow)" if not np.isfinite(m) or m > 1e3 else f"max|Psi| =
    {m:.4f}, regolare"))
147         except RuntimeError as e:
148             print(f"      dT = {fatt:.1f} x limite: instabile ({e})")
149
150 # ============================================================ Es. 6: LU contro
    Thomas
151 titolo("Es. 6 -- Burgers AB-CN: fattorizzazione LU contro algoritmo di Thomas")
152 import prog4_4_burgers_abcn as b4
153 x1, u1, _ = b4.integra("abcn", 201, 0.005, 1.0, risolutore="thomas")
154 x2, u2, _ = b4.integra("abcn", 201, 0.005, 1.0, risolutore="lu")
155 print(f"   differenza massima fra le due soluzioni: {np.max(np.abs(u1 - u2)):.2e}
    (precisione di macchina)")
156 n = 199
157 print(f"   operazioni per passo (n = {n} incognite): Thomas ~ 8n = {8*n} flop
    (rifa' ogni volta l'eliminazione);")
158 print(f"   LU: fattorizzazione una volta (~3n), poi ~5n = {5*n} flop per passo
    (sostituzioni in avanti e all'indietro).")
159
160 # ============================================================ Es. 7: coordinate
    del cilindro
161 titolo("Es. 7 -- cilindro: laplaciano e termine convettivo nelle coordinate xi,
    eta")
162 xi, eta = sp.symbols("xi eta", real=True)
163 r = sp.exp(sp.pi * xi); th = sp.pi * eta
164 f = sp.Function("f")(xi, eta)
165 # derivate rispetto a r e theta tramite la regola della catena: d/dr = (1/(pi r))
    d/dxi, d/dtheta = (1/pi) d/deta
166 d_r = lambda F: sp.diff(F, xi) / (sp.pi * r)
167 d_th = lambda F: sp.diff(F, eta) / sp.pi
168 lap_polare = d_r(d_r(f)) + d_r(f) / r + d_th(d_th(f)) / r**2
169 g = sp.pi**2 * sp.exp(2 * sp.pi * xi)
170 print("   nabla^2 f - (f_xixi + f_etaeta)/g =", sp.simplify(lap_polare -
    (sp.diff(f, xi, 2) + sp.diff(f, eta, 2)) / g))
171 psi, om = sp.Function("psi")(xi, eta), sp.Function("omega")(xi, eta)
172 vr = d_th(psi) / r; vth = -d_r(psi)
173 conv = vr * d_r(om) + vth / r * d_th(om)
174 print("   v_r omega_r + (v_theta/r) omega_theta - (psi_eta omega_xi - psi_xi
    omega_eta)/g =",
175       sp.simplify(conv - (sp.diff(psi, eta) * sp.diff(om, xi) - sp.diff(psi, xi) *
    sp.diff(om, eta)) / g))
176 print("   v_r = psi_eta/(pi e^{pi xi}):", sp.simplify(vr - sp.diff(psi, eta) /
    (sp.pi * r)) == 0,
177       ";  v_theta = -psi_xi/(pi e^{pi xi}):", sp.simplify(vth + sp.diff(psi, xi) /
    (sp.pi * r)) == 0)
178 # condizione al contorno esterno (4.6.58): psi potenziale
179 print("   psi potenziale r sin(theta)(1 - 1/r^2) = 2 sin(pi eta) sinh(pi xi):",
180       sp.simplify(sp.expand((r * sp.sin(th) * (1 - 1 / r**2) - 2 * sp.sin(sp.pi *
    eta) * sp.sinh(sp.pi * xi)).rewrite(sp.exp))) == 0)
181
182 # ============================================================ Es. 8: funzione di
    prova per piccoli Re
183 titolo("Es. 8 -- funzione di prova (4.1.52) per Re -> 0 e soluzione di Stokes")
184 rr, zz = sp.symbols("r zeta", positive=True)
185 A1, A2, A3, A4, B1, B2, B3, B4 = sp.symbols("A1:5 B1:5")
186 f_r = rr**2 / 2 + A1 * rr + A2 + A3 / rr + A4 / rr**2
187 g_r = B1 * rr + B2 + B3 / rr + B4 / rr**2
188 psi_t = f_r * (1 - zz**2) + g_r * zz * (1 - zz**2)
189 D2 = lambda F: sp.diff(F, rr, 2) + (1 - zz**2) / rr**2 * sp.diff(F, zz, 2)
190 D4 = sp.simplify(D2(D2(psi_t)))
```

```
print("   D^4 psi =", sp.factor(D4))
print("   D^4 psi = 0 per ogni r richiede A2 = A4 = 0 e B1 = B3 = 0; le condizioni di aderenza sulla parte")
print("   dispari danno allora B2 = B4 = 0 (nessuna asimmetria avanti-dietro nel limite di Stokes)")
sol = sp.solve([sp.Eq(f_r.subs(rr, 1), 0), sp.Eq(sp.diff(f_r, rr).subs(rr, 1), 0), A2, A4], [A1, A2, A3, A4], dict=True)[0]
print("   con le condizioni di aderenza psi(1) = psi_r(1) = 0:", sol, " -> psi = (r^2/2 - 3r/4 + 1/(4r)) sin^2 theta  (Stokes)")
cd_stokes = sp.Rational(24, 1)
print("   resistenza di Stokes: D = 6 pi mu a U, C_D = 24/Re (Re basato sul diametro)")

# ============================================================ Es. 9: problema di Neumann
titolo("Es. 9 -- equazione di Poisson con condizioni di Neumann su 4 nodi interni")
A = np.array([[-2, 1, 1, 0], [1, -2, 0, 1], [1, 0, -2, 1], [0, 1, 1, -2]], float)
w = np.sort(np.linalg.eigvalsh(A))
print("   autovalori di A:", np.round(w, 4), " -> uno nullo: matrice singolare (rango 3)")
print("   vettore nullo: (1,1,1,1)/2 (la pressione e' definita a meno di una costante); la compatibilita'")
print("   richiede che il termine noto sia ortogonale a (1,1,1,1): somma delle r_ij = 0")
A1m = A.copy(); A1m[0, :] = [1, 0, 0, 0]
w1 = np.linalg.eigvals(A1m)
print("   fissando phi_11 = 0: autovalori", np.round(np.sort(w1.real), 4), f", rcond = {1/np.linalg.cond(A1m, 1):.4f}")
# esempio su griglia N x N: confronto del condizionamento fra Dirichlet e Neumann fissato
for N in (8, 16, 32):
    n = N * N; idx = lambda i, j: i * N + j
    An = np.zeros((n, n)); Ad = np.zeros((n, n))
    for i in range(N):
        for j in range(N):
            p = idx(i, j); Ad[p, p] = -4
            for (a, b) in ((i + 1, j), (i - 1, j), (i, j + 1), (i, j - 1)):
                if 0 <= a < N and 0 <= b < N:
                    An[p, idx(a, b)] = 1; Ad[p, idx(a, b)] = 1; An[p, p] -= 1
    An[0, :] = 0; An[0, 0] = 1
    print(f"   N = {N:2d}: numero di condizionamento (norma 2): Neumann fissato {np.linalg.cond(An):9.1f}, Dirichlet {np.linalg.cond(Ad):9.1f}")

# ------------------------------------------------------------ figura riassuntiva (Es. 1-4)
fig, ax = plt.subplots(1, 3, figsize=(8.4, 2.6))
for c, col in zip((0.25, 0.5, 0.75, 1.0, 1.25), (GRIGIO, VERDE, ARANCIO, BLU, ROSSO)):
    ax[0].plot(bs, np.abs(1 - c * (1 - np.exp(-1j * bs))), color=col, label=f"$C={c:g}$".replace(".", "{,}"))
ax[0].axhline(1, color="k", lw=0.5); ax[0].set_xlabel("$\\beta = kh$"); ax[0].set_ylabel("$|G|$ upwind"); ax[0].legend(fontsize=6)
for c, col in zip((0.1, 0.5, 1.0), (VERDE, ARANCIO, BLU)):
    ax[1].plot(bs, np.sqrt(1 + c * c * np.sin(bs)**2), color=col, label=f"$C={c:g}$".replace(".", "{,}"))
ax[1].axhline(1, color="k", lw=0.5); ax[1].set_xlabel("$\\beta$"); ax[1].set_ylabel("$|G|$ FTCS"); ax[1].legend(fontsize=6)
Rs = np.linspace(0, 1, 201)
ax[2].plot(Rs, np.abs(1 - 4 * Rs), color=BLU, label="von Neumann $|G|_{\\max}=|1-4R|$")
```

```
ax[2].plot(Rs, np.abs(1 - 2 * Rs), "--", color=ARANCIO, label="pert. discreta $|1-2R|$")
ax[2].plot(Rs, 6 * (1 / 6 - Rs), ":", color=GRIGIO, label="Hirt: coeff. di $u_{yyyy}$, $6(1/6-R)$")
ax[2].axhline(1, color="k", lw=0.5); ax[2].axhline(0, color="k", lw=0.5)
ax[2].set_xlabel("$R=\\nu\\tau/h^2$"); ax[2].set_ylim(-1.2, 3); ax[2].legend(fontsize=6)
fig.tight_layout()
salva(fig, "esercizi_stabilita")
```

Listato A.3: `esercizi_svolti.py`: verifiche numeriche e simboliche degli Esercizi 1–9.

A.3 Come riprodurre i risultati

Istruzioni

1. Requisiti: Python ≥ 3.9 con `numpy`, `scipy`, `matplotlib` e `sympy` (`pip install numpy scipy matplotlib sympy`); una distribuzione TEX con `pdflatex` e i pacchetti `tcolorbox`, `pgfplots`, `siunitx`, `listings`, `qrcode`, `marginnote`, `babel-italian` (tutti inclusi in TEX Live / MacTeX completo).
2. Dalla cartella `codice/`: `bash esegui_tutti.sh` esegue tutti i programmi e gli esercizi (circa dieci minuti, di cui quattro per il cilindro del Programma 4.6b e due per la cavità esplicita del Programma 4.5) e rigenera le figure in `figure/`. Ogni programma si può anche eseguire da solo con `python3 nome.py`; i programmi di approfondimento importano funzioni da quelli principali (per esempio `prog4_5b` e `prog4_5c` da `prog4_5`), quindi vanno eseguiti dalla stessa cartella.
3. Dalla cartella principale: `pdflatex main.tex` due volte (o `latexmk -pdf main.tex`).

A.4 Codici in rete e QR code

I programmi sono pubblicati all'indirizzo

`https://braucci.github.io/CFD/monografia4/codici/`

La pagina elenca tutti i file con le rispettive dipendenze, li mostra con l'evidenziazione della sintassi e permette di scaricarli singolarmente o in un unico archivio. Il QR code nel margine accanto a ogni listato codifica l'indirizzo del file corrispondente e, nel PDF, è anche un collegamento cliccabile.

Per l'autore: configurazione e pubblicazione

1. In `main.tex` la macro `\nomemonografia` (qui `monografia4`) determina tutti gli indirizzi: `\urlcodici` vale `https://braucci.github.io/CFD/<nome>/codici/` e `\urlfile{file.py}` aggiunge il nome del file.
2. Un listato completo si inserisce con `\listatopy[opzioni]{file.py}` (legge `codice/file.py` e mette il QR nel margine); prima di un estratto in `lstlisting` si scrive `\qrlistato{file.py}`.

3. `python3 pubblica_github.py` crea `github/monografia4/` con il PDF, la cartella `codici/`, la pagina `index.html` e l'archivio zip: va copiata così com'è nella cartella `CFD/` del repository, accanto a `monografia1/`, `monografia2/` e `monografia3/`; il portale `index.html` della cartella `CFD/` va aggiornato con la scheda del modulo 4.

A.5 Come sostituire le figure

Le figure del documento sono di due tipi, gestite da due macro definite in `main.tex`.

`\figuraesterna[larghezza]{nome}{testo}` per le figure schematiche: geometrie, griglie e molecole di calcolo. Se nella cartella `immagini/` esiste il file `nome.png` (oppure `.pdf` o `.jpg`) viene incluso con la larghezza indicata; altrimenti compare un *segnaposto* grigio con il nome del file atteso e una breve descrizione. Basta copiare in `immagini/` le immagini con i nomi della Tab. A.3 e ricompilare.

`\figuragenerata[larghezza]{nome}` grafici prodotti dagli script Python, che il documento legge dalla cartella `figure/`. Anche questi si possono sostituire: se esiste `immagini/nome.png`, ha la precedenza sul PDF generato.

Tabella A.3: Figure con segnaposto e nome del file atteso in `immagini/`.

Figura	File	Contenuto
1.1	`fig_4_1_1.png`	corrente uniforme intorno a una sfera, coordinate sferiche
1.2	`fig_4_1_2.png`	sforzi sull'elemento di superficie della sfera
1.3	`fig_4_1_3.png`	bolla di ricircolo dietro la sfera
1.4	`fig_4_1_4.png`	costruzione del metodo di Newton–Raphson
1.7	`fig_4_1_8.png`	corrente radiale fra pareti inclinate
2.1	`fig_4_2_1.png`	molecola dello schema upwind
3.1	`fig_4_3_1.png`	schema del problema di Bénard (metà scatola)
3.7	`fig_4_3_2.png`	tubo rotante con restringimento
5.1	`fig_4_5_1.png`	griglia collocata unidimensionale
5.2	`fig_4_5_2.png`	griglia sfalsata unidimensionale
5.3	`fig_4_5_3.png`	cella MAC
5.4	`fig_4_5_4.png`	condizioni per u vicino a una parete orizzontale
5.5	`fig_4_5_5.png`	cella di pressione adiacente a una parete
5.6	`fig_4_5_8.png`	quadrato con quattro nodi interni
5.7	`fig_4_5_9.png`	cavità con coperchio mobile
6.1	`fig_4_6_1.png`	griglia vicino alla parete inferiore
6.2	`fig_4_6_2.png`	griglia per l'equazione di ψ (Fig. 4.6.3 della fonte)
6.4	`fig_4_6_4.png`	geometria del cilindro e della bolla
6.5	`fig_4_6_5.png`	cilindro nelle coordinate trasformate

Appendice B

Esercizi svolti

Gli esercizi completano i capitoli: i primi nove sono problemi di analisi (stabilità, algoritmi, trasformazioni di coordinate), i dieci seguenti sono i *progetti*, veri e propri calcoli che estendono i programmi, e l'ultimo è la derivazione della formula della resistenza del cilindro. Ogni esercizio è svolto con lo stesso schema dei moduli precedenti: *testo*, *impostazione* (che cosa ci si aspetta e perché, prima di calcolare), *svolgimento* passo per passo, *risultato* e *commento*. Le verifiche numeriche e simboliche degli Esercizi 1–9 sono nel file `esercizi_svolti.py` (Listato A.3), che le stampa nell'ordine; i progetti sono risolti dai programmi di approfondimento dei capitoli, cui si rimanda per i listati, e qui se ne discute l'impostazione e il risultato.

B.1 Esercizio 1 – Von Neumann per lo schema upwind

Testo

Mostrare che l'analisi di stabilità di von Neumann applicata allo schema upwind (2.3) fornisce lo stesso criterio (2.7), $0 < C \leq 1$, ottenuto con l'analisi della perturbazione discreta.

Impostazione. L'analisi di von Neumann segue un'onda $\zeta_{i,j} = G^j e^{\mathrm{i}\beta i}$ ($\beta = kh$ fase per nodo) e richiede $|G| \leq 1$ per ogni β. Ci si aspetta che il fattore di amplificazione sia una media pesata di 1 e di $e^{-\mathrm{i}\beta}$, che sta dentro il cerchio unitario se i pesi sono non negativi.

Svolgimento.

1. Sostituendo nella (2.3): $G^{j+1}e^{\mathrm{i}\beta i} = (1-C)G^j e^{\mathrm{i}\beta i} + CG^j e^{\mathrm{i}\beta(i-1)}$, da cui
$$G = 1 - C + Ce^{-\mathrm{i}\beta} = 1 - C\left(1 - e^{-\mathrm{i}\beta}\right).$$
2. Il modulo quadro: $|G|^2 = (1 - C + C\cos\beta)^2 + C^2\sin^2\beta = 1 - 2C(1-C)(1-\cos\beta)$, come verifica SymPy nel programma.
3. Poiché $1 - \cos\beta \in [0, 2]$, $|G|^2 \leq 1$ per ogni β se e solo se $C(1-C) \geq 0$, cioè $0 \leq C \leq 1$. Per $C > 1$ il modo $\beta = \pi$ ha $|G| = |1 - 2C| > 1$; per $C < 0$ (downwind) $|G|^2 = 1 + 2|C|(1+|C|)(1-\cos\beta) > 1$.

Risultato

$|G|^2 = 1 - 2C(1-C)(1-\cos\beta) \le 1 \iff 0 \le C \le 1$: lo stesso criterio della perturbazione discreta. Il programma stampa $\max_\beta |G| = 1$ per $C = 0{,}25$, 0,5, 0,75, 1 e 1,2, 2 per $C = 1{,}1$, 1,5 (Fig. B.1, a sinistra). Per $C = 1$, $G = e^{-\mathrm{i}\beta}$: modulo unitario e fase esatta, lo schema trasla senza errori.

Commento. Geometricamente G è il punto che divide il segmento fra 1 e $e^{-\mathrm{i}\beta}$ (entrambi sul cerchio unitario) nel rapporto $C : (1-C)$: per $0 \le C \le 1$ sta su una corda, dentro il cerchio; per C fuori da $[0,1]$ sta sul prolungamento, fuori. L'accordo fra i due metodi non è generale: l'Esercizio 4 mostra un caso in cui divergono.

B.2 Esercizio 2 – Lo schema centrato per la convezione

Testo

Con la differenza in avanti nel tempo e centrata nello spazio la (2.1) diventa

$$\zeta_{i,j+1} = \zeta_{i,j} - \tfrac{1}{2}C\left(\zeta_{i+1,j} - \zeta_{i-1,j}\right).$$

Mostrare con l'analisi della perturbazione discreta che lo schema è stabile solo per $C \to 0$ e che non possiede la proprietà trasportiva.

Impostazione. Lo schema usa l'informazione da entrambi i lati: una perturbazione in x_m deve comparire anche in x_{m-1}, contro corrente, e con segno opposto a quella in x_{m+1}. L'ampiezza totale, somma dei moduli, cresce.

Svolgimento.

1. Con $\zeta = 0$ ovunque e ϵ in x_m: al passo successivo $\zeta_{m,j+1} = \epsilon$ (il nodo stesso non cambia), $\zeta_{m+1,j+1} = +\tfrac{1}{2}C\epsilon$, $\zeta_{m-1,j+1} = -\tfrac{1}{2}C\epsilon$.
2. Il nodo *a monte* riceve il disturbo: non c'è proprietà trasportiva. Al passo successivo $\zeta_{m,j+2} = \epsilon(1 - \tfrac{1}{2}C^2)$, $\zeta_{m\pm1,j+2} = \pm\tfrac{1}{2}C\epsilon$ (dalle due direzioni), $\zeta_{m\pm2,j+2} = \tfrac{1}{4}C^2\epsilon$ con segni; il programma stampa i valori per $C = 0{,}5$: $(-0{,}25,\ 1,\ 0{,}25)$, poi $(0{,}062, -0{,}5, 0{,}875, 0{,}5, 0{,}062)$, poi $(-0{,}016, 0{,}188, -0{,}703, 0{,}625, 0{,}703, 0{,}188, 0{,}016)$.
3. Il criterio $|\zeta_{m\pm1,j+1}/\epsilon| = \tfrac{1}{2}C \le 1$ darebbe $C \le 2$, ma la *somma* dei moduli al passo n, che è ciò che misura l'energia del disturbo, vale $(1+C)^n$ e cresce per ogni $C > 0$. Con von Neumann: $G = 1 - \mathrm{i}C\sin\beta$, $|G|^2 = 1 + C^2\sin^2\beta \ge 1$, con uguaglianza solo per $C = 0$: lo schema è incondizionatamente instabile.

Risultato

$|G| = \sqrt{1 + C^2\sin^2\beta} > 1$ per ogni $C \neq 0$ e $\beta \neq 0, \pi$ (Fig. B.1, al centro): stabile solo nel limite $C \to 0$. Nel Programma 4.3 la gaussiana trasportata con $C = 0{,}5$ esplode (varianza 31,8 contro 0,45) e perfino con $C = 0{,}05$ l'errore a $t = 8$ è del 6%.

Commento. L'instabilità di FTCS per la convezione è "debole" ($|G|^2 - 1 \propto C^2$) ma inesorabile: dopo n passi il modo peggiore è cresciuto di $(1 + C^2)^{n/2}$. La cura è aggiungere diffusione (fisica o artificiale) oppure usare uno schema del secondo ordine nel tempo come Lax–Wendroff, che aggiunge esattamente la diffusione $\frac{1}{2}C^2$ che cancella l'antidiffusione di Eulero (Cap. 2).

B.3 Esercizio 3 – Analisi di Hirt per l'equazione della diffusione

Testo

Applicare l'analisi di Hirt allo schema esplicito FTCS per $u_t = \nu u_{yy}$, $u_i^{n+1} = u_i^n + R(u_{i+1}^n - 2u_i^n + u_{i-1}^n)$ con $R = \nu\tau/h^2$, e mostrare che, trattenendo i termini fino a $\mathcal{O}(\tau, h^2)$, l'equazione modificata è

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial y^2} + \frac{1}{2}\nu h^2 \left(\frac{1}{6} - R\right) \frac{\partial^4 u}{\partial y^4}.$$

Secondo il testo i termini positivi di derivate pari, come quello del secondo ordine, smorzano, e la condizione di stabilità risultante sarebbe $R \le 1/6$, più restrittiva della $R \le 1/2$ di von Neumann. Mostrare anche che per lo schema implicito BTCS lo stesso procedimento dà un coefficiente sempre positivo, coerentemente con la stabilità per ogni R.

Impostazione. Sostituendo gli sviluppi di Taylor e eliminando u_{tt} con $u_{tt} = \nu^2 u_{yyyy}$ si trova il coefficiente del quarto ordine. Ma attenzione all'interpretazione: per il modo e^{iky} la derivata quarta ha simbolo $+k^4$, quindi un termine $+a\,u_{yyyy}$ con $a > 0$ *amplifica* (è un'antidiffusione di ordine superiore), mentre con $a < 0$ smorza (iperviscosità). L'affermazione del testo sul segno è corretta per la derivata seconda ma non per la quarta; vedremo che il risultato giusto è un'altra conclusione, altrettanto istruttiva.

Svolgimento.

1. Sviluppi: $u_i^{n+1} = u + \tau u_t + \frac{1}{2}\tau^2 u_{tt} + \dots$ e $u_{i\pm1}^n = u \pm h u_y + \frac{1}{2}h^2 u_{yy} \pm \frac{1}{6}h^3 u_{yyy} + \frac{1}{24}h^4 u_{yyyy} + \dots$, da cui $u_{i+1} - 2u_i + u_{i-1} = h^2 u_{yy} + \frac{1}{12}h^4 u_{yyyy} + \mathcal{O}(h^6)$.
2. Sostituendo: $u_t + \frac{1}{2}\tau u_{tt} + \mathcal{O}(\tau^2) = \nu u_{yy} + \frac{1}{12}\nu h^2 u_{yyyy} + \mathcal{O}(h^4)$.
3. Dall'equazione al primo ordine, $u_{tt} = \nu u_{yyt} = \nu^2 u_{yyyy}$; quindi

   $$u_t = \nu u_{yy} + \left(\tfrac{1}{12}\nu h^2 - \tfrac{1}{2}\tau\nu^2\right) u_{yyyy} = \nu u_{yy} + \tfrac{1}{2}\nu h^2 \left(\tfrac{1}{6} - R\right) u_{yyyy},$$

   che è l'equazione richiesta (SymPy: $-h^2\nu(6R-1)/12$). Per BTCS si sviluppa intorno al livello $n+1$: $u^n = u - \tau u_t + \frac{1}{2}\tau^2 u_{tt}$, e il segno del termine in τ si inverte: coefficiente $\frac{1}{2}\nu h^2(\frac{1}{6} + R)$, sempre positivo (SymPy: $h^2\nu(6R+1)/12$).
4. *Interpretazione.* Per e^{iky} il tasso di crescita è $\sigma = -\nu k^2 + a k^4$ con $a = \frac{1}{2}\nu h^2(\frac{1}{6} - R)$. Se si ragionasse come il testo (coefficiente positivo = smorzamento) si concluderebbe $R \le 1/6$; con il segno corretto si concluderebbe invece che per $R < 1/6$ c'è antidiffusione, ma essa prevale sulla diffusione solo per $k^2 > \nu/a = 2/[h^2(1/6 - R)]$, cioè $kh > \sqrt{12/(1-6R)} \ge \sqrt{12} > \pi$: oltre il numero d'onda massimo rappresentabile sulla griglia. L'equazione modificata troncata al quarto ordine *non prevede alcuna instabilità* per nessun $R \ge 0$, né per FTCS né per BTCS.

5. *Verifica numerica.* Il programma integra $\sin(\pi y)$ su $[0,1]$ con 41 nodi fino a $t = 0{,}1$: stabile ed accurato per $R = 0{,}1,\ 1/6,\ 0{,}3,\ 0{,}45,\ 0{,}5$ (errori da 8×10^{-5} a 4×10^{-4}) e instabile per $R = 0{,}55$. A $R = 1/6$ l'errore è $2{,}6 \times 10^{-8}$, tre ordini di grandezza inferiore: il termine del quarto ordine si annulla e lo schema diventa del *quarto* ordine nello spazio, il caso speciale $R = 1/6$ già incontrato nel modulo 3.

Risultato

L'equazione modificata è quella richiesta, con coefficiente $\frac{1}{2}\nu h^2(\frac{1}{6} - R)$ per FTCS e $\frac{1}{2}\nu h^2(\frac{1}{6} + R)$ per BTCS. Ma la conclusione corretta non è "$R \le 1/6$ per la stabilità": $R = 1/6$ è il valore di *massima accuratezza*, non un limite di stabilità, e lo schema è stabile fino a $R = 1/2$ (von Neumann, confermato dall'esperimento). Il metodo di Hirt, troncato al primo termine, qui non è in grado di prevedere la stabilità, perché il termine dominante dell'errore è del quarto ordine e il suo effetto è sottodominante rispetto alla diffusione fisica in tutto il campo dei numeri d'onda rappresentabili.

Commento. È il caso discusso da Warming e Hyett [7]: l'analisi euristica basata sul segno del primo termine dell'equazione modificata fallisce per l'equazione del calore, e serve la serie completa (che riproduce von Neumann). Il testo dell'esercizio, che deduce $R \le 1/6$, confonde il ruolo dei termini del quarto ordine; l'abbiamo svolto fino in fondo proprio perché mostra i limiti di un metodo apparentemente semplice. La morale generale (Roache [3]): ogni analisi di stabilità poggia su un'ipotesi fisica, e va capito quale.

B.4 Esercizio 4 – Perturbazione discreta per lo schema FTCS della diffusione

Testo

Mostrare che l'analisi della perturbazione discreta applicata allo schema esplicito $u_i^{n+1} = u_i^n + R(u_{i+1}^n - 2u_i^n + u_{i-1}^n)$ fornisce la condizione $R \le 1/2$, in accordo con il metodo di von Neumann.

Impostazione. Un disturbo ϵ in x_m si ripartisce al passo successivo fra x_m e i due vicini con pesi $1 - 2R$ e R, R; la somma dei pesi è 1 (conservazione). La condizione che il disturbo non cresca in modulo darebbe $|1 - 2R| \le 1$, cioè $R \le 1$: troppo permissiva. La condizione di non cambiare segno, $1 - 2R \ge 0$, dà il risultato giusto.

Svolgimento.

1. Con $u = 0$ ovunque e $u_m = \epsilon$: $u_m^{n+1} = (1 - 2R)\epsilon$, $u_{m\pm1}^{n+1} = R\epsilon$.
2. *Criterio di ampiezza*: $|1 - 2R| \le 1$ e $R \le 1$ danno $0 \le R \le 1$. Ma per $1/2 < R \le 1$ il disturbo nel nodo perturbato cambia segno a ogni passo: una "diffusione" che inverte il segno non è fisica, e corrisponde al modo a scacchiera $\beta = \pi$ di von Neumann, per cui $G = 1 - 4R$ e $|G| > 1$ se $R > 1/2$.
3. *Criterio di positività*: tutti i pesi ≥ 0, cioè $1 - 2R \ge 0$, $R \le 1/2$. Con pesi non negativi e somma 1 la soluzione resta una media dei valori precedenti (principio del massimo

discreto) e non può crescere: è una condizione *sufficiente* di stabilità, che qui coincide con quella necessaria di von Neumann.

4. Esperimento: dopo 200 passi il disturbo massimo vale 0,031 per $R = 0{,}4$, 0,054 per $R = 0{,}5$, 5×10^{27} per $R = 0{,}6$, 10^{67} per $R = 0{,}8$.

Risultato

$R \leq 1/2$ (Fig. B.1, a destra: le curve $|1 - 4R|$ di von Neumann e $|1 - 2R|$ della perturbazione discreta superano 1 rispettivamente a $R = 1/2$ e $R = 1$; la retta di Hirt si annulla a $R = 1/6$). Il criterio di ampiezza della perturbazione discreta, da solo, avrebbe dato $R \leq 1$, sbagliato di un fattore 2: l'analisi di un disturbo puntuale non "vede" il modo a scacchiera, che è una distribuzione di disturbi di segno alterno.

Commento. Tre metodi, tre risposte diverse per lo stesso schema: $R \leq 1/2$ (von Neumann, esatto), $R \leq 1$ o $R \leq 1/2$ (perturbazione discreta, secondo il criterio), nessuna restrizione o $R \leq 1/6$ (Hirt, secondo l'interpretazione). Il motivo del disaccordo è che ciascun metodo richiede qualcosa di diverso: che nessuna onda cresca, che un disturbo puntuale non cresca, che il coefficiente di diffusione sia positivo. Per gli schemi lineari a coefficienti costanti il metodo di von Neumann è necessario e sufficiente ed è quello da usare; gli altri due servono dove von Neumann non si applica (coefficienti variabili, non linearità, condizioni al contorno) e danno indicazioni, non teoremi.

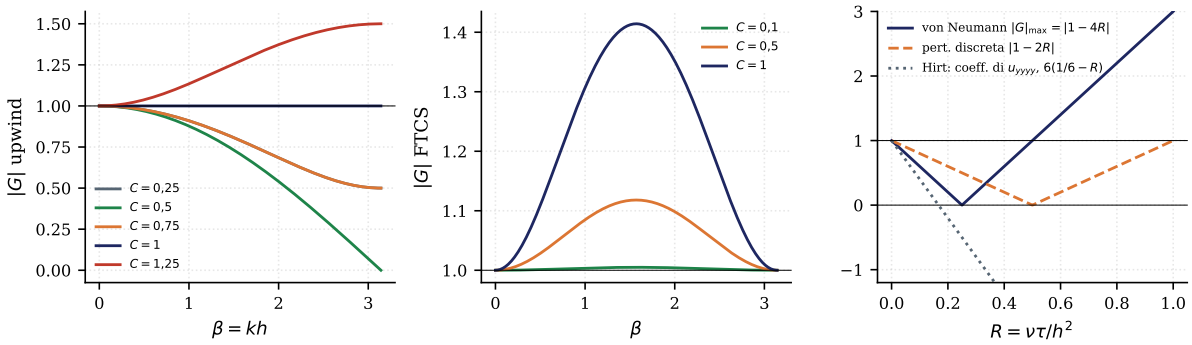


Figura B.1: Esercizi 1–4: modulo del fattore di amplificazione dello schema upwind (a sinistra) e di FTCS per la convezione (al centro) in funzione della fase β; a destra i criteri per FTCS della diffusione in funzione di R.

B.5 Esercizio 5 – Stabilità dello schema di Torrance

Testo

Applicato all'equazione dell'energia (3.9), lo schema (3.18) si scrive $\theta'_{i,j} = a_1\theta_{i+1,j} + a_2\theta_{i-1,j} + a_3\theta_{i,j} + a_4\theta_{i,j+1} + a_5\theta_{i,j-1}$. Mostrare che tutti i coefficienti tranne a_3 sono positivi qualunque sia il verso della corrente, e che la condizione $a_3 \geq 0$ (analisi quasi-lineare di Lax e Richtmyer) dà il criterio (3.19).

Impostazione. Lo schema è una combinazione lineare dei cinque valori vicini; se tutti i coefficienti sono non negativi e la loro somma è 1, il nuovo valore è una media pesata e

non può uscire dall'intervallo dei valori vecchi (principio del massimo): è la stessa logica dell'Esercizio 4. La non negatività dei coefficienti "laterali" è garantita dalla costruzione upwind; quella del coefficiente centrale limita il passo.

Svolgimento.

1. Dalle (3.14)–(3.18) con $A = 0$ e $B = 1/Pr$, raccogliendo i coefficienti di ciascun valore:

$$a_1 = -\frac{\Delta T}{2h}(U_f - |U_f|) + \frac{\Delta T}{h^2 Pr}, \qquad a_2 = \frac{\Delta T}{2h}(U_b + |U_b|) + \frac{\Delta T}{h^2 Pr},$$
$$a_4 = -\frac{\Delta T}{2h}(V_f - |V_f|) + \frac{\Delta T}{h^2 Pr}, \qquad a_5 = \frac{\Delta T}{2h}(V_b + |V_b|) + \frac{\Delta T}{h^2 Pr},$$
$$a_3 = 1 - \frac{\Delta T}{2h}\left(U_f + |U_f| - U_b + |U_b|\right) - \frac{\Delta T}{2h}\left(V_f + |V_f| - V_b + |V_b|\right) - \frac{4\Delta T}{h^2 Pr}.$$

2. Poiché $|U| - U \geq 0$ e $|U| + U \geq 0$ per ogni segno di U, $a_1, a_2, a_4, a_5 \geq 0$ sempre: il flusso upwind preleva sempre dal nodo a monte con peso positivo.

3. La somma vale $a_1 + \cdots + a_5 = 1 - (\Delta T/h)(U_f - U_b + V_f - V_b)$ (SymPy lo conferma): è 1 se la divergenza discreta $U_f - U_b + V_f - V_b$ è nulla, cioè se il campo di velocità è solenoidale, come garantisce la funzione di corrente a meno di errori di discretizzazione.

4. $a_3 \geq 0$ equivale a

$$\Delta T \leq \left[\frac{1}{2h}\left(U_f + |U_f| - U_b + |U_b| + V_f + |V_f| - V_b + |V_b|\right) + \frac{4}{h^2 Pr}\right]^{-1},$$

   che è la (3.19) con $B = 1/Pr$. Per l'equazione della vorticità il termine diffusivo ha $B = 1$ e la condizione analoga è più restrittiva se $Pr > 1$.

5. Verifica: nel Programma 4.3 il limite calcolato lungo la simulazione è $\Delta T_{\max} = 0{,}0034$; con $\Delta T = 0{,}9\Delta T_{\max}$ e $1{,}1\Delta T_{\max}$ (su 400 passi) il calcolo resta regolare, con $1{,}5\Delta T_{\max}$ il SOR non converge più dopo pochi passi (la vorticità è esplosa).

Risultato

Il criterio (3.19) è dimostrato; $\Delta T = 0{,}0025$ del programma è sotto il limite con margine del 30%, e la sua violazione del 50% porta all'instabilità. La violazione del 10% è tollerata perché la condizione di positività è sufficiente ma non necessaria, e perché il limite è raggiunto solo in pochi nodi e per breve tempo.

Commento. Il criterio è una condizione locale che dipende dalla soluzione: il programma può calcolare a ogni passo il ΔT massimo ammissibile e usarlo (è ciò che fa il Programma 4.3c della tazza), guadagnando nelle fasi lente e proteggendosi in quelle rapide. Per le equazioni della vorticità il termine di sorgente $Gr\,\partial\theta/\partial X$ non entra nel criterio ma, se molto grande, può produrre crescite rapide che il criterio non prevede: ΔT va comunque verificato per tentativi, come fece la fonte.

B.6 Esercizio 6 – Fattorizzazione LU al posto dell'algoritmo di Thomas

Testo

Il Programma 4.4 risolve l'equazione di Burgers con lo schema AB–CN e l'algoritmo di Thomas. Risolvere lo stesso problema con la fattorizzazione LU al posto di Thomas.

Impostazione. La matrice del sistema (4.17) ha diagonale $1 + d$ e fuori diagonale $-d/2$, costanti nel tempo: la fattorizzazione (4.22) si calcola una volta e a ogni passo restano le due sostituzioni. Ci si aspetta la stessa soluzione (alla precisione di macchina) e un risparmio di tempo di circa $3/8$ delle operazioni del risolutore.

Svolgimento.

1. Si scrivono `lu_tridiag(a, b, c)` (restituisce l_j e u_j) e `lu_risolvi(a, l, u, f)` (sostituzioni in avanti e all'indietro) secondo le (4.22)–(4.24).
2. Nella funzione `integra` del Programma 4.4 il parametro `risolutore` sceglie fra `thomas` e `lu`; con `lu` la fattorizzazione è eseguita prima del ciclo temporale.
3. Si confrontano le soluzioni (201 nodi, $\Delta t = 0{,}005$) e i tempi (2001 nodi, 1000 passi).

Risultato

La differenza massima fra le due soluzioni è 0 (identiche bit per bit: le operazioni sono le stesse nello stesso ordine); i tempi passano da 3,2 a 2,5 s. Per $n = 199$ incognite Thomas costa $\sim 8n = 1592$ operazioni per passo, LU $\sim 5n = 995$ più la fattorizzazione iniziale $\sim 3n$.

Commento. Il risparmio reale (un quarto, in Python puro) è inferiore al teorico perché il costo è dominato dall'interprete, non dalle operazioni in virgola mobile; in un linguaggio compilato o con `scipy.linalg.solve_banded` la differenza è quella teorica. Il principio vale però in generale ed è decisivo per le matrici sparse bidimensionali dei Programmi 4.5–4.6b, dove la fattorizzazione costa N^4 e le sostituzioni N^3.

B.7 Esercizio 7 – Le equazioni del cilindro nelle coordinate trasformate

Testo

Verificare che con la trasformazione (6.25) le equazioni (6.23)–(6.24) diventano le (6.27)–(6.29), che le velocità sono date dalle (6.26), e che la funzione di corrente potenziale sul contorno esterno è la (6.31).

Impostazione. La trasformazione è la mappa conforme $\ln z$: ci si aspetta che il laplaciano si trasformi senza termini del primo ordine e con il solo fattore di scala $1/g$, e che lo jacobiano convettivo $v_r\omega_r + (v_\theta/r)\omega_\theta$ si trasformi con lo stesso fattore.

Svolgimento.

1. Regola della catena: $\partial/\partial r = (1/\pi r)\,\partial/\partial\xi$ e $\partial/\partial\theta = (1/\pi)\,\partial/\partial\eta$. Allora
$$\frac{\partial^2}{\partial r^2} = \frac{1}{\pi r}\frac{\partial}{\partial\xi}\left(\frac{1}{\pi r}\frac{\partial}{\partial\xi}\right) = \frac{1}{\pi^2 r^2}\frac{\partial^2}{\partial\xi^2} - \frac{1}{\pi r^2}\frac{\partial}{\partial\xi}$$
(perché $\partial_\xi(1/\pi r) = -1/r$), e il termine $(1/r)\partial_r = (1/\pi r^2)\partial_\xi$ cancella esattamente la derivata prima: $\nabla^2 = (1/\pi^2 r^2)(\partial_{\xi\xi} + \partial_{\eta\eta}) = (1/g)(\partial_{\xi\xi} + \partial_{\eta\eta})$ con $g = \pi^2 e^{2\pi\xi}$.
2. $v_r = (1/r)\psi_\theta = \psi_\eta/(\pi r)$ e $v_\theta = -\psi_r = -\psi_\xi/(\pi r)$; il termine convettivo diventa
$$v_r\frac{\partial\omega}{\partial r} + \frac{v_\theta}{r}\frac{\partial\omega}{\partial\theta} = \frac{1}{\pi^2 r^2}\left(\psi_\eta\omega_\xi - \psi_\xi\omega_\eta\right).$$
3. $\psi = r\sin\theta(1 - 1/r^2) = \sin(\pi\eta)(e^{\pi\xi} - e^{-\pi\xi}) = 2\sin(\pi\eta)\sinh(\pi\xi)$.
4. SymPy conferma le tre identità (differenze simboliche nulle, stampate dal programma).

Risultato

Le equazioni trasformate sono le (6.27)–(6.29); la vorticità alla parete, dove $\psi_\eta = \psi_{\eta\eta} = 0$ e $g = \pi^2$, è $\omega = -\psi_{\xi\xi}/\pi^2$, da cui la formula di Thom (6.30).

Commento. La cancellazione del termine del primo ordine non è un caso: per ogni mappa conforme il laplaciano si trasforma con il solo fattore $1/|F'|^2$. È questo che rende le coordinate ξ–η così comode: la griglia uniforme nel piano trasformato è una griglia polare con spaziatura radiale esponenziale, fitta vicino al corpo e rada lontano, e le equazioni discretizzate sono quelle cartesiane con un coefficiente variabile.

B.8 Esercizio 8 – La funzione di prova di Kawaguti per piccoli numeri di Reynolds

Testo

Per $Re < 2$ Kawaguti usò la funzione di prova
$$\psi = \left(\tfrac{1}{2}r^2 + A_1 r + A_2 + \frac{A_3}{r} + \frac{A_4}{r^2}\right)(1-\zeta^2) + \left(B_1 r + B_2 + \frac{B_3}{r} + \frac{B_4}{r^2}\right)\zeta(1-\zeta^2).$$
Mostrare che per $Re \to 0$ essa si riduce alla soluzione di Stokes.

Impostazione. Per $Re \to 0$ l'equazione (1.7) si riduce a $D^4\psi = 0$, lineare: la funzione di prova la soddisfa esattamente se i coefficienti dei termini che l'operatore non annulla sono nulli, e le condizioni di aderenza fissano i restanti. Ci si aspetta $\psi = (\frac{1}{2}r^2 - \frac{3}{4}r + \frac{1}{4r})(1-\zeta^2)$ e nessuna asimmetria avanti–dietro ($B = 0$).

Svolgimento.

1. Per $\psi = f(r)(1-\zeta^2)$ si ha $D^2\psi = (1-\zeta^2)(f'' - 2f/r^2)$ (perché $\partial_{\zeta\zeta}(1-\zeta^2) = -2$), e $D^4\psi = (1-\zeta^2)L^2 f$ con $L = \mathrm{d}^2/\mathrm{d}r^2 - 2/r^2$. Su r^2: $Lr^2 = 2-2 = 0$; su r: $-2/r$; su 1: $-2/r^2$; su $1/r$: $2/r^3 - 2/r^3 = 0$; su $1/r^2$: $6/r^4 - 2/r^4 = 4/r^4$. Quindi $Lf = -2A_1/r - 2A_2/r^2 + 4A_4/r^4$ e $L^2 f = -8A_2/r^4 + 72A_4/r^6$ (perché $L(1/r) = 0$, $L(1/r^2) = 4/r^4$, $L(1/r^4) = 18/r^6$).
2. Per la parte dispari, $\psi = g(r)\zeta(1-\zeta^2)$: $\partial_{\zeta\zeta}[\zeta(1-\zeta^2)] = -6\zeta$, quindi $D^2\psi = \zeta(1-\zeta^2)(g'' - 6g/r^2)$ con $L_2 = \mathrm{d}^2/\mathrm{d}r^2 - 6/r^2$, che annulla r^3 e $1/r^2$ ma non r e $1/r$: $L_2^2 g \propto B_1 r^{-3}\cdot(\dots) + B_3 r^{-5}\cdot(\dots)$. SymPy dà complessivamente
$$D^4\psi = \frac{8(1-\zeta^2)}{r^6}\left(-A_2 r^2 + 9A_4 + 3B_1 r^3\zeta - 3B_3 r\zeta\right).$$
3. $D^4\psi = 0$ per ogni r e ζ richiede $A_2 = A_4 = 0$ e $B_1 = B_3 = 0$. Le condizioni di aderenza sulla parte dispari, $B_1 + B_2 + B_3 + B_4 = 0$ e $B_1 - B_3 - 2B_4 = 0$, danno allora $B_2 = B_4 = 0$: nessuna asimmetria. Sulla parte pari, $\frac{1}{2} + A_1 + A_3 = 0$ e $1 + A_1 - A_3 = 0$ danno $A_1 = -\frac{3}{4}$, $A_3 = \frac{1}{4}$.

Risultato

$\psi = (\frac{1}{2}r^2 - \frac{3}{4}r + \frac{1}{4r})\sin^2\theta$: la funzione di corrente di Stokes per la sfera, da cui $C_D = 24/Re$. La funzione di prova (1.11) del capitolo, priva del termine in r, non può riprodurla: ecco perché a $Re = 5$ sovrastima la resistenza del 38%.

Commento. Il termine $A_1 r$, che a grande distanza cresce linearmente, è lo "Stokeslet": rappresenta la forza concentrata che la sfera esercita sul fluido e decade come $1/r$ nella velocità. Nella funzione di prova per Re finiti è assente perché, con l'inerzia, la perturbazione decade più in fretta fuori dalla scia (regime di Oseen); le due funzioni di prova sono adatte a regimi diversi, e nessuna delle due a entrambi. È la limitazione del metodo di Galerkin discussa nel Cap. 1: bisogna sapere in anticipo come è fatta la soluzione.

B.9 Esercizio 9 – Il problema di Neumann su quattro nodi

Testo

Calcolare gli autovalori della matrice $\mathbf{A}$ del sistema 4×4 dell'esempio del Cap. 5 (quadrato con $N = 2$), verificare che uno è nullo, fissare $\phi_{11} = 0$ e ricalcolare autovalori e condizionamento; confrontare il condizionamento del problema di Neumann "fissato" con quello di Dirichlet al crescere della griglia.

Impostazione. La matrice $\mathbf{A}$ dell'esempio del Cap. 5 ha righe a somma nulla: $(1,1,1,1)$ è un autovettore con autovalore 0. Fissando un nodo si elimina la singolarità ma non il cattivo condizionamento, che per il laplaciano discreto cresce come N^2 (rapporto fra gli autovalori estremi $\sim 8/(\pi^2 h^2)$).

Svolgimento.

1. Gli autovalori di $\mathbf{A}$ sono -4, -2, -2, 0 (la fonte riporta -4 per i primi tre: refuso); il vettore nullo è $(1,1,1,1)/2$ e la compatibilità richiede $\sum r_{ij} = 0$.

2. Con la prima riga sostituita da $\phi_{11} = 0$: autovalori $-3{,}4142$, -2, $-0{,}5858$ e 1 (riga fittizia), rcond $= 1/16 = 0{,}0625$ in norma 1 (la fonte: 0,125 con una diversa normalizzazione).
3. Su griglie $N \times N$ il numero di condizionamento in norma 2 vale, per Neumann fissato contro Dirichlet: 593 contro 32 ($N = 8$), 3295 contro 117 ($N = 16$), 16 802 contro 441 ($N = 32$): entrambi crescono come N^2, ma quello di Neumann è circa 40 volte maggiore.

Risultato

Matrice singolare di rango 3; dopo il fissaggio, soluzione unica ma condizionamento alto: i metodi iterativi convergono circa $\sqrt{40} \approx 6$ volte più lentamente (gradiente coniugato) o 40 volte (Jacobi) che per il problema di Dirichlet, e un metodo diretto sparso è la scelta giusta per griglie moderate.

Commento. Fissare un nodo è semplice ma rompe la simmetria della matrice e peggiora il condizionamento; alternative migliori sono imporre media nulla (sostituendo un'equazione con $\sum \phi_{ij} = 0$), oppure lasciare la matrice singolare e usare un metodo iterativo (gradiente coniugato) con termine noto compatibile, che converge alla soluzione a media nulla: la singolarità è innocua se il termine noto è ortogonale al vettore nullo, come sulla griglia MAC.

B.10 Esercizio 10 (progetto) – Corrente di Jeffery–Hamel con il metodo di Galerkin

Testo

Valutare analiticamente gli integrali (1.44) e la condizione di portata per $n = 2$, scrivere un programma per $Re = 14{,}164$ e $\alpha = 0{,}36$ rad, calcolare la velocità adimensionale $u_r \rho r / Q$ e la differenza di pressione $(p - p_0)\rho r^2/(Q\mu)$ in funzione di θ e confrontare con la soluzione esatta di Rosenhead.

Impostazione. Con $n = 2$, $f = C_1(\alpha^2 - \theta^2) + C_2(\alpha^2 - \theta^2)^2$ e il residuo (1.43) è un polinomio pari in θ di grado 8; gli integrali $\int_0^\alpha E(\theta)(\alpha^2 - \theta^2)^k \mathrm{d}\theta$ sono polinomi in α con coefficienti quadratici in C_0, C_1, C_2. Tre equazioni (una lineare, due quadratiche) per tre incognite: Newton–Raphson. La soluzione esatta si ottiene integrando numericamente la (1.45).

Svolgimento.

1. Condizione di portata: $2\int_0^\alpha f \,\mathrm{d}\theta = 2C_1 \cdot \frac{2}{3}\alpha^3 + 2C_2 \cdot \frac{8}{15}\alpha^5 = 1$ (con $\int_0^\alpha (\alpha^2 - \theta^2)\mathrm{d}\theta = \frac{2}{3}\alpha^3$ e $\int_0^\alpha (\alpha^2 - \theta^2)^2 \mathrm{d}\theta = \frac{8}{15}\alpha^5$).
2. Residuo: $E = Ref^2 + 2C_0 + 4f + f''$ con $f'' = -2C_1 + C_2(12\theta^2 - 4\alpha^2)$. Gli integrali pesati con $(\alpha^2 - \theta^2)$ e $(\alpha^2 - \theta^2)^2$ si calcolano in forma chiusa (il Programma 4.1c li ricava con SymPy e li risolve con Newton usando lo jacobiano esatto).
3. La soluzione esatta per tiro: incognite $f(0)$ e C_0, con $f'(0) = 0$ per simmetria, condizioni $f(\alpha) = 0$ e portata unitaria.

Risultato

$C_0 = -11{,}10$, $C_1 = 7{,}78$, $C_2 = 79{,}97$; $f(0) = u_r \rho r / Q|_{\theta=0} = 2{,}352$ contro 2,404 esatto; errore massimo sul profilo 0,052 (2%); la pressione $(p - p_0)\rho r^2/(Q\mu) = C_0 + 2f$ vale $-6{,}4$ sull'asse e $-11{,}1$ alle pareti (esatto: $-6{,}5$ e $-11{,}3$). Con $n = 3$ e 4 l'errore scende a 0,006 e 0,0007 (Fig. 1.8).

Commento. "Eccellente accordo" con la soluzione esatta, come affermano Snyder et al., significa 2% con due termini: per un problema con soluzione analitica e funzioni di prova polinomiali la convergenza è esponenziale. La pressione adimensionale $C_0 + 2f$ è massima sull'asse ($-6{,}5$) e minima alle pareti ($-11{,}3$), dove $f = 0$: un effetto puramente viscoso, perché nella corrente radiale il termine $2\mu\, \partial u_r/\partial\theta / r^2$ della componente θ dell'equazione del moto (1.40) richiede un gradiente di pressione trasversale.

B.11 Esercizio 11 (progetto) – Scatola riscaldata dall'alto con superficie libera

Testo

Ripetere l'esperimento del Programma 4.3 con $\theta = 0$ ovunque all'inizio tranne $\theta = 1$ sulla superficie superiore, mantenendo poi le temperature al contorno; la superficie superiore è una superficie libera rigida su cui vorticità e sforzo tangenziale si annullano; fluido inizialmente in quiete.

Impostazione. Stratificazione stabile: niente instabilità di Bénard. Il moto nasce dove la superficie calda tocca la parete fredda, per il termine $Gr\, \partial\theta/\partial X$, ed è confinato dalla stratificazione vicino alla superficie. Si aspettano due celle (una per metà), con moto discendente lungo le pareti, più deboli di quelle di Bénard.

Svolgimento. Nel Programma 4.3d, funzione `scatola_dall_alto`: temperatura iniziale $\theta = 1$ sulla riga $j = n$; vorticità sulla superficie $\Omega_{i,n} = 0$ invece della (3.21); tutto il resto invariato (stesso `pnew`, stesso `sorlx`).

Risultato

$|\Psi|_{\max} = 0{,}033$ a $T = 0{,}05$, 0,17 a 0,25, 0,39 a 1, 0,46 a 5, tutta positiva nella metà sinistra (moto antiorario: discesa lungo la parete fredda, risalita al centro). La cella, inizialmente confinata nello spigolo superiore, si estende a tutta la lunghezza e a tutta l'altezza, con le isoterme che si inarcano verso il basso al centro (Fig. 3.6a); a $T = 5$ il SOR converge in una sola iterazione: stato stazionario raggiunto.

Commento. È un flusso di convezione *forzata dal contorno*, non un'instabilità: esiste per ogni Gr, senza soglia, perché un gradiente orizzontale di temperatura genera sempre vorticità. L'intensità, metà di quella di Bénard allo stesso Gr, e la stazionarietà raggiunta rapidamente sono tipiche dei moti stabilmente stratificati.

B.12 Esercizio 12 (progetto) – Strato con gradiente orizzontale periodico

Testo

Fluido fra due piastre orizzontali indefinite; all'inizio $\theta = 0$ ovunque tranne sulla piastra superiore, dove si impone e si mantiene $\theta = 1 + 0{,}5\sin(2\pi X/\lambda)$; piastra inferiore a $\theta = 0$. Condizioni periodiche in X sul tratto $0 \le X \le \lambda$, con $S_{1,j} = S_{m,j}$, $S_{0,j} = S_{m-1,j}$, $S_{2,j} = S_{m+1,j}$ per ogni variabile S, cosicché $V_{1,j} = -(\Psi_{2,j} - \Psi_{m-1,j})/2h$. Con $Pr = 6{,}75$, $Gr = 1000$ e $\lambda = 2{,}5$, tracciare linee di corrente e isoterme a vari istanti.

Impostazione. Il gradiente orizzontale sulla piastra superiore genera una coppia di celle per lunghezza d'onda: fluido che sale verso la parte calda e scende sotto quella fredda. La periodicità richiede di modificare il risolutore di Poisson (il SOR con le condizioni di Fig. 3.1 non va più bene) e le differenze ai bordi.

Svolgimento. Nel Programma 4.3d, funzione `strato_periodico`: nodi $X_i = ih$, $i = 0, \dots, N_X - 1$ con $N_X = \lambda/h = 20$ e periodicità gestita con `np.roll` e con una colonna fantasma per lato negli array passati a `pnew`; equazione di Poisson con matrice sparsa periodica fattorizzata una volta; vorticità su entrambe le piastre dalle (3.21).

Risultato

Due celle controrotanti di uguale intensità per lunghezza d'onda, $|\Psi|_{\max} = 0{,}049$ a $T = 0{,}05$, 0,187 a 0,25 e 1, 0,208 a 5 (Fig. 3.6b); le isoterme, orizzontali all'inizio, vengono deformate dalle celle in modo da trasportare calore verso il basso sotto la zona calda.

Commento. La condizione periodica è la più naturale per gli strati indefiniti e sarà usata di nuovo nei vortici di Taylor. La circolazione raggiunge il regime in un tempo $T \approx 0{,}3$, dell'ordine di h^2 di diffusione viscosa su mezza altezza: con $Pr = 6{,}75$ la quantità di moto si diffonde quasi sette volte più in fretta del calore, e la temperatura continua ad aggiustarsi quando il moto è già stazionario.

B.13 Esercizio 13 (progetto) – Vortici di Taylor

Testo

Integrare le equazioni (3.24)–(3.27) partendo dalla corrente di Couette circolare (3.29) perturbata con la (3.30), con $A = 2$, $B = 0$, $\lambda = 4$, $Re = 1000$, $h = 0{,}125$, $\Delta T = 0{,}02$, e mostrare lo sviluppo dei vortici anulari di Taylor.

Impostazione. Per il criterio di Rayleigh la corrente con il cilindro esterno fermo è instabile a ogni velocità in assenza di viscosità; a $Re = 1000$ si è ben oltre la soglia viscosa e la perturbazione deve crescere esponenzialmente, poi saturare in celle stazionarie. Il momento torcente sul cilindro interno deve aumentare.

Svolgimento. Programma 4.3b (Cap. 3): schema di Torrance per Ω e W, Poisson diretta periodica per Ψ, formula di Thom alle pareti, coppia da $\partial W/\partial R - 2W/R$ con differenza a tre punti.

Risultato

Crescita esponenziale con $\sigma = 0{,}187\omega_i$ fino a $T \approx 20$, saturazione a $|\Psi|_{\max} = 0{,}10$; due celle controrotanti per lunghezza d'onda; coppia da $-16{,}8$ (Couette) a $-48{,}2$ a $T = 60$ (Fig. 3.4). Con $\lambda = 2$: $\sigma = 0{,}356$, coppia $-51{,}7$.

Commento. Il moto meridiano resta debole (0,1 in unità di $r_i^2 \cdot r_i\omega_i$) rispetto al moto azimutale, ma triplica la coppia: il trasporto convettivo del momento angolare è molto più efficace della diffusione. Il risultato dipende dalla lunghezza d'onda imposta dalla periodicità, perché la cella non può scegliere la propria dimensione: con $\lambda = 4$ si ottengono celle allungate, con $\lambda = 2$ celle quasi quadrate e più efficienti. Nel modulo precedente la teoria lineare dava le soglie; qui si vede lo stato finito.

B.14 Esercizio 14 (progetto) – Tubo rotante con restringimento

Testo

Trovare la soluzione stazionaria della corrente in un tubo rotante con brusco restringimento (Fig. 3.7): all'ingresso rotazione rigida ω_0 e velocità assiale uniforme u_0; $Re = u_0 r_0/\nu = 50$; tracciare il moto meridiano per $Ro = 2$, 1, 0,5, 0,2.

Impostazione. Le equazioni sono le (3.31) e (3.27), con Re e Ro come parametri. Al decrescere del numero di Rossby la rotazione domina sul moto assiale e ci si aspetta, a monte del restringimento, una regione di ricircolo sull'asse (rottura del vortice).

Svolgimento (impostazione del calcolo).

1. *Dominio e griglia.* Tratto di tubo di lunghezza L_1 (per esempio $2{,}5r_0$) con raggio 1 seguito da un tratto di raggio 0,2; griglia uniforme con $h = 0{,}05$ nel piano (Z, R) in cui i nodi del tratto stretto con $R > 0{,}2$ sono esclusi (parete a gradino): è una geometria "a scalino", trattabile con la stessa griglia cartesiana marcando i nodi solidi.
2. *Condizioni al contorno.* Asse: $\Psi = 0$, $\Omega = 0$, $W = 0$, velocità assiale dalla $U_{i,1} = 2\Psi_{i,2}/h^2$; parete (compreso il gradino): $\Psi = \frac{1}{2}$ (portata adimensionale $\int_0^1 UR\,\mathrm{d}R = \frac{1}{2}$ per $U = 1$), $W = R^2$ (rotazione rigida della parete), vorticità dalla formula di Thom con la distanza normale appropriata; ingresso: $\Psi = \frac{1}{2}R^2$, $W = R^2$, $\Omega = 0$; uscita: $\partial\Psi/\partial Z = 0$, $\partial\Omega/\partial Z = \partial W/\partial Z = 0$.
3. *Avanzamento.* Schema di Torrance per Ω e W con il passo scelto dal criterio (3.19) e il termine di accoppiamento $-(1/2Ro^2)(W/R^3)\partial W/\partial Z$; Poisson diretta per Ψ (matrice con i nodi solidi eliminati); iterazione fino allo stato stazionario, con continuazione in Ro dal valore più alto al più basso.
4. *Diagnostica.* Il segno di U sull'asse a monte del restringimento: $U < 0$ indica la bolla di ricircolo; la sua lunghezza in funzione di Ro.

Risultato atteso

Per $Ro = 2$ e 1 la corrente converge regolarmente verso il restringimento accelerando (e, per conservazione del momento angolare, aumentando la velocità di rotazione nel tratto stretto fino a 25 volte, W/R^2 con $R = 0{,}2$); per $Ro = 0{,}5$ e 0,2 la rotazione impedisce al fluido vicino all'asse di contrarsi e compare una bolla di ricircolo stazionaria sull'asse a monte del gradino, che si allunga al decrescere di Ro, come nella soluzione non viscosa di Yih per il pozzo rotante. La realizzazione numerica è lasciata come estensione: richiede la gestione del gradino nella matrice di Poisson e nella vorticità di parete, non diversa in linea di principio da quella delle cavità.

Commento. La fisica è quella del *vortex breakdown*: in un vortice con moto assiale, quando la rotazione è forte rispetto all'avanzamento (Ro piccolo, o "numero di swirl" grande), le onde inerziali possono propagarsi contro corrente e la corrente diventa "subcritica", in analogia con il risalto idraulico; un restringimento a valle produce allora una bolla di ricircolo a monte. È il fenomeno che limita la portanza delle ali a delta ad alta incidenza, quando i vortici del bordo d'attacco "esplodono" sopra l'ala.

B.15 Esercizio 15 (progetto) – Il caffè sulla scala a chiocciola

Testo

Verificare al calcolatore l'affermazione della cameriera: caffè inizialmente fermo in una tazza di raggio $r_0 = 4$ cm e altezza $2r_0$ messa improvvisamente in rotazione a $\omega_0 = 0{,}1\pi$ rad/s, $\nu = 10^{-6}$ m^2/s ($Re = 500$), superficie libera piana senza sforzi; ricavare due espressioni per ΔT analoghe alla (3.19) dalle equazioni (3.26) e (3.27), usare a ogni passo il minimo su tutti i nodi e seguire il moto fino a $T = 8$.

Impostazione. L'avviamento avviene per pompaggio di Ekman, su un tempo $\sqrt{Re}/\omega_0 \approx 22/\omega_0$: a $T = 8$ ci si aspetta che solo la parte esterna del liquido ruoti. I due criteri di stabilità: per la (3.26) e la (3.27) i coefficienti diffusivi sono entrambi $1/Re$ e i termini convettivi hanno la stessa forma, quindi i due limiti coincidono, $\Delta T \le [\,(\ldots)/2h + 4/(h^2 Re)\,]^{-1}$, con l'avvertenza che i termini $-\Omega/R^2$ e $-VW/R$ aggiungono contributi al coefficiente del nodo centrale ($-\Delta T/(ReR^2)$ e $-\Delta T\,V/R$) di cui si può tenere conto nel criterio o che si trascurano con un fattore di sicurezza.

Svolgimento. Programma 4.3c: griglia 41×81 ($h = 0{,}025$), ΔT dal criterio con fattore 0,8 a ogni passo, Poisson diretta, Thom sulle pareti, $\Omega = 0$ e $\partial W/\partial Z = 0$ sulla superficie libera.

Risultato

ΔT fra 0,012 e 0,034; momento angolare acquisito in frazione della rotazione rigida: 0,11 a $T = 0{,}5$, 0,25 a 2, 0,51 a 8 (con $h = 0{,}05$: 0,50); a $T = 8$ la circolazione secondaria occupa il terzo esterno della tazza e il centro è fermo ($u_\theta = 1{,}5\%$ del valore rigido a metà raggio). La cameriera ha torto: dopo 25 s il caffè non è "mescolato" (Fig. 3.5).

Commento. Due osservazioni per la didattica. La prima: il tempo di avviamento scala come $\sqrt{Re}$ e non come Re, grazie alla circolazione secondaria; senza di essa (per esempio in un contenitore con pareti laterali ma senza fondo, una corona infinita) l'avviamento sarebbe puramente diffusivo e 20 volte più lento. La seconda: la risoluzione dello strato di Ekman ($\delta_E = 0{,}045 r_0$, due celle) è marginale, ma il momento angolare totale è insensibile al dettaglio dello strato, perché è governato dal bilancio globale della coppia alle pareti; i dettagli del moto secondario lo sono di più. È il tipo di verifica (griglia dimezzata, confronto delle grandezze integrali e locali) che ogni calcolo dovrebbe includere.

B.16 Esercizio 16 (progetto) – Cavità con lo schema AB–CN a passi frazionari

Testo

Integrare le equazioni (5.1)–(5.2) per la cavità con coperchio mobile a $Re = 100$ e 400 con lo schema semi-implicito Crank–Nicolson/Adams–Bashforth a passi frazionari su griglia MAC 21×21; risolvere l'equazione di Neumann per la pressione con un metodo diretto fissando $P = 0$ in un nodo; assumere lo stato stazionario quando la norma dell'errore fra iterati successivi è scesa di tre ordini di grandezza; calcolare ψ e ω nei nodi di pressione e confrontare con Ghia et al. e Kim e Moin.

Impostazione. Rispetto al Programma 4.5 cambiano il predittore (sistema di Helmholtz per componente, con matrice costante) e il passo (solo convettivo). Con 21×21 celle ci si aspetta un errore di qualche percento a $Re = 100$ e maggiore a $Re = 400$, dove gli strati limite sono più sottili della cella.

Svolgimento. Programma 4.5c: matrici di Helmholtz con i nodi fittizi (`matrice_viscosa`), Poisson con nodo fissato (`matrice_pressione` del Programma 4.5), AB con avvio di Eulero, criterio di arresto sul residuo $\max|\Delta\mathbf{V}|/\Delta t < 10^{-6}$ (più severo dei tre ordini di grandezza richiesti).

Risultato

21×21: $\psi_c = -0{,}1000$, $\omega_c = -2{,}98$ a $Re = 100$ ($-0{,}1034$, $-3{,}17$ Ghia), $\psi_c = -0{,}0989$, $\omega_c = -2{,}10$ a $Re = 400$ ($-0{,}1139$, $-2{,}29$); meno di un secondo di calcolo. 65×65: stessi valori dello schema esplicito con 2–3 volte meno passi, e 56 volte meno a $Re = 1$.

Commento. Il progetto chiede anche la forma della norma dell'errore: la norma del massimo $\|F\|_\infty = \max_i \sum_j |f_{ij}|$ o quella di Frobenius $\|F\|_f = (\sum_{ij} f_{ij}^2)^{1/2}$ della differenza fra iterati; il programma usa il massimo del modulo diviso per Δt, che è una stima di $\|\partial\mathbf{V}/\partial t\|_\infty$ e non dipende dal passo. La forma incrementale del metodo (Approfondimenti del Cap. 5) elimina l'errore di separazione e andrebbe sempre preferita.

B.17 Esercizio 17 (progetto) – La cavità mossa dal vento

Testo

Risolvere la cavità con sforzo tangenziale costante $\tau = \rho_a u_*^2$ imposto sulla superficie superiore (interfaccia aria–acqua senza onde), con u_* e L come scale e $Re = u_* L/\nu$.

Impostazione. La condizione è sulla derivata: $\mu\,\partial u/\partial y = \rho_a u_*^2$, cioè $\partial u/\partial y = (\rho_a/\rho)Re$ in unità di u_* e L. Sulla griglia MAC si impone con il nodo fittizio $u_{N+1} = u_N + h\,\partial u/\partial y$. Le velocità attese sono piccole ($\rho_a/\rho = 10^{-3}$) e la struttura simile a quella del coperchio mobile.

Svolgimento. Programma 4.5b, che riusa la funzione `cavita` del Programma 4.5 con il parametro `sforzo`; $Re = 100$ e 2000, griglia 41×41, vento verso sinistra.

Risultato

Velocità superficiale massima $0{,}014u_*$ ($Re = 100$) e $0{,}15u_*$ ($Re = 2000$); $\Delta\psi = 1{,}3 \times 10^{-3}$ e $1{,}4 \times 10^{-2}$; corrente di ritorno in profondità (Fig. 5.10).

Commento. Una stima a mano: in uno strato di Couette di spessore δ la velocità superficiale è $u_s = \tau\delta/\mu = (\rho_a/\rho)Re\,\delta\,u_*$; con $\delta \approx 0{,}1L$ a $Re = 100$ si ha $u_s \approx 0{,}01u_*$, e con $\delta \approx 0{,}08L$ a $Re = 2000$, $u_s \approx 0{,}16u_*$: coerente con il calcolo. In una vasca di laboratorio profonda 1 cm con $u_* = 0{,}2$ m/s ($Re = 2000$) la corrente superficiale è di circa 3 cm/s; nei laghi reali, dove la corrente è turbolenta e il modello laminare non vale, la regola empirica dei naviganti dà una deriva superficiale del 2–3% della velocità del vento.

B.18 Esercizio 18 (progetto) – Cavità ω–ψ con il metodo ADI

Testo

Modificare il risolutore della cavità in formulazione vorticità–funzione di corrente per risolvere l'equazione di ψ con il metodo ADI di Peaceman–Rachford su 31×31 nodi interni per $Re = 1$, 100, 400; usare $\rho \approx 8/N$ oppure la successione di Wachspress (6.21)–(6.22), e confrontare le velocità di convergenza con i due criteri (parametro fisso e successione) ai vari Re.

Impostazione. La Poisson va risolta a convergenza a ogni passo; la stima iniziale (il ψ precedente) è buona e pochi passi ADI bastano. La successione di Wachspress deve ridurre di molto il numero di iterazioni rispetto a ρ fisso, soprattutto partendo da zero.

Svolgimento. Programma 4.6: `poisson_adi` con i sistemi tridiagonali per righe e per colonne risolti con `solve_banded`; ρ scalare o successione ciclica; parametri di Wachspress da (6.21) (per $M = 11$ riproduce 0,51033610, 1,85949096, 6,77535187, 24,68707504).

Risultato

Da zero, tolleranza 10^{-6}: SOR ottimo 72 iterazioni, ADI $\rho = 8/M$ 61, ADI $\rho = 1$ 205, ADI Wachspress 11. Nel ciclo temporale: 4–6 doppi passi per passo contro 9 del SOR, e tempo totale 2 s contro 36 a $Re = 100$. Il numero di passi ADI per passo temporale cresce poco con Re (4,1 a $Re = 1$, 5,6 a 100, 5,3 a 400).

Commento. Con la successione di Wachspress il costo del risolutore ellittico diventa trascurabile rispetto al resto, e si può pensare di usare lo stesso ADI anche per l'equazione della vorticità in forma implicita (schema ADI di Peaceman–Rachford nel tempo fisico), eliminando la condizione diffusiva. Per la precisione: il criterio di "tre ordini di grandezza" suggerito dalla fonte è meno severo del 10^{-6} assoluto usato qui, e richiede 1–2 cicli di Wachspress.

B.19 Esercizio 19 (progetto) – Corrente intorno al cilindro

Testo

Risolvere le (6.27)–(6.29) con le condizioni (6.30), $\psi = \omega = 0$ sugli assi, $\omega = 0$ e (6.31) sul contorno esterno, con un metodo di avanzamento temporale a scelta; determinare il numero di Reynolds di separazione (esperimenti: 5–7), la lunghezza della bolla in funzione di Re (confronto con Panton) e la resistenza per $2 \le Re \le 38$ (confronto con i dati).

Impostazione. Eulero esplicito è sufficiente (il passo è limitato dalla diffusione alla parete, dove g è minimo); la Poisson con coefficiente g a termine noto ha matrice costante. Separazione: cambio di segno della vorticità di parete partendo dal punto posteriore; bolla: cambio di segno di ψ sulla prima linea sopra l'asse posteriore; resistenza dalla (6.33). Ci si aspetta $Re_{\text{sep}} \approx 6$, bolla lineare in Re, C_D decrescente da ~ 8 a $\sim 1{,}6$.

Svolgimento. Programma 4.6b, con le verifiche di griglia ($N = 40$, 60, 80) e di posizione del contorno esterno (ξ_∞ da 0,75 a 1,5).

Risultato

$Re_{\text{sep}} = 6{,}3$; $\theta_s = 29{,}3°$, 43,5°, 52,1° e $L/d = 0{,}24$, 0,88, 1,86 a $Re = 10$, 20, 36; $C_D = 7{,}87$, 3,03, 2,16, 1,67 a $Re = 2$, 10, 20, 36, del 6% superiori a Dennis–Chang per effetto del contorno esterno a $23a$; con il contorno a $111a$, $C_D = 2{,}02$ e $L/a = 1{,}83$ a $Re = 20$ (Fornberg: 2,00, 1,82). Figg. 6.6–6.7.

Commento. Il progetto insegna una lezione di validazione: tre fonti di errore (griglia, contorno esterno, schema) vanno separate, e solo la seconda conta qui. La lunghezza della bolla segue la legge empirica $L/d \approx 0{,}05(Re - 6)$ riportata da Panton; la resistenza di pressione supera quella d'attrito già a $Re = 2$ (per la sfera succedeva a $Re \approx 90$), perché il cilindro bidimensionale ha una scia relativamente più larga.

B.20 Esercizio 20 – Derivazione della formula della resistenza del cilindro

Testo

Ricavare la (6.33) valutando le componenti normale e azimutale dell'equazione del moto sulla superficie del cilindro e trasformandole nelle coordinate ξ–η; ricavare anche la (6.34).

Impostazione. Sulla superficie, per l'aderenza, i termini convettivi sono nulli e l'equazione del moto lega il gradiente di pressione al termine viscoso, che in due dimensioni si esprime con la sola vorticità: $\nabla^2\mathbf{V} = -\nabla\times(\omega\mathbf{k})$. La resistenza è l'integrale della pressione (componente x della normale) e dello sforzo tangenziale $\mu\omega_w$.

Svolgimento.

1. *Sforzo alla parete.* In coordinate polari $\tau_{r\theta} = \mu[r\,\partial(v_\theta/r)/\partial r + (1/r)\partial v_r/\partial\theta]$; sulla parete $v_r = v_\theta = 0$ per ogni θ, quindi $\tau_{r\theta} = \mu\,\partial v_\theta/\partial r$, e la vorticità $\omega = \partial v_\theta/\partial r + v_\theta/r - (1/r)\partial v_r/\partial\theta$ si riduce a $\partial v_\theta/\partial r$: $\tau_w = \mu\omega_w$.

2. *Attrito.* Con θ misurato dal punto posteriore (asse x nel verso della corrente), lo sforzo τ_w agisce lungo $\hat{\boldsymbol{\theta}}$, la cui componente x è $-\sin\theta$. Sull'intero cilindro, $D_s = -\int_0^{2\pi}\mu\omega_w a\sin\theta\,\mathrm{d}\theta = -2\mu a\int_0^{\pi}\omega_w\sin\theta\,\mathrm{d}\theta$ (simmetria). Con $\omega_w = (U_\infty/a)\tilde\omega$, $\theta = \pi\eta$, $\mu/(\rho U_\infty a) = 2/Re$:

$$\frac{D_s}{\rho U_\infty^2 a} = -\frac{2\mu}{\rho U_\infty a}\pi\int_0^1\tilde\omega\sin(\pi\eta)\,\mathrm{d}\eta = -\frac{4\pi}{Re}\int_0^1\tilde\omega\sin(\pi\eta)\,\mathrm{d}\eta.$$

   Nelle zone attaccate della superficie superiore il fluido scorre nel verso $-\hat{\boldsymbol{\theta}}$ (dal punto anteriore a quello posteriore) e $\omega_w < 0$: $D_s > 0$.

3. *Pressione.* La forza di pressione sul cilindro è $-\oint p\,\mathbf{n}\,\mathrm{d}s$ con $\mathbf{n} = \hat{\mathbf{r}}$, la cui componente x è $\cos\theta$: $D_p = -\int_0^{2\pi} p\cos\theta\,a\,\mathrm{d}\theta$, cioè, con $P = p/(\rho U_\infty^2)$, $D_p/(\rho U_\infty^2 a) = -2\pi\int_0^1 P\cos(\pi\eta)\,\mathrm{d}\eta$. Sommando si ha la (6.33), e dividendo per $\frac{1}{2}\rho U_\infty^2 d = \rho U_\infty^2 a$ si ha C_D, la (6.32).

4. *Gradiente di pressione.* La componente θ dell'equazione del moto (6.1) sulla parete, con $\mathbf{V} = 0$, è $(1/r)\partial p/\partial\theta = \mu(\nabla^2\mathbf{V})_\theta$. In due dimensioni $\nabla^2\mathbf{V} = \nabla(\nabla\cdot\mathbf{V}) - \nabla\times\nabla\times\mathbf{V} = -\nabla\times(\omega\mathbf{k})$, e $\nabla\times(\omega\mathbf{k}) = (1/r)(\partial\omega/\partial\theta)\hat{\mathbf{r}} - (\partial\omega/\partial r)\hat{\boldsymbol{\theta}}$: quindi $(\nabla^2\mathbf{V})_\theta = \partial\omega/\partial r$ e $\partial p/\partial\theta = \mu a\,\partial\omega/\partial r$. Adimensionalizzando ($p = \rho U_\infty^2 P$, $\theta = \pi\eta$, $r = ae^{\pi\xi}$, $\partial/\partial r = (1/\pi a)\partial/\partial\xi$ in $\xi = 0$): $\rho U_\infty^2\pi^{-1}\partial P/\partial\eta = \mu a(U_\infty/a)(1/\pi a)\partial\tilde\omega/\partial\xi$, da cui

$$\frac{\partial P}{\partial\eta} = \frac{\mu}{\rho U_\infty a}\frac{\partial\tilde\omega}{\partial\xi} = \frac{2}{Re}\frac{\partial\tilde\omega}{\partial\xi},$$

   la (6.34). La componente radiale, $\partial p/\partial r = \mu(\nabla^2\mathbf{V})_r = -(\mu/r)\partial\omega/\partial\theta$, dà il gradiente normale di pressione alla parete, non nullo a Re finito: è il motivo per cui, a differenza dello strato limite, la pressione sulla superficie non coincide con quella appena fuori.

Risultato

$C_D = -\dfrac{4\pi}{Re}\displaystyle\int_0^1 \omega \sin(\pi\eta)\,\mathrm{d}\eta - 2\pi \int_0^1 P\cos(\pi\eta)\,\mathrm{d}\eta$ con $\partial P/\partial\eta = (2/Re)\partial\omega/\partial\xi$: la costante di integrazione di P non contribuisce ($\int_0^1 \cos\pi\eta\,\mathrm{d}\eta = 0$), e la pressione di ristagno assunta nel nodo $(1,1)$ serve solo per il grafico di C_p. Il Programma 4.6b usa esattamente queste formule, con la regola dei trapezi, e il segno dei risultati ($C_D > 0$, $\omega_w < 0$ nelle zone attaccate) ne conferma le convenzioni.

Commento. La formula mostra che in formulazione ω–ψ *tutta* la resistenza si ricava dalla vorticità di parete: l'attrito direttamente, la pressione per integrazione della sua derivata normale. È una proprietà generale dei flussi bidimensionali, sfruttata nei metodi a vortici, e spiega perché l'accuratezza della vorticità alla parete (la formula di Thom) sia così importante.

Bibliografia

Testi di riferimento del modulo

[1] S. Biringen, C.-Y. Chow, *An Introduction to Computational Fluid Mechanics by Example*, Wiley, Hoboken, 2011. Capitolo 4: *Numerical Solution of the Incompressible Navier–Stokes Equation*, pp. 215–297. Programmi MATLAB su `www.wiley.com/go/biringen`.

[2] C.-Y. Chow, *An Introduction to Computational Fluid Mechanics*, Wiley, New York, 1979 (edizione originale, con programmi FORTRAN).

Testi generali di fluidodinamica numerica

[3] P. J. Roache, *Computational Fluid Dynamics*, Hermosa, Albuquerque, 1972 (ed. riveduta 1976; nuova edizione *Fundamentals of Computational Fluid Dynamics*, 1998). Cap. 3: schemi upwind, proprietà trasportiva, viscosità artificiale, analisi di Hirt e della perturbazione discreta (Sez. A-5, A-8–A-11), condizioni al contorno per la vorticità.

[4] R. Peyret, T. D. Taylor, *Computational Methods for Fluid Flow*, Springer, New York, 1983. Cap. 6: metodi in variabili primitive, griglia MAC, condizioni di stabilità (p. 148), nodi fittizi (p. 150); Sez. 6.4: corrente intorno al cilindro (pp. 207–212).

[5] J. H. Ferziger, M. Perić, *Computational Methods for Fluid Dynamics*, 3ª ed., Springer, Berlin, 2002; J. H. Ferziger, *Numerical Methods for Engineering Application*, 2ª ed., Wiley, New York, 1998 (metodi ADI, scelta del parametro, p. 265; costo della fattorizzazione LU, p. 5).

[6] A. R. Mitchell, *Computational Methods in Partial Differential Equations*, Wiley, London, 1969. Problema di Neumann e singolarità della matrice (pp. 118–121); parametri ADI (p. 111).

[7] R. F. Warming, B. J. Hyett, *The modified equation approach to the stability and accuracy analysis of finite-difference methods*, Journal of Computational Physics **14** (1974) 159–179.

[8] C. W. Hirt, *Heuristic stability theory for finite-difference equations*, Journal of Computational Physics **2** (1968) 339–355.

[9] A. Thom, C. J. Apelt, *Field Computations in Engineering and Physics*, Van Nostrand, London, 1961 (analisi della perturbazione discreta; formula di Thom per la vorticità di parete, 1933).

Metodo di Galerkin e sfera

[10] M. Kawaguti, *The critical Reynolds number for the flow past a sphere*, Journal of the Physical Society of Japan **10** (1955) 694–699.

[11] S. Taneda, *Experimental investigation of the wake behind a sphere at low Reynolds numbers*, Journal of the Physical Society of Japan **11** (1956) 1104–1108.

[12] L. J. Snyder, T. W. Spriggs, W. E. Stewart, *Solution of the equations of change by Galerkin's method*, AIChE Journal **10** (1964) 535–540.

[13] L. Rosenhead, *The steady two-dimensional radial flow of viscous fluid between two inclined plane walls*, Proceedings of the Royal Society A **175** (1940) 436–467.

[14] L. D. Landau, E. M. Lifshitz, *Fluid Mechanics*, Pergamon, London, 1959 (2ª ed. 1987), §23: corrente fra pareti inclinate (soluzione di Jeffery–Hamel in funzioni ellittiche, p. 81).

Instabilità, convezione e fluidi in rotazione

[15] S. Chandrasekhar, *Hydrodynamic and Hydromagnetic Stability*, Clarendon Press, Oxford, 1961 (Dover, 1981). Cap. II: convezione di Bénard, approssimazione di Boussinesq (Sez. II.8); Cap. VII: stabilità della corrente di Couette fra cilindri rotanti, criterio di Rayleigh, numero di Taylor.

[16] P. G. Drazin, W. H. Reid, *Hydrodynamic Stability*, 2ª ed., Cambridge University Press, 2004. Cap. 2: convezione termica; Cap. 3: instabilità centrifuga, vortici di Taylor e di Görtler.

[17] K. E. Torrance, *Comparison of finite-difference computations of natural convection*, Journal of Research of the National Bureau of Standards B **72** (1968) 281–301; K. E. Torrance, J. A. Rockett, *Numerical study of natural convection in an enclosure with localized heating from below*, Journal of Fluid Mechanics **36** (1969) 33–54.

[18] H. P. Greenspan, *The Theory of Rotating Fluids*, Cambridge University Press, 1968. Strati di Ekman, spin-up, onde inerziali.

Variabili primitive e metodi di proiezione

[19] A. J. Chorin, *Numerical solution of the Navier–Stokes equations*, Mathematics of Computation **22** (1968) 745–762.

[20] J. Kim, P. Moin, *Application of a fractional-step method to incompressible Navier–Stokes equations*, Journal of Computational Physics **59** (1985) 308–323.

[21] A. Huser, S. Biringen, *Calculation of two-dimensional shear-driven cavity flows at high Reynolds numbers*, International Journal for Numerical Methods in Fluids **14** (1992) 1087–1109.

[22] F. H. Harlow, J. E. Welch, *Numerical calculation of time-dependent viscous incompressible flow of fluid with free surface*, Physics of Fluids **8** (1965) 2182–2189 (metodo MAC).

[23] S. V. Patankar, *Numerical Heat Transfer and Fluid Flow*, Hemisphere, Washington, 1980 (algoritmo SIMPLE; con D. B. Spalding, International Journal of Heat and Mass Transfer **15** (1972) 1787–1806).

[24] U. Ghia, K. N. Ghia, C. T. Shin, *High-Re solutions for incompressible flow using the Navier–Stokes equations and a multigrid method*, Journal of Computational Physics **48** (1982) 387–411.

Metodi ADI e corrente intorno al cilindro

[25] D. W. Peaceman, H. H. Rachford, *The numerical solution of parabolic and elliptic differential equations*, Journal of the Society for Industrial and Applied Mathematics **3** (1955) 28–41.

[26] E. L. Wachspress, *CURE: a generalized two-space-dimension multigroup coding for the IBM-704*, rapporto KAPL-1724, Knolls Atomic Power Laboratory, 1957; E. L. Wachspress, *Iterative Solution of Elliptic Systems*, Prentice-Hall, Englewood Cliffs, 1966 (parametri ottimi per l'ADI).

[27] Ta Phuoc Loc, *Numerical analysis of unsteady secondary vortices generated by an impulsively started circular cylinder*, Journal of Fluid Mechanics **100** (1980) 111–128.

[28] S. C. R. Dennis, G.-Z. Chang, *Numerical solutions for steady flow past a circular cylinder at Reynolds numbers up to 100*, Journal of Fluid Mechanics **42** (1970) 471–489.

[29] B. Fornberg, *A numerical study of steady viscous flow past a circular cylinder*, Journal of Fluid Mechanics **98** (1980) 819–855.

[30] R. L. Panton, *Incompressible Flow*, Wiley, New York, 1984 (4ª ed. 2013). Cap. 15: corrente intorno al cilindro a bassi numeri di Reynolds, lunghezza della bolla, resistenza (Fig. 15-15, p. 387).

[31] E. R. Benton, G. W. Platzman, *A table of solutions of the one-dimensional Burgers equation*, Quarterly of Applied Mathematics **30** (1972) 195–212.